

PART VIII - SECURITY AND TRUST ACROSS SERVICES

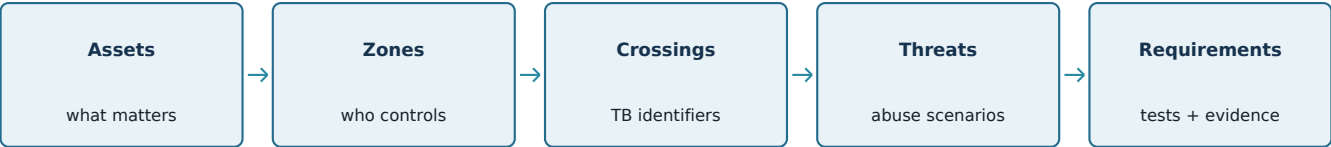
43. Threat and Trust Boundaries Before Mechanisms

NEXUS-1 names every distributed trust crossing before choosing authentication, authorization, certificate, secret or network mechanisms. Human, workload, data, messaging, telemetry and administrative authority remain separate.

CHAPTER DECISION

ADR-043 defines twelve stable trust boundaries across the operator/BFF path, owner services, RabbitMQ, owner databases, telemetry, diagnostics, repair, migration, deployment and configuration. Trust is explicit, local, bounded, fresh, revocable and never transitive. A focused ledger links each abuse scenario to assets, requirements, planned verification and residual uncertainty. Mechanism implementation is deferred to Chapters 44-48, and production security maturity is not claimed.

PROOF PATH 43-A - ARCHITECTURE TO SECURITY OBLIGATION



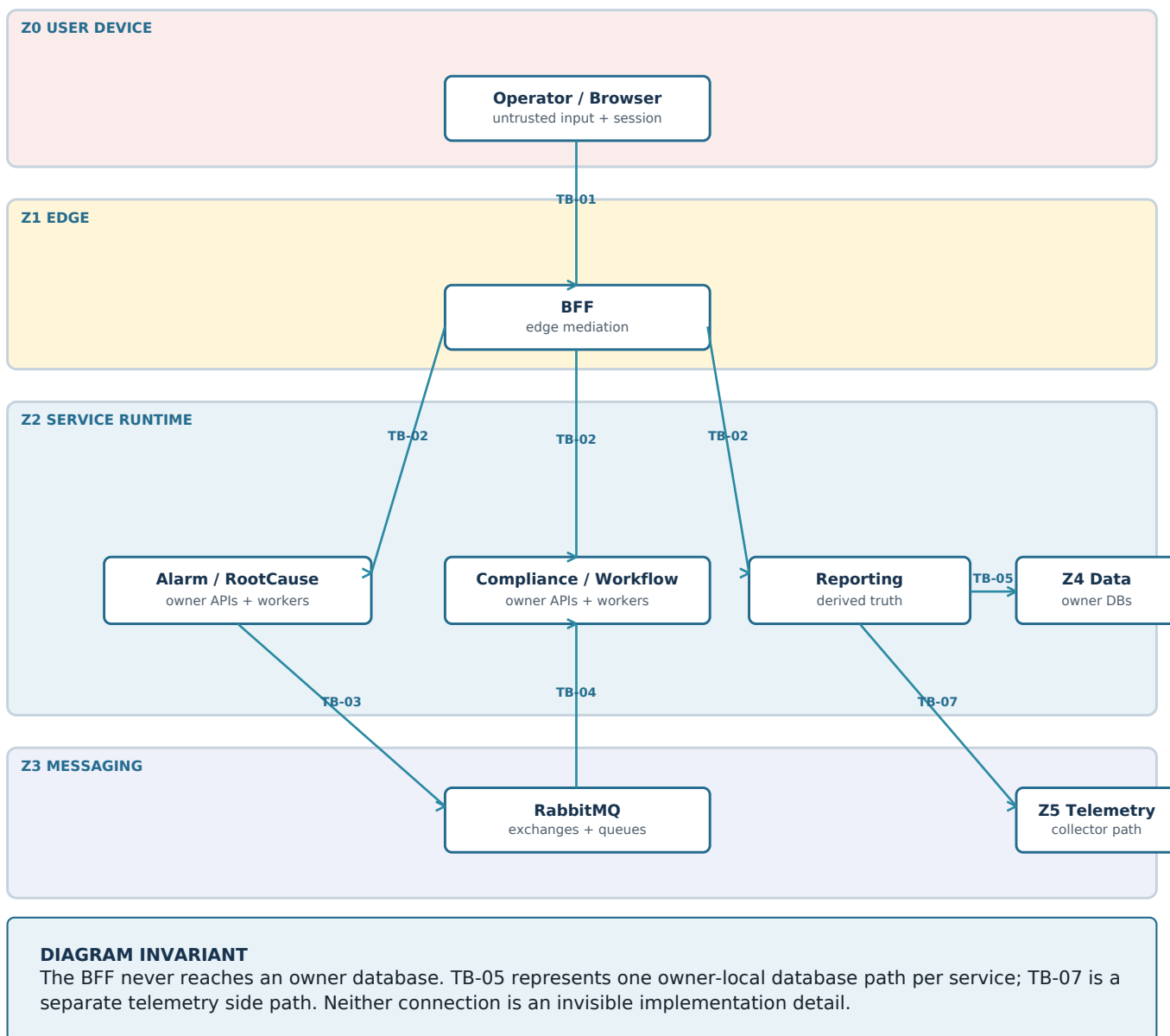
GUARANTEE LEDGER 43-A

Specified	Explicitly rejected
two security data-flow diagrams	one trusted internal network
human/workload identity separation	user token as service trust
owner-local authorization	BFF as global authority
telemetry/admin paths modeled	invisible management plane
PENDING mechanism evidence	production security claim

DISTRIBUTED TRUST-BOUNDARY DIAGRAM

Runtime flow crosses user, edge, service, messaging, data and telemetry zones

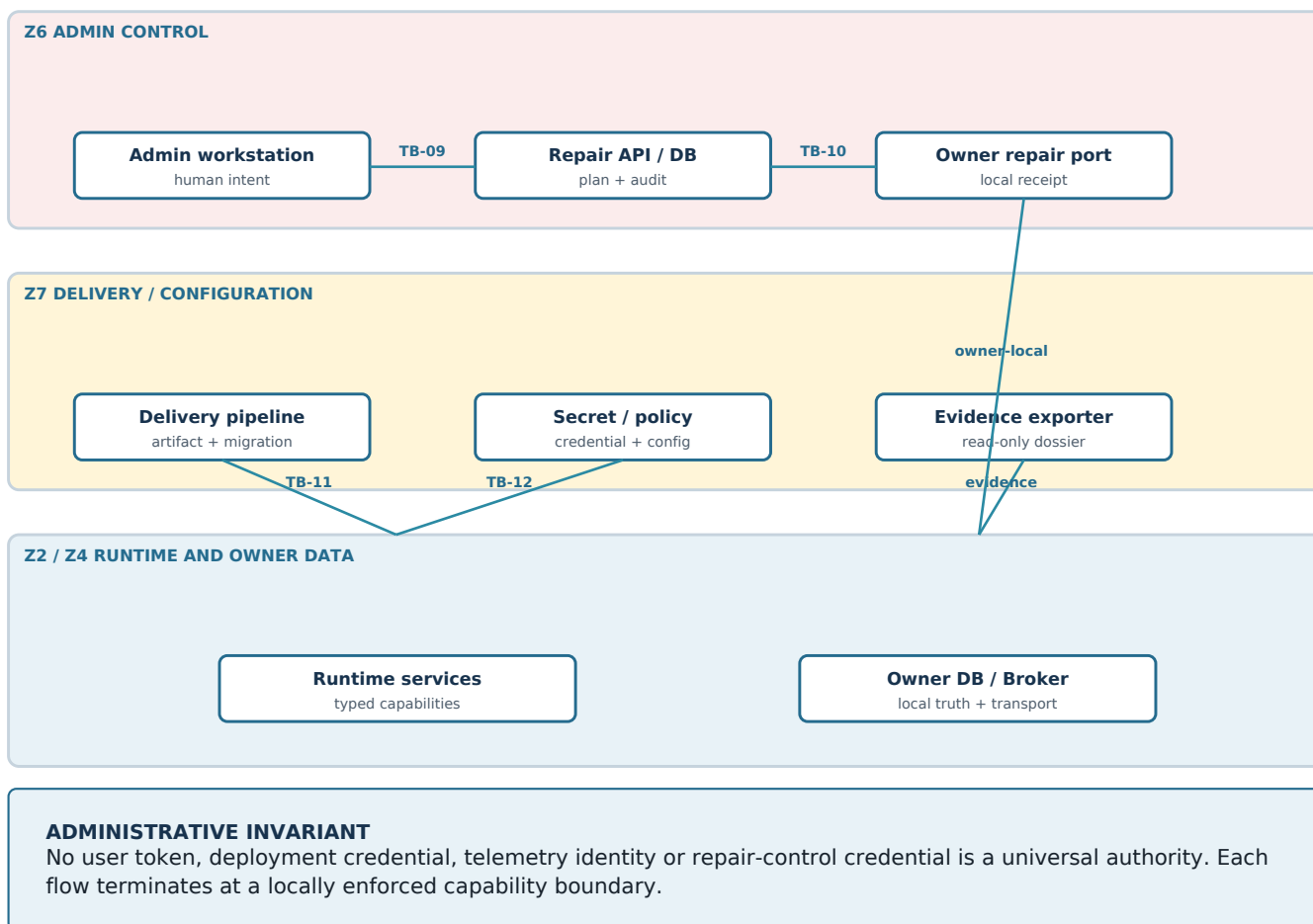
The diagram is a security data-flow model, not a deployment topology. Labeled crossings name where principal, data handling or authority assumptions change. Databases remain owner-local; telemetry is a side path; the broker is not inside any service trust zone.



ADMINISTRATIVE TRUST-BOUNDARY DIAGRAM

Repair, delivery and secrets form a second control plane

The administrative model is separate because its principals can approve recovery, invoke owner-local repair, change schema, deploy artifacts or issue credentials. These flows are not ordinary operator requests and must not inherit their session or service-account assumptions.



PART VIII PREMISE

Distribution replaces one perimeter with many decisions

Phase-0 originally protected one demonstrator boundary. Services, a BFF, RabbitMQ, owner databases, telemetry and repair tooling now exchange data across independently privileged processes. Security therefore becomes a property of each crossing, principal, asset and failure mode rather than one firewall-shaped assumption.

DECISION LEDGER 43-4

Monolith assumption	Distributed reality	Required response
inside is trusted	every process can be compromised	authorize per resource
one user session	human and workload identities differ	preserve both contexts
one database	owner stores and migration paths	separate credentials
logs are local	telemetry crosses another boundary	classify and protect

PART RULE

No network position, process name, broker connection or successful deployment is sufficient proof of trust.

CHAPTER BOUNDARY**Mechanisms are deferred until the trust problem is exact**

Chapter 43 does not choose an identity provider, token format, certificate authority, secret store or network product. It identifies assets, actors, zones, flows, trust changes and abuse cases, then turns them into security requirements that Chapters 44-48 must implement and verify.

DECISION LEDGER 43-5

Defined here	Deferred
trust zones and TB identifiers	edge authentication mechanism
principal and asset catalogue	workload identity mechanism
focused threat ledger	claim mapping implementation
security requirements	secret/certificate handling
evidence obligations	runtime security test bundle

ORDER MATTERS

A product default can be useful only after the model proves which threat and boundary it is intended to control.

SECURITY MATURITY**A distributed boundary is not production security maturity**

The manuscript specifies a coherent Phase-0 security boundary. It does not claim penetration testing, hardened infrastructure, managed key custody, production monitoring, vulnerability response, regulatory certification or operational staffing. Those claims require environment-specific artifacts and accountable owners.

DECISION LEDGER 43-6

Claim	Chapter status
distributed trust boundaries identified	SPECIFIED
threats and requirements traceable	SPECIFIED
mechanisms selected and configured	PENDING Chapters 44-47
adversarial tests retained	PENDING Chapter 48
production security maturity	NOT CLAIMED

HONEST BOUNDARY

Architecture can reduce ambiguity now without pretending that documentation neutralizes operational risk.

THREAT-MODEL LOOP

Scope, decompose, threaten, require, verify and revisit

The model follows a repeatable engineering loop. Scope names the system and assumptions. Decomposition draws data flows and trust changes. Threat analysis asks what can go wrong. Requirements state the intended response. Verification names evidence. Change triggers reopen the model.

DECISION LEDGER 43-7

Step	Durable artifact	Failure to avoid
scope	model identity + exclusions	protecting an undefined system
decompose	zones, flows, stores, principals	missing crossings
threaten	focused abuse ledger	generic checklist only
require	SR-043 requirements	product-first selection
verify	test/evidence obligations	control by assertion
revisit	change triggers + review record	stale model

LIVING MODEL

A threat model frozen before the architecture changes is historical documentation, not a current security boundary.

TRUST VOCABULARY

Trust means a bounded accepted claim

NEXUS-1 uses trust narrowly: a component accepts a specific claim from a specific principal for a specific resource, action, audience, scope and time under stated evidence. Trust is not a general property that one service can lend to another.

DECISION LEDGER 43-8

Term	Meaning
principal	human or workload asking to act
asset	something whose loss or misuse matters
trust zone	area with one accountable control posture
trust boundary	crossing where assumptions or authority change
trust decision	accept/reject one bounded claim
residual risk	known exposure remaining after intended controls

NO TRANSITIVITY

If the BFF trusts an operator identity and RootCause trusts the BFF workload, RootCause still needs a local decision about the requested capability.

ZERO-TRUST PRINCIPLE

Network location supplies context, never authorization

A container network, private subnet, localhost binding or cluster membership may reduce exposure, but none proves who is calling or what the caller may do. Resource access is evaluated at the boundary with current principal, action, target and policy context.

DECISION LEDGER 43-9

Signal	May contribute	Cannot prove
source network	risk/context	identity
process/service name	routing	authorization
broker virtual host	isolation context	message meaning
database endpoint	resource location	least privilege
TLS session	protected peer channel	business permission

SCOPE CAUTION

The chapter adopts the resource-focused principle; it does not claim a complete enterprise zero-trust architecture.

INHERITED ARCHITECTURE

Security must preserve ownership and repair invariants

The security model is not allowed to collapse the architecture it protects. Service-owned truth, exact public message bytes, owner-local transactions, at-least-once delivery, append-only correction and compare-plan-authorize-execute-verify remain binding.

DECISION LEDGER 43-10

Invariant	Security consequence
service-owned truth	no cross-owner database credential
exact envelope bytes	transport context cannot rewrite meaning
human/workload separation	user token alone is not service trust
owner-local repair	repair host has no business-table write
dual control	requester cannot approve high-risk replay
honest uncertainty	identity/control outage fails closed

SECURITY IS CONSTRAINED DESIGN

A centralized super-credential is not acceptable merely because it simplifies enforcement.

ADR-043

Trust is explicit at every crossing and never transitive

ADR-043 accepts twelve named trust boundaries, separate human and workload contexts, resource-local authorization requirements, protection of telemetry and administrative paths, and mechanism selection only after threat traceability. It rejects implicit internal trust and production-maturity claims.

DECISION LEDGER 43-11

ADR field	Decision
model	nexus1/distributed-trust-boundaries/1
ledger	nexus1/threat-ledger/phase0/1
trust	explicit, local, bounded, revocable
transitivity	rejected
mechanisms	deferred to Chapters 44-48
production maturity	not claimed
evidence	PENDING

ADR DIGEST

sha-256+jcs 4060479ea89e0f5e4fe3c8aa71d432918ba7d6576ed7bb92ca2f44adbc2d95ce

ASSET CATALOGUE

Threats are meaningful only against valued assets

The model begins with assets whose unauthorized use, disclosure, alteration, destruction or unavailability would matter to NEXUS-1. Components and technologies are not assets by default; the truth, authority and evidence they hold are.

DECISION LEDGER 43-12

Asset class	Examples	Primary concern
business truth	alarms, verdicts, reviews	integrity + ownership
public fact history	envelopes, ordering, lineage	integrity + availability
identity/authority	subjects, workloads, capabilities	spoofing + elevation
repair authority	plans, approvals, receipts	integrity + repudiation
evidence	audit, manifests, test artifacts	tamper resistance
confidential material	credentials, sensitive payloads	disclosure
service capacity	DB, broker, workers, APIs	availability

SAFETY LANGUAGE

NEXUS-1 remains an educational Phase-0 demonstrator; the model protects demonstrator assets and does not assert real-plant control authority.

BUSINESS TRUTH ASSETS

Integrity includes owner and history, not only bytes

A verdict copied byte-for-byte into the wrong owner table is still a security and architecture failure. Business integrity therefore includes who may decide, who may write, which contract carries the decision and whether corrections retain the original history.

DECISION LEDGER 43-13

Asset	Integrity condition
AlarmManagement decision	only AlarmManagement policy/transaction writes
RootCause case/verdict	only RootCause decides and persists
Compliance review	only Compliance owns lifecycle
Reporting projection	derived, generation-scoped, not owner truth
Workflow progress	derived from approved public facts
correction history	append/supersede; no destructive edit

OWNERSHIP IS A SECURITY CONTROL

A credential that can cross these write boundaries defeats both least privilege and domain meaning.

IDENTITY ASSETS

Identity context is security-sensitive data

Human subject, workload identity, authentication context, audience, site/tenant scope, delegated capability, issue/expiry time and policy result can authorize consequential actions. They require integrity, minimization and bounded retention even when they are not secrets.

DECISION LEDGER 43-14

Identity coordinate	Why protected
human subject	accountability and privacy
workload principal	service authority
audience/resource	prevents cross-service replay
capability/scope	bounds action and target
site/tenant	prevents horizontal access
authentication context	risk/assurance evidence
issue/expiry/revocation	freshness and trust decay

TOKEN IS A CONTAINER

No token format is trusted merely because it deserializes or has a familiar claim name.

REPAIR ASSETS

Administrative intent can be more dangerous than business traffic

Repair plans, approval identities, canonical digests, idempotency keys, owner receipts, generation lineage and verification results decide whether privileged recovery work occurs. Tampering can manufacture authority while leaving ordinary service traffic apparently healthy.

DECISION LEDGER 43-15

Repair asset	Protected property
candidate snapshot	freshness and integrity
canonical plan	exact scope and action
approvals	distinct subjects, policy, expiry
RepairKey	idempotency binding
owner receipt	action/result lineage
audit chain	ordering and non-repudiation evidence
verification	independent convergence result

CHAPTER 42 CONTINUITY

Security must preserve operations/reconciliation-repair/1 and cannot turn repair into a privileged bypass.

EVIDENCE ASSETS

Telemetry, audit and acceptance evidence are different

Logs and traces support diagnosis but may be sampled, delayed or exported. Repair audit owns administrative history. Owner databases and inbox/outbox rows own local effects. Test manifests bind claims to a run. Treating these as interchangeable makes tampering and omission harder to detect.

DECISION LEDGER 43-16

Evidence	Authority	Known limit
trace/log	diagnostic	sampled/exported
metric	aggregate operation	low-cardinality only
repair audit	administrative decision history	not business truth
owner receipt	local repair action	not end-to-end success
database row	owner-local state	not broker observation
evidence manifest	claim-to-artifact binding	requires validator

NO SECURITY BY DASHBOARD

A green dashboard is an observation, not proof that every authoritative record is intact.

AVAILABILITY ASSETS

Security includes bounded access to progress and recovery

A service can preserve confidentiality while still be insecure if an attacker can exhaust its API, broker, database, telemetry pipeline or repair workers. Availability requirements therefore protect normal-path progress and prevent recovery tooling from worsening an incident.

DECISION LEDGER 43-17

Capacity asset	Abuse consequence
BFF/API budget	operator cannot observe or act
broker memory/disk	delivery stalls or rejects
owner DB connections	transactions starve
consumer concurrency	backlog/duplicate pressure
telemetry collector	diagnostic blindness or backpressure
repair executor	incident amplification

PRIORITY RULE

Repair and telemetry traffic must never crowd out the business path they are intended to diagnose or restore.

DATA CLASSIFICATION

Classification follows meaning and use

The model assigns a minimum handling class to every flow and store. Deployments may strengthen classes; they may not downgrade an asset because it appears only in logs, headers or a derived projection.

DECISION LEDGER 43-18

Class	Examples	Default handling intent
C0 Public	published documentation	integrity
C1 Internal	topology names, build identities	bounded disclosure
C2 Operational	message IDs, health, workflow state	authorized access
C3 Sensitive	incident payload, user context, evidence	minimize + protect
C4 Secret	private keys, passwords, tokens	never log/export

CONTEXT MATTERS

A correlation ID may be operational alone and sensitive when joined with incident or actor data.

HUMAN PRINCIPALS

Operator roles are not one generic user

The threat model distinguishes ordinary console readers, business command actors, repair requesters, repair approvers, high-risk approvers, auditors and platform administrators. Combining them hides separation-of-duty threats.

DECISION LEDGER 43-19

Principal	Intended authority	Must not inherit
operator reader	bounded status/query	repair execution
business actor	declared command capability	DB/broker admin
repair requester	prepare exact plan	self-approval
repair approver	approve immutable plan	edit plan
repair executor	start approved job	expand scope
auditor	read protected evidence	runtime mutation
platform admin	infrastructure operation	domain decision

NO ROLE COMPRESSION

One test account with every permission cannot prove the intended production separation.

WORKLOAD PRINCIPALS

Each runtime function receives its own identity boundary

AlarmManagement, RootCause, Compliance, Audit, Reporting, Workflow, BFF, Repair Control, repair workers, outbox publishers, consumers and telemetry collectors are separate workload principals even when deployed on one machine.

DECISION LEDGER 43-20

Workload	Minimum resource class
BFF	service APIs; no owner database
service runtime	own DB DML + declared broker operations
outbox publisher	own outbox read/update + publish routes
consumer	declared queue consume/ack + own DB
Reporting	ReportingDb + declared subscriptions
Repair Control	RepairControlDb + typed owner ports
telemetry collector	telemetry receive/export only

DEPLOYMENT CO-LOCATION

Sharing a host does not merge the workload identities or permission manifests.

ADMINISTRATIVE PRINCIPALS

Runtime, migration and platform authority stay distinct

Migration agents require schema authority unavailable to runtime services. Broker topology provisioning differs from message publish/consume. Deployment identities can change artifacts but must not become business actors. Evidence exporters read artifacts without mutating the system under test.

DECISION LEDGER 43-21

Principal	Allowed	Denied
DB migration	one owner DB DDL	runtime business requests
broker provisioner	declared topology/policy	business event production
deployment identity	verified artifact/config rollout	Domain commands
secret authority	issue/rotate scoped material	business data reads
evidence collector	read named artifacts	repair execution

STANDING WARNING

An emergency super-admin path is a separate high-risk boundary, not an undocumented exception.

THREAT ACTORS

The model includes compromise and misuse, not only outsiders

Threat sources include an unauthenticated external actor, a stolen human session, a compromised workload, an over-privileged administrator, malicious or accidental insider action, a tampered artifact, an untrusted dependency and an overwhelmed component.

DECISION LEDGER 43-22

Actor model	Representative capability
external actor	send crafted/voluminous edge traffic
stolen session	act as a valid human until rejected/revoked
compromised BFF	alter downstream context
compromised service	use its legitimate network position
insider/admin misuse	exercise real privileges improperly
supply-chain tamper	replace code/config before runtime
dependency failure	return false/stale identity or policy data

AUTHORIZED DOES NOT MEAN BENIGN

The ledger asks what a principal can abuse after successful authentication, not only how spoofing might occur.

TRUST ZONES

A zone groups one control posture, not one subnet

NEXUS-1 defines zones by accountable authority and data handling: user device, edge, service runtime, messaging, owner data, observability, administrative control, delivery/configuration and external dependencies. Co-location does not erase their logical boundaries.

DECISION LEDGER 43-23

Zone	Primary contents	Default stance
Z0 User device	browser/session/UI	untrusted input
Z1 Edge	BFF/public API	authenticate and mediate
Z2 Service runtime	owner APIs/workers	local authorization
Z3 Messaging	exchanges/queues	route-scoped access
Z4 Owner data	service databases	owner-only write
Z5 Observability	collector/backend	sensitive diagnostic
Z6 Admin control	repair/platform paths	highest scrutiny
Z7 Delivery/config	artifacts/secrets/policy	supply authority

BOUNDARY TEST

If data or authority changes its accepted assumptions, a trust boundary exists even on localhost.

BOUNDARY CATALOGUE

Twelve crossings make the distributed trust model reviewable

Each TB identifier is stable across the diagram, threat ledger, security requirements, tests and evidence. New flows cannot be added as anonymous arrows.

DECISION LEDGER 43-24

Boundary	Source -> destination	Flow
TB-01	operator device -> BFF edge	interactive request/response
TB-02	BFF edge -> owning service API	bounded user context + service call
TB-03	producer service -> RabbitMQ	public fact publication
TB-04	RabbitMQ -> consumer service	at-least-once delivery
TB-05	runtime service -> owned database	owner-local DML
TB-06	migration agent -> owned database	bounded DDL

CATALOGUE DIGEST

sha-256+jcs 651328e6e3191afb2de82372102a581b5fbe950aa951293590d0d2ae0fb73f93

BOUNDARY CATALOGUE CONTINUED

Operational and administrative flows are first-class

The second half of the catalogue makes telemetry, diagnostics, repair, deployment and secret/configuration delivery visible. These paths are often omitted from application-only diagrams even though they carry broad authority or sensitive evidence.

DECISION LEDGER 43-25

Boundary	Source -> destination	Flow
TB-07	service/collector -> telemetry pipeline	logs, metrics and traces
TB-08	diagnostic caller -> health/evidence edge	bounded operational read
TB-09	admin workstation -> repair admin API	request/approve/execute
TB-10	repair control -> owner repair port	typed privileged action
TB-11	delivery pipeline -> runtime host	artifact/config deployment
TB-12	secret/config authority -> runtime host	credential and policy material

NO INVISIBLE MANAGEMENT PLANE

A path used only during incidents, migration or deployment still belongs in the threat model.

TB-01 OPERATOR EDGE

The browser is an untrusted execution and storage environment

The BFF treats every request, route value, header, cookie/token, body and browser-generated correlation field as untrusted input. Client-side visibility controls and local state cannot authorize a server action.

DECISION LEDGER 43-26

Threat	Security requirement
spoofed/stolen session	fresh authenticated subject context
cross-site action	request binding and safe session semantics
tampered site/tenant	server derives/validates scope
hidden-button bypass	server capability enforcement
large/slow input	limits, deadlines and rejection
sensitive response caching	bounded response/cache policy

CHAPTER 44 TARGET

The exact edge authentication and operator authorization mechanism is deferred; the required behaviors are fixed here.

SPOOFING**Identity must be proven for the intended resource**

A valid credential for another audience, tenant, site, environment or principal type does not establish the requested identity. The receiving boundary validates issuer/authority, audience, subject/workload type, freshness and policy context before using identity data.

DECISION LEDGER 43-27

Spoofing path	Rejected inference
foreign audience credential	signature means accepted here
user credential at service port	human identity is workload identity
service name header	caller chose its own identity
trace baggage subject	telemetry context authenticates
private network address	network location identifies principal

SR-043-001

Every protected crossing must name accepted principal type, authority, audience, freshness and failure behavior.

TAMPERING**Integrity covers context, routing and policy inputs**

An attacker need not alter a domain payload if they can replace a site claim, routing key, repair plan digest, policy identity, configuration provider or active deployment artifact. Integrity requirements therefore bind both data and the coordinates that determine its meaning.

DECISION LEDGER 43-28

Tamper target	Protected binding
request target/body	capability + resource + representation
identity context	issuer/audience/subject/scope
message properties/body	route + envelope coordinates + digest
repair plan	canonical bytes + approval digest
configuration	provider order + identity + value-free manifest
artifact	build/deployment identity

SR-043-002

Every security-sensitive decision must identify the exact inputs whose integrity it depends on.

REPUDIATION

Accountability requires authoritative events, not verbose logs

The system must later distinguish who requested, approved and executed an action; which workload performed the owner-local step; what plan or command was accepted; and which outcome was observed. Trace text alone is insufficient because it may be sampled or mutable.

DECISION LEDGER 43-29

Action	Required authoritative evidence
business command	owner transaction + actor/context reference
message publication	outbox identity + publisher attempt
message consumption	inbox/owner effect + disposition
repair	plan, approvals, receipt, verification, audit chain
migration/deployment	artifact/config identity + actor/run
evidence access/export	purpose + subject + artifact digest

SR-043-003

Consequential actions must create durable, minimally sensitive and tamper-detectable accountability evidence.

INFORMATION DISCLOSURE

Operational data can expose topology, incidents and authority

Problem details, health endpoints, traces, broker headers, audit exports and repair views can leak identities, message content, queue names, resource topology or credentials. Response contracts and telemetry require explicit allow lists.

DECISION LEDGER 43-30

Surface	Must exclude by default
HTTP problems	stack, SQL, token, internal endpoint
health/readiness	secret values and dependency credentials
telemetry	raw bodies, tokens, high-cardinality personal data
repair console	poison bytes and secret configuration
audit export	unnecessary payload/credential material
broker management	broad topology and message inspection

SR-043-004

Every outward representation declares a data-classification ceiling and an allow-list serializer.

DENIAL OF SERVICE

Authentication does not make expensive work safe

Authenticated callers and compromised workloads can exhaust queues, database pools, projection reducers, repair scanners or telemetry exporters. Each crossing needs budgets, concurrency limits, deadlines, backpressure and priority rules independent of identity success.

DECISION LEDGER 43-31

Abuse	Required bound
large/complex request	body/query/result budgets
high request rate	principal/resource quotas
message flood	queue length, producer policy, consumer capacity
telemetry cardinality	attribute allow list and size limit
repair scan/replay	blast radius + lower priority
dependency stall	timeout, circuit/bulkhead and honest status

SR-043-005

Every protected operation names a cost budget and preserves normal-path capacity under failure.

ELEVATION OF PRIVILEGE

Valid access to one capability must not expand sideways

Elevation occurs when a user becomes a workload, a reader becomes a repair executor, a runtime principal becomes a migration principal, one service writes another owner's data, or telemetry access becomes payload access. Local policy and credential separation contain these moves.

DECISION LEDGER 43-32

Elevation path	Required separation
reader -> command actor	capability policy
requester -> approver	distinct subject/role
user -> service trust	separate workload identity
runtime -> migration	separate credentials and lifecycle
service A -> owner B DB	no credential/network grant
collector -> admin	receive/export only

SR-043-006

Privilege manifests must enumerate allowed capabilities and demonstrate absence of prohibited ones.

TB-02 BFF TO SERVICE

The BFF is a mediator, not an authorization oracle

The BFF authenticates the edge and shapes the operator experience, but an owning service enforces its own capability and resource policy. The downstream request distinguishes human context from the BFF workload principal and rejects absent or overbroad context.

DECISION LEDGER 43-33

Context	Owner-service use
BFF workload	authenticate calling service
human subject	accountability and capability mapping
audience	bind request to owner service
site/tenant	resource-scope decision
command/query	local policy
correlation	diagnosis only; not authorization

NO TRANSITIVE AUTHORIZATION

A BFF allow decision is useful upstream evidence, not a substitute for the owner's local decision.

CONFUSED DEPUTY

A trusted BFF can still act for the wrong subject or target

The service must prevent the BFF from using its broader workload authority to perform a command outside the human subject's capability or site scope. The target is derived from the route/body and compared with trusted context; it is not copied blindly from a header.

EXECUTABLE ASSET 43-A - LOCAL DECISION INPUTS

```
public sealed record BoundaryDecisionInput(  
    WorkloadPrincipal Caller,  
    HumanSubject? OnBehalfOf,  
    Capability Capability,  
    ResourceCoordinate Resource,  
    Audience Audience,  
    Instant ObservedAt,  
    string PolicyIdentity);  
  
public interface IBoundaryPolicy  
{  
    BoundaryDecision Evaluate(BoundaryDecisionInput input);  
}
```

MECHANISM NEUTRAL

The type fixes the decision inputs. Chapters 44-46 decide how trustworthy values reach it.

HUMAN VERSUS WORKLOAD

Two identities answer two different questions

The workload identity answers which service made the network call. Human context answers on whose behalf an operator action is requested. Service-initiated background work may have no human subject; it still requires a workload capability and explicit policy path.

DECISION LEDGER 43-35

Question	Identity context
who opened the connection/call?	workload principal
who initiated this operator action?	human subject
what may the service do?	workload capability
what may the human do?	operator capability/resource scope
is this scheduled background work?	service policy path; no fake user
who is accountable?	both contexts where applicable

CHAPTER 45 TARGET

A user token alone must never be treated as proof that the caller is an authorized service workload.

CLAIMS MINIMIZATION

Downstream context is a purpose-built projection

The BFF forwards only the human coordinates required by the target capability. It does not pass the entire external identity document, unrelated group membership or upstream authorization database semantics into every service.

DECISION LEDGER 43-36

May forward	Must not forward by default
stable subject reference	raw external identity document
target capability	all upstream groups/roles
site/tenant scope	unrelated organizational attributes
authentication context reference	credential or refresh material
decision/correlation reference	sensitive profile data

CHAPTER 46 TARGET

Local policy mapping preserves service autonomy and prevents one service from becoming the authorization database for all others.

TB-03 PRODUCER TO BROKER**Broker acceptance is a narrow transport trust decision**

RabbitMQ may authenticate a connection and authorize exchange/routing operations. The producer still owns the outbox record and exact public envelope. The broker does not decide whether the underlying Domain fact was legitimate.

DECISION LEDGER 43-37

Layer	Accepted claim	Not accepted
producer Domain	owner fact exists	broker delivery
outbox	one frozen public intent	consumer effect
broker access	principal may publish route	payload meaning
publisher confirm	broker accepted attempt	all consumers completed
message digest	bytes unchanged	producer authorized decision

TRUST BOUNDARY

Connection trust, route permission, envelope integrity and Domain authority are separate proofs.

UNAUTHORIZED PUBLICATION**A compromised producer must not publish every contract**

Broker permissions and topology manifests scope each publisher to declared exchanges and routing patterns. Consumer admission also validates producer identity, contract coordinates and body fingerprint so a route grant alone cannot impersonate another owner.

DECISION LEDGER 43-38

Threat	Required evidence
service publishes foreign event	publisher permission matrix
route typo/misdirection	mandatory routing + confirm outcome
body/type mismatch	consumer admission corpus
reused MessageId with changed bytes	inbox identity/digest conflict
topology drift	value-free topology manifest/digest

THR-043-011

A compromised workload may use its legitimate connection; least-privilege route grants limit lateral publication.

MESSAGE INTEGRITY**Transport protection and semantic integrity are distinct**

A protected connection reduces in-transit tampering risk between peers. The consumer still validates the exact envelope grammar, producer, event type, version, MessageId, correlation/causation coordinates, payload invariants and content fingerprint.

DECISION LEDGER 43-39

Control layer	Protects	Does not replace
channel peer protection	connection confidentiality/integrity	contract admission
broker authorization	route operation	producer Domain authority
content fingerprint	canonical content integrity	identity authorization
inbox receipt	duplicate/conflict handling	publisher confirm
owner transaction	local effect atomicity	broker acknowledgement

NO SINGLE MAGIC LAYER

Removing contract validation because the connection is protected would merge unrelated trust claims.

BROKER AUTHORITY**Management, provisioning and runtime permissions differ**

The broker management plane can observe or alter topology and must not be exposed through normal service credentials. Topology provisioning can declare queues/exchanges/policies. Runtime publishers and consumers receive only their route-specific operations.

DECISION LEDGER 43-40

Principal	Allowed	Denied
topology provisioner	declared configure operations	business publish/consume
publisher	write declared routes	configure/read other queues
consumer	read/ack declared queue	publish foreign routes
status collector	bounded statistics	message consume/configure
repair replay	approved normal-route publish	broker-wide admin

THR-043-012

A broad broker credential can turn one compromised service into a cross-service publisher, consumer or topology administrator.

TB-04 BROKER TO CONSUMER**Delivery is untrusted until admission and local commit**

A consumer copies the delivery, authenticates/authorizes the producer path, validates properties and body, translates into local language, commits inbox plus effect, then settles. A broker-delivered message is not automatically an accepted Domain input.

DECISION LEDGER 43-41

Stage	Security decision
delivery	copy bytes/properties within callback lifetime
route admission	expected exchange/routing/producer path
contract admission	strict schema and fingerprint
semantic validation	consumer-owned invariants
idempotency	MessageId + body digest conflict policy
local transaction	receipt/effect/outbox atomicity
settlement	ack only known durable outcome

UNTRUSTED QUEUE

The queue is a transport buffer across a trust boundary, not a trusted invocation stack.

POISON AND REPLAY**Terminal evidence cannot become an administrative bypass**

Invalid or unsupported deliveries move to durable poison evidence. Replay requires diagnosis, immutable bytes, remediation, approval and normal-route re-admission. The repair path may not invoke a Domain handler directly or mutate rejected bytes.

DECISION LEDGER 43-42

Threat	Boundary response
poison flood	bounded terminal storage and alert
hostile bytes replay	eligibility and integrity gate
self-approved replay	dual control
replay bypasses inbox	normal route mandatory
replay loop	bounded generations and new plan
evidence deletion	append disposition; retain original

THR-043-013

Administrative re-admission is one of the highest-risk crossings and inherits every Chapter 42 guard.

TB-05 RUNTIME TO DATABASE

A database connection is an ownership decision

Each service runtime performs DML only in its owned database with a runtime principal. The BFF, Process Manager, Reporting and Repair Control receive no credentials for owner tables merely because a query would be convenient.

DECISION LEDGER 43-43

Caller	Permitted store	Explicit denial
AlarmManagement	AlarmManagementDb	RootCause/Compliance DML
RootCause	RootCauseDb	AlarmManagement reads
Compliance	ComplianceDb	RootCause writes
Reporting	ReportingDb	owner DB joins/writes
Workflow	WorkflowDb	owner DB polling
Repair Control	RepairControlDb	owner business-table DML

THR-043-014

Cross-owner credentials silently recreate the shared-database trust boundary the architecture removed.

DATABASE LEAST PRIVILEGE

Schema ownership and runtime operations are separate manifests

Runtime principals receive only required DML and execution rights. Migration principals receive bounded DDL for one database and are absent from normal runtime configuration. Backup, restore and audit readers are distinct administrative paths.

DECISION LEDGER 43-44

Principal class	Capability	Lifecycle
runtime	owner DML only	service lifetime
migration	one owner DDL	deployment window
read-only diagnostic	approved safe views	incident/time bound
backup/restore	platform operation	controlled admin
evidence reader	named artifacts/snapshots	run scoped

ABSENCE PROOF

The environment manifest must prove forbidden connection authorities are not present, without exposing secret values.

TB-06 MIGRATION PATH

Deployment authority is powerful, temporary and auditable

A migration can change schema and therefore application meaning. The delivery pipeline must bind repository/artifact identity, migration list, target database, environment, actor/workload, approval and result. Runtime services never auto-acquire broad DDL to simplify startup.

DECISION LEDGER 43-45

Threat	Requirement
migration targets wrong DB	owner/environment binding
unreviewed migration	artifact/migration identity and gate
runtime applies DDL	separate migration principal
rollback corrupts owner state	forward/rollback plan + evidence
credential persists	short lifecycle/rotation proof
history tampered	database migration ledger + deployment record

THR-043-015

A green application startup is not evidence that the intended migration identity changed the intended database.

TB-07 TELEMETRY

Observability crosses a confidentiality and integrity boundary

Services emit data to a collector that can aggregate and export system-wide operational context. That path can leak sensitive data, accept spoofed signals, create high-cardinality denial of service or become a lateral movement surface.

DECISION LEDGER 43-46

Telemetry threat	Requirement
secret/payload leakage	allow-list attributes and scrub
spoofed service signal	authenticated workload context
trace-context injection	diagnostic only; validate grammar
metric cardinality attack	bounded label sets
collector compromise	receive/export least privilege
export outage	no correctness dependency; bounded buffer

THR-043-016

Telemetry is a distributed data pipeline and must be threat-modeled like one.

TELEMETRY CONFIDENTIALITY**Instrumentation cannot decide sensitivity automatically**

Domain payloads, user identity, credentials, connection strings and repair notes are excluded from ordinary telemetry by construction. Message IDs and repair IDs may appear in bounded logs/traces for diagnosis but never as metric labels.

DECISION LEDGER 43-47

Signal	Allowed	Excluded
metric	bounded service/result/reason	IDs, payload, actor
trace	safe lineage IDs and state	token/body/secret
structured log	reason code + safe reference	raw exception secrets
audit event	actor/plan/action/digest	credential material
evidence export	named artifact + SHA-256	unapproved raw data

DATA MINIMIZATION

Redaction after collection is a backstop; the preferred control is not to emit unnecessary sensitive fields.

TELEMETRY INTEGRITY**A trace can be forged without changing business truth**

Traceparent, baggage, log attributes and service-name labels aid diagnosis but do not authorize actions or determine message identity. The system correlates telemetry with durable owner/inbox/outbox/audit evidence and records gaps or exporter outages explicitly.

DECISION LEDGER 43-48

Observation	Authoritative source
span says publish succeeded	outbox/publisher confirm state
span says consumer completed	inbox + owner transaction
log says repair approved	RepairControlDb approval/audit
metric says zero failures	discovery + retained test/artifact set
trace parent claims actor	accepted identity context
telemetry missing	correctness remains owner-local

THR-043-017

Compromised or missing telemetry may impair detection, but it must not manufacture a business or repair outcome.

TB-08 DIAGNOSTICS

Health and evidence endpoints reveal only bounded operational truth

Liveness remains process-local. Readiness exposes safe dependency dispositions. Evidence endpoints return model/configuration identities and references, never secrets, full topology, payload bodies or raw failure internals. Administrative diagnostics require separate authorization.

DECISION LEDGER 43-49

Surface	May expose	Must not expose
liveness	process alive	dependency details
readiness	ready/not ready + safe reason	credentials/endpoints
model status	ADR/model digest	policy values/secrets
evidence status	PENDING/PROVED + references	raw protected artifacts
admin diagnostics	authorized detailed state	business mutation

THR-043-018

Operational transparency is designed through safe contracts, not by returning exceptions or configuration dumps.

TB-09 ADMINISTRATIVE EDGE

The repair console is a separate high-risk experience

An admin workstation and human session can request, approve and execute plans with operational consequences. The boundary requires stronger separation, freshness, explicit intent, exact scope confirmation, protected evidence access and denial of normal browser shortcuts.

DECISION LEDGER 43-50

Threat	Required control objective
stolen admin session	fresh high-assurance context
requester self-approves	distinct identities
plan changes after approval	canonical digest binding
stale screen executes	ETag/fresh precondition
scope expansion	server-owned plan + blast limits
clickjacking/automation	explicit protected transition
evidence exfiltration	purpose-bound minimal view

THR-043-019

The operator console and repair console may share design language; they must not share implicit authority.

REPAIR SEPARATION OF DUTIES

Request, approve and execute are independent decisions

The model treats role combination, delegated identities and automation as explicit risks. High-risk replay requires distinct human approvers; a service identity cannot masquerade as human approval; an executor cannot edit the authorized plan.

DECISION LEDGER 43-51

Stage	Principal	Security binding
compare	read-only scanner	snapshot/digest
request	repair requester	candidate + reason
approve	distinct approver(s)	exact plan digest + expiry
execute	executor workload/human action	valid approvals + ETag
owner action	owner repair workload	RepairKey + target precondition
verify	read-only oracle	new snapshot/digest

NO EMERGENCY AMBIGUITY

Any break-glass exception must be a separately modeled path with bounded activation, evidence and post-event review.

REPAIR PLAN TAMPERING

Approval binds canonical bytes, not a database row label

A plan's kind, owner, semantic target, snapshot basis, digest, limits, mode, policy and expiry are canonicalized and hashed. Every approval and execution request names the same digest. Reuse with changed bytes fails conflict.

EXECUTABLE ASSET 43-B - SECURITY-SENSITIVE PLAN BINDING

```
public sealed record ApprovedPlanReference(  
    Guid RepairPlanId,  
    string PlanSha256,  
    string PolicyIdentity,  
    Instant ExpiresAt,  
    IReadOnlySet<ApprovalSubject> Approvers);  
  
public static void DemandExact(  
    CanonicalRepairPlan plan, ApprovedPlanReference approved, Instant now)  
{  
    if (!CryptographicOperations.FixedTimeEquals(  
        Convert.FromHexString(plan.Sha256),  
        Convert.FromHexString(approved.PlanSha256)))  
        throw Denied("plan-digest-mismatch");  
    if (now >= approved.ExpiresAt) throw Denied("approval-expired");  
}
```

NO MUTABLE APPROVAL

Changing scope, target, evidence basis or limits creates a new plan and requires new authorization.

TB-10 OWNER REPAIR PORT**Repair Control cannot become an owner super-service**

Repair Control authenticates to one typed owner repair endpoint and presents the approved action context. The owner validates its local target, plan digest, fresh precondition and RepairKey, then stores a local receipt. Generic SQL or broker clients are prohibited.

DECISION LEDGER 43-53

Threat	Owner-local requirement
repair calls wrong owner	audience/endpoint/action binding
action outside plan	typed contract + plan digest
duplicate call	unique RepairKey receipt
stale evidence	fresh owner precondition
broad repair credential	capability per repair kind/owner
response loss	receipt lookup before retry

THR-043-020

A compromised Repair Control must face owner-local policy rather than inherit direct mutation authority.

TB-11 DELIVERY PIPELINE**Build and deployment are trust boundaries into runtime**

Source revision, dependency locks, build outputs, container/image identity, configuration manifest and deployment target determine which code and policy run. A valid application credential cannot compensate for a tampered artifact.

DECISION LEDGER 43-54

Threat	Required binding
wrong source/dirty build	commit + dirty flag
dependency drift	lockfile/package digest
artifact substitution	artifact identity/signature policy
wrong environment	target/configuration identity
unreviewed config change	provider/value-free manifest
rollback to vulnerable build	approved version/reversal evidence

THR-043-021

The exact supply-chain mechanism is deferred, but every runtime claim already requires build and configuration identity.

TB-12 SECRET AND CONFIGURATION AUTHORITY**Configuration changes behavior; secrets grant capabilities**

A configuration provider can change routes, audiences, timeouts, feature policy or security behavior. A secret or certificate can impersonate a principal. Values stay out of manifests, while provider order, key presence, policy identity, rotation state and permission scope remain verifiable.

DECISION LEDGER 43-55

Asset	Threat	Requirement
secret	disclosure/reuse	scoped storage and redaction
certificate/key	impersonation/expiry	lifecycle and peer binding
provider order	override policy	deterministic manifest
feature flag	bypass control	authorized change + fail closed
endpoint/audience	redirect trust	validated immutable policy
rotation	outage/stale trust	overlap/recovery evidence

CHAPTER 47 TARGET

The chapter fixes what must be controlled; Chapter 47 specifies concrete secret, certificate and configuration handling.

TRUST CONTRACT**Every crossing has an executable requirement record**

The trust contract names source/destination zones, accepted principal types, assets, operations, data class, integrity/freshness expectations, availability budget, audit result and failure disposition. Mechanism fields can remain PENDING without weakening the requirement.

EXECUTABLE ASSET 43-C - BOUNDARY CONTRACT

```
public sealed record TrustBoundaryContract(  
    string BoundaryId,  
    string SourceZone,  
    string DestinationZone,  
    IReadOnlySet<PrincipalType> AcceptedPrincipals,  
    IReadOnlySet<string> Operations,  
    IReadOnlySet<string> Assets,  
    DataClass MaximumDataClass,  
    FreshnessRule Freshness,  
    AvailabilityBudget Budget,  
    string FailureDisposition,  
    IReadOnlySet<string> RequirementIds,  
    string EvidenceStatus);
```

MACHINE-CHECKABLE BOUNDARY

A flow without a contract is an architecture finding, not a harmless diagram omission.

BOUNDARY REGISTRY

Stable identifiers connect diagrams, code and evidence

The registry is canonicalized and hashed. CI compares the expected catalogue with discovered endpoints, workers, broker routes, database principals and telemetry exporters. Discovery cannot add a new trust crossing silently.

EXECUTABLE ASSET 43-D - REGISTRY FIXTURE

```
var registry = new TrustBoundaryRegistry(THREAT_MODEL_ID)
.Add("TB-01", Zone.UserDevice, Zone.Edge, "interactive-request")
.Add("TB-02", Zone.Edge, Zone.ServiceRuntime, "downstream-call")
.Add("TB-03", Zone.ServiceRuntime, Zone.Messaging, "publish")
.Add("TB-04", Zone.Messaging, Zone.ServiceRuntime, "delivery")
.Add("TB-05", Zone.ServiceRuntime, Zone.OwnerData, "runtime-dml")
.Add("TB-06", Zone.Delivery, Zone.OwnerData, "migration-ddl")
.Add("TB-07", Zone.ServiceRuntime, Zone.Observability, "telemetry")
.Add("TB-08", Zone.Operations, Zone.ServiceRuntime, "diagnostics")
.Add("TB-09", Zone.AdminDevice, Zone.AdminControl, "repair-admin")
.Add("TB-10", Zone.AdminControl, Zone.ServiceRuntime, "owner-repair")
.Add("TB-11", Zone.Delivery, Zone.ServiceRuntime, "deployment")
.Add("TB-12", Zone.Configuration, Zone.ServiceRuntime, "secret-config");
```

REFERENCE DIGEST

651328e6e3191afb2de82372102a581b5fbe950aa951293590d0d2ae0fb73f93

THREAT RECORD

A threat is a scenario with a traceable response

The ledger avoids one-line labels such as 'spoofing possible.' A record names actor, precondition, path, targeted asset, consequence, affected boundary, STRIDE category, likelihood/impact rationale, security requirements, verification obligation and residual uncertainty.

EXECUTABLE ASSET 43-E - THREAT LEDGER ENTRY

```
public sealed record ThreatRecord(
    string ThreatId,
    string BoundaryId,
    string Title,
    string Actor,
    string Preconditions,
    string AbusePath,
    IReadOnlySet<string> Assets,
    StrideCategory Category,
    OrdinalRisk Likelihood,
    OrdinalRisk Impact,
    IReadOnlySet<string> RequirementIds,
    IReadOnlySet<string> VerificationIds,
    string ResidualRisk,
    ThreatDisposition Disposition);
```

NO ORPHAN MITIGATION

A mechanism or test without a linked threat and boundary is ungrounded; a high-priority threat without a requirement is unresolved.

STRIDE COVERAGE**STRIDE checks categories; the domain supplies meaning**

STRIDE is applied per element and crossing to reduce blind spots. The final threat wording remains NEXUS-1 specific: verdict ownership, poison replay, projection generation, repair approval and evidence tampering are more actionable than category names alone.

DECISION LEDGER 43-59

Category	Question	NEXUS-1 example
Spoofing	who/what is impersonated?	user or producer workload
Tampering	what meaning can change?	plan digest or message route
Repudiation	what action can be denied?	repair approval/execution
Disclosure	what data leaks?	incident/identity/secret
DoS	what capacity is exhausted?	broker/DB/collector/repair
Elevation	what authority expands?	runtime to migration/admin

COVERAGE, NOT SCORING

Six categories do not rank severity and do not prove that a mitigation works.

FOCUSED THREAT LEDGER I**Edge and service-call threats**

The first ledger group protects the operator-to-owner path. It distinguishes identity spoofing, scope tampering, confused-deputy behavior, overbroad claims, stale identity and request exhaustion.

DECISION LEDGER 43-60

Threat	Boundary	Risk	Requirement
THR-043-001 stolen/forged user context	TB-01	H/H	SR-001, SR-007
THR-043-002 client scope tamper	TB-01	M/H	SR-002, SR-008
THR-043-003 hidden-action bypass	TB-01	H/H	SR-006
THR-043-004 BFF confused deputy	TB-02	M/H	SR-006, SR-009
THR-043-005 user token used as service	TB-02	M/H	SR-001, SR-010
THR-043-006 identity outage fails open	TB-01/02	M/H	SR-011

DISPOSITION

All six are OPEN requirements until Chapters 44-46 retain implementation and adversarial evidence.

FOCUSED THREAT LEDGER II

Messaging and owner-data threats

This group protects producer authority, frozen public facts, consumer admission, idempotent effects and database ownership.

DECISION LEDGER 43-61

Threat	Boundary	Risk	Requirement
THR-043-011 foreign event publication	TB-03	M/H	SR-006, SR-012
THR-043-012 broker over-privilege	TB-03/04	M/H	SR-006
THR-043-013 unsafe poison replay	TB-04/09/10	M/H	SR-013
THR-043-014 cross-owner DB write	TB-05	M/H	SR-006, SR-014
THR-043-015 runtime gets DDL	TB-06	M/H	SR-015
THR-043-022 message/body identity conflict	TB-04	M/H	SR-002, SR-016

ARCHITECTURE LEVERAGE

Owner databases, outbox/inbox transactions and strict admission already supply boundaries; security must preserve and prove them.

FOCUSED THREAT LEDGER III

Telemetry and diagnostics threats

Observability can leak sensitive data, accept spoofed signals, exhaust shared capacity or mislead incident response. Diagnostics can reveal topology or credentials even when business endpoints are protected.

DECISION LEDGER 43-62

Threat	Boundary	Risk	Requirement
THR-043-016 sensitive telemetry leak	TB-07	H/H	SR-004, SR-017
THR-043-017 telemetry tamper/spoof	TB-07	M/M	SR-002, SR-018
THR-043-018 diagnostic disclosure	TB-08	M/H	SR-004
THR-043-023 cardinality exhaustion	TB-07	M/H	SR-005
THR-043-024 collector lateral movement	TB-07	L/H	SR-006
THR-043-025 telemetry mistaken for audit	TB-07	M/H	SR-003, SR-018

OBSERVATION BOUNDARY

Security monitoring is essential, but telemetry never becomes authority for business or repair truth.

FOCUSED THREAT LEDGER IV

Administrative and delivery threats

The final group covers repair authority, self-approval, plan tampering, owner-port misuse, supply-chain substitution and secret/configuration compromise.

DECISION LEDGER 43-63

Threat	Boundary	Risk	Requirement
THR-043-019 stolen admin context	TB-09	M/H	SR-007, SR-013
THR-043-020 owner repair-port misuse	TB-10	M/H	SR-006, SR-013
THR-043-021 artifact/config substitution	TB-11/12	M/H	SR-002, SR-019
THR-043-026 self/duplicate approval	TB-09	M/H	SR-013
THR-043-027 secret disclosure/reuse	TB-12	M/H	SR-020
THR-043-028 emergency path unrecorded	TB-09/10	L/H	SR-003, SR-013

HIGHEST CONSEQUENCE

These paths can alter topology, schema, deployment or recovery behavior and therefore receive explicit separation and audit requirements.

ABUSE CASES

Security tests begin as attacker goals

Each high-priority threat becomes a misuse narrative with preconditions, action sequence, expected refusal or containment, durable evidence and recovery. This produces more useful tests than isolated 'invalid token' cases.

EXECUTABLE ASSET 43-F - ABUSE-CASE CONTRACT

```
public sealed record AbuseCase(  
    string Id,  
    string ThreatId,  
    string Actor,  
    IReadOnlyList<string> Preconditions,  
    IReadOnlyList<string> Steps,  
    string ExpectedBoundaryDecision,  
    IReadOnlyList<string> ForbiddenEffects,  
    IReadOnlyList<string> RequiredEvidence,  
    string RecoveryExpectation);
```

NEGATIVE EFFECTS MATTER

A refusal test asserts zero Domain calls, zero owner writes, zero publication and one safe accountability outcome where appropriate.

ABUSE CASE 43-01**A reader calls a hidden repair endpoint directly**

The actor has a valid ordinary operator session and discovers the administrative route. The request supplies a plausible plan ID and idempotency key. The boundary must deny before RepairControlDb transition or owner call and return a safe problem response.

DECISION LEDGER 43-65

Assertion	Expected result
authentication	valid human subject, wrong capability
authorization	deny Repair.Execute
RepairControlDb	no transition/intent
owner port	zero calls
audit	bounded denied-action evidence
response	403 without plan/credential details

THREAT COVERAGE

THR-043-003, THR-043-019 and SR-043-006/007/013.

ABUSE CASE 43-02**A user credential is replayed as a service identity**

An attacker presents an otherwise valid operator credential directly to an owning service or broker-facing path. The boundary must reject the principal type/audience even when the human would have the business capability through the BFF.

DECISION LEDGER 43-66

Assertion	Expected result
credential integrity	may be structurally valid
accepted principal type	workload required
audience	owner/broker resource binding required
Domain/application	zero invocation
database/broker	zero command
evidence	stable denial reason; no token content

THREAT COVERAGE

THR-043-005 and the exact Chapter 45 acceptance criterion.

ABUSE CASE 43-03**A compromised service publishes a foreign owner fact**

The compromised workload uses its legitimate broker connection but attempts another service's exchange/routing contract. Broker permission denies it; if traffic is misrouted, consumer admission rejects producer/type coordinates before Domain work.

DECISION LEDGER 43-67

Layer	Expected containment
publisher permission	route denied
topology	foreign binding absent
consumer admission	producer/type mismatch rejected
consumer Domain	zero calls
consumer DB	no inbox/business effect for rejected body
security evidence	safe principal/route/reason

LAYERED CONTROL

The consumer does not assume the broker permission is perfect; the broker does not parse Domain meaning.

ABUSE CASE 43-04**A telemetry field carries a canary secret**

The test injects a configured canary into exception text, configuration, baggage and message metadata. Exported logs, traces, metrics, problem details and evidence bundles must not contain the canary. Detection is a failed run, not a warning.

DECISION LEDGER 43-68

Surface	Required result
structured logs	no canary
trace attributes/events	no canary
metric labels/exemplars	no canary
HTTP problem	no canary
audit export	no credential material
artifact scan	zero configured secret patterns

THREAT COVERAGE

THR-043-016/018/027 and SR-043-004/017/020.

ABUSE CASE 43-05**An approved repair plan is altered before execution**

The actor changes one target, limit or snapshot coordinate while preserving the plan ID. Approval and execution digests no longer match. The administrative API rejects the request, stores no execution intent and calls no owner port.

DECISION LEDGER 43-69

Assertion	Expected result
plan ID	same
canonical bytes	different
digest comparison	constant-time mismatch
HTTP	409/412 stable problem
owner calls	zero
audit	tamper/stale-plan denial

THREAT COVERAGE

THR-043-020/026 and SR-043-002/013.

RISK ORDERING**Ordinal reasoning is explicit and revisable**

The ledger uses Low/Medium/High likelihood and impact with written rationale. It does not multiply subjective numbers into false precision. Priority also considers detectability, blast radius, reversibility, exposure duration and whether a boundary can affect owner truth or privileged repair.

DECISION LEDGER 43-70

Factor	Question
likelihood	what capability and exposure are required?
impact	which assets/owners/sites can be affected?
detectability	would authoritative evidence reveal it?
reversibility	can history be corrected without loss?
blast radius	one request, service, site or system?
duration	one attempt or persistent credential/config?

PHASE-0 LIMIT

Risk labels support engineering order; they are not enterprise risk acceptance or safety certification.

THREAT DISPOSITION

Every threat has one accountable state

Threats remain Open until a requirement owner and planned chapter exist. Mitigated requires implemented controls and evidence. Accepted requires an accountable decision and bounded rationale. Transferred names the external dependency and contract. NotApplicable needs proof that the flow or asset is absent.

DECISION LEDGER 43-71

Disposition	Required record
Open	owner + next action/chapter
Planned	security requirement + verification obligation
Mitigated	control identity + same-run evidence
Accepted	accountable approver + rationale + expiry
Transferred	dependency/contract + residual risk
NotApplicable	architecture/discovery proof

NO CLOSED BY PROSE

Writing a recommendation in this chapter cannot move a threat to Mitigated.

SECURITY REQUIREMENTS I

Identity, integrity and accountability requirements

Requirements are stable, test-oriented statements independent of a specific product. Later chapters may select mechanisms only if they preserve these outcomes.

DECISION LEDGER 43-72

Requirement	Outcome
SR-043-001	accepted principal type/authority/audience/freshness per boundary
SR-043-002	bind security decision to exact integrity-sensitive inputs
SR-043-003	durable accountability for consequential actions
SR-043-004	allow-list outward data by classification
SR-043-005	cost budgets and normal-path capacity protection
SR-043-006	local least privilege and absence of forbidden capabilities

MECHANISM NEUTRAL

A requirement remains valid whether Phase-0 uses one host, containers or a later orchestrated deployment.

SECURITY REQUIREMENTS II

Context, separation and failure requirements

The second group governs privileged and cross-service behavior, including human/workload separation and fail-closed dependency handling.

DECISION LEDGER 43-73

Requirement	Outcome
SR-043-007	fresh protected context for privileged human actions
SR-043-008	server-owned resource/site/tenant scope
SR-043-009	owner service re-authorizes delegated human action
SR-043-010	user identity is never sufficient workload trust
SR-043-011	identity/policy uncertainty fails closed and stays visible
SR-043-012	publisher/consumer routes match declared ownership
SR-043-013	repair preserves plan binding, dual control and owner receipt

NO HIDDEN AVAILABILITY OVERRIDE

Fail closed may reduce availability; the system reports the blocked state rather than silently weakening authority.

SECURITY REQUIREMENTS III

Data, telemetry, deployment and secret requirements

The final group protects owner storage, message admission, observability, artifacts and credential material.

DECISION LEDGER 43-74

Requirement	Outcome
SR-043-014	runtime credentials cannot cross owner databases
SR-043-015	migration DDL is separate, bounded and auditable
SR-043-016	strict route/envelope/inbox admission before Domain work
SR-043-017	sensitive data excluded/minimized in telemetry
SR-043-018	telemetry never becomes business/audit authority
SR-043-019	runtime binds to reviewed artifact/config identity
SR-043-020	secret/certificate lifecycle prevents logging/reuse

TRACEABILITY

Each requirement maps back to one or more TB/THR entries and forward to a Chapter 44-48 verification obligation.

NEGATIVE REQUIREMENTS

Security is also defined by forbidden authority

Positive permissions are incomplete unless prohibited paths are testable. The architecture manifest therefore records absence of owner DB credentials, foreign broker routes, generic repair clients, runtime DDL, unbounded telemetry attributes and anonymous mutation endpoints.

EXECUTABLE ASSET 43-G - PROHIBITED AUTHORITY MANIFEST

```
public sealed record ProhibitedAuthority(  
    string RequirementId,  
    string Principal,  
    string ResourcePattern,  
    string Operation,  
    string DiscoverySource);  
  
var prohibited = new[]  
{  
    new("SR-043-014", "repair-control", "*-owner-db", "DML", "secret-inventory"),  
    new("SR-043-015", "*-runtime", "*-owner-db", "DDL", "grant-inventory"),  
    new("SR-043-012", "root-cause", "compliance.*", "publish", "broker-permissions"),  
    new("SR-043-013", "repair-control", "*", "generic-sql", "service-graph")  
};
```

ABSENCE IS EVIDENCE

A manifest that lists only granted permissions cannot prove that a dangerous grant is absent.

ASSUMPTION REGISTER

Assumptions are named, owned and expiring

The model separates verified facts from assumptions about identity authority, time, host isolation, broker durability, database grants, telemetry export and administrator devices. Every assumption names an owner, validation path, expiry and consequence if false.

EXECUTABLE ASSET 43-H - ASSUMPTION RECORD

```
public sealed record TrustAssumption(  
    string AssumptionId,  
    string Statement,  
    string Owner,  
    string ValidationMethod,  
    Instant ReviewBy,  
    string ConsequenceIfFalse,  
    AssumptionStatus Status);  
  
public enum AssumptionStatus  
{  
    Proposed, AcceptedForPhase0, Verified, Invalidated, Expired  
}
```

NO PERMANENT ASSUMPTION

An expired or invalidated assumption reopens linked threats and can block deployment or privileged execution.

NON-ASSUMPTIONS

The model explicitly refuses convenient trust shortcuts

Several properties are never accepted as security proof. Writing them down prevents future implementation choices from smuggling them back as defaults.

DECISION LEDGER 43-77

Not assumed	Reason
internal network is trusted	compromise/lateral movement remains possible
BFF authorization is final	owner policy/resource context required
broker message is valid	strict consumer admission required
database login means owner authority	grant may be overbroad
TLS means authorized business action	channel and permission differ
telemetry is complete/untampered	sample/export/collector boundary
admin is benign	misuse/compromise and separation matter

REVIEW VALUE

A future chapter must justify any exception through a new ADR and updated threat ledger.

TRUST FRESHNESS

Every accepted context decays with time and change

Human sessions expire or are revoked; workload instances and keys rotate; plans expire; configuration and policy versions change; admin approvals become stale; deployment artifacts are superseded. Boundaries name the freshness source and behavior when it is unavailable.

DECISION LEDGER 43-78

Context	Freshness coordinate
human identity	issued/expiry/revocation/auth context
workload identity	instance/key/cert lifetime
authorization	policy identity + decision time
repair plan	snapshot digest + expiry + ETag
configuration	configuration identity/version
artifact	build/deployment identity
threat model	architecture/review version

SR-043-021

Security-sensitive context must have explicit freshness and revocation semantics; cache success is not indefinite trust.

TIME BOUNDARY

Clock uncertainty can change security outcomes

Expiry, approval windows, nonce/replay windows, certificate validity, repair plans and evidence staleness depend on time. The system uses server-side time sources and defined skew budgets; browser time and message arrival order cannot authorize an action.

DECISION LEDGER 43-79

Threat	Requirement
clock rollback extends trust	monotonic/reference time policy
skew rejects valid context	bounded skew and explicit failure
browser changes timestamp	server-owned decision time
delayed message appears fresh	occurred/observed/received separated
old approval replay	expiry + plan/resource binding
time source unavailable	fail closed for privileged actions

NO TIME MAGIC

Time is a dependency with an uncertainty budget, not a universally trusted scalar.

FAILURE SEMANTICS

Security uncertainty is not converted to anonymous access

When identity, policy, secret, certificate status, clock or authorization dependencies are unavailable, protected operations fail closed. Read-only experiences may return an explicit unavailable/unknown state where safe. No cached allow result is extended beyond its declared policy.

DECISION LEDGER 43-80

Failure	Protected operation outcome
identity authority unavailable	deny/503; no Domain call
policy mapping unknown	deny; stable reason
workload identity unavailable	no downstream/broker call
secret/certificate invalid	startup/readiness failure
repair approval status unknown	no execution
telemetry unavailable	correctness continues; visibility degraded

AVAILABILITY HONESTY

A blocked operation is visible as denied or unavailable; it is never reported as successful for continuity.

BOUNDARY DECISION EVIDENCE**Record enough to explain without leaking credentials**

Security decisions emit bounded reason code, policy identity, principal type and pseudonymous/stable subject reference where justified, resource scope, decision time and correlation reference. Credentials and raw claims are not logged.

EXECUTABLE ASSET 43-I - SAFE DECISION OBSERVATION

```
public sealed record BoundaryDecisionObserved(  
    string BoundaryId,  
    string Decision,  
    string ReasonCode,  
    string PolicyIdentity,  
    string PrincipalType,  
    string? SubjectReference,  
    string ResourceClass,  
    string ScopeDigest,  
    DateTimeOffset ObservedAtUtc,  
    string CorrelationReference);
```

AUDIT SEPARATION

This observation supports diagnosis. Consequential business/repair actions still require their authoritative owner/audit records.

SECURITY TELEMETRY**Signals are bounded, useful and non-authoritative**

Metrics aggregate decision, boundary, reason class and principal type. Traces link safe boundary decisions. Logs carry protected references. Security signals never include raw tokens, message bodies, secrets or unbounded actor/resource identifiers.

DECISION LEDGER 43-82

Signal	Bounded dimensions
boundary_decisions_total	TB, allow/deny, reasonClass
identity_validation_total	principalType, outcome
authorization_denied_total	capabilityClass, reasonClass
broker_permission_denied_total	producerClass, routeClass
secret_redaction_failures_total	surface
admin_actions_current	state, riskClass

CARDINALITY DEFENSE

Subject, MessageId, RepairId and resource IDs belong only in controlled traces/logs when required.

DETECTION VERSUS PREVENTION

Every high-impact threat names both where practical

Prevention limits what can occur; detection reveals attempts, drift or compromise; response contains and recovers. A threat is not mitigated by alerting alone when the action can alter owner truth, replay poison or deploy code.

DECISION LEDGER 43-83

Threat	Prevent	Detect/respond
cross-owner DB	no credential/grant	grant inventory drift alert
foreign publish	route permission	deny/route anomaly
plan tamper	digest binding	audit denial
secret leak	do not emit	canary artifact scan
artifact substitution	verified identity/gate	runtime manifest mismatch
DoS	budgets/bulkheads	pressure alert + kill switch

RESPONSE OWNERSHIP

Alerts without an owner, threshold, safe action and evidence path are incomplete controls.

THREAT-TO-TEST TRACEABILITY

Every planned mitigation becomes an adversarial obligation

Chapter 48 will validate identity failures, authorization boundaries, route and database least privilege, secret redaction, telemetry isolation, repair separation and configuration/artifact binding. The traceability matrix prevents a green suite with no coverage of high-risk scenarios.

DECISION LEDGER 43-84

Threat group	Required verification class
TB-01/02 identity	valid/expired/missing/insufficient/cross-audience
TB-03/04 messaging	permission + admission + duplicate/conflict
TB-05/06 database	grant inventory + forbidden access probes
TB-07/08 telemetry	canary/redaction/tamper/cardinality
TB-09/10 admin	separation/digest/stale/duplicate/unknown
TB-11/12 delivery	artifact/config/secret identity and rotation

ZERO DISCOVERY FAILS

A security suite that discovers no threat-linked tests is a failed evidence run.

SECURITY EVIDENCE

The boundary is proved by correlated artifacts

Required evidence includes endpoint/policy inventory, accepted principal matrix, broker permissions, database grants, secret-free configuration manifest, telemetry export scan, adversarial test results, audit chain, artifact identity and negative-effect snapshots.

DECISION LEDGER 43-85

Claim	Evidence
edge protected	route/policy inventory + identity matrix tests
service identity distinct	cross-principal rejection tests
broker least privilege	permission inventory + probes
DB owner isolation	grant inventory + forbidden connection tests
telemetry safe	canary scan + collector config identity
repair separated	approval/owner/audit fault evidence
runtime intended	build/config/deployment identity

SAME-RUN RULE

Artifacts must share environment, configuration and run identity; evidence from unrelated runs cannot prove one boundary.

THREAT-MODEL MANIFEST

The security model has a machine-readable checkpoint

The manifest names diagram/catalog/ledger/ADR digests, open threat counts, requirement coverage, evidence status and production-maturity non-claim. It changes whenever a boundary, asset, principal or threat changes.

EXECUTABLE ASSET 43-J - MODEL MANIFEST

```
{
  "chapter": 43,
  "modelId": "nexus1/distributed-trust-boundaries/1",
  "adr": "ADR-043",
  "adrSha256": "4060479ea89e0f5e4fe3c8aa71d432918ba7d6576ed7bb92ca2f44adbc2d95ce",
  "boundaryCatalogSha256": "651328e6e3191afb2de82372102a581b5fbe950aa951293590d0d2ae0fb73f93",
  "boundaryCount": 12,
  "openThreatCount": 28,
  "securityRequirementCount": 21,
  "mechanismEvidence": "PENDING",
  "productionSecurityClaim": false,
  "status": "SPECIFIED_RUNTIME_PENDING"
}
```

NO IMPLIED HARDENING

A complete threat model is a design artifact, not proof that controls have been implemented or operated.

MANIFEST VALIDATOR

Missing boundaries and orphan threats fail the build

The validator requires unique TB/THR/SR identities, diagram-to-catalog parity, threat-to-boundary references, requirement ownership, high-risk verification obligations and an explicit non-production claim while evidence is pending.

EXECUTABLE ASSET 43-K - FAIL-CLOSED MODEL GATE

```
public ThreatModelVerdict Validate(ThreatModel model)
{
    Require(model.Boundaries.Count > 0);
    RequireUnique(model.Boundaries.Select(x => x.Id));
    Require(model.DiagramBoundaryIds.SetEquals(model.BoundaryIds));
    Require(model.Threats.All(t => model.BoundaryIds.Contains(t.BoundaryId)));
    Require(model.Threats.All(t => t.RequirementIds.Count > 0));
    Require(model.HighThreats.All(t => t.VerificationIds.Count > 0));
    Require(model.Requirements.All(r => r.Owner is not null));
    Require(model.EvidenceStatus != "PROVED" || model.RuntimeArtifacts.Valid);
    Require(!model.ProductionSecurityClaim);
    return ThreatModelVerdict.Specified(model.CanonicalSha256);
}
```

NO ORPHAN ROWS

A diagram-only boundary, ledger-only threat or mechanism-only requirement is a failed model.

ARCHITECTURE DISCOVERY

The runtime inventory must match the model

Static and production-host discovery inventories endpoints, authorization metadata, HttpClient destinations, broker producers/consumers, DbContexts/connections, hosted workers, telemetry exporters and administrative routes. Unknown edges become findings.

DECISION LEDGER 43-88

Discovery source	Compared with
endpoint inventory	TB-01/08/09 routes and policies
typed clients	TB-02/10 destinations
broker topology	TB-03/04 publishers/consumers
DbContext/connection inventory	TB-05/06 owner grants
hosted workers	workload principals/capabilities
telemetry config	TB-07 exporters/processors
deployment manifest	TB-11/12 inputs

DRIFT FINDING

A newly discovered connection is not auto-trusted; the model and review must change first.

THREAT REVIEW

Review includes owners of every side of the boundary

A meaningful review includes service/domain owners, Edge, messaging, database, platform, observability, security, test/evidence and repair-operation perspectives. One team cannot silently accept risk owned by another boundary.

DECISION LEDGER 43-89

Reviewer	Required perspective
Domain/service owner	business authority and invariants
Edge/UI	human context and browser threats
messaging	route, delivery, poison and replay
data/platform	grants, migration and backup
observability	collection/export/privacy/capacity
security	threat completeness and control fit
test/evidence	adversarial proof and chain-of-custody

DECISION RECORD

The review records GO, CONDITIONAL GO, HOLD or NO-GO with open threats, owners and expiry; attendance alone is not approval.

CHANGE TRIGGERS

Security review reopens when trust changes

The model is revisited when a new service/route/store/exporter appears, identity or policy mapping changes, a secret/certificate mechanism changes, admin capability is added, data classification rises, deployment topology changes or an incident invalidates an assumption.

DECISION LEDGER 43-90

Trigger	Required action
new endpoint/client/queue/DB	new or updated TB contract
new principal/capability	principal and threat review
identity provider/claim change	TB-01/02/09 reassessment
broker/DB grant change	least-privilege comparison
telemetry field/export change	classification/redaction review
repair kind added	high-risk abuse and dual-control review
incident or failed assumption	reopen linked threats

VERSIONING

Diagram, boundary catalogue, threat ledger and requirement matrix move as one versioned set.

SECURITY AND RESILIENCE

Protective controls must fail within bounded resource use

Identity validation, policy mapping, secret retrieval, telemetry scrubbing and audit emission consume resources and can fail. Their timeouts, caches, retry behavior and bulkheads belong in the boundary contract so a security dependency cannot create unbounded systemic failure.

DECISION LEDGER 43-91

Control dependency	Resilience requirement
identity validation	bounded cache/freshness; fail closed
policy mapping	local deterministic policy where possible
key/secret source	startup/rotation/readiness semantics
audit store	business action atomicity where required
telemetry exporter	no correctness dependency; bounded buffer
admin approval lookup	no execution on uncertainty

NO RETRY STORM

A security-service outage must not trigger synchronized unbounded retries across every workload.

DEPLOYMENT SEQUENCE

Observe boundary parity before enforcing mechanisms

Part VIII rollout begins with the registry and threat ledger, then production-path discovery. Edge, workload, claims and secret mechanisms are introduced incrementally with deny/audit modes only where risk permits, adversarial tests and rollback/lockout recovery.

DECISION LEDGER 43-92

Stage	Capability	Exit gate
0	boundary/threat catalogue	reviewed model and open owners
1	runtime discovery	diagram parity; unknown edges resolved
2	edge identity/policy	Chapter 44 adversarial matrix
3	workload identity	Chapter 45 cross-principal proof
4	claims minimization	Chapter 46 contract tests
5	secret/cert/config	Chapter 47 lifecycle evidence
6	security evidence gate	Chapter 48 same-run dossier

LOCKOUT PLANNING

Rollback restores the prior reviewed policy/configuration; it never enables anonymous or broad fallback authority.

INCIDENT RUNBOOK

Contain the boundary without destroying evidence

When compromise is suspected, operators identify affected TB/principal/assets, stop or narrow new authority, preserve owner and audit evidence, rotate/revoke through approved procedures, compare expected/discovered grants and replay only through Chapter 42 controls.

DECISION LEDGER 43-93

Order	Action
1	name TB, principal, resource, time and evidence scope
2	contain capability/credential/route; protect normal path
3	preserve owner, inbox/outbox, broker and audit artifacts
4	compare permission/config/artifact identities
5	rotate/revoke/redeploy through accountable path
6	reconcile business/projection/repair consequences
7	update threats, assumptions, tests and evidence

DO NOT CLEAN FIRST

Deleting poison, logs, audit rows or failed generations before preservation destroys the evidence needed to understand the incident.

REJECTED SHORTCUTS

Security slogans cannot replace boundary decisions

The chapter rejects one perimeter, one super-token, one shared service account, one database admin credential, one generic broker user, one telemetry dump and one emergency replay button. Each shortcut hides an authority transition.

DECISION LEDGER 43-94

Shortcut	Hidden failure	Required correction
internal = trusted	lateral movement	per-boundary decision
BFF already authorized	owner bypass	local capability policy
same token everywhere	audience/principal confusion	bounded contexts
shared DB/broker admin	cross-owner authority	principal manifests
TLS solves security	channel != authorization	layered claims
log everything	disclosure/cardinality	classified allow lists
admin knows best	misuse/repudiation	separation + audit

DESIGN SMELL

A mechanism described without its TB, threat, asset, principal and failure behavior is not yet a security design.

PART VIII ROADMAP

The remaining chapters implement the requirements in order

Chapter 43 supplies the common model. Chapter 44 secures human entry and operator capabilities. Chapter 45 establishes workload identity. Chapter 46 minimizes and maps claims locally. Chapter 47 governs secrets, certificates and configuration. Chapter 48 proves the boundary with adversarial evidence.

DECISION LEDGER 43-95

Chapter	Primary boundaries	Deliverable
44	TB-01, TB-09	edge authentication/operator authorization
45	TB-02, TB-03/04, TB-10	service-to-service identity
46	TB-01/02/09/10	minimal context and local mapping
47	TB-06/11/12	secrets, certificates, configuration
48	all TBs	security tests and audit evidence

SEQUENCE DISCIPLINE

Later chapters may refine the model, but they cannot silently contradict its non-transitivity, ownership or evidence rules.

CHAPTER 44 BOUNDARY

Human identity enters at the Edge; owner policy remains local

Chapter 44 must authenticate at the BFF, preserve safe session behavior, map the operator to explicit capabilities, and enforce those capabilities again in owning services. It must test valid, expired, missing, insufficient and wrong-scope identities without weakening availability semantics.

DECISION LEDGER 43-96

Chapter 43 requirement	Chapter 44 implementation proof
TB-01 accepted human context	valid/expired/missing identity matrix
server-owned resource scope	cross-site/tenant denial
BFF not final authority	owner-service re-authorization
fresh privileged context	repair/admin action requirements
safe denial	zero effects + bounded problem/audit
no production claim	mechanism limits documented

PROTECTED HANDOFF

The mechanism must satisfy the model; the model is not rewritten to match whichever mechanism is easiest to configure.

OFFICIAL TECHNICAL ANCHORS

Threat modeling remains evidence-led and resource-focused

- NIST SP 800-207 Zero Trust Architecture: <https://csrc.nist.gov/pubs/sp/800/207/final>
- NIST SP 800-154 data-centric threat modeling: <https://csrc.nist.gov/pubs/sp/800/154/ipd>
- NIST SP 1800-35 Zero Trust practice guide:
<https://www.nccoe.nist.gov/projects/implementing-zero-trust-architecture>
- CISA Zero Trust Maturity Model: <https://www.cisa.gov/zero-trust-maturity-model>
- OWASP Threat Modeling Project: <https://owasp.org/www-project-threat-modeling/>
- OWASP Threat Modeling Cheat Sheet:
https://cheatsheetseries.owasp.org/cheatsheets/Threat_Modeling_Cheat_Sheet.html
- Microsoft STRIDE threat categories:
<https://learn.microsoft.com/azure/security/develop/threat-modeling-tool-threats>
- RabbitMQ access control: <https://www.rabbitmq.com/docs/access-control>
- RabbitMQ TLS support: <https://www.rabbitmq.com/docs/ssl>
- OpenTelemetry security guidance: <https://opentelemetry.io/docs/security/>
- OpenTelemetry sensitive-data guidance: <https://opentelemetry.io/docs/security/handling-sensitive-data/>

WHAT THIS CHAPTER HAS PROVED

Chapter 43 specifies the Part VIII security premise, ADR-043, asset/principal/data-classification catalogues, nine logical zones, twelve stable trust boundaries, separate runtime and administrative diagrams, STRIDE coverage, twenty-eight focused threats, abuse cases, twenty-one mechanism-neutral security requirements, negative authority rules, assumptions, freshness/failure semantics, threat-to-test/evidence traceability, runtime discovery, review/change triggers, deployment and incident runbooks. Mechanisms and runtime evidence remain PENDING; production security maturity is explicitly not claimed.

CHAPTER CHECKPOINT

- Why does a private network not establish identity or authorization?
- Which assets matter beyond payload confidentiality?
- Why must human and workload identity remain separate?
- What does broker authentication prove, and what remains for consumer admission?
- Why are telemetry and audit not interchangeable?
- Which crossing makes Repair Control dangerous even without direct database access?
- How does an approval bind exact plan bytes?
- What evidence can prove absence of cross-owner database or broker authority?
- When must identity or policy uncertainty fail closed?
- Which artifacts are needed before a threat can move from Planned to Mitigated?

NEXT - CHAPTER 44

Edge Authentication and Operator Authorization will implement TB-01 and the human side of TB-09. It will authenticate at the BFF, derive bounded operator capability context, enforce it again at owning services, and test valid, expired, missing, insufficient and wrong-scope identities while keeping human trust separate from service-to-service identity.

PART VIII - SECURITY AND TRUST ACROSS SERVICES

44. Edge Authentication and Operator Authorization

The BFF authenticates the human and owns the browser session. Angular receives no OAuth tokens. Owning services independently authorize the bounded human context against current resources, while workload trust remains a separate mandatory gate.

CHAPTER DECISION

ADR-044 selects an OIDC confidential Authorization Code flow with PKCE, a server-side revocable BFF session, an opaque Secure/HttpOnly/SameSite cookie, anti-forgery for every cookie-authenticated mutation, typed operator capabilities and owner-local resource authorization. The browser never receives access, ID or refresh tokens. A valid user identity never satisfies service trust. The production BFF workload mechanism remains PENDING for Chapter 45, full external claim mapping remains Chapter 46 work, and production security maturity is not claimed.

PROOF PATH 44-A - HUMAN ENTRY TO OWNER DECISION



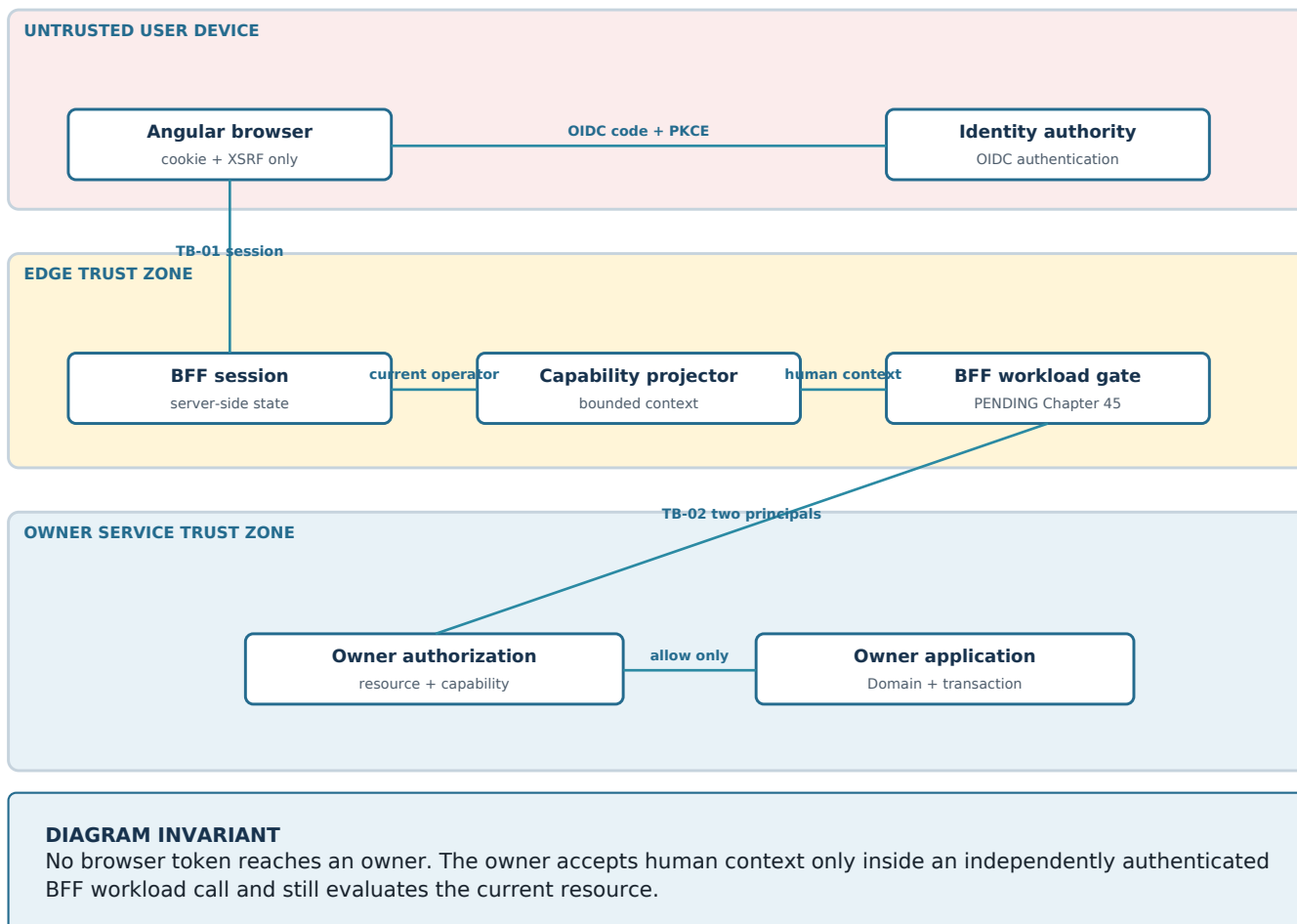
GUARANTEE LEDGER 44-A

Specified	Explicitly rejected
valid/expired/missing/insufficient matrix	login equals authorization
revocable server session	token in browser storage
owner resource reauthorization	BFF as global authority
anti-forgery on mutations	SameSite as sole CSRF control
PENDING workload/evidence gates	user token as service trust

EDGE TRUST-BOUNDARY DIAGRAM

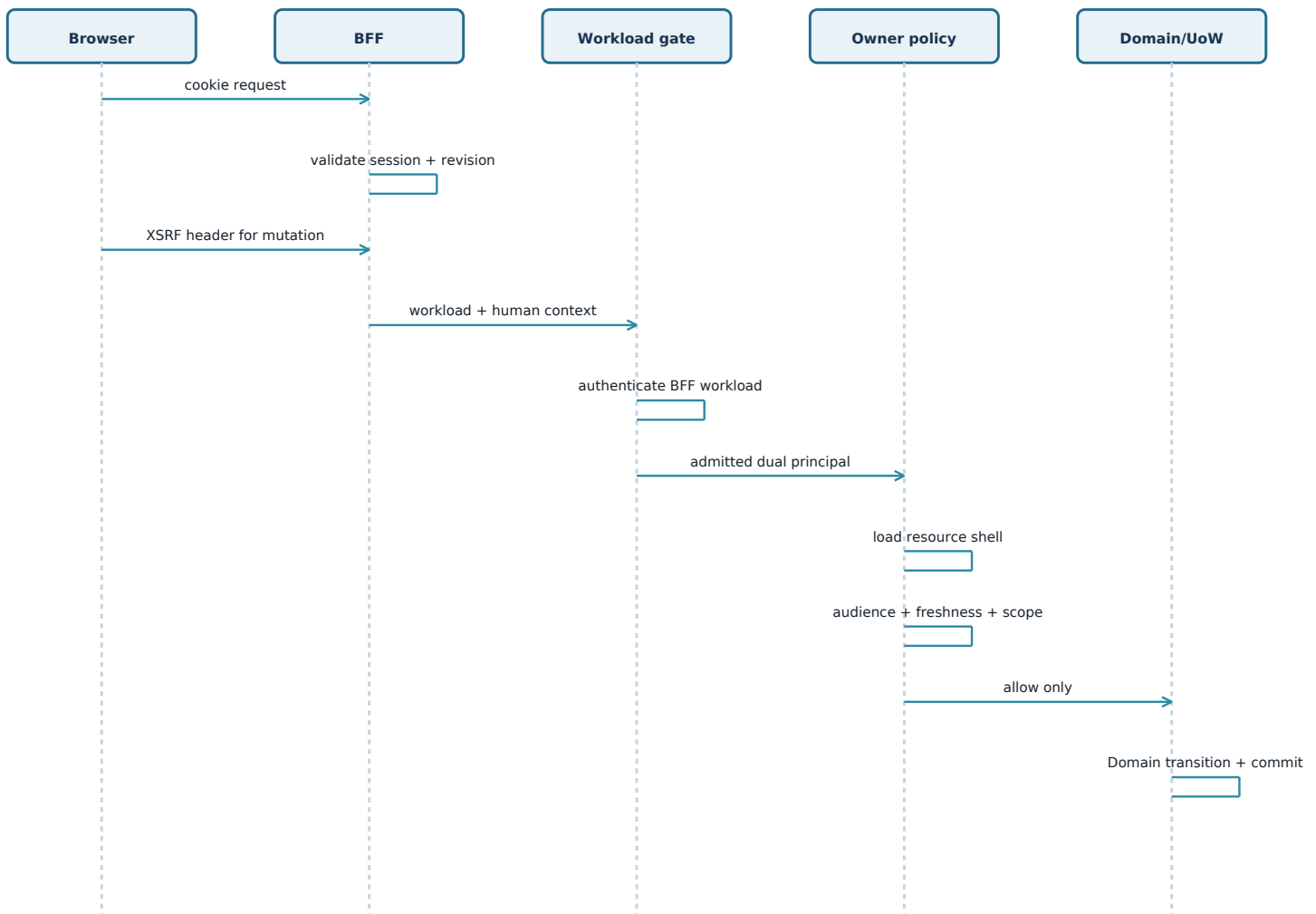
The browser terminates at the BFF; owner truth remains behind a second gate

The diagram shows security decisions rather than network hops. OIDC authenticates the human into a BFF-owned session. Angular receives only a safe summary. A protected downstream call must later carry an authenticated BFF workload plus bounded human context, and the owner reauthorizes the exact resource.



DUAL-GATE DECISION SEQUENCE**An allow decision is the intersection of session, request, workload and owner policy**

The sequence makes short-circuit order testable. Authentication and anti-forgery happen at the edge. Workload trust admits the caller at the owner. Human context is then checked against current resource state. Any failure stops all later work.

**SHORT-CIRCUIT INVARIANT**

Missing session, invalid XSRF, absent workload trust, stale human context, insufficient capability or wrong resource scope produces zero Domain calls and zero owner commits.

CHAPTER BOUNDARY

The chapter secures the human path without inventing workload trust

Chapter 44 implements TB-01 and the human side of TB-09. It also specifies the human context needed at TB-02, but an owner may consume that context only after an independent BFF workload gate. The production mechanism for that gate remains the explicit subject of Chapter 45.

DECISION LEDGER 44-4

Implemented here	Held for later	Reason
operator authentication	workload authentication	different principals
BFF session	service credential	browser is not a service
typed capabilities	full external claim mapping	Chapter 46 owns mapping
owner human-policy check	production edge-call trust	Chapter 45 gate
test seams and matrices	retained adversarial dossier	Chapter 48 gate

NO SCOPE THEFT

A complete Chapter 44 makes Chapter 45 necessary and testable; it does not make it redundant.

INHERITED MODEL

ADR-043 requirements become concrete refusal points

The mechanism is acceptable only if it preserves the model established in Chapter 43. Authentication, authorization, resource scope, freshness, workload trust and audit are separate decisions with separate evidence.

DECISION LEDGER 44-5

Inherited requirement	Chapter 44 response
SR-043-001	issuer, audience, subject and freshness validation
SR-043-007	recent authentication for privileged human actions
SR-043-008	server-owned site and tenant scope
SR-043-009	owner service reauthorizes the human action
SR-043-010	user identity never satisfies workload trust
SR-043-011	identity and policy uncertainty fail closed
SR-043-021	expiry, revocation and policy revision are explicit

TRACEABLE MECHANISM

Every configuration and test in this chapter names the requirement it is intended to satisfy.

SECURITY MATURITY

A coherent edge design is not production security maturity

The manuscript selects a defensible Phase-0 pattern and provides compile-oriented reference code. It does not claim a configured identity provider, protected production key ring, penetration test, monitored revocation channel or retained runtime evidence.

DECISION LEDGER 44-6

Claim	Status
edge and owner authorization design	SPECIFIED
reference code and test catalogue	SPECIFIED
configured IdP interoperability	PENDING
production workload gate	PENDING Chapter 45
claim minimization implementation	PENDING Chapter 46
runtime adversarial evidence	PENDING Chapter 48
production security maturity	NOT CLAIMED

HONEST EVIDENCE

Reference code demonstrates design intent. Only retained build, test and environment artifacts can prove execution.

IDENTITY VOCABULARY

Principal, session, capability and resource are not synonyms

Precise language prevents a valid login from being promoted into unlimited authority. The BFF knows a bounded operator session; the owner knows a bounded action on a loaded resource; neither fact creates workload trust.

DECISION LEDGER 44-7

Term	Meaning
subject	external human identity reference
operator session	BFF-owned current login state
authentication context	how and when identity was established
capability	typed action class exposed by local policy
scope	server-owned site/tenant/resource coordinates
workload principal	calling process identity; Chapter 45
resource decision	owner-local allow or deny for one target

NO ROLE MAGIC

A role-like external claim can be input to mapping later; it is never the final owner-service decision.

THREAT RESPONSE

The edge controls are selected against named abuse paths

Mechanisms are not copied from a checklist. Each one responds to a concrete spoofing, replay, elevation, scope, CSRF or disclosure threat at TB-01, TB-02 or TB-09.

DECISION LEDGER 44-8

Abuse	Control objective	Refusal evidence
stolen bearer in browser	no browser token	storage and response tests
session fixation	rotate on sign-in/change	session-id transition
CSRF mutation	anti-forgery	zero owner calls
wrong site	owner resource scope	404/zero effects
stale capability	revision/freshness	stable denial code
user token as service	workload gate	independent rejection
UI bypass	server checks	direct-request test

CONTROL PLACEMENT

A browser control can improve safety, but only BFF and owner controls can authorize the request.

ADR-044

Human identity terminates at the BFF and owners reauthorize

ADR-044 fixes the trust shape while leaving provider and workload mechanisms replaceable. The digest binds the decision record used by later evidence.

REFERENCE ASSET 44-A - CANONICAL DECISION

```
public sealed record EdgeAuthorizationDecision(  
    string AdrId,  
    string SessionProfile,  
    string OperatorContextProfile,  
    bool BrowserReceivesTokens,  
    bool OwnerReauthorizes,  
    bool UserIdentitySatisfiesWorkloadTrust,  
    string EvidenceStatus,  
    bool ProductionSecurityClaim);  
  
var decision = new EdgeAuthorizationDecision(  
    "ADR-044",  
    "nexus1/edge-operator-session/1",  
    "nexus1/operator-context/v1",  
    BrowserReceivesTokens: false,  
    OwnerReauthorizes: true,  
    UserIdentitySatisfiesWorkloadTrust: false,  
    EvidenceStatus: "PENDING",  
    ProductionSecurityClaim: false);
```

ADR DIGEST

sha-256+jcs d63cd73aeee79027c27aec1dd3542fcf1bd253b2e60ae7f665a01ba0d049771c

DECISION OWNERSHIP

Each decision has one accountable component

The design avoids an authorization database disguised as an edge service. The identity provider authenticates, the BFF owns browser session state, capability mapping is bounded, and the owner service decides on its resource.

DECISION LEDGER 44-10

Decision	Owner	Not delegated to
external authentication	identity authority	Angular
browser session validity	BFF	owner database
safe UI capability summary	BFF projector	route guard
resource existence/scope	owning service	BFF
business permission	owning service	identity provider
workload trust	destination boundary	human session
audit truth	owner/admin authority	telemetry

LOCAL AUTHORITY

The BFF can deny early; it cannot manufacture an owner allow decision.

DENY TAXONOMY

Unauthenticated, unauthorized, hidden and stale are different

Stable semantics help clients recover without leaking protected details. The route contract chooses whether wrong-scope resources are hidden, and every mutating denial proves that no owner command or write occurred.

DECISION LEDGER 44-11

Condition	HTTP	Client action
missing/expired/revoked session	401	offer sign-in
authenticated, capability absent	403	explain unavailable action
resource outside visible scope	404	do not confirm existence
stale ETag/version	412	refresh before retry
valid request conflicts with state	409	show current state
identity/policy dependency unavailable	503	safe retry only

NO REDIRECT TRAP

Protected API routes return bounded problem responses; they never turn 401 or 403 into an HTML identity-provider redirect.

BFF PATTERN

The browser receives a session, not OAuth tokens

The BFF is the confidential interactive client. It performs OIDC Authorization Code flow with PKCE, validates the authentication response, projects a minimal operator session and returns only an opaque protected cookie to the browser.

DECISION LEDGER 44-12

Layer	Receives	Must not receive
browser	HttpOnly session cookie	access/id/refresh token
Angular	safe session summary	raw claims or credentials
BFF	OIDC response and session record	global owner authority
owner service	trusted workload + bounded human context	browser cookie
telemetry	reason/policy/reference	token or full claim set

BFF INVARIANT

The browser cannot copy a token it never receives. XSS risk remains, but token extraction from Web Storage is removed from the design.

BFF COMPOSITION

Cookie authentication handles the session; OIDC handles challenge

The .NET 8 host uses an explicit cookie scheme for authentication and an explicit OIDC scheme for challenge. The configuration uses code flow with PKCE and does not persist tokens into the authentication ticket.

REFERENCE ASSET 44-B - AUTHENTICATION SCHEMES

```
const string SessionScheme = "OperatorSession";
const string OidcScheme = "OperatorOidc";

services.AddAuthentication(options =>
{
    options.DefaultAuthenticateScheme = SessionScheme;
    options.DefaultSignInScheme = SessionScheme;
    options.DefaultChallengeScheme = OidcScheme;
})
.AddCookie(SessionScheme, options => ConfigureSessionCookie(options))
.AddOpenIdConnect(OidcScheme, options =>
{
    options.Authority = oidc.Authority;
    options.ClientId = oidc.ClientId;
    options.ClientSecret = oidc.ClientSecret;
    options.ResponseType = OpenIdConnectResponseType.Code;
    options.UsePkce = true;
    options.SaveTokens = false;
    options.MapInboundClaims = false;
    options.GetClaimsFromUserInfoEndpoint = false;
});
```

CONFIGURATION BOUNDARY

Authority, client identity, redirect URIs and credential material are validated configuration. Secret lifecycle remains Chapter 47 work.

OIDC VALIDATION

Issuer, audience, nonce, state and lifetime remain middleware concerns

The application does not hand-parse ID tokens. It configures the protocol handler, requires HTTPS metadata outside isolated tests and maps only the identifiers needed to project the operator session.

REFERENCE ASSET 44-C - BOUNDED VALIDATION

```
options.RequireHttpsMetadata = true;
options.TokenValidationParameters = new TokenValidationParameters
{
    ValidateIssuer = true,
    ValidIssuer = oidc.ExpectedIssuer,
    ValidateAudience = true,
    ValidAudience = oidc.ClientId,
    ValidateLifetime = true,
    ClockSkew = TimeSpan.FromSeconds(30),
    NameClaimType = "name",
    RoleClaimType = "nexus1-unused-role"
};

options.Scope.Clear();
options.Scope.Add("openid");
options.Scope.Add("profile");
```

ROLE CLAIM INTENTIONALLY UNUSED

The external provider authenticates the subject. Chapter 46 will map selected external inputs to local capabilities; no generic role string authorizes an owner resource here.

OIDC TRANSACTION

Every redirect parameter has a bounded job

The BFF relies on maintained protocol middleware and exact redirect registration. State protects request correlation, nonce binds the ID token transaction, and PKCE binds authorization and token exchange. None of them authorizes a business action.

DECISION LEDGER 44-15

Coordinate	Protects	Does not prove
state	request/response correlation	operator capability
nonce	ID token replay context	resource ownership
code_verifier	authorization code exchange	client identity by itself
redirect URI	registered callback target	safe arbitrary return URL
ID token	authentication result	service-to-service trust
session cookie	BFF session reference	owner authorization

PROTOCOL BOUNDARY

OAuth and OIDC transaction protections are necessary, but owner authorization is a separate local decision.

LOGIN ENDPOINT

Return locations are local, canonical and allow-listed

The login endpoint accepts only a local UI return path. It stores the validated return path in authentication properties and lets the OIDC handler create state and correlation artifacts.

REFERENCE ASSET 44-D - SAFE CHALLENGE

```
app.MapGet("/bff/login", (
    string? returnUrl,
    LinkGenerator links,
    HttpContext http) =>
{
    var fallback = links.GetPathByName("operator-home", values: null!);
    var target = IsSafeLocalUiPath(returnUrl) ? returnUrl! : fallback;

    return Results.Challenge(
        new AuthenticationProperties { RedirectUri = target },
        authenticationSchemes: ["OperatorOidc"]);
})
.AllowAnonymous();
```

NO OPEN REDIRECT

Scheme-relative paths, absolute URLs, encoded control characters and paths outside the UI allow-list are rejected before the challenge begins.

REDIRECT ALLOW-LIST

Local syntax alone is not the complete return-path policy

A path can be syntactically local and still lead into a sensitive or protocol route. The BFF combines local-URL validation with an explicit UI prefix allow-list and canonical decoding.

DECISION LEDGER 44-17

Input	Disposition	Reason
/alarms/active	accept	allowed UI route
/repair/queue	accept if capability later	safe return only
//evil.example	reject	scheme-relative redirect
https://evil.example	reject	absolute external URL
/%2f%2fevil.example	reject	decoded scheme-relative path
/signin-oidc	reject	protocol callback
/ui-api/v1/session	reject	API is not UI return target

REDIRECT IS NAVIGATION ONLY

A safe return path never carries capability or authorization meaning.

OIDC SUCCESS

The protocol principal is projected into a BFF-owned session

Authentication success is a transition, not a permanent principal. The projector validates mandatory coordinates, creates a new opaque session identifier and persists a bounded record before issuing the cookie.

REFERENCE ASSET 44-E - SUCCESS EVENT

```
options.Events.OnTokenValidated = async context =>
{
    var projection = context.HttpContext.RequestServices
        .GetRequiredService<IOperatorSessionProjector>();

    var result = await projection.ProjectAsync(
        context.Principal!,
        context.SecurityToken.ValidFrom,
        context.SecurityToken.ValidTo,
        context.HttpContext.RequestAborted);

    context.Principal = result.CookiePrincipal;
    context.Properties.IsPersistent = false;
    context.Properties.AllowRefresh = false;
};
```

PROJECTION LIMIT

The session projector accepts only the current minimal input profile. Full provider-specific claim mapping remains Chapter 46.

PROTOCOL FAILURE

Authentication failure returns a bounded edge result

Remote errors can contain provider details, correlation values or user-controlled text. The BFF records a safe reason class and returns to a fixed local error page without reflecting raw protocol parameters.

DECISION LEDGER 44-19

Failure	BFF result	Audit/telemetry
state/correlation invalid	fixed login failure	protocol-correlation-invalid
nonce invalid	fixed login failure	protocol-nonce-invalid
issuer/audience invalid	fixed login failure	token-context-invalid
lifetime invalid	fixed login failure	identity-expired
provider unavailable	503-safe login page	identity-authority-unavailable
unexpected exception	generic failure	internal reference only

NO REFLECTED DETAIL

Raw error descriptions, tokens and claims never appear in the browser response, ordinary logs or metric labels.

COOKIE CONFIGURATION

The authentication cookie is host-scoped, secure and opaque

The cookie carries only the protected reference created by the cookie handler and server-side ticket store. It is never readable by Angular and is never accepted over plaintext HTTP.

REFERENCE ASSET 44-F - SESSION COOKIE

```
static void ConfigureSessionCookie(CookieAuthenticationOptions options)
{
    options.Cookie.Name = "__Host-NEXUS1-Operator";
    options.Cookie.Path = "/";
    options.Cookie.HttpOnly = true;
    options.Cookie.SecurePolicy = CookieSecurePolicy.Always;
    options.Cookie.SameSite = SameSiteMode.Lax;
    options.ExpireTimeSpan = TimeSpan.FromMinutes(30);
    options.SlidingExpiration = false;
    options.LoginPath = "/bff/login";
    options.AccessDeniedPath = "/bff/forbidden";
    options.Events = ApiCookieEvents.Create();
}
```

SAMESITE CHOICE

Lax is a deliberate default compatible with top-level sign-in navigation. Anti-forgery still protects mutations because SameSite is defense in depth, not the sole CSRF control.

COOKIE CONTRACT

Every attribute closes a different failure path

Cookie settings are reviewed as one contract. A secure flag does not compensate for script access; SameSite does not replace anti-forgery; and an expiry attribute does not prove server-side revocation.

DECISION LEDGER 44-21

Attribute	Purpose	Residual concern
__Host- name	host-only, path root, Secure	subdomain compromise remains relevant
HttpOnly	blocks normal script reads	XSS can still act as the user
Secure	HTTPS-only transmission	TLS endpoint compromise
SameSite=Lax	reduces cross-site sends	top-level navigation allowed
non-persistent	browser-session lifetime	browser restore behavior
opaque reference	small/revocable cookie	session-store availability

LAYERED DEFENSE

Cookie flags reduce exposure; they do not authorize resources or remove the need for XSS, CSRF and revocation controls.

SERVER-SIDE SESSION

The durable session record is explicit and revocable

The record contains only the coordinates needed to validate the current human session and project bounded capabilities. It separates absolute, idle and revocation state and carries a concurrency token for race-safe updates.

REFERENCE ASSET 44-G - SESSION RECORD

```
public sealed record OperatorSessionRecord(
    SessionId Id,
    SubjectReference Subject,
    string Issuer,
    Instant AuthenticatedAt,
    Instant IssuedAt,
    Instant AbsoluteExpiresAt,
    Instant IdleExpiresAt,
    Instant? RevokedAt,
    long IdentityRevision,
    long CapabilityRevision,
    string AuthenticationContext,
    byte[] ConcurrencyToken);

public bool IsCurrent(Instant now) =>
    RevokedAt is null &&
    now < AbsoluteExpiresAt &&
    now < IdleExpiresAt;
```

NO TOKEN ARCHIVE

The Phase-0 operator session does not require access, ID or refresh tokens after projection. If a later integration retains tokens, they remain server-side protected and separately threat-modeled.

TICKET STORE

The cookie handler stores only a protected session reference

An `ITicketStore` adapter binds the framework authentication ticket to the BFF session repository. Missing, expired or revoked records return no ticket and therefore no authenticated principal.

REFERENCE ASSET 44-H - ITICKETSTORE ADAPTER

```
public sealed class BffTicketStore(
    IOperatorSessionRepository sessions,
    TimeProvider time) : ITicketStore
{
    public Task<string> StoreAsync(AuthenticationTicket ticket) =>
        sessions.CreateAsync(ticket, time.GetUtcNow());

    public Task RenewAsync(string key, AuthenticationTicket ticket) =>
        sessions.RenewAsync(key, ticket, time.GetUtcNow());

    public Task<AuthenticationTicket?> RetrieveAsync(string key) =>
        sessions.RetrieveCurrentAsync(key, time.GetUtcNow());

    public Task RemoveAsync(string key) =>
        sessions.RevokeAsync(key, time.GetUtcNow());
}
```

FRAMEWORK SEAM

The repository enforces absolute expiry and revocation. The adapter never extends trust merely because a cookie decrypts.

SESSION LIFECYCLE

Creation, use, rotation and revocation are state transitions

Session behavior is designed as a lifecycle rather than an expiry setting. Every transition has a cause, a persisted result and a testable browser effect.

DECISION LEDGER 44-24

Transition	Required action	Result
sign-in success	create fresh opaque ID	old anonymous state discarded
normal request	validate record/revisions	bounded idle touch
privilege change	rotate ID and refresh context	old ID revoked
subject disabled	revoke all subject sessions	next request 401
logout	revoke record before clear	replay rejected
absolute expiry	no renewal	new authentication required
store uncertainty	fail closed	503/401 by route contract

NO SILENT EXTENSION

Sliding UI convenience cannot move absolute authentication or privileged-action freshness boundaries.

CURRENT-SESSION VALIDATION

The cookie principal is checked against server-owned state

ValidatePrincipal reloads current session state and revision coordinates. A missing or non-current record rejects the principal and deletes the browser cookie. Validation errors never fall back to the embedded claims.

REFERENCE ASSET 44-I - COOKIE VALIDATION

```
options.Events.OnValidatePrincipal = async context =>
{
    var validator = context.HttpContext.RequestServices
        .GetRequiredService<ICurrentOperatorSessionValidator>();

    var result = await validator.ValidateAsync(
        context.Principal!,
        context.HttpContext.RequestAborted);

    if (!result.IsCurrent)
    {
        context.RejectPrincipal();
        await context.HttpContext.SignOutAsync("OperatorSession");
        return;
    }

    context.ReplacePrincipal(result.RefreshedPrincipal);
    context.ShouldRenew = result.RotateCookie;
};
```

FAIL CLOSED

Repository, revision or policy uncertainty cannot be converted into a previously cached allow decision.

SESSION TIMING

Absolute, idle and privileged freshness clocks differ

One timeout cannot express all security decisions. The BFF uses server time and records explicit values so tests can advance each boundary independently.

DECISION LEDGER 44-26

Clock	Example Phase-0 value	Effect at boundary
absolute session	8 hours	new login required
idle session	30 minutes	inactive session rejected
cookie ticket	not beyond session	browser reference invalid
privileged auth age	10 minutes	step-up/re-auth required
capability revision	event/version based	refresh or deny
clock skew	30 seconds	bounded validation tolerance

VALUES ARE POLICY

These are illustrative Phase-0 defaults. Accepted values require environment review, risk ownership and configuration identity.

LOGOUT

Revocation precedes cookie deletion

A logout mutation validates anti-forgery, revokes the server-side record, clears the local authentication cookie and only then initiates any optional identity-provider sign-out flow.

REFERENCE ASSET 44-J - LOCAL LOGOUT

```
app.MapPost("/bff/logout", async (
    HttpContext http,
    IAntiforgery antiforgery,
    IOperatorSessionRevoker revoker,
    CancellationToken ct) =>
{
    await antiforgery.ValidateRequestAsync(http);
    await revoker.RevokeCurrentAsync(http.User, ct);
    await http.SignOutAsync("OperatorSession");
    return Results.NoContent();
})
.RequireAuthorization();
```

LOGOUT BOUNDARY

Local logout can be proved by replay rejection. Global identity-provider logout is provider-specific and is not silently claimed.

REVOCATION RACES

Logout cannot erase a request already committed

The design states the race boundary honestly. Revocation prevents later authorization, but an operation that already passed owner authorization may complete. High-risk actions therefore perform a fresh owner-side check immediately before the state transition.

DECISION LEDGER 44-28

Race	Defined outcome	Evidence
logout before BFF validation	401	zero owner calls
logout after BFF, before owner	owner rejects stale context	zero Domain calls
logout after owner authorization	operation may complete	authoritative receipt
concurrent logout requests	idempotent revoke	one final state
replayed old cookie	401	revoked session reference

NO IMPOSSIBLE PROMISE

The system defines and observes the commit point; it does not claim that logout reverses already authorized history.

SESSION-STORE FAILURE

Availability loss is visible and never anonymous

The BFF session store is now a critical dependency. The runtime distinguishes invalid authentication from an unavailable validation dependency so operators can recover safely without weakening the trust boundary.

DECISION LEDGER 44-29

Condition	Read route	Mutation route
session absent	401	401; zero effects
session expired	401	401; zero effects
session revoked	401	401; zero effects
store timeout	503 safe problem	503; zero effects
corrupt record	401 + security signal	deny + quarantine
revision source unavailable	503/deny	deny

AVAILABILITY HONESTY

A store outage is not reported as bad credentials and is never handled by extending a stale session indefinitely.

API DENIAL

Cookie events return status codes instead of browser redirects

Interactive navigation can use the login endpoint, but /ui-api and /bff mutation routes receive stable API semantics. This prevents fetch calls from following an HTML sign-in redirect and appearing successful.

REFERENCE ASSET 44-K - API COOKIE EVENTS

```
public static CookieAuthenticationEvents Create() => new()
{
    OnRedirectToLogin = context =>
    {
        if (IsApi(context.Request))
        {
            context.Response.StatusCode = StatusCodes.Status401Unauthorized;
            return Task.CompletedTask;
        }
        context.Response.Redirect(context.RedirectUri);
        return Task.CompletedTask;
    },
    OnRedirectToAccessDenied = context =>
    {
        context.Response.StatusCode = StatusCodes.Status403Forbidden;
        return Task.CompletedTask;
    }
};
```

CLIENT CONTRACT

Angular reacts to 401 and 403 explicitly. It never infers authentication from a 200 response containing an HTML login page.

DATA PROTECTION

Cookie protection needs a durable, isolated key contract

ASP.NET Core Data Protection protects cookie tickets and anti-forgery material. Multi-instance and restart behavior require a persistent key ring, one application discriminator and protected key access. Key deletion can invalidate sessions and must be operationally planned.

DECISION LEDGER 44-31

Decision	Requirement	Evidence
key persistence	survive restart/replicas	restart test
application name	isolate NEXUS1 BFF	configuration manifest
key protection	restrict at-rest access	provider/permission record
rotation	overlap valid keys	old/new cookie test
loss/corruption	fail closed/recover	runbook exercise
environment split	no cross-environment cookie	negative test

CHAPTER 47 DEPENDENCY

The key-ring contract is specified here because sessions depend on it; concrete secret and certificate lifecycle evidence remains Chapter 47.

ANTI-FORGERY

Cookie-authenticated mutations require an independent request token

Angular reads only the non-authentication XSRF cookie and echoes it in a custom header. The authentication cookie remains HttpOnly. The BFF validates the request token before any application or owner call.

REFERENCE ASSET 44-L - ANTI-FORGERY CONFIGURATION

```
services.AddAntiforgery(options =>
{
    options.Cookie.Name = "__Host-NEXUS1-XSRF";
    options.Cookie.Path = "/";
    options.Cookie.HttpOnly = false;
    options.Cookie.SecurePolicy = CookieSecurePolicy.Always;
    options.Cookie.SameSite = SameSiteMode.Strict;
    options.HeaderName = "X-XSRF-TOKEN";
});

app.MapGet("/bff/antiforgery", (
    HttpContext http, IAntiforgery xsrf) =>
{
    xsrf.GetAndStoreTokens(http);
    return Results.NoContent();
})
.RequireAuthorization();
```

NOT AN AUTH TOKEN

The readable XSRF token is safe only as a request-correlation secret paired with the protected server cookie. It is never an operator credential.

ANGULAR XSRF

The client uses the same-origin cookie and header contract

Angular HttpClient is configured with the BFF anti-forgery names. Relative same-origin mutation requests receive the header automatically; cross-origin requests are outside the UI contract.

REFERENCE ASSET 44-M - ANGULAR PROVIDER

```
export const appConfig: ApplicationConfig = {
    providers: [
        provideHttpClient(
            withXsrfConfiguration({
                cookieName: '__Host-NEXUS1-XSRF',
                headerName: 'X-XSRF-TOKEN'
            })
        )
    ]
};

// Authentication tokens are never read or written by Angular.
// All protected calls use relative /ui-api or /bff URLs.
```

SAME-ORIGIN BOUNDARY

CORS is not used to widen the operator API. The browser, Angular assets and BFF share one controlled origin in this Phase-0 design.

CSRF COVERAGE

Every state-changing route is inventoried

Anti-forgery is applied by route convention and verified by endpoint inventory. Safe-method labels are not trusted blindly: any GET with side effects is an architecture defect and must be redesigned.

DECISION LEDGER 44-34

Route class	Session	XSRF	Owner policy
GET safe query	required	not required	required
POST command	required	required	required
PUT/PATCH	required	required	required
DELETE	required	required	required
logout	required	required	session revoke
login challenge	anonymous	n/a	n/a
OIDC callback	protocol	correlation	n/a

ZERO-DISCOVERY GATE

The inventory fails if no mutation endpoints are discovered; a vacuous pass is not evidence.

CSRF FAILURE

Invalid anti-forgery material stops before application work

The BFF distinguishes missing, malformed and invalid request tokens only in protected diagnostics. The public response stays bounded and the test proves zero owner-client invocations.

DECISION LEDGER 44-35

Mutation input	HTTP	Effects
missing token	400/403 contract	0 owner calls
header only	400/403	0 owner calls
cookie only	400/403	0 owner calls
mismatched pair	400/403	0 owner calls
token from old session	400/403	0 owner calls
valid pair, expired session	401	0 owner calls

ORDER IS EVIDENCE

Authentication and anti-forgery checks run before body-to-command translation and before any downstream effect.

BROWSER SECURITY

XSS is reduced, not solved, by the BFF pattern

An HttpOnly cookie prevents ordinary script extraction, but injected script can still call same-origin endpoints as the operator. Content Security Policy, output encoding, dependency hygiene and explicit intent remain necessary defense in depth.

DECISION LEDGER 44-36

Threat	BFF benefit	Remaining control
token theft from storage	token absent	storage contract test
script reads auth cookie	HttpOnly	cookie attribute test
script submits mutation	no prevention alone	CSP + XSRF + intent
DOM injection	no prevention alone	Angular safe binding/CSP
malicious dependency	smaller credential surface	supply-chain controls
clickjacking	no prevention alone	frame-ancestors policy

NO SILVER BULLET

The chapter claims elimination of browser token storage, not elimination of browser compromise.

BROWSER HEADERS

The edge emits a reviewed response-header profile

Security headers are applied at the BFF because it owns the browser origin. Values are tested as exact behavior and reviewed against the Angular build rather than copied without compatibility evidence.

DECISION LEDGER 44-37

Header	Phase-0 intent
Content-Security-Policy	allow-list scripts/connect/frame ancestors
Strict-Transport-Security	HTTPS persistence after safe rollout
X-Content-Type-Options	nosniff
Referrer-Policy	minimize cross-origin referrer
Permissions-Policy	disable unused browser capabilities
Cache-Control	no-store for session/security responses

HEADER EVIDENCE

A header exists only when a host test observes the expected value on the intended route and deployment path.

SESSION SUMMARY

Angular receives only safe, display-oriented state

The session endpoint returns a bounded contract. It does not return raw external claims, tokens, issuer metadata, session identifiers or owner-resource lists.

REFERENCE ASSET 44-N - SAFE UI CONTRACT

```
public sealed record OperatorSessionSummaryV1(
    bool Authenticated,
    string DisplayName,
    DateTimeOffset SessionExpiresAtUtc,
    DateTimeOffset AuthenticatedAtUtc,
    IReadOnlyList<string> Capabilities,
    string PolicyIdentity);

app.MapGet("/ui-api/v1/session", (
    CurrentOperator operatorContext) =>
    Results.Ok(operatorContext.ToSafeSummary()))
    .RequireAuthorization();
```

DELIBERATE OMISSION

The response contains no `access_token`, `id_token`, `refresh_token`, raw claims, cookie value, subject identifier or session-store key.

TYPED CAPABILITY

Capabilities are closed vocabulary, not free-form strings

A typed capability prevents route code from inventing ad hoc role names. Parsing is strict and unknown values fail closed. The external-to-local mapping that creates these values is completed in Chapter 46.

REFERENCE ASSET 44-O - CAPABILITY VOCABULARY

```
public readonly record struct OperatorCapability
{
    private OperatorCapability(string value) => Value = value;
    public string Value { get; }

    public static readonly OperatorCapability AlarmRead =
        new("Alarm.Read");
    public static readonly OperatorCapability RootCauseCaseRead =
        new("RootCause.Case.Read");
    public static readonly OperatorCapability RepairRequest =
        new("Repair.Request");
    public static readonly OperatorCapability RepairApprove =
        new("Repair.Approve");
    public static readonly OperatorCapability RepairExecute =
        new("Repair.Execute");
}
```

CLOSED SET

Capability evolution is a reviewed contract change with route, owner-policy, UI and evidence updates.

CAPABILITY CATALOGUE

Capabilities describe actions, not organizational status

The Phase-0 vocabulary is intentionally narrow. Read capabilities do not imply mutation capabilities, and repair stages remain separate to preserve Chapter 42 separation of duties.

DECISION LEDGER 44-40

Capability	Meaning	Owner
Alarm.Read	read visible alarm state	owning service
RootCause.Case.Read	read visible case state	owning service
Compliance.Review.Read	read visible review state	owning service
Workflow.Read	read visible workflow projection	owning service
Repair.Read	read repair plan/evidence summary	owning service
Repair.Request	request bounded repair plan	owning service
Repair.Approve	approve exact plan digest	owning service
Repair.Execute	execute approved plan transition	owning service

NO CAPABILITY INHERITANCE

Repair.Execute does not imply Repair.Approve; Alarm.Read does not imply RootCause.Case.Read.

OPERATOR CONTEXT

The human context is bounded, versioned and expiring

The BFF projects an internal context for downstream authorization. It names the human subject by a safe reference, accepted audiences, site and tenant scope, capability set, policy identity, time bounds and revisions.

REFERENCE ASSET 44-P - BOUNDED HUMAN CONTEXT

```
public sealed record OperatorContextV1(
    SubjectReference Subject,
    string Issuer,
    IReadOnlySet<string> Audiences,
    IReadOnlySet<SiteId> Sites,
    IReadOnlySet<TenantId> Tenants,
    IReadOnlySet<OperatorCapability> Capabilities,
    Instant AuthenticatedAt,
    Instant ExpiresAt,
    long IdentityRevision,
    long CapabilityRevision,
    string PolicyIdentity);

public bool IsFreshFor(Instant now, Duration maximumAge) =>
    now < ExpiresAt && now - AuthenticatedAt <= maximumAge;
```

NOT YET A TRUSTED TRANSPORT

This type defines human meaning. Chapter 45 must establish the BFF workload and channel that make a received context trustworthy.

CAPABILITY RESOLUTION SEAM

External claims stop before local policy

Chapter 44 defines the seam required to produce a local capability snapshot but keeps provider-specific mapping out of route code. Chapter 46 will implement minimization, mapping and drift evidence behind this interface.

REFERENCE ASSET 44-Q - MAPPING SEAM

```
public interface IOperatorCapabilityResolver
{
    Task<CapabilitySnapshot> ResolveAsync(
        AuthenticatedExternalSubject subject,
        CancellationToken cancellationToken);
}

public sealed record CapabilitySnapshot(
    IReadOnlySet<OperatorCapability> Capabilities,
    IReadOnlySet<SiteId> Sites,
    IReadOnlySet<TenantId> Tenants,
    long Revision,
    string PolicyIdentity,
    Instant ValidUntil);
```

FAIL-CLOSED SEAM

Unknown claims, mapping failure, stale revision or unavailable policy produce no capability snapshot and therefore no protected request.

SCOPE OWNERSHIP

Site and tenant coordinates are server-owned

Angular may request a selected site for convenience, but the BFF and owner intersect it with the operator's server-owned scope. Client input can narrow scope; it can never widen it.

DECISION LEDGER 44-43

Input	Use	Forbidden interpretation
route siteId	requested target	proof of access
query tenantId	requested filter	tenant membership
UI selected site	display preference	authorization source
session site set	early BFF filter	owner resource truth
owner resource site	final local coordinate	client assertion

INTERSECTION RULE

requested scope must be a subset of session scope and must still match the loaded owner resource.

BFF EARLY POLICY

The edge can refuse obvious capability failures

An early BFF policy avoids unnecessary owner calls and provides consistent UX. It does not replace the owner decision because it does not own current resource state or business invariants.

REFERENCE ASSET 44-R - EDGE REQUIREMENT

```
public sealed record OperatorCapabilityRequirement(
    OperatorCapability Capability) : IAuthorizationRequirement;

public sealed class OperatorCapabilityHandler
    : AuthorizationHandler<OperatorCapabilityRequirement>
{
    protected override Task HandleRequirementAsync(
        AuthorizationHandlerContext context,
        OperatorCapabilityRequirement requirement)
    {
        if (CurrentOperator.From(context.User)
            .Has(requirement.Capability))
            context.Succeed(requirement);

        return Task.CompletedTask;
    }
}
```

OPTIMIZATION, NOT AUTHORITY

An edge allow means only that the request may proceed to the owner boundary.

ROUTE INVENTORY

Every operator route names authentication, capability and mutation protection

The route catalogue is a review and test input. Discovery compares the running endpoint graph with this catalogue and fails on missing, duplicate or unexpectedly anonymous entries.

DECISION LEDGER 44-45

Method	Route	Edge requirement	Owner result
GET	/bff/login	anonymous	OIDC challenge
GET	/signin-oidc	protocol callback	OIDC middleware
POST	/bff/logout	session + XSRF	revoke then clear
GET	/ui-api/v1/session	session	safe summary
GET	/ui-api/v1/alarms	Alarm.Read	owner recheck
GET	/ui-api/v1/root-cause/{id}	RootCause.Case.Read	owner recheck
POST	/ui-api/v1/repair/requests	Repair.Request + XSRF	owner recheck
POST	/ui-api/v1/repair/{id}/approve	Repair.Approve + XSRF	owner recheck
POST	/ui-api/v1/repair/{id}/execute	Repair.Execute + XSRF	owner recheck

INVENTORY IS EXECUTABLE

A new route without a catalogue entry, policy and negative test blocks the security gate.

ENDPOINT COMPOSITION

Policies are visible at the route boundary

The route maps a safe request DTO, requires the session and early capability, validates anti-forgery, and then invokes a typed owner client. Hidden middleware is not the only place where authority is declared.

REFERENCE ASSET 44-S - REPAIR REQUEST ROUTE

```
app.MapPost("/ui-api/v1/repair/requests", async (
    CreateRepairRequestV1 request,
    HttpContext http,
    IAntiforgery xsrf,
    IRepairOwnerClient owner,
    CancellationToken ct) =>
{
    await xsrf.ValidateRequestAsync(http);
    var result = await owner.RequestAsync(
        request.ToOwnerCommand(), ct);
    return RepairHttp.Map(result);
})
.RequireAuthorization("Operator:Repair.Request")
.WithName("repair-request-create");
```

SECOND GATE STILL REQUIRED

IRepairOwnerClient must establish BFF workload trust and carry the bounded operator context; direct user tokens are prohibited.

CONTEXT PROPAGATION

The owner receives two independent principals

The downstream call must represent both who is calling as a workload and on whose behalf it acts as a human operation. These coordinates are evaluated independently and recorded without forwarding the browser cookie or external user token.

DECISION LEDGER 44-47

Coordinate	Source	Destination use
workload principal	Chapter 45 mechanism	authenticate BFF caller
human subject	validated BFF session	accountability/policy
capability	local projection	requested action class
site/tenant	server-owned session scope	intersect with resource
freshness/revision	BFF policy state	reject stale context
policy identity	mapping snapshot	decision evidence
correlation reference	request context	diagnosis only

TWO GATES

valid human context plus missing workload trust is denied; valid workload plus missing human context is also denied.

TRUSTED EDGE CALL

Only boundary middleware may construct accepted caller context

Application handlers receive a `TrustedEdgeCall` only after workload authentication and human-context admission. The type keeps the two principals separate so tests can prove that neither substitutes for the other.

REFERENCE ASSET 44-T - DUAL-PRINCIPAL CONTEXT

```
public sealed record TrustedEdgeCall(
    WorkloadPrincipal Caller,
    OperatorContextV1 OnBehalfOf,
    string BoundaryId,
    Instant ObservedAt);

public interface ITrustedEdgeCallAccessor
{
    TrustedEdgeCall Current { get; }
}

// Production construction belongs to authenticated boundary middleware.
// Tests may inject a named test-only workload authenticator.
```

CONSTRUCTION BOUNDARY

Route bodies and user-supplied headers cannot construct a `TrustedEdgeCall`.

CHAPTER 45 GATE

Production owner calls fail closed until workload trust exists

Chapter 44 supplies a deliberate pending implementation. It allows independent human-policy tests with a test-only scheme, while the production registration rejects every edge call until Chapter 45 replaces the gate.

REFERENCE ASSET 44-U - PENDING WORKLOAD GATE

```
public sealed class PendingBffWorkloadAuthenticator
    : IBffWorkloadAuthenticator
{
    public ValueTask<WorkloadAuthenticationResult> AuthenticateAsync(
        HttpContext http,
        CancellationToken cancellationToken) =>
        ValueTask.FromResult(
            WorkloadAuthenticationResult.Denied(
                "workload-trust-pending-chapter-45"));
}

services.AddSingleton<IBffWorkloadAuthenticator,
    PendingBffWorkloadAuthenticator>();
```

HONEST INCOMPLETENESS

End-to-end production authorization remains PENDING. Tests that bypass this gate must be unmistakably test-only and absent from the production descriptor graph.

OWNER ADMISSION

Authentication is completed before human policy evaluation

The owner boundary rejects malformed or untrusted calls before decoding human authority. This order prevents an attacker from using arbitrary operator headers to probe resource policy.

DECISION LEDGER 44-50

Stage	Failure	Next stage
workload authenticate	401/deny	none
operator context decode	400/deny	none
audience/freshness	403/deny	none
load safe resource shell	404/deny	resource policy
resource authorize	403/404	application handler
precondition	412/409	Domain transition
commit	owner outcome	receipt/audit

SHORT CIRCUIT

Every denied stage proves that all later stages, Domain calls, writes and public facts are absent.

OWNER RESOURCE

Authorization uses current owner state

The owner loads only the minimum resource coordinates needed for policy, then invokes resource-based authorization. Attribute authorization alone cannot decide scope because attributes run before the target resource is loaded.

REFERENCE ASSET 44-V - RESOURCE COORDINATE

```
public sealed record AlarmResource(  
    AlarmId Id,  
    SiteId SiteId,  
    TenantId TenantId,  
    AlarmState State,  
    long Version);  
  
public sealed record OwnerCapabilityRequirement(  
    OperatorCapability Capability,  
    string ExpectedAudience,  
    Duration MaximumAuthenticationAge)  
    : IAuthorizationRequirement;
```

MINIMUM LOAD

Loading a safe authorization shell must not disclose the full resource before scope is established.

OWNER POLICY HANDLER

Capability, audience, freshness and scope succeed together

The handler checks one trusted edge call against the current resource. It does not call remote systems or infer permission from a UI route. Failure records a bounded reason and leaves the requirement unsatisfied.

REFERENCE ASSET 44-W - RESOURCE AUTHORIZATION

```
protected override Task HandleRequirementAsync(
    AuthorizationHandlerContext context,
    OwnerCapabilityRequirement requirement,
    AlarmResource resource)
{
    var call = trusted.Current;
    var human = call.OnBehalfOf;
    var now = time.UtcNow();

    if (call.Caller.Service != "nexus1-bff") return Done();
    if (!human.Audiences.Contains(requirement.ExpectedAudience)) return Done();
    if (!human.IsFreshFor(now, requirement.MaximumAuthenticationAge)) return Done();
    if (!human.Capabilities.Contains(requirement.Capability)) return Done();
    if (!human.Sites.Contains(resource.SiteId)) return Done();
    if (!human.Tenants.Contains(resource.TenantId)) return Done();

    context.Succeed(requirement);
    return Done();
}
```

ALL COORDINATES REQUIRED

A valid subject with one missing coordinate is denied. There is no best-effort or majority authorization.

IMPERATIVE AUTHORIZATION

The owner checks after loading and before invoking Domain behavior

The endpoint loads a safe resource shell, calls `IAuthorizationService` with the exact resource and maps failure according to the disclosure policy. Only success reaches the application handler.

REFERENCE ASSET 44-X - OWNER ENDPOINT

```
var resource = await resources.FindAuthorizationShellAsync(id, ct);
if (resource is null)
    return Results.NotFound();

var authorization = await auth.AuthorizeAsync(
    http.User,
    resource,
    new OwnerCapabilityRequirement(
        OperatorCapability.AlarmRead,
        "alarm-management",
        Duration.FromHours(8)));

if (!authorization.Succeeded)
    return OwnerDenial.For(resource, trusted.Current);

return AlarmHttp.Map(await handler.HandleAsync(id, ct));
```

NO PRE-AUTHORIZED DTO

A client-supplied `siteld` is never substituted for the owner resource used by the policy.

RESOURCE DISCLOSURE

403 and 404 are chosen by an explicit policy

Some routes may reveal that a resource exists to authenticated operators; others hide cross-site or cross-tenant existence. The choice is consistent per resource class and tested for both status and response shape.

DECISION LEDGER 44-54

Case	Public result	Internal reason
resource absent	404	resource-not-found
resource outside site/tenant	404	resource-scope-denied
resource visible, capability absent	403	capability-denied
context expired/revoked	401/403 contract	human-context-stale
workload not trusted	401	workload-denied
policy dependency unavailable	503	policy-unavailable

CONSISTENT SHAPE

A hidden-resource 404 must not leak existence through body, headers, cache behavior or an obviously different code path.

OWNER POLICY MATRIX

Human-context cases are tested independently

The matrix fixes the expected decision for the exact cases required by the contents and extends them with revocation, audience, scope, revision and workload separation.

DECISION LEDGER 44-55

Identity case	Workload	Owner result
valid, capable, in scope	valid test BFF	allow
missing	valid test BFF	deny
expired	valid test BFF	deny
revoked	valid test BFF	deny
insufficient capability	valid test BFF	403
wrong audience	valid test BFF	deny
wrong site/tenant	valid test BFF	404
stale revision	valid test BFF	deny
valid human	missing workload	deny
missing human	valid workload	deny

REQUIRED FOUR PLUS MORE

Valid, expired, missing and insufficient identities are mandatory acceptance cases, not examples.

POLICY UNIT TEST

A valid identity succeeds only for the exact resource

Deterministic time, resources and contexts make the policy result reproducible. The test checks the requested capability and exact current resource coordinates.

REFERENCE ASSET 44-Y - VALID RESOURCE TEST

```
[Fact]
public async Task OWNER_AUTH_001_valid_context_allows_exact_resource()
{
    var resource = Resources.Alarm(site: "ATH-01", tenant: "NEXUS-1");
    var call = Calls.ValidBffOperator(
        capability: "Alarm.Read",
        site: "ATH-01",
        tenant: "NEXUS-1",
        expiresAt: Time.Now.Plus(Duration.FromMinutes(5)));

    var result = await Harness.AuthorizeAsync(call, resource);

    result.Succeeded.Should().BeTrue();
}
```

EXACT TARGET

Changing capability, audience, site, tenant, expiry or caller changes the result to denied.

POLICY NEGATIVE TEST

Denial proves absence of application and persistence effects

A status assertion is not enough. The harness records application calls, database commits, outbox additions and audit outcomes so a denial cannot hide partial work.

REFERENCE ASSET 44-Z - ZERO-EFFECT DENIAL

```
[Theory]
[MemberData(nameof(InvalidOperatorContexts))]
public async Task OWNER_AUTH_002_denial_has_zero_effects(
    OperatorContextV1? context)
{
    var result = await Harness.ExecuteAsync(
        Calls.WithHumanContext(context),
        Commands.RequestRepair);

    result.IsAllowed.Should().BeFalse();
    Harness.ApplicationCalls.Should().Be(0);
    Harness.Commits.Should().Be(0);
    Harness.OutboxAdds.Should().Be(0);
    Harness.OwnerReceipts.Should().BeEmpty();
}
```

NO PARTIAL AUTHORITY

A rejected security decision must happen before any Domain transition or durable business effect.

WORKLOAD SEPARATION TEST

A user token alone is rejected as service trust

Chapter 44 locks the negative invariant before Chapter 45 chooses its mechanism. A request containing valid-looking human material but no accepted BFF workload cannot create a TrustedEdgeCall.

REFERENCE ASSET 44-AA - INDEPENDENT GATES

```
[Fact]
public async Task OWNER_AUTH_003_user_identity_is_not_workload_trust()
{
    var request = OwnerRequests.WithValidHumanContext()
        .WithoutWorkloadCredential();

    var response = await Owner.SendAsync(request);

    response.StatusCode.Should().Be(HttpStatusCode.Unauthorized);
    Owner.TrustedEdgeCalls.Should().Be(0);
    Owner.ApplicationCalls.Should().Be(0);
}
```

CHAPTER 45 ACCEPTANCE GATE

The test remains and must still pass after the real workload mechanism is implemented.

CAPABILITY DRIFT

A long-lived session cannot freeze authority indefinitely

Identity and capability revisions make disablement and policy changes observable. The BFF can refresh a safe snapshot, but privileged operations reject stale revisions instead of trusting a session created before the change.

DECISION LEDGER 44-59

Change	BFF behavior	Owner behavior
subject disabled	revoke session	reject context
capability removed	refresh/rotate	reject old revision
site removed	refresh/rotate	404 old scope
policy version changed	new snapshot	reject incompatible identity
resolver unavailable	503/deny	deny
revision event delayed	bounded stale window	privileged fresh check

FRESHNESS BUDGET

Any bounded cache window is a documented residual risk with an owner and privileged-operation override.

READ VERSUS MUTATION

Authorization strength follows action consequence

Read paths still require current session, workload and resource scope. Mutations add anti-forgery, exact preconditions and stronger freshness. Repair actions add plan-digest binding and separation of duties.

DECISION LEDGER 44-60

Action	Additional requirement
read alarm/case	session + capability + owner scope
request repair	XSRF + recent operator context
approve repair	distinct subject + exact plan digest
execute repair	fresh context + approvals + ETag
read protected evidence	purpose + bounded projection
download artifact	size/classification + audit

NO READ SHORTCUT

Read-only does not mean public. Disclosure and cross-tenant enumeration are security effects.

PRIVILEGED FRESHNESS

Repair actions require recent authentication context

The owner requirement can demand a shorter authentication age than ordinary session validity. The BFF initiates re-authentication or step-up, but the owner verifies the resulting timestamp and context again.

REFERENCE ASSET 44-AB - PRIVILEGED REQUIREMENT

```
var requirement = new OwnerCapabilityRequirement(  
    OperatorCapability.RepairExecute,  
    ExpectedAudience: "repair-control",  
    MaximumAuthenticationAge: Duration.FromMinutes(10));  
  
var result = await authorization.AuthorizeAsync(  
    http.User,  
    repairPlanShell,  
    requirement);  
  
if (!result.Succeeded)  
    return Results.Problem(  
        statusCode: StatusCodes.Status403Forbidden,  
        type: "urn:nexus1:problem:recent-authentication-required");
```

FRESHNESS IS NOT UI STATE

A hidden Execute button cannot prove recent authentication; the owner evaluates the accepted context at execution time.

REPAIR SEPARATION

Request, approve and execute remain distinct capabilities

Chapter 44 applies the Chapter 42 repair policy to human authorization. The same subject cannot satisfy a required independent approval, and changing the plan digest invalidates prior authorization.

DECISION LEDGER 44-62

Stage	Human requirement	Bound object
view	Repair.Read	safe plan summary
request	Repair.Request	candidate and reason
approve	Repair.Approve + distinct subject	canonical plan digest
execute	Repair.Execute + recent auth	approved digest + ETag
verify	Repair.Read	owner receipt/new snapshot

NO ADMIN UMBRELLA

There is no single Admin role that collapses request, approval and execution.

PLAN AUTHORIZATION RESOURCE

Approval binds exact canonical plan identity

The resource passed to authorization contains the plan digest, requester, risk class, expiry and owner coordinate. Policy can enforce distinct subjects and freshness without loading mutable command fields from the browser.

REFERENCE ASSET 44-AC - REPAIR AUTHORIZATION SHELL

```
public sealed record RepairPlanAuthorizationResource(  
    RepairPlanId Id,  
    string CanonicalSha256,  
    SubjectReference RequestedBy,  
    RepairRiskClass Risk,  
    OwnerService Owner,  
    Instant ExpiresAt,  
    long Version);  
  
public static bool IsDistinctApprover(  
    TrustedEdgeCall call,  
    RepairPlanAuthorizationResource plan) =>  
    call.OnBehalfOf.Subject != plan.RequestedBy;
```

SERVER-OWNED PLAN

The browser names a plan ID and precondition. It never submits a replacement plan body for approval or execution.

PRIVILEGED DENIAL

Repair failures preserve history and authority

A failed privileged authorization cannot modify the plan, approvals, owner resource or audit chain. Security denial is recorded as a bounded attempt, not as an approved business transition.

DECISION LEDGER 44-64

Failure	Result	Durable effect
same requester/approver	403	denial observation only
authentication too old	403 re-auth required	none
plan digest changed	409/412	none
approval expired	403	none
wrong owner audience	403	none
workload gate absent	401	none
owner policy unavailable	503	none

AUDIT DISTINCTION

A denial observation supports investigation; it does not become a repair approval or owner receipt.

SAFE DECISION EVIDENCE

Authorization observations explain decisions without leaking identity material

The observation records boundary, decision, reason, policy, safe subject reference, resource class and scope digest. Raw tokens, cookies, claims, names and resource bodies are excluded.

REFERENCE ASSET 44-AD - DECISION OBSERVATION

```
public sealed record OperatorAuthorizationObserved(  
    string BoundaryId,  
    string Decision,  
    string ReasonCode,  
    string PolicyIdentity,  
    string SubjectReference,  
    string Capability,  
    string ResourceClass,  
    string ScopeDigest,  
    DateTimeOffset ObservedAtUtc,  
    string CorrelationReference);
```

DIAGNOSTIC, NOT AUTHORITY

The observation explains an attempt. Owner state, repair audit and receipts remain the authoritative outcome records.

REASON CODES

Public problems are bounded; protected evidence is more precise

Stable reason classes support tests and operations without revealing sensitive distinctions to an attacker. Public responses use a smaller vocabulary than protected decision evidence.

DECISION LEDGER 44-66

Protected reason	Public problem
session-missing	authentication-required
session-expired	authentication-required
session-revoked	authentication-required
capability-denied	forbidden
resource-scope-denied	not-found
operator-context-stale	recent-authentication-required
workload-trust-pending	service-unavailable/unauthorized
policy-unavailable	service-unavailable

NO ENUMERATION BY ERROR

Response status, body, headers and cache policy are consistent within each disclosure class.

SECURITY TELEMETRY

Metrics remain low-cardinality and non-authoritative

Metrics expose boundary, decision and bounded reason class. Per-subject, session, plan and resource identifiers remain in controlled logs or traces only when justified.

DECISION LEDGER 44-67

Signal	Allowed labels
edge_authentication_total	outcome, reasonClass
operator_session_current	state
operator_authorization_total	boundary, decision, capabilityClass
antiforgery_rejected_total	routeClass, reasonClass
owner_authorization_total	owner, decision, reasonClass
workload_gate_denied_total	owner, reasonClass
session_store_latency	operation, result

NO IDENTITY LABELS

Subject, session ID, cookie, token, site ID, tenant ID and resource ID are prohibited metric labels.

ANGULAR SESSION STATE

The UI recovers authentication from the BFF after every reload

The client does not persist an authentication boolean. It queries the safe session endpoint, represents unknown/loading/authenticated/anonymous states explicitly and treats 401 as a transition to anonymous.

REFERENCE ASSET 44-AE - SESSION SERVICE

```
type SessionState =
  | { kind: 'loading' }
  | { kind: 'anonymous' }
  | { kind: 'authenticated'; value: OperatorSessionSummaryV1 }
  | { kind: 'unavailable' };

@Injectable({ providedIn: 'root' })
export class OperatorSessionService {
  readonly state = signal<SessionState>({ kind: 'loading' });

  async refresh(): Promise<void> {
    const value = await firstValueFrom(
      this.http.get<OperatorSessionSummaryV1>('/ui-api/v1/session'));
    this.state.set({ kind: 'authenticated', value });
  }
}
```

HARD-REFRESH PROOF

Reloading the browser restores state from the server session; no token or durable auth flag is read from Web Storage.

ANGULAR ROUTE GUARD

A guard improves navigation but does not grant authority

The guard waits for known session state and checks the safe capability summary only to choose navigation. Direct requests still reach server authorization and are denied when the guard is bypassed.

REFERENCE ASSET 44-AF - UX-ONLY GUARD

```
export const capabilityGuard = (capability: Capability): CanMatchFn =>
  async () => {
    const sessions = inject(OperatorSessionService);
    const router = inject(Router);
    await sessions.ensureKnown();

    const state = sessions.state();
    if (state.kind === 'anonymous')
      return router.parseUrl('/sign-in');
    if (state.kind !== 'authenticated')
      return router.parseUrl('/unavailable');

    return state.value.capabilities.includes(capability)
      ? true
      : router.parseUrl('/forbidden');
  };
};
```

UX BOUNDARY

Route guards prevent confusing navigation. They are not counted as a security authorization control.

BUTTON VISIBILITY

Visible actions explain capability; hidden actions do not enforce it

The Angular application uses the safe summary to render relevant controls. Every command remains callable only through a protected BFF route and owner resource decision.

DECISION LEDGER 44-70

UI behavior	Value	Security limit
hide Execute	reduces error/confusion	request can be forged
disable during refresh	prevents duplicate click	not authorization
show scope label	explains current view	not scope proof
show re-auth prompt	guides recovery	owner rechecks age
display 403	explains capability gap	does not reveal hidden resource
display 401	starts sign-in flow	does not extend session

DIRECT-REQUEST TEST

Every hidden or guarded action is tested with a manually constructed request that bypasses Angular.

CLIENT STORAGE CONTRACT

No authentication token or session secret enters Web Storage

The browser test inspects localStorage, sessionStorage, IndexedDB and relevant response bodies. Non-security UI preferences may be stored, but token-like keys and values are prohibited.

DECISION LEDGER 44-71

Surface	Allowed	Forbidden
localStorage	theme, density	access/id/refresh token
sessionStorage	ephemeral UI state	auth/session secret
IndexedDB	approved offline non-sensitive data	credential material
session response	safe summary	raw claims/tokens
URL/history	safe route	token/session/identity payload
service worker cache	versioned static assets	security responses

PRECISE TEST

The test forbids authentication material; it does not make the false claim that all browser storage must always be empty.

CACHING

Security-sensitive responses are not shared or replayed

Session summaries, authorization problems and protected resource responses use explicit cache policy. Service workers must not cache /bff or /ui-api responses unless a separately reviewed offline contract exists.

DECISION LEDGER 44-72

Response	Cache policy
session summary	no-store
401/403 problem	no-store
repair plan summary	private, no-store
protected owner query	private; ETag only if reviewed
static Angular assets	content-hashed immutable
OIDC callback/error	no-store

CACHE IS ANOTHER BOUNDARY

A correct authorization decision can still leak data if a shared cache serves its response to another session.

ENDPOINT ARCHITECTURE TESTS

The running route graph must match the protected catalogue

Host tests inspect endpoint metadata and route patterns. They fail on anonymous mutations, missing capability policy, missing anti-forgery convention, duplicate names or a zero-endpoint discovery run.

DECISION LEDGER 44-73

Gate	Failure condition
route discovery	zero discovered endpoints
anonymous allow-list	unknown AllowAnonymous endpoint
mutation protection	POST/PUT/PATCH/DELETE lacks XSRF marker
capability policy	operator route lacks named policy
owner boundary	route bypasses typed owner client
route names	duplicate or unnamed protected endpoint
callback isolation	protocol route exposed as UI API

RUNTIME GRAPH

Source-code attributes are not enough; the test inspects the composed host used by production startup.

BFF HOST TEST

Anonymous API requests receive 401, not an OIDC redirect

The test disables automatic redirect following and asserts both status and absence of a Location header. The login endpoint is tested separately for an explicit challenge.

REFERENCE ASSET 44-AG - API REDIRECT TEST

```
[Fact]
public async Task BFF_AUTH_001_anonymous_api_is_401_not_redirect()
{
    using var client = factory.CreateClient(
        new WebApplicationFactoryClientOptions
        {
            AllowAutoRedirect = false
        });

    var response = await client.GetAsync("/ui-api/v1/session");

    response.StatusCode.Should().Be(HttpStatusCode.Unauthorized);
    response.Headers.Location.Should().BeNull();
    response.Content.Headers.ContentType?.MediaType
        .Should().NotBe("text/html");
}
```

SEPARATE CHALLENGE TEST

GET /bff/login is expected to challenge the OIDC scheme and emit a validated provider redirect.

COOKIE HOST TEST

The emitted cookie attributes are observed over HTTPS

The integration test completes a test-only authentication flow and inspects Set-Cookie. It asserts the exact name and security attributes rather than assuming AddCookie defaults.

REFERENCE ASSET 44-AH - COOKIE ATTRIBUTES

```
[Fact]
public async Task BFF_AUTH_002_cookie_is_host_secure_and_http_only()
{
    var response = await Login.AsValidOperatorAsync();
    var cookie = response.Headers.GetValues("Set-Cookie")
        .Single(x => x.StartsWith("__Host-NEXUS1-Operator="));

    cookie.Should().Contain("Path=/");
    cookie.Should().Contain("Secure");
    cookie.Should().Contain("HttpOnly");
    cookie.Should().Contain("SameSite=Lax");
    cookie.Should().NotContain("Domain=");
}
```

OBSERVED CONFIGURATION

The test runs through the composed host and HTTPS client so middleware and proxy assumptions are visible.

SESSION-RESPONSE TEST

The safe contract cannot grow token fields silently

A JSON contract test compares the exact member set and scans values for token-shaped leakage. Additions require explicit review rather than a permissive deserializer.

REFERENCE ASSET 44-AI - SAFE SUMMARY CORPUS

```
[Fact]
public async Task BFF_AUTH_003_session_summary_contains_no_tokens()
{
    var json = await Login.AsValidOperator()
        .GetStringAsync("/ui-api/v1/session");
    using var doc = JsonDocument.Parse(json);

    doc.RootElement.EnumerateObject().Select(x => x.Name)
        .Should().BeEquivalentTo(
            "authenticated", "displayName",
            "sessionExpiresAtUtc", "authenticatedAtUtc",
            "capabilities", "policyIdentity");

    json.Should().NotContain("access_token");
    json.Should().NotContain("id_token");
    json.Should().NotContain("refresh_token");
}
```

CLOSED RESPONSE

Raw subject, session ID and external claims are excluded even when they are not literally named token.

ANTI-FORGERY HOST TEST

A valid session without XSRF cannot mutate

The test uses an authenticated browser client but omits the request token. It asserts denial and verifies that the fake owner client observed zero calls.

REFERENCE ASSET 44-AJ - XSRF REFUSAL

```
[Fact]
public async Task BFF_XSRF_001_missing_token_stops_before_owner()
{
    using var browser = await Login.AsValidOperatorAsync(
        capabilities: ["Repair.Request"]);

    var response = await browser.PostAsJsonAsync(
        "/ui-api/v1/repair/requests",
        Fixtures.ValidRepairRequest);

    response.StatusCode.Should().BeOneOf(
        HttpStatusCode.BadRequest,
        HttpStatusCode.Forbidden);
    OwnerClient.Calls.Should().Be(0);
}
```

VALID PAIR COUNTERPART

A second test obtains the XSRF cookie, echoes the request token and proves exactly one owner-client call.

LOGOUT HOST TEST

A captured old cookie fails after logout

The test retains the pre-logout cookie, performs a valid XSRF-protected logout and then replays the captured cookie from a fresh client.

REFERENCE ASSET 44-AK - REPLAY AFTER LOGOUT

```
[Fact]
public async Task BFF_SESSION_001_logout_revokes_old_cookie()
{
    var login = await Login.CaptureAsync();
    await login.Browser.PostAsync(
        "/bff/logout",
        Xsrf.WithValidPair(login.Xsrf));

    using var replay = Clients.WithCookie(login.AuthenticationCookie);
    var response = await replay.GetAsync("/ui-api/v1/session");

    response.StatusCode.Should().Be(HttpStatusCode.Unauthorized);
    Sessions.IsRevoked(login.SessionId).Should().BeTrue();
}
```

SERVER-SIDE PROOF

Clearing the browser cookie alone would not make this test pass; the session record must be revoked.

RETURN-PATH TEST

Open-redirect mutations are rejected before OIDC challenge

The challenge test covers absolute, scheme-relative, encoded and protocol-route inputs. A rejected return path resolves to a fixed local fallback.

REFERENCE ASSET 44-AL - RETURN URL CORPUS

```
[Theory]
[InlineData("https://evil.example/")]
[InlineData("//evil.example/")]
[InlineData("/%2f%2fevil.example/")]
[InlineData("/signin-oidc")]
[InlineData("/ui-api/v1/session")]
public async Task BFF_OIDC_001_unsafe_return_path_is_not_used(
    string returnUrl)
{
    var response = await Client.GetAsync(
        "/bff/login?returnUrl=" + Uri.EscapeDataString(returnUrl));

    response.Headers.Location!.Query
        .Should().NotContain(Uri.EscapeDataString(returnUrl));
}
```

PROVIDER REDIRECT REMAINS EXACT

The test also validates the registered callback URI and does not treat an arbitrary local path as an OIDC redirect URI.

IDENTITY TEST CORPUS

Required user-identity cases have exact expected effects

The acceptance corpus is shared by policy unit tests, BFF host tests and owner host tests. It prevents one layer from using a different definition of expired, missing or insufficient.

DECISION LEDGER 44-80

Case ID	Input	Expected
ID-VALID	current, capable, in scope	200/allow
ID-MISSING	no session/context	401/deny
ID-EXPIRED	expiresAt <= now	401/deny
ID-REVOKED	server revocation	401/deny
ID-INSUFFICIENT	capability missing	403/deny
ID-AUDIENCE	wrong owner audience	403/deny
ID-SCOPE	wrong site/tenant	404/deny
ID-REVISION	stale identity/policy	403/503 deny

ZERO EFFECTS IN EVERY DENIAL

All denied cases assert zero Domain calls, commits, outbox facts and owner receipts.

FAULT LABORATORY

Identity infrastructure failures exercise action, not just status

Fault tests inject timeouts, partial failures and races at the BFF and owner boundaries. Each test observes bounded response, zero unauthorized effects and a recoverable operational signal.

DECISION LEDGER 44-81

Fault	Expected action
IdP metadata unavailable	login unavailable; existing local policy unchanged
session store timeout	fail closed; no owner call
capability resolver timeout	deny/503; no cached allow extension
Data Protection key mismatch	session rejected; readiness signal
clock advanced	expiry boundary rejects
clock rolled back	absolute issued/revision checks apply
logout/request race	defined commit boundary
owner policy unavailable	503; no Domain call

FAULT EVIDENCE

The controller's action and downstream call count matter more than a lone HTTP status assertion.

BROWSER E2E

The real operator journey proves UX and security boundaries together

Browser tests use a controllable test identity authority and the production-path BFF composition. They inspect navigation, storage, cookies, XSRF and direct-request refusal.

DECISION LEDGER 44-82

Journey	Proof
sign in and hard refresh	session restored from BFF
session expires	401 leads to sign-in; no stale UI allow
capability removed	UI refreshes; owner already denies
button hidden	forged request still 403
wrong site URL	404 and no existence disclosure
logout then back	old page cannot mutate
storage inspection	no authentication material
CSP report/test	expected Angular bundle executes

E2E LIMIT

A browser pass proves the exercised deployment path, not every provider, browser version or adversarial technique.

TEST-ONLY AUTHENTICATION

The test scheme cannot leak into production composition

Fast host tests need deterministic identities, but a permissive test handler is dangerous if registered outside tests. Architecture gates compare production descriptors and fail on the test scheme or bypass implementation.

DECISION LEDGER 44-83

Control	Required proof
named test scheme	only test host environment
test BFF workload	only owner test harness
production descriptor scan	no test handler types
environment manifest	no bypass flag/key
startup guard	production fails on bypass configuration
evidence record	host factory and environment identity

NO CONVENIENT BYPASS

A test seam is explicit dependency replacement, not a runtime feature flag that can enable anonymous trust.

ROUTE-TO-TEST TRACEABILITY

Every protected route has positive and negative evidence

The catalogue links route, capability, owner, session cases, anti-forgery cases and disclosure policy. A route is incomplete until all required cells point to retained tests.

DECISION LEDGER 44-84

Route class	Positive	Negative
session summary	valid session	missing/expired/revoked
safe owner query	capable/in-scope	insufficient/wrong scope
repair request	valid + XSRF	missing XSRF/stale auth
repair approval	distinct approver	self-approval/digest change
repair execute	fresh + approved	expired/ETag/workload missing
logout	revoke and clear	replay/concurrency
login	valid challenge	unsafe return/provider fault

NO ORPHAN ROUTE

Discovery fails when a protected endpoint has no catalogue row or when a catalogue row has no running endpoint.

EVIDENCE MANIFEST

Security claims are bound to exact runs and artifacts

The manifest records source, SDK, configuration, identity fixtures, route catalogue, policy digests, test counts and artifacts. Status stays PENDING until validators confirm non-zero discovery and retained files.

REFERENCE ASSET 44-AM - EVIDENCE RECORD

```
public sealed record EdgeAuthorizationEvidence(  
    string Chapter,  
    string AdrSha256,  
    string SourceRevision,  
    string SdkVersion,  
    string ConfigurationIdentity,  
    string RouteCatalogueSha256,  
    string CapabilityPolicySha256,  
    int DiscoveredEndpoints,  
    int DiscoveredTests,  
    IReadOnlyList<EvidenceArtifact> Artifacts,  
    string WorkloadTrustStatus,  
    string EvidenceStatus,  
    bool ProductionSecurityClaim);
```

FAIL-CLOSED VALIDATOR

Zero endpoints, zero tests, missing artifacts, digest mismatch or a production claim with PENDING workload trust invalidates the dossier.

EVIDENCE CLAIMS

Each claim names the artifact that could prove it

The chapter separates manuscript specification from runtime evidence. The table is a delivery contract for Chapters 45-48 and for any real repository implementation.

DECISION LEDGER 44-86

Claim	Required retained artifact	Current
OIDC code+PKCE configured	host options snapshot + provider test	PENDING
browser receives no tokens	E2E storage/response report	PENDING
sessions revocable	logout/replay component result	PENDING
mutations require XSRF	route inventory + host tests	PENDING
owners reauthorize	owner policy matrix + call trace	PENDING
user is not workload trust	negative owner test	PENDING
production security maturity	out of scope	NOT CLAIMED

REFERENCE STATUS

The manuscript specifies these artifacts; it does not fabricate them.

DEPLOYMENT GATES

Security configuration moves through staged evidence

The rollout starts with provider and callback validation, then session/key continuity, then read routes, then low-risk mutations, and finally repair operations. Each stage has an observable reversal that never enables anonymous fallback.

DECISION LEDGER 44-87

Stage	Gate	Safe reversal
0 options	issuer/audience/redirect validation	do not start
1 login	provider flow + cookie attributes	disable login entry
2 session	replica/restart/revocation	re-authenticate users
3 reads	dual policy + scope tests	read UI unavailable
4 mutations	XSRF + zero-effect denial	disable commands
5 repair	freshness/dual control/workload	repair unavailable

NO INSECURE ROLLBACK

Rollback may remove availability or require re-login; it never disables authentication or owner authorization.

OPERATIONAL READINESS

The BFF exposes safe state for operators, not credentials

Readiness checks distinguish identity metadata, session store, key-ring continuity and capability resolver health. Public health responses stay bounded; protected diagnostics carry references and configuration identities.

DECISION LEDGER 44-88

Dependency	Readiness rule	Public detail
identity metadata	fresh and valid	identity-ready/not-ready
session store	read/write within budget	session-ready/not-ready
Data Protection	key ring usable	protection-ready/not-ready
capability resolver	current policy available	policy-ready/not-ready
owner clients	bounded route health	owner class only
workload trust	Chapter 45 gate current	trust-ready/not-ready

LIVENESS REMAINS LOCAL

An unavailable identity provider or session store affects readiness, not process liveness.

CAPACITY AND ABUSE

Authentication and policy paths have explicit budgets

Login, session validation, anti-forgery, capability refresh and owner authorization can be abused to consume capacity. Budgets are enforced without using subject or resource identifiers as metric labels.

DECISION LEDGER 44-89

Surface	Bounded control
login challenge	per-origin/risk rate limit
OIDC callback	correlation lifetime + one-time use
session lookup	timeout/cache with freshness bound
capability refresh	single-flight + revision cache
anti-forgery issuance	authenticated and rate limited
owner authorization	bulkhead and deadline
repair commands	stricter rate/burst limits

SECURITY AVAILABILITY

Rate limiting cannot create an allow path; overload returns bounded denial or unavailability before business effects.

INCIDENT ACTIONS

Compromise response preserves evidence and closes authority

Runbooks distinguish subject compromise, BFF compromise, key-ring loss, policy corruption and provider outage. Each action identifies what is revoked, what remains uncertain and what evidence must be retained.

DECISION LEDGER 44-90

Incident	Immediate action	Recovery proof
subject compromise	revoke subject sessions	replay rejected
BFF compromise	isolate, rotate edge trust	new workload/key evidence
session store leak	revoke/rotate sessions	old references rejected
Data Protection loss	invalidate sessions	new key-ring continuity
policy corruption	fail closed, restore reviewed version	digest match
IdP outage	block new login	provider recovery test

NO TOKEN HUNTING IN LOGS

Because raw tokens and cookies are excluded from telemetry, incident response relies on safe references, revocation state and authoritative provider evidence.

RESIDUAL RISKS

Known limits stay visible after the design is complete

The BFF pattern reduces token exposure and centralizes browser session controls, but it remains a high-value component. XSS can act within a session, policy caches have bounded staleness, and owner services depend on the correctness of Chapter 45 workload trust.

DECISION LEDGER 44-91

Residual risk	Current treatment	Next owner
BFF compromise	least privilege + owner recheck	Chapters 45/48
XSS acting as user	CSP/XSRF/intent/freshness	security program
capability cache staleness	revision + bounded TTL	Chapter 46
key/secret lifecycle	configuration contract	Chapter 47
workload impersonation	pending deny gate	Chapter 45
provider-specific logout	local revoke guaranteed	environment owner
production maturity	not claimed	Chapter 48+operations

RESIDUAL RISK IS NOT A FOOTNOTE

Every accepted residual risk has a bounded consequence, owner, review trigger and evidence obligation.

IMPLEMENTATION SEQUENCE

Build the refusal path before the happy path

The sequence prevents a working login from being mistaken for complete authorization. Each stage adds a new gate and keeps prior negative tests green.

DECISION LEDGER 44-92

Order	Deliverable
1	ADR, route and capability catalogues
2	OIDC challenge and safe callback policy
3	server-side session and hardened cookie
4	safe session summary and Angular recovery
5	anti-forgery on every mutation
6	bounded operator-context model
7	owner resource-based authorization
8	pending production workload gate
9	identity/fault/browser matrices
10	evidence validator and deployment runbook

DEFINITION OF DONE

Chapter 44 is complete as a specification only when the next workload-identity task is explicit and no security claim exceeds its evidence.

CHAPTER 45 HANDOFF

Workload identity must make the dual-principal path real

Chapter 45 replaces the pending workload authenticator at TB-02 and extends workload identity to service APIs, RabbitMQ and owner repair ports. All Chapter 44 human-context and zero-effect tests remain unchanged.

DECISION LEDGER 44-93

Chapter 44 leaves	Chapter 45 must provide
TrustedEdgeCall shape	authenticated BFF workload construction
pending production gate	Phase-0 workload mechanism
valid-human/missing-workload test	continued rejection
audience requirement	service-specific workload audience
safe decision evidence	workload identity reference
owner authorization handler	trusted caller principal
no user token rule	mechanism-level proof

PROTECTED HANDOFF

A user token alone must still fail after Chapter 45. The new mechanism supplies workload trust; it does not weaken human authorization.

OFFICIAL TECHNICAL ANCHORS

The pattern is grounded in maintained platform and protocol guidance

- ASP.NET Core OIDC web authentication:
<https://learn.microsoft.com/aspnet/core/security/authentication/configure-oidc-web-authentication>
- ASP.NET Core cookie authentication:
<https://learn.microsoft.com/en-us/aspnet/core/security/authentication/cookie?view=aspnetcore-8.0>
- ASP.NET Core SameSite guidance:
<https://learn.microsoft.com/en-us/aspnet/core/security/samesite?view=aspnetcore-8.0>
- ASP.NET Core policy authorization:
<https://learn.microsoft.com/en-us/aspnet/core/security/authorization/policies?view=aspnetcore-8.0>
- ASP.NET Core resource authorization:
<https://learn.microsoft.com/en-us/aspnet/core/security/authorization/resource-based?view=aspnetcore-8.0>
- ASP.NET Core anti-forgery:
<https://learn.microsoft.com/en-us/aspnet/core/security/anti-request-forgery?view=aspnetcore-8.0>
- ASP.NET Core Data Protection:
<https://learn.microsoft.com/aspnet/core/security/data-protection/configuration/overview>
- Angular security and XSRF guidance: <https://angular.dev/best-practices/security>
- OAuth 2.0 Security Best Current Practice: <https://www.rfc-editor.org/rfc/rfc9700>
- PKCE: <https://www.rfc-editor.org/rfc/rfc7636>
- OpenID Connect Core 1.0: https://openid.net/specs/openid-connect-core-1_0.html
- OWASP Session Management:
https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html
- OWASP CSRF Prevention:
https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html

WHAT THIS CHAPTER HAS PROVED

Chapter 44 specifies ADR-044, the BFF/OIDC/session pattern, cookie and Data Protection contracts, safe redirects, local logout and revocation, anti-forgery, Angular session and storage behavior, typed capabilities, server-owned scope, a bounded operator context, dual workload/human gates, owner resource authorization, exact 401/403/404/409/412/503 semantics, privileged repair freshness and separation, zero-effect negative tests, browser and fault laboratories, evidence manifests, staged deployment and incident runbooks. Reference implementation and evidence remain PENDING; production security maturity is not claimed.

CHAPTER CHECKPOINT

- Why does Angular receive a session summary instead of OAuth tokens?
- Which cookie attributes are necessary, and what does each fail to prove?
- Why is anti-forgery still required with SameSite cookies?
- Why can an edge allow decision never replace owner authorization?
- How are human and workload principals kept independent?
- Which conditions map to 401, 403, 404, 409, 412 and 503?
- What proves that a denied request had zero business effects?
- Why must repair request, approval and execution remain distinct?
- Which test proves that a user token is not service trust?
- What evidence is still missing before any runtime security claim?

NEXT - CHAPTER 45

Service-to-Service Identity will replace the pending BFF workload gate with a Phase-0 workload identity mechanism across service APIs, RabbitMQ and owner repair ports. It will document mechanism limits and retain the Chapter 44 invariant that a valid user token alone is rejected as service trust.

PART VIII - SECURITY BOUNDARIES BEFORE SECURITY MECHANISMS

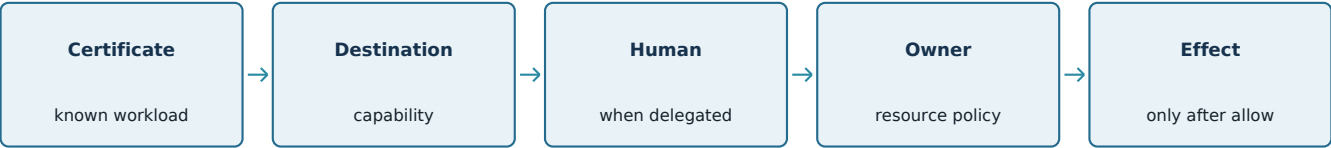
45. Service-to-Service Identity

A Phase-0 mutual-TLS mechanism gives each service a named workload identity, lets every destination authorize that identity locally, and proves that a human token alone is never accepted as service trust.

CHAPTER DECISION

Use one dedicated X.509 workload certificate per service role and environment. HTTP destinations require direct mTLS, exact URI-SAN identity and local capability policy; RabbitMQ uses TLS EXTERNAL authentication plus per-resource permissions. Human and workload principals remain independent, lifecycle evidence remains pending, and production security maturity is not claimed.

PROOF PATH 45-A - AUTHENTICATED CALLER TO OWNER EFFECT



GUARANTEE LEDGER 45-A

Boundary	Mechanism	Allowed claim	Still not allowed
TLS	custom-root mTLS	recognized peer	business authority
service API	local capability	allowed destination action	global trust
delegation	two principals	bounded operator call	user token as workload
broker	EXTERNAL + ACL	allowed topology action	valid event meaning

CHAPTER BOUNDARY

One mechanism, three proofs, bounded claims

Chapter 45 implements a Phase-0 workload identity mechanism for synchronous service APIs, RabbitMQ connections and owner repair ports. The mechanism is mutual TLS with a dedicated X.509 certificate for each named service identity. The receiving destination authenticates that workload and then applies its own capability and resource policy.

This chapter does not turn a certificate into universal internal trust. It does not replace the human principal from Chapter 44, and it does not claim production-grade issuance, storage, rotation or revocation. Those lifecycle controls are specified as requirements here and implemented in Chapter 47.

PROOF OBLIGATIONS 45-A

Proof	Required observation	Forbidden inference
workload authentication	recognized certificate and private-key possession	business permission
local authorization	destination capability and resource allow	global internal role
dual-principal call	workload plus bounded human context	user token is service trust
broker connection	named workload and least broker permissions	message semantics are valid
evidence	retained negative and recovery results	production maturity

NON-NEGOTIABLE INVARIANT

A valid user token without an accepted workload certificate is rejected before TrustedEdgeCall construction, owner policy evaluation and every Domain effect.

DECISION RECORD

ADR-045 - Workloads authenticate cryptographically and authorize locally

The Phase-0 demonstrator uses direct mTLS on internal HTTPS listeners and RabbitMQ TLS listeners. Each runtime role receives its own certificate. HTTP identity is the single exact URI subject alternative name; common name is never an HTTP fallback. RabbitMQ EXTERNAL maps its protocol login to a checked workload catalogue and permissions remain per vhost, exchange and queue.

DECISION 45-B

Decision	Selected	Rejected shortcut
credential	dedicated X.509 per service role/environment	shared certificate
HTTP channel	direct mTLS to Kestrel	untrusted forwarded cert header
identity name	exact URI SAN	CN/substring/regex guess
authorization	destination capability + resource	authenticated means allowed
broker	EXTERNAL + least permissions	shared username/password
maturity	Phase-0 specified and testable	production security claim

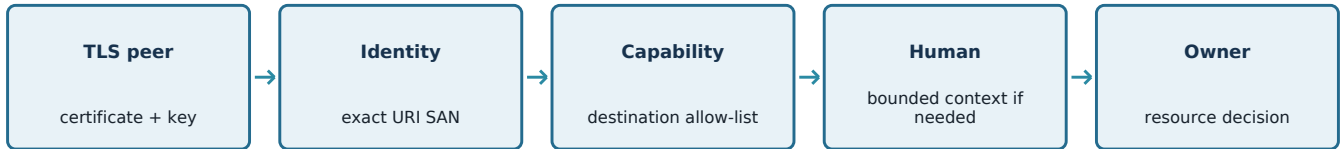
DECISION IDENTITY

ADR-045 canonical SHA-256: 76caa8c29183193bdf6c6aeb0308e69919995092d2c3ba51eed0fc9494a311b5

TRUST FLOW

HTTP trust is a sequence of independent gates

The owner service first accepts the TLS peer, then maps the certificate to a known workload, then evaluates a destination-specific capability. A delegated operator call adds Chapter 44 human authentication and owner resource authorization after the workload gates.

PROOF PATH 45-C - INTERNAL HTTPS**GATE OWNERSHIP 45-D**

Gate	Owner	Failure
TLS certificate	listener	handshake rejection or no client certificate
identity mapping	certificate handler	authentication failure
capability	destination	403 and zero effects
human context	owner Edge	401/403 and zero effects
resource policy	owner Application	404/403 and zero effects

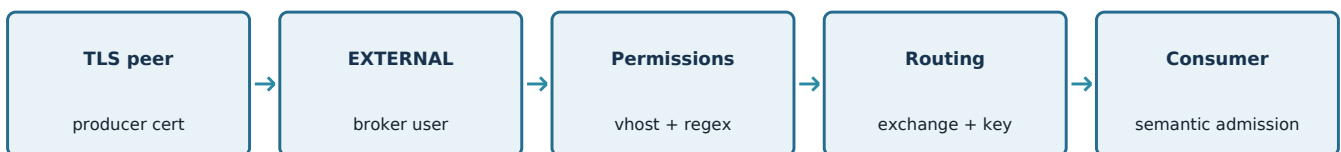
NO COLLAPSED TRUST

Passing one gate supplies input to the next. It never short-circuits later authorization.

BROKER TRUST FLOW

A broker identity controls connection and topology, not event truth

RabbitMQ verifies the client certificate and maps it to a broker user through EXTERNAL authentication. Vhost and resource permissions limit what that workload can configure, publish and consume. The consumer still performs strict route, envelope, fingerprint, schema and owner-policy admission.

PROOF PATH 45-E - RABBITMQ**BROKER CLAIM LEDGER 45-F**

Observed	May claim	May not claim
TLS accepted	known broker workload connected	body is valid
publish allowed	resource permission matched	producer owns event meaning
User ID checked	declared broker identity matches connection	human identity
delivery consumed	queue permission matched	business effect committed
envelope admitted	known V1 contract meaning	exactly-once transport

TWO DEFENSES

Broker authorization reduces reachable topology. Consumer admission protects semantic ownership even when topology or credentials are misconfigured.

VOCABULARY

Human, workload, deployment and message identities remain distinct

Identity words are dangerous when they hide different evidence. The implementation uses separate types and separate telemetry fields so that an operator subject cannot be mistaken for a calling service, and a certificate cannot be mistaken for a deployed binary attestation.

IDENTITY CATALOGUE 45-G

Identity	Evidence	Used for	Not evidence of
human subject	OIDC session	operator intent	caller workload
workload	mTLS certificate	service caller	operator authority
deployment	environment manifest	expected configuration	private-key possession
broker user	EXTERNAL mapping	topology permissions	message meaning
message producer	admitted envelope	public fact provenance	current connection

TYPE THE DISTINCTIONS

Use `WorkloadPrincipal`, `OperatorContext`, `DeploymentIdentity` and `AcceptedEnvelope`. Do not pass an undifferentiated `ClaimsPrincipal` through every layer.

THREAT BOUNDARY

The mechanism closes specific paths and leaves others visible

The mechanism addresses impersonation by an unauthenticated process, shared service credentials, accidental broad broker access and the Chapter 44 user-token shortcut. It does not prove runtime integrity, protect a stolen private key after compromise or create production-ready certificate operations.

THREAT-TO-CONTROL MAP 45-H

Threat	Control	Residual
user token sent to owner	mTLS required before human context	BFF key compromise
service impersonation	custom-root chain + exact SAN	role-level, not instance attestation
wrong destination	audience/capability allow-list	catalogue correctness
broker lateral movement	per-user/vhost/resource permissions	broker admin compromise
stale credential	validity/revocation/rotation policy	Chapter 47 lifecycle pending
forged cert header	direct TLS only	future proxy design required

PHASE-0 BOUNDARY

A control is not mature merely because the API compiles. Runtime evidence, key custody and operational recovery remain explicit obligations.

CERTIFICATE PROFILE

One constrained certificate represents one service role

The certificate profile is intentionally narrow. It is an end-entity certificate, valid for client authentication, signed by the Phase-0 workload root, and contains exactly one recognized URI SAN. Environment is part of the identity string so credentials cannot silently cross environments.

PROFILE 45-I

Field	Required rule	Reject when
Basic Constraints	CA=false	CA=true or missing policy
Key Usage	Digital Signature	incompatible use
Extended Key Usage	TLS Web Client Authentication	clientAuth absent
URI SAN	one exact Nexus workload URI	zero, many or unknown URI SANs
issuer	configured Phase-0 workload root	public/system root only
validity	current UTC within short profile	not-yet-valid/expired

CN IS METADATA

The HTTP identity reader never falls back to subject common name. A certificate without the required URI SAN is not a workload identity.

CANONICAL IDENTITY

URI SAN values are exact, versioned and environment-bound

The book uses a Nexus-specific URN rather than pretending that Phase-0 implements SPIFFE or SPIRE. The grammar is small enough to parse without substring matching, host-name guessing or case-folding surprises.

EXECUTABLE ASSET 45-A - TYPED WORKLOAD IDENTITY

```
public readonly record struct WorkloadId(string Value)
{
    private const string Prefix = "urn:nexus1:workload:v1:";

    public static WorkloadId ParseExact(string value)
    {
        ArgumentException.ThrowIfNullOrWhiteSpace(value);
        if (!value.StartsWith(Prefix, StringComparison.Ordinal))
            throw new WorkloadIdentityException("unsupported-workload-id");
        if (value.Any(char.IsWhiteSpace) || value != value.ToLowerInvariant())
            throw new WorkloadIdentityException("non-canonical-workload-id");
        return new(value);
    }
}

public static class Phase0Workloads
{
    public static readonly WorkloadId Bff =
        WorkloadId.ParseExact("urn:nexus1:workload:v1:phase0:nexus1-bff");
    public static readonly WorkloadId RootCause =
        WorkloadId.ParseExact("urn:nexus1:workload:v1:phase0:root-cause");
}
```

NO IMPLIED STANDARD

The URN is a Phase-0 project identifier. A later adoption of SPIFFE requires an explicit migration and evidence; names that merely resemble SPIFFE IDs are not conformance.

IDENTITY CATALOGUE

Every runtime role has a distinct identity and owner

A version-controlled catalogue is the only source for recognized workload IDs, certificate profile, environment and owning team. Runtime configuration contains catalogue identity and digest, not an ad hoc list assembled in each service.

PHASE-0 WORKLOADS 45-J

Role	Canonical workload ID	Primary use
NEXUS-1 BFF	...:phase0:nexus1-bff	delegated operator calls
AlarmManagement	...:phase0:alarm-management	owner API and fact publisher
RootCause	...:phase0:root-cause	owner API and fact publisher
Compliance	...:phase0:compliance	verdict consumer/owner
Reporting	...:phase0:reporting	delayed-truth consumer
Workflow	...:phase0:workflow	process-manager calls
RepairControl	...:phase0:repair-control	typed owner repair ports

NO IDENTITY RECYCLING

Retired roles are tombstoned. A later workload never inherits an old certificate subject or broker username.

CATALOGUE SCHEMA

Recognition and authorization data are separate

The certificate catalogue says which canonical identity a certificate may represent. A destination policy says what that identity may do. Keeping them separate prevents certificate issuance from silently granting every internal capability.

EXECUTABLE ASSET 45-B - CATALOGUE AND BINDING

```
public sealed record WorkloadCatalogueEntry(  
    WorkloadId Id,  
    string Environment,  
    string CertificateProfile,  
    string BrokerUser,  
    bool Enabled);  
  
public sealed record CertificateBinding(  
    string ThumbprintSha256,  
    WorkloadId Workload,  
    DateTimeOffset NotBeforeUtc,  
    DateTimeOffset NotAfterUtc,  
    string Revision);  
  
public interface IWorkloadCatalogue  
{  
    WorkloadCatalogueEntry RequireEnabled(WorkloadId id);  
    CertificateBinding RequireBinding(  
        string certificateSha256, DateTimeOffset observedAtUtc);  
}
```

SEPARATION 45-K

Catalogue owns	Destination policy owns
canonical identity	allowed capability
environment and profile	target audience
certificate binding	resource constraints
enabled/retired state	delegation and freshness rules

ISSUANCE IS NOT AUTHORIZATION

Adding a certificate binding changes recognized identity. It must not mutate destination capability policy as a side effect.

TRUST STORE

Custom trust excludes ambient public roots

Internal client authentication validates against an explicit Phase-0 workload root set. The operating system's public web roots are not sufficient evidence for a Nexus workload. Trust configuration is immutable for one decision and identified by revision and digest.

EXECUTABLE ASSET 45-C - CERTIFICATE AUTHENTICATION OPTIONS

```
services.AddAuthentication(CertificateAuthenticationDefaults.AuthenticationScheme)
    .AddCertificate(options =>
    {
        options.AllowedCertificateTypes = CertificateTypes.Chained;
        options.ChainTrustValidationMode = X509ChainTrustMode.CustomRootTrust;
        options.CustomTrustStore.Add(workloadRoot);
        options.ValidateCertificateUse = true;
        options.ValidateValidityPeriod = true;
        options.RevocationMode = X509RevocationMode.Online;
        options.EventsType = typeof(NexusCertificateEvents);
    });

services.AddScoped<NexusCertificateEvents>();
```

FAIL CLOSED

If the required trust or revocation evidence cannot be evaluated within its budget, the authentication decision fails. Availability does not create an allow path.

CHAIN VALIDATION

The chain result is retained as bounded evidence

A certificate handler records only safe decision metadata: trust revision, certificate reference, chain status classes, validity window class and decision code. It never logs the private key, full certificate, raw human token or complete subject distinguished name.

VALIDATION MATRIX 45-L

Check	Allow condition	Denial code
chain	builds to configured custom root	workload-untrusted-chain
time	notBefore <= now < notAfter	workload-time-invalid
usage	clientAuth + digitalSignature	workload-usage-invalid
revocation	policy result current	workload-revocation-unknown/revoked
SAN	one exact recognized URI	workload-san-invalid
binding	enabled catalogue revision	workload-disabled/unknown

ONE DECISION CLOCK

All time comparisons for one authentication decision use the same injected UTC observation, making boundary tests deterministic.

SAN EXTRACTION

Certificate parsing is a tested adapter, not string formatting

SubjectAlternativeName is ASN.1 data. The adapter reads URI GeneralName entries from extension 2.5.29.17, rejects malformed encodings, demands exactly one URI and passes that value to the canonical parser. Display strings and localized certificate formatting are not parser inputs.

EXECUTABLE ASSET 45-D - STRICT READER CONTRACT

```
public interface IWorkloadUriSanReader
{
    IReadOnlyList<Uri> ReadUriSans(X509Certificate2 certificate);
}

public static WorkloadId RequireSingleWorkloadUri(
    X509Certificate2 certificate,
    IWorkloadUriSanReader reader)
{
    var values = reader.ReadUriSans(certificate);
    if (values.Count != 1)
        throw new WorkloadIdentityException("workload-uri-san-count-invalid");

    var uri = values.Single();
    if (!uri.IsAbsoluteUri || uri.Fragment.Length != 0 || uri.Query.Length != 0)
        throw new WorkloadIdentityException("workload-uri-san-shape-invalid");

    return WorkloadId.ParseExact(uri.OriginalString);
}
```

COMPILE-ORIENTED SEAM

The ASN.1 reader has its own corpus tests. The policy layer depends on its typed result and never tries to parse certificate display text.

CERTIFICATE HANDLER

Authentication produces one bounded workload principal

The ASP.NET Core certificate event translates validated certificate evidence into a principal with a workload identifier, authentication method, binding revision and safe certificate reference. Capability claims are deliberately absent; authorization resolves them at the destination.

EXECUTABLE ASSET 45-E - CERTIFICATE VALIDATION EVENT

```
public sealed class NexusCertificateEvents(
    IWorkloadCertificateAuthenticator authenticator)
    : CertificateAuthenticationEvents
{
    public override async Task CertificateValidated(
        CertificateValidatedContext context)
    {
        var result = await authenticator.AuthenticateAsync(
            context.ClientCertificate, context.HttpContext.RequestAborted);

        if (!result.IsAccepted)
        {
            context.Fail(result.SafeCode);
            return;
        }

        context.Principal = WorkloadClaimsPrincipal.Create(result.Principal!);
        context.Success();
    }
}
```

AUTHENTICATION OUTPUT

The principal answers 'which recognized workload presented this certificate?' It does not answer 'may it perform this operation?'

CLAIMS SHAPE

A workload principal carries references, not portable authority

Only claims required to reconstruct the typed workload principal are emitted. No wildcard roles, human subject, group membership or destination-independent permission appears in the identity.

WORKLOAD CLAIMS 45-M

Claim	Value class	Why retained	Why bounded
sub	canonical workload ID	stable caller identity	one exact value
amr	mTLS	authentication method	not authorization
nexus:env	phase0	environment binding	exact match
nexus:cert_ref	safe digest prefix/reference	incident correlation	not full cert
nexus:binding_rev	catalogue revision	evidence identity	not capability

CHAPTER 46 HANDOFF

Claims minimization and external-to-local policy mapping are expanded next. Chapter 45 emits only the minimum needed to authenticate the workload.

INTERNAL LISTENER

Service endpoints use a dedicated direct-mTLS listener

Phase-0 keeps the internal listener separate from public health and edge routes. Client certificates are required during TLS negotiation. Direct Kestrel termination avoids trusting arbitrary forwarding headers or creating an undocumented proxy-to-service hop.

EXECUTABLE ASSET 45-F - KESTREL INTERNAL HTTPS

```
builder.WebHost.ConfigureKestrel((context, kestrel) =>
{
    kestrel.ListenAnyIP(7443, listen =>
    {
        listen.UseHttps(https =>
        {
            https.ServerCertificate = serverCertificates.Current();
            https.ClientCertificateMode =
                ClientCertificateMode.RequireCertificate;
        });
    });
});

app.MapGroup("/service-api/v1")
    .RequireAuthorization(WorkloadPolicies.Authenticated)
    .MapOwnerServiceEndpoints();
```

TOPOLOGY IS A CONTROL

The internal port is not exposed as a public browser endpoint. Network placement reduces reachability but never replaces mTLS and local authorization.

FORWARDED CERTIFICATES

A header is not a certificate-authenticated peer

If TLS terminates at a reverse proxy, the owner service authenticates the proxy connection, not automatically the original client. Accepting X-Client-Cert from arbitrary callers would recreate the internal=trusted shortcut. Phase-0 therefore uses direct mTLS and rejects forwarded certificate headers.

PROXY DECISION 45-N

Deployment	Peer proved	Phase-0 status
direct Kestrel mTLS	calling workload certificate	implemented path
proxy terminates TLS	proxy workload only	new trust design required
untrusted cert header	nothing about original caller	rejected
proxy re-mTLS + signed metadata	proxy and bounded assertion	future ADR/evidence

FAIL ON TOPOLOGY DRIFT

Startup compares the declared TLS termination mode to the deployment manifest. A proxy insertion without the matching trust design makes readiness fail.

AUTHENTICATION SCHEMES

The internal host does not accept bearer tokens as workload identity

The service-to-service listener registers certificate authentication as its workload scheme. A browser or operator bearer token is not a fallback scheme. If a future service accepts an OAuth workload token, it must be a separate mechanism with its own issuer, audience and workload semantics.

EXECUTABLE ASSET 45-G - EXPLICIT WORKLOAD SCHEME

```
services.AddAuthentication(options =>
{
    options.DefaultAuthenticateScheme =
        CertificateAuthenticationDefaults.AuthenticationScheme;
    options.DefaultChallengeScheme =
        CertificateAuthenticationDefaults.AuthenticationScheme;
}).AddCertificate();

services.AddAuthorization(options =>
{
    options.AddPolicy(WorkloadPolicies.Authenticated, policy =>
        policy.AddAuthenticationSchemes(
            CertificateAuthenticationDefaults.AuthenticationScheme)
            .RequireAuthenticatedUser()
            .AddRequirements(new KnownWorkloadRequirement()));
});
```

NEGATIVE REGISTRATION TEST

The production internal host test fails if JwtBearer is registered as an alternative workload authentication scheme for these routes.

AUTHENTICATION RESULT

Typed outcomes preserve why trust was refused

Authentication returns a closed result family. Adapters map each result to safe transport behavior and evidence; application code receives only an accepted `WorkloadPrincipal`.

EXECUTABLE ASSET 45-H - CLOSED OUTCOME

```
public abstract record WorkloadAuthenticationResult
{
    public sealed record Accepted(WorkloadPrincipal Principal)
        : WorkloadAuthenticationResult;
    public sealed record MissingCertificate : WorkloadAuthenticationResult;
    public sealed record UntrustedChain(string SafeCode) : WorkloadAuthenticationResult;
    public sealed record InvalidProfile(string SafeCode) : WorkloadAuthenticationResult;
    public sealed record UnknownIdentity : WorkloadAuthenticationResult;
    public sealed record DisabledIdentity : WorkloadAuthenticationResult;
    public sealed record EvidenceUnavailable(string SafeCode)
        : WorkloadAuthenticationResult;
}
```

RESULT OWNERSHIP 45-O

Result	May enter Application?
Accepted	yes, after destination authorization
Missing/Untrusted/Invalid	no
Unknown/Disabled	no
EvidenceUnavailable	no; bounded unavailable response

NO EXCEPTION-DRIVEN ALLOW

Unexpected parsing, chain or catalogue failures become denial/unavailability and zero effects; they are never caught as anonymous success.

CAPABILITIES

Capabilities describe one destination action

A capability name is owned by the receiving boundary and is narrower than a role. The same caller can be allowed to query one owner while being unable to command it, repair another owner or publish to an unrelated exchange.

CAPABILITY CATALOGUE 45-P

Destination	Capability	Allowed caller
AlarmManagement API	alarm.operator.read	NEXUS-1 BFF
AlarmManagement API	alarm.operator.command	NEXUS-1 BFF
RootCause API	root-cause.operator.read	NEXUS-1 BFF
Workflow API	workflow.transition.invoke	Workflow only where owned
owner repair port	repair.owner.invoke	RepairControl
RabbitMQ exchange	fact.publish.<owner>	that owner only

NO INTERNAL ROLE

The policy catalogue contains no internal=true, trusted-service, super-token or all-services wildcard capability.

DESTINATION POLICY

The receiver decides; the caller cannot mint its permission

A request names the intended audience and required capability through endpoint metadata. The destination resolves the authenticated WorkloadId against its local policy revision. Caller-supplied role or capability headers are ignored and rejected when present.

EXECUTABLE ASSET 45-I - DESTINATION REQUIREMENT

```
public sealed record WorkloadCapabilityRequirement(
    ServiceAudience Audience,
    WorkloadCapability Capability) : IAuthorizationRequirement;

public sealed record WorkloadPolicyDecisionInput(
    WorkloadId Caller,
    ServiceAudience Destination,
    WorkloadCapability Capability,
    string ResourceClass,
    string PolicyRevision,
    DateTimeOffset ObservedAtUtc);

public interface IWorkloadPolicy
{
    WorkloadPolicyDecision Evaluate(WorkloadPolicyDecisionInput input);
}
```

LOCAL SOURCE OF AUTHORITY

The authenticated caller supplies identity evidence. The receiving owner supplies audience, capability and resource policy.

AUTHORIZATION HANDLER

Authorization consumes a typed workload principal

The handler reconstructs the typed principal only from the certificate authentication scheme, reads fixed endpoint metadata and asks the local policy. It succeeds on an explicit allow and records a bounded decision reference.

EXECUTABLE ASSET 45-J - CAPABILITY HANDLER

```
public sealed class WorkloadCapabilityHandler(
    IWorkloadPrincipalReader principals,
    IWorkloadPolicy policy,
    ISecurityDecisionRecorder recorder)
    : AuthorizationHandler<WorkloadCapabilityRequirement>
{
    protected override Task HandleRequirementAsync(
        AuthorizationHandlerContext context,
        WorkloadCapabilityRequirement requirement)
    {
        var principal = principals.RequireCertificatePrincipal(context.User);
        var decision = policy.Evaluate(WorkloadPolicyInputs.Create(
            principal, requirement));

        recorder.Record(decision.ToSafeEvidence());
        if (decision.IsAllowed) context.Succeed(requirement);
        return Task.CompletedTask;
    }
}
```

NO SILENT DEFAULT

Unknown identity, unknown capability, wrong audience, stale policy and malformed endpoint metadata all deny.

ENDPOINT COMPOSITION

Each internal route declares audience and capability

Route metadata makes the policy surface inventoryable. Architecture tests enumerate every service endpoint and fail if it lacks a workload requirement, owner resource authorization or the documented exception for public-local health.

EXECUTABLE ASSET 45-K - SERVICE ROUTE DECLARATION

```
group.MapPost("/alarms/{alarmId}/acknowledge", AcknowledgeAlarm)
    .RequireAuthorization(new AuthorizeAttribute
    {
        Policy = WorkloadPolicies.AlarmOperatorCommand
    })
    .WithMetadata(new ServiceAudienceMetadata("alarm-management"),
        new CapabilityMetadata("alarm.operator.command"));

group.MapPost("/repair/reconcile-outbox", ReconcileOutbox)
    .RequireAuthorization(WorkloadPolicies.OwnerRepairInvoke)
    .WithMetadata(new ServiceAudienceMetadata("alarm-management"),
        new CapabilityMetadata("repair.owner.invoke"));
```

INVENTORY GATE

A route added without destination audience and capability metadata fails the architecture test before deployment.

POLICY DIGEST

A decision records the exact local policy identity

Authorization evidence carries the destination policy revision and canonical digest. It never stores a raw certificate or operator token. This makes environment comparison and incident reconstruction possible without turning logs into a credential store.

DECISION EVIDENCE 45-Q

Field	Example	Cardinality
decision_ref	wd_01j...	one per decision
caller_ref	wl:nexus1-bff	bounded service set
destination	alarm-management	bounded service set
capability	alarm.operator.command	catalogue set
result_code	allow/wrong-audience/...	closed set
policy_revision	workload-policy/7	deployment value
cert_ref	sha256:12-byte-prefix	logs only safe reference

NO RESOURCE LABELS

Metrics use bounded result and capability classes. High-cardinality resource, subject and certificate references stay in protected traces/evidence.

POLICY EXAMPLES

The same caller receives different answers at different owners

Destination ownership is visible in the matrix. A BFF certificate can be recognized everywhere while receiving only the operator proxy capabilities each owner explicitly exposes. RepairControl can reach repair ports but cannot impersonate the BFF or publish owner facts.

LOCAL POLICY EXAMPLES 45-R

Caller	Destination	Capability	Decision
NEXUS-1 BFF	AlarmManagement	alarm.operator.read	allow
NEXUS-1 BFF	AlarmManagement	repair.owner.invoke	deny
RepairControl	AlarmManagement	repair.owner.invoke	allow
RepairControl	RootCause	root-cause.operator.read	deny
RootCause	AlarmManagement	alarm.operator.command	deny
unknown	any owner	any	deny

RECOGNITION IS BROAD; AUTHORITY IS NARROW

Knowing a workload ID does not make it an allowed caller for every destination.

BFF HANDOFF

Chapter 45 replaces the pending workload gate

Chapter 44 intentionally left PendingBffWorkloadAuthenticator as a denial boundary. The real adapter now accepts only the BFF certificate identity and only for the declared owner audience/capability. Human context remains a separate, bounded argument.

EXECUTABLE ASSET 45-L - REAL BFF WORKLOAD GATE

```
public sealed class CertificateBackedBffAuthenticator(
    IWorkloadPrincipalReader workload,
    IWorkloadPolicy policy) : IBffWorkloadAuthenticator
{
    public BffWorkloadContext Require(
        ClaimsPrincipal transportPrincipal,
        ServiceAudience owner,
        WorkloadCapability capability)
    {
        var caller = workload.RequireCertificatePrincipal(transportPrincipal);
        if (caller.Id != Phase0Workloads.Bff)
            throw WorkloadDenied.WrongCaller();

        policy.Demand(caller, owner, capability);
        return new(caller.Id, caller.CertificateReference,
            caller.BindingRevision);
    }
}
```

PRESERVED CHAPTER 44 CONTRACT

The implementation changes Pending to certificate-backed. It does not alter operator freshness, intent, scope or owner authorization rules.

DUAL PRINCIPAL

TrustedEdgeCall exists only after both identities are valid

The owner Edge composes the workload and human contexts after each has independently passed its boundary. Neither context contains the other's claims, and application handlers accept only the typed composite.

EXECUTABLE ASSET 45-M - TWO-PRINCIPAL CONSTRUCTION

```
public TrustedEdgeCall RequireTrustedEdgeCall(
    HttpContext http,
    OwnerOperation operation)
{
    var workload = bffWorkload.Require(
        http.User, operation.Audience, operation.WorkloadCapability);
    var human = operatorContext.Require(
        http.Request.Headers, operation.HumanRequirement);

    return new TrustedEdgeCall(
        workload,
        human,
        operation.Intent,
        decisionClock.UtcNow);
}
```

CONSTRUCTION MATRIX 45-S

Workload	Human	Result
valid BFF	valid operator	construct and continue
valid BFF	missing/expired	deny; zero effects
missing/invalid	valid operator	deny; zero effects
wrong service	valid operator	deny; zero effects
valid BFF	wrong scope/intent	deny; zero effects

THE REQUIRED PROOF

The valid-human/missing-workload row is a release-blocking test. It proves that a user token alone is not service trust.

NO TOKEN FORWARDING

The BFF does not turn an operator token into an internal credential

Browser OAuth tokens remain absent from Angular and are not forwarded to owner services. The BFF authenticates itself with mTLS and sends only the bounded human context contract defined in Chapter 44. Owners validate that contract and reauthorize resources.

BOUNDARY PAYLOAD 45-T

Value	Sent to owner?	Reason
BFF client certificate	TLS-layer only	workload authentication
bounded operator context	yes	human decision input
raw access/id token	no	not owner credential
browser cookie	no	edge session only
caller capability header	no	destination resolves policy
decision/correlation references	yes, bounded	evidence continuity

NO CONFUSED DEPUTY

The BFF cannot gain owner authority merely by presenting a powerful human token. The owner requires the BFF workload identity and its own local policy.

OUTGOING MTLS

Typed HttpClient attaches only the caller's current certificate

Each service owns a typed client per destination. The primary handler receives the service's client certificate while normal server certificate and hostname validation remains enabled. No accept-any validator appears in production composition.

EXECUTABLE ASSET 45-N - TYPED MTLS CLIENT

```
services.AddHttpClient<AlarmManagementOwnerClient>((sp, client) =>
{
    var endpoints = sp.GetRequiredService<OwnerEndpoints>();
    client.BaseAddress = endpoints.AlarmManagement;
    client.Timeout = TimeSpan.FromSeconds(3);
})
.ConfigurePrimaryHttpMessageHandler(sp =>
{
    var source = sp.GetRequiredService<IWorkloadCertificateSource>();
    var handler = new HttpClientHandler();
    handler.ClientCertificates.Add(source.CurrentClientCertificate());
    return handler; // default server chain and hostname checks remain active
})
.SetHandlerLifetime(TimeSpan.FromMinutes(5));
```

RELEASE GATE

Static analysis and composition tests reject DangerousAcceptAnyServerCertificateValidator and custom callbacks that return true without chain and hostname proof.

SERVER IDENTITY

Mutual TLS authenticates both directions

The client also validates the destination server certificate against the configured internal server trust and expected DNS name. A valid caller certificate sent to the wrong server must not create a successful connection or leak request data.

TWO DIRECTIONS 45-U

Direction	Identity	Validation owner
client -> server	calling workload URI SAN	receiving service
server -> client	destination DNS SAN	typed client handler
request -> endpoint	audience and capability	receiving service
response -> caller	contract and correlation	calling adapter

AUDIENCE IS STILL REQUIRED

Correct server TLS identity proves the network peer. Endpoint authorization still checks the destination audience and required capability.

CERTIFICATE SOURCE

Private-key access is isolated behind a narrow port

Application and Domain code never receive `X509Certificate2` or private-key handles. Host composition supplies a certificate source to transport adapters. The source exposes current identity, validity and safe reference, and can overlap old/new generations during rotation.

EXECUTABLE ASSET 45-O - CERTIFICATE SOURCE PORT

```
public interface IWorkloadCertificateSource
{
    WorkloadCertificateLease LeaseCurrent(DateTimeOffset observedAtUtc);
}

public sealed class WorkloadCertificateLease : IDisposable
{
    public required X509Certificate2 Certificate { get; init; }
    public required WorkloadId Identity { get; init; }
    public required string SafeReference { get; init; }
    public required string Generation { get; init; }
    public required DateTimeOffset NotAfterUtc { get; init; }
    public void Dispose() => Certificate.Dispose();
}
```

CHAPTER 47 OWNS CUSTODY

This port defines what transport needs. File permissions, secret stores, HSM/KMS choices, issuance and rotation automation are Chapter 47 concerns.

HANDLER POOLING

Certificate rotation accounts for connection and handler reuse

HttpClientFactory pools handlers and existing TLS connections can outlive a configuration change. Rotation therefore has a planned overlap: publish trust for the new generation, activate new clients, drain old connections, verify traffic, then remove old trust.

ROTATION SEQUENCE 45-V

Stage	Caller	Receiver	Evidence
prepare	new cert present, inactive	trusts old + new	profile/chain test
activate	new handler generation	accepts both	new cert_ref observed
drain	old connections expire	accepts both	old use approaches zero
retire	old key inaccessible	old binding disabled	old negative test
close	only new generation	only new trust	manifest complete

NO INSTANT-ROTATION FICTION

Changing a file does not replace live TLS sessions. Evidence must observe the active certificate generation on new connections.

DEADLINE AND CANCELLATION

Trust dependencies are bounded before business work

Chain building, revocation lookup, catalogue resolution and downstream TLS connection all consume bounded time. The request deadline and cancellation token flow through these adapters. A timeout returns safe unavailability and cannot be converted into authenticated anonymous access.

BUDGET MAP 45-W

Operation	Budget rule	On exhaustion
chain/revocation	authentication sub-budget	fail closed
catalogue	bounded local/config lookup	fail closed
policy	bounded local evaluation	fail closed
owner call	remaining request budget	503/504 contract
evidence write	non-secret bounded sink	security alert; no allow mutation

AVAILABILITY SEMANTICS

Unavailable security evidence is not equivalent to a denied identity and never equivalent to an allow.

RABBITMQ LISTENER

The broker requires a certificate before protocol authentication

The Phase-0 broker exposes a TLS listener, verifies the client chain and refuses peers without certificates. Plain AMQP is disabled for the demonstrator network. TLS establishes the connection peer; SASL EXTERNAL maps that peer to a RabbitMQ user.

CONFIGURATION ASSET 45-A - RABBITMQ TLS AND EXTERNAL

```
listeners.tcp = none
listeners.ssl.default = 5671

ssl_options.cacertfile = /run/nexus/trust/workload-ca.pem
ssl_options.certfile   = /run/nexus/broker/server-cert.pem
ssl_options.keyfile    = /run/nexus/broker/server-key.pem
ssl_options.verify     = verify_peer
ssl_options.fail_if_no_peer_cert = true

auth_mechanisms.1 = EXTERNAL
ssl_cert_login_from = common_name
```

PROTOCOL ADAPTER LIMIT

RabbitMQ's configured certificate-to-user extraction is not the canonical HTTP identity parser. A checked binding manifest maps the broker username to the same canonical WorkloadId.

RABBITMQ PLUGIN

EXTERNAL authentication is explicit deployment state

The rabbitmq_auth_mechanism_ssl plugin is enabled before startup and included in the environment manifest. A missing plugin, TLS listener or EXTERNAL mechanism makes broker readiness fail; clients do not fall back to password authentication.

CONFIGURATION ASSET 45-B - ENABLED PLUGIN

```
[rabbitmq_management, rabbitmq_auth_mechanism_ssl].
```

STARTUP GATES 45-X

Observed	Required	Failure
TLS listener	5671 active; 5672 disabled	broker not ready
peer verification	verify_peer + fail_if_no_peer_cert	broker not ready
EXTERNAL	only configured client mechanism	client refuses startup
plugin	auth_mechanism_ssl enabled	deployment gate fails
trust digest	matches environment manifest	environment mismatch

NO PASSWORD FALLBACK

An EXTERNAL failure does not trigger PLAIN with a shared secret. The workload remains disconnected.

BROKER IDENTITY MAPPING

Protocol usernames map to canonical workload identities exactly

The broker certificate profile includes a protocol-specific subject name used by RabbitMQ and the canonical URI SAN used by service APIs. A reviewed manifest binds both to one WorkloadId. Duplicate, unknown or cross-environment mappings fail validation.

CONFIGURATION ASSET 45-C - CHECKED BROKER BINDING

```
{
  "revision": "broker-identities/4",
  "bindings": [
    {
      "rabbitMqUser": "nexus1.phase0.root-cause",
      "certificateCommonName": "nexus1.phase0.root-cause",
      "workloadId": "urn:nexus1:workload:v1:phase0:root-cause",
      "enabled": true
    }
  ]
}
```

NO PARSING EQUIVALENCE CLAIM

The HTTP adapter authorizes from the URI SAN. The broker adapter uses its documented extraction setting and this exact manifest; it does not pretend that both protocols parse certificates identically.

BROKER USERS

One service role receives one broker user

The broker has no shared nexus-service user. Producers, consumers and repair tooling connect as distinct workload identities. The built-in guest account is unavailable outside loopback and is not part of any deployment manifest.

BROKER IDENTITIES 45-Y

Workload	Broker user	Permitted posture
AlarmManagement	nexus1.phase0.alarm-management	publish owned alarm facts
RootCause	nexus1.phase0.root-cause	consume alarm facts; publish root-cause facts
Compliance	nexus1.phase0.compliance	consume verdict facts; publish owned facts
Reporting	nexus1.phase0.reporting	consume reporting queues only
Workflow	nexus1.phase0.workflow	process-manager queues only
RepairControl	nexus1.phase0.repair-control	no ordinary fact impersonation

ACCOUNT IDENTITY IS NOT OWNERSHIP

Broker usernames are adapters for authenticated connections. Event producer ownership is still enforced by topology and consumer admission.

VHOST BOUNDARY

Phase-0 traffic lives inside an explicit broker namespace

All NEXUS-1 runtime resources live in a named vhost. Access to another vhost is absent, not merely unused. The deployment test opens an allowed connection and proves that the same certificate cannot open the default or foreign vhost.

VHOST TESTS 45-Z

Attempt	Expected
known workload -> nexus1-phase0	connect if user and certificate match
known workload -> /	access refused
known workload -> nexus1-prod	access refused
unknown certificate -> nexus1-phase0	TLS/authentication refused
no certificate -> any vhost	TLS peer refused

ENVIRONMENT ISOLATION

A Phase-0 identity string, certificate chain and broker permission set never authorize a production-named vhost.

BROKER PERMISSIONS

Configure, write and read permissions are distinct

RabbitMQ permissions are regular expressions over resources. Each identity gets only the configure, write and read patterns its topology role requires. Regexes are anchored and generated from a typed topology catalogue rather than handwritten broad wildcards.

EXAMPLE PERMISSION SET 45-AA

RootCause action	Pattern	Reason
configure	^root-cause\.	owner-declared queues/bindings only
write	^nexus\.events\$	publish through shared fact exchange
read	^root-cause\.	consume owner queues only
foreign queue	no match	cannot drain Reporting/Compliance
administration	no tag	cannot manage users/policies

SHARED EXCHANGE NEEDS SEMANTIC ADMISSION

A shared events exchange can be writable by several producers. Routing-key and producer ownership therefore remain consumer admission rules.

TOPIC PERMISSIONS

Publishing is constrained by routing-key ownership

Where RabbitMQ topic authorization is enabled, the broker restricts a user to its owner-prefixed routing keys. This complements resource permissions and makes a RootCause certificate unable to publish an AlarmManagement route.

TOPIC POLICY 45-AB

Caller	Allowed routing-key family	Denied example
AlarmManagement	alarm-management.#	root-cause.case-opened.v1
RootCause	root-cause.#	alarm-management.alarm-flood-detected.v1
Compliance	compliance.#	root-cause.verdict-issued.v1
Reporting	reporting.# when it publishes	compliance.#
Workflow	workflow.#	owner facts

DEFENSE IN DEPTH

If topic authorization is unavailable or misconfigured, consumers still reject producer/route mismatches before Domain work.

AMQP USER ID

The message User ID can bind declaration to the connection

A publisher sets the AMQP User ID property to its authenticated broker username. RabbitMQ validates the property against the connection user when present. Consumers retain it as physical provenance and compare it with the expected producer path.

EXECUTABLE ASSET 45-P - PUBLISHER PROPERTIES

```
var properties = new BasicProperties
{
    Persistent = true,
    ContentType = "application/json",
    ContentEncoding = "utf-8",
    MessageId = envelope.MessageId,
    Type = envelope.EventType,
    AppId = "root-cause",
    UserId = "nexus1.phase0.root-cause",
    CorrelationId = envelope.CorrelationId
};
```

BOUNDED MEANING

A validated User ID proves the publisher declaration matched the broker connection. It does not validate the envelope, public-fact owner or payload semantics.

CONSUMER PROVENANCE

The consumer joins broker and contract evidence without conflating them

The consumer's accepted-delivery type contains physical provenance separately from the versioned envelope. The admission function checks expected exchange, routing key, broker User ID, app ID, envelope producer, event type and fingerprint before translating to an owner-local command.

EXECUTABLE ASSET 45-Q - PRODUCER-PATH CHECK

```
public void DemandExpectedProducer(
    RawDelivery raw,
    EnvelopeV1 envelope,
    ProducerPath expected)
{
    DemandEqual(expected.Exchange, raw.Exchange, "exchange");
    DemandEqual(expected.RoutingKey, raw.RoutingKey, "routing-key");
    DemandEqual(expected.BrokerUser, raw.UserId, "broker-user");
    DemandEqual(expected.AppId, raw.AppId, "app-id");
    DemandEqual(expected.Producer, envelope.Producer, "producer");
    DemandEqual(expected.EventType, envelope.EventType, "event-type");
}
```

NO SINGLE MAGIC FIELD

Provenance is a conjunction of connection, topology, properties and envelope evidence. Every mismatch is terminal before Domain invocation.

BROKER CONNECTION FACTORY

Clients demand TLS and EXTERNAL explicitly

The RabbitMQ client configuration names the expected broker host, enables TLS, supplies the workload certificate and selects EXTERNAL. Automatic recovery may recreate connections, but it cannot change identity or fall back to a different mechanism.

EXECUTABLE ASSET 45-R - RABBITMQ CLIENT IDENTITY

```
var factory = new ConnectionFactory
{
    HostName = options.Host,
    Port = 5671,
    VirtualHost = "nexus1-phase0",
    AuthMechanisms = [new ExternalMechanismFactory()],
    Ssl = new SslOption
    {
        Enabled = true,
        ServerName = options.ExpectedDnsName,
        CertPath = options.ClientCertificatePath,
        CertPassphrase = options.ClientCertificatePassword
    },
    AutomaticRecoveryEnabled = true,
    NetworkRecoveryInterval = TimeSpan.FromSeconds(5)
};
```

SECRET-HANDLING SEAM

File paths and passphrases are illustrative adapter inputs. Chapter 47 replaces deployment-local loading with the approved custody mechanism and proves permissions/redaction.

CONNECTION RECOVERY

Recovery reauthenticates; it does not preserve old authority forever

A recovered connection must use the currently active certificate generation and the same expected identity. Recovery evidence records connection generation and certificate reference. Repeated authentication failures open a bounded circuit and surface readiness failure.

RECOVERY STATES 45-AC

State	Allowed action	Forbidden action
connected	publish/consume within permissions	identity mutation
recovering	bounded reconnect	ack unknown deliveries
auth refused	alert and remain unavailable	password fallback
new generation	redeclare/check topology	reuse stale identity evidence
retired generation	close session	continue publishing

UNKNOWN SETTLEMENT REMAINS UNKNOWN

Connection recovery does not change publisher-confirm, consumer-ack or transaction semantics established in Chapters 27-30.

TLS SESSION REUSE

Revocation expectations include session resumption behavior

Existing TLS sessions and resumed sessions may continue without a fresh full handshake. If an incident requires immediate trust removal, the runbook disables session reuse where supported, closes affected connections and verifies that the retired certificate cannot reconnect.

REMOVAL PROOF 45-AD

Action	Required observation
disable catalogue binding	new HTTP auth denied
remove broker permission/user	new broker auth denied
close live connections	connection generation terminated
control session reuse	no resumed acceptance of retired identity
run negative probe	old cert fails API and broker

OPERATIONAL TRUTH

Updating a trust store is not proof that all active sessions stopped using the old credential.

REPAIR BOUNDARY**RepairControl receives narrow owner-specific capabilities**

Chapter 42's repair jobs call typed owner repair ports. The RepairControl certificate is allowed only on those ports and only for named repair operations. It cannot issue operator commands, read broad owner data or publish owner facts.

REPAIR CAPABILITIES 45-AE

Operation	Destination	Additional owner gate
reconcile outbox	producing owner	bounded range + dry-run/execute
rebuild projection	projection owner	checkpoint + generation
replay dead letter	consumer owner	message allow-list + approval
resume workflow	Workflow	instance version + approval
cross-owner SQL	none	forbidden

IDENTITY IS ONE GATE

Owner repair authorization still verifies request, approval, freshness, resource scope, idempotency and current state.

PURE WORKLOAD CALLS**Not every service call carries a human principal**

Scheduled reconciliation, workflow transitions and internal reads may be initiated by a workload. Their endpoints require a workload capability and owner resource policy but no fabricated operator subject. Delegated operator operations require both principals.

PRINCIPAL MATRIX 45-AF

Operation class	Workload	Human	Example
delegated operator	required	required	BFF acknowledge alarm
workflow automation	required	absent	advance process state
repair execution	required	approval context required	replay bounded DLQ
message consumption	broker workload	absent	consume public fact
public health	not required	absent	bounded liveness

NO SYNTHETIC HUMANS

A machine operation records its initiating workload and policy; it does not invent an operator identity to satisfy a generic authorization API.

AUDIENCE BINDING

A certificate is not a bearer credential for every destination

The same workload certificate may authenticate the service role to several destinations, but each endpoint policy has one audience and capability. Typed clients fix the base address and expected audience; destinations reject mismatched audience metadata or request assertions.

EXECUTABLE ASSET 45-S - OWNER REQUEST CONTEXT

```
public sealed record WorkloadCallContext(
    WorkloadId Caller,
    ServiceAudience Destination,
    WorkloadCapability Capability,
    string CertificateReference,
    string BindingRevision,
    DateTimeOffset AuthenticatedAtUtc);

public static void DemandDestination(
    WorkloadCallContext call, ServiceAudience local)
{
    if (call.Destination != local)
        throw WorkloadDenied.WrongAudience();
}
```

NOT TOKEN AUDIENCE

This Phase-0 audience is endpoint policy metadata, not a claim trusted from the certificate or caller header.

HTTP FAILURE SEMANTICS

Failures reveal little and occur before effects

A missing client certificate can fail the TLS handshake, producing no application HTTP response. A certificate that reaches the authentication handler but fails validation is rejected without entering application code. An authenticated workload without capability receives bounded 403 behavior.

FAILURE MAP 45-AG

Failure	Transport observation	Domain invoked?
no client certificate	TLS handshake fails	no
untrusted/expired/wrong use	TLS/authentication failure	no
unknown/disabled SAN	authentication failure	no
wrong capability/audience	403	no
human missing/invalid	401/403 by Chapter 44 contract	no
resource hidden	404	no mutation

DO NOT OVER-SPECIFY THE WIRE

TLS stacks may expose different handshake details. Tests assert connection failure and zero effects, not a fragile application status for a request that never entered ASP.NET Core.

SAFE DENIAL

Clients receive stable problem classes, not certificate diagnostics

When an HTTP response exists, the service returns a stable problem type and correlation reference. Detailed chain status, subject data, trust paths and certificate references remain protected evidence. This prevents identity enumeration and diagnostic leakage.

RESPONSE BOUNDARY 45-AH

Audience	Visible	Hidden
caller	status, safe problem type, correlation	chain details, trust anchors
operations	result code, service/capability class	private key, raw token
security evidence	safe cert ref, revision, decision	secret material
metrics	bounded denial class	subject/resource/cert labels

CONSISTENT REFUSAL

Unknown caller and disabled caller are not distinguished to an untrusted client, while protected evidence retains the operational reason.

ZERO-EFFECT CONTRACT

Every denied workload path proves absence of side effects

Status assertions are insufficient. Negative tests observe the application handler spy, database row counts, outbox/inbox state, broker publications, workflow transitions and audit records. Security decision evidence may be written; business effects must remain zero.

EFFECT PROBES 45-AI

Surface	Denied expectation
Application handler	not invoked
owner database	no new/changed aggregate rows
outbox/inbox	no business message state
RabbitMQ	no publication/ack caused by request
process manager	no transition/timer
security evidence	one bounded denial record allowed

SECURITY EVIDENCE IS NOT A DOMAIN EFFECT

Recording a refusal is required. It must use a separate bounded sink and never participate in business truth.

USER-TOKEN-ONLY PROOF

The critical negative test starts with a valid operator

The test deliberately uses a fully valid Chapter 44 operator context. It omits the BFF workload certificate and calls the owner route through the production listener. Success would prove the exact confused-deputy shortcut the chapter must prevent.

EXECUTABLE TEST 45-A - VALID HUMAN IS NOT SERVICE TRUST

```
[Fact]
public async Task Valid_operator_without_workload_certificate_is_rejected()
{
    await using var host = await OwnerHost.StartProductionPathAsync();
    var before = await host.Effects.SnapshotAsync();

    using var client = host.CreateClientWithoutCertificate();
    using var request = OperatorRequests.ValidAcknowledgeAlarm();
    var attempt = await HttpAttempt.CaptureAsync(() => client.SendAsync(request));

    attempt.MustBeTlsOrAuthenticationFailure();
    (await host.Effects.SnapshotAsync()).Should().BeEquivalentTo(before);
    host.ApplicationProbe.Invocations.Should().Be(0);
}
```

RELEASE BLOCKER

This test must run through the same listener and authentication registrations as production composition. An in-memory handler that bypasses TLS is not sufficient evidence.

WRONG WORKLOAD PROOF

A valid certificate for another service still fails

Trusting the CA is not enough. A RootCause certificate is cryptographically valid but lacks the BFF operator-proxy capability at AlarmManagement. The request is denied before human context can create authority.

EXECUTABLE TEST 45-B - VALID WRONG SERVICE

```
[Fact]
public async Task RootCause_certificate_cannot_proxy_operator_command()
{
    await using var host = await AlarmManagementHost.StartProductionPathAsync();
    using var client = host.CreateMtlsClient(Certificates.RootCause);
    using var request = OperatorRequests.ValidAcknowledgeAlarm();

    var response = await client.SendAsync(request);

    response.StatusCode.Should().Be(HttpStatusCode.Forbidden);
    host.ApplicationProbe.Invocations.Should().Be(0);
    (await host.Effects.BusinessDeltaAsync()).Should().BeEmpty();
}
```

AUTHENTICATION IS NOT AUTHORIZATION

This case must reach certificate authentication successfully and then fail destination authorization.

CERTIFICATE CORPUS

Profile failures are generated, not hand-waved

A test CA fixture issues short-lived certificates with one controlled defect at a time. The corpus verifies the actual parser, chain builder and ASP.NET Core authentication event rather than mocking an IsValid boolean.

CERTIFICATE CORPUS 45-AJ

Variant	Expected code	Effects
valid known URI SAN	accepted	only after policy
unknown root	untrusted-chain	zero
expired/not-yet-valid	time-invalid	zero
serverAuth only	usage-invalid	zero
CA=true	profile-invalid	zero
no URI SAN	san-count-invalid	zero
two URI SANs	san-count-invalid	zero
malformed/uppercase/foreign env	identity-invalid	zero

ONE DEFECT PER FIXTURE

Single-defect certificates make failure attribution stable and keep test diagnostics useful.

EPHEMERAL CA FIXTURE

Tests build a real chain with constrained extensions

The fixture creates an ephemeral root and end-entity certificate in memory, adds the exact extensions and returns separate public/private instances. It never ships test private keys with the runtime image.

EXECUTABLE TEST ASSET 45-C - CERTIFICATE PROFILE FIXTURE

```
public IssuedWorkloadCertificate Issue(WorkloadId id, ProfileMutation mutation)
{
    using var key = RSA.Create(2048);
    var request = new CertificateRequest(
        "CN=nexus1.phase0.test", key, HashAlgorithmName.SHA256,
        RSASignaturePadding.Pkcs1);
    request.CertificateExtensions.Add(new X509BasicConstraintsExtension(false, false, 0, true));
    request.CertificateExtensions.Add(new X509KeyUsageExtension(
        X509KeyUsageFlags.DigitalSignature, true));
    request.CertificateExtensions.Add(ClientAuthenticationEku());
    request.CertificateExtensions.Add(SubjectAlternativeName(id, mutation));

    var serial = RandomNumberGenerator.GetBytes(16);
    var publicCert = request.Create(root, NotBefore(mutation), NotAfter(mutation), serial);
    return new(publicCert.CopyWithPrivateKey(key), root.PublicOnly());
}
```

TEST-ONLY AUTHORITY

The ephemeral CA is a deterministic test instrument, not an example of production certificate issuance or custody.

PARSER TESTS

The URI SAN reader has an adversarial corpus

Identity parsing tests operate on encoded certificates, not pre-parsed strings. The corpus covers unknown GeneralName types, duplicate URI entries, malformed ASN.1, invisible whitespace, percent-encoding surprises, query/fragment additions and subject-only identities.

PARSER MATRIX 45-AK

Input	Expected
one canonical URI SAN	typed WorkloadId
CN only	reject
DNS SAN only	reject for HTTP workload identity
URI SAN + second URI SAN	reject ambiguous
URI with query/fragment	reject shape
mixed case or whitespace	reject non-canonical
malformed extension	reject parsing

NO DISPLAY PARSER

A test proves that changing the localized certificate display string cannot change the extracted identity.

CHAIN TESTS

Custom-root validation excludes system-root surprises

Provider-backed tests use an isolated custom trust store and demonstrate that a certificate chaining to a normally trusted public/system root is still rejected as a Nexus workload. Intermediate ordering and missing intermediates are exercised explicitly.

CHAIN MATRIX 45-AL

Chain	Expected
leaf -> Phase-0 intermediate -> configured root	accept profile, continue
leaf -> public/system root	reject
leaf -> foreign environment root	reject
missing intermediate	reject unless explicitly supplied by policy
expired intermediate	reject
unconstrained CA misuse	reject profile/policy

TRUST IS DEPLOYMENT DATA

The test records the exact root-set digest; 'works on my machine' system trust is not accepted evidence.

REVOCATION TESTS

Revoked, unknown and unavailable are different observations

The validator exposes distinct safe codes for explicitly revoked credentials, inability to establish revocation status and stale evidence. The Phase-0 policy fails closed for protected internal operations; the operational response may differ by cause.

REVOCATION MATRIX 45-AM

Observation	Authentication	Operation
good, current evidence	continue	normal
revoked	deny	isolate workload/incident
status unknown	deny/unavailable	security dependency degraded
responder timeout	deny/unavailable	bounded retry; no effect
evidence stale	deny	refresh required
policy configured offline	only if explicit Phase-0 profile	residual risk recorded

NO SOFT-FAIL DEFAULT

A network error while checking revocation never becomes a successful authentication by exception handling.

POLICY TESTS

Authorization is a Cartesian matrix, not one happy path

Table-driven tests enumerate caller, destination, capability, resource class, environment, policy revision and enabled state. The test suite fails if any unknown coordinate defaults to allow.

EXECUTABLE TEST 45-D - WORKLOAD POLICY MATRIX

```
[Theory]
[MemberData(nameof(WorkloadPolicyCases.All), MemberType = typeof(WorkloadPolicyCases))]
public void Policy_is_exact(PolicyCase test)
{
    var decision = policy.Evaluate(test.Input);

    decision.IsAllowed.Should().Be(test.ExpectedAllow);
    decision.SafeCode.Should().Be(test.ExpectedCode);
    decision.PolicyRevision.Should().Be("workload-policy/7");
}

[Fact]
public void Unknown_coordinates_never_allow() =>
    policy.Evaluate(PolicyInputs.UnknownEverything()).IsAllowed.Should().BeFalse();
```

MUTATION GUARD

Policy mutation tests remove audience, invert caller, broaden wildcard and stale the revision. Each mutation must make at least one test fail.

ROUTE INVENTORY TESTS

Every service endpoint is accounted for

A production-composition test enumerates endpoint metadata. Every internal owner route has the certificate scheme, one service audience, one workload capability and the owner resource gate. Public-local health endpoints are the only documented exception.

EXECUTABLE TEST 45-E - ENDPOINT INVENTORY

```
[Fact]
public async Task Every_service_endpoint_declares_workload_policy()
{
    await using var host = await ProductionHost.StartAsync();
    var endpoints = host.Services.GetRequiredService<EndpointDataSource>()
        .Endpoints.OfType<RouteEndpoint>()
        .Where(e => e.RoutePattern.RawText!.StartsWith("/service-api/"));

    foreach (var endpoint in endpoints)
    {
        endpoint.Metadata.GetMetadata<ServiceAudienceMetadata>().Should().NotBeNull();
        endpoint.Metadata.GetMetadata<CapabilityMetadata>().Should().NotBeNull();
        endpoint.Metadata.GetOrderedMetadata<IAuthorizeData>().Should().NotBeEmpty();
    }
}
```

ZERO DISCOVERY FAILS

The test first asserts that at least one service endpoint was discovered, preventing a broken filter from producing a false green result.

COMPOSITION TESTS

Production registrations contain no bypass scheme

Descriptor tests inspect the host registrations and options used by the real internal listener. They detect duplicate authenticators, a permissive fallback scheme, an accept-any server certificate callback or a test certificate source registered in production.

COMPOSITION GATES 45-AN

Gate	Expected
workload authenticator	one certificate-backed implementation
PendingBffWorkloadAuthenticator	absent
JWT bearer fallback	absent on service routes
client certificate mode	RequireCertificate
server validation bypass	absent
test CA/source	absent from production container
trust revision	matches environment manifest

SAME COMPOSITION PATH

Tests may override certificate bytes and clocks, but they build the same listener, authentication and authorization registrations as the runtime.

HTTP INTEGRATION MATRIX

The production listener proves handshake and policy behavior

Integration tests open real TLS sockets to the internal listener. They distinguish handshake rejection from authenticated authorization denial and verify safe response bodies when an HTTP response exists.

HOST MATRIX 45-AO

Certificate	Capability	Expected
none	n/a	connection/handshake failure
untrusted	n/a	connection/auth failure
expired/wrong EKU	n/a	authentication failure
known wrong service	missing	403
BFF	read	allowed read path
BFF	repair	403
RepairControl	repair	continue to owner repair policy

ASSERT BOTH SIDES

The test records server decision evidence and the client-visible observation; either alone is incomplete.

TWO-PRINCIPAL MATRIX

Workload and operator validity vary independently

The BFF-owner component suite crosses all meaningful combinations. It prevents later refactoring from reintroducing an edge allow shortcut or treating a human context as sufficient when mTLS is absent.

DUAL-PRINCIPAL TESTS 45-AP

Workload	Human	Owner policy	Expected
valid BFF	valid	allow	one intended effect
valid BFF	expired	n/a	deny; zero effects
valid BFF	wrong scope	n/a	deny; zero effects
missing	valid	n/a	TLS/auth fail; zero effects
wrong service	valid	n/a	403; zero effects
valid BFF	valid	resource deny	404/403; zero effects

ONLY ONE POSITIVE CELL

The happy path is the conjunction of independent workload, human and owner-resource allows.

RABBITMQ COMPONENT TESTS**A real broker proves certificate and permission behavior**

Container-backed tests start RabbitMQ with the actual plugin, TLS listener and generated permission manifest. They connect with distinct certificates and attempt real configure, publish and consume operations.

BROKER MATRIX 45-AQ

Attempt	Expected
no/unknown cert	TLS/auth refused
known cert + wrong vhost	access refused
RootCause declares Reporting queue	permission refused
RootCause publishes owner route	allowed + confirmed
RootCause publishes AlarmManagement route	topic/consumer refusal
Reporting consumes RootCause queue	permission refused
User ID mismatches connection	publish refused/channel closed

REAL PROVIDER REQUIRED

A mocked connection factory cannot prove TLS negotiation, EXTERNAL mapping, vhost permission or broker User-ID validation.

BROKER SEMANTIC TESTS**Authenticated publishers can still send invalid messages**

A test using a valid producer certificate publishes a body with the wrong producer, event type, fingerprint or schema. The broker may route it, but the consumer must reject it before Domain invocation and retain terminal admission evidence.

AUTHENTICATED-INVALID MATRIX 45-AR

Broker identity	Message defect	Consumer result
AlarmManagement	producer=RootCause	terminal reject
AlarmManagement	wrong routing key	not routed or terminal reject
AlarmManagement	fingerprint mismatch	terminal reject
AlarmManagement	unknown schema	compatibility policy
AlarmManagement	valid duplicate	Inbox/semantic duplicate path

CONNECTION TRUST IS NOT CONTENT TRUST

This suite protects the boundary even after a legitimate producer process is buggy or compromised.

ROTATION TEST

Old and new generations overlap intentionally, then converge

The rotation laboratory holds an old connection open, introduces a new certificate generation, creates new HTTP and AMQP connections, drains old handlers/connections and finally disables the old binding. Evidence distinguishes accepted overlap from completed retirement.

ROTATION ASSERTIONS 45-AS

Moment	Old	New
before prepare	accepted	unknown
trust overlap	accepted	accepted
caller activation	existing sessions only	new sessions observed
drain complete	no active sessions	accepted
retirement	new connection rejected	accepted

NO GAP, NO INDEFINITE OVERLAP

The runbook defines maximum overlap and retirement evidence. Both simultaneous rejection and permanent dual trust are failures.

FAULT LABORATORY

Security dependencies fail without granting authority

Fault injection interrupts trust-store loading, catalogue refresh, revocation evidence, certificate source access, DNS, TLS handshakes and broker authentication. Each fault has a bounded timeout, readiness consequence and zero-effect assertion.

FAULT SET 45-AT

Fault	Expected behavior	Recovery proof
trust file missing	host not ready/start fails	correct digest loaded
catalogue stale	new decisions fail closed	revision advances
revocation offline	bounded unavailable	current evidence observed
private key inaccessible	client not ready	new handshake succeeds
DNS/wrong server cert	client TLS fails	expected name succeeds
broker rejects EXTERNAL	publisher/consumer unavailable	named user reconnects

LIVENESS REMAINS LOCAL

External trust and broker faults affect readiness and operations; they do not make the process itself dead or create anonymous fallback.

PROXY INJECTION FAULT

Forged certificate headers are ignored and rejected

A negative test connects without a client certificate and supplies certificate-shaped headers. The direct-mTLS listener rejects the connection before header parsing. A second test uses a valid wrong workload certificate plus a forged BFF header; authorization follows the actual TLS principal and denies.

HEADER INJECTION 45-AU

TLS identity	Header	Expected
none	X-Client-Cert=BFF	handshake failure
RootCause	X-Client-Cert=BFF	RootCause principal; 403
BFF	X-Client-Cert=RepairControl	BFF principal; normal policy
BFF	caller-capability=*	header ignored/rejected

PEER EVIDENCE WINS

Only the certificate from the authenticated TLS connection can construct the workload principal in Phase-0.

ENVIRONMENT EQUIVALENCE

Trust policy is compared without exporting secrets

Each environment produces a value-free manifest containing trust-root fingerprints, identity/policy/catalogue digests, listener mode, broker mechanism, certificate profile and lifecycle policy IDs. Private keys, passwords and full certificates are excluded.

EVIDENCE ASSET 45-D - VALUE-FREE TRUST MANIFEST

```
{
  "schema": "nexus1.workload-trust-manifest/1",
  "environment": "phase0",
  "httpMode": "direct-mtls",
  "clientCertificateMode": "require-certificate",
  "identityGrammar": "urn:nexus1:workload:v1",
  "trustRootSha256": ["sha256:<full-root-fingerprint>"],
  "catalogueDigest": "sha256:<digest>",
  "policyDigest": "sha256:<digest>",
  "brokerMechanism": "EXTERNAL",
  "productionSecurityClaimed": false
}
```

COMPARABLE, NOT SECRET

CI can compare manifests across environments without receiving any private key or credential value.

OBSERVABILITY**Telemetry explains decisions without exposing credentials**

Traces carry a bounded workload identity class, destination, capability, result code, policy revision and decision reference. Logs never contain PEM, PFX, private-key paths with secrets, passphrases, raw operator tokens or certificate serial/subject values unless a protected evidence policy explicitly permits a safe reference.

TELEMETRY FIELDS 45-AV

Field	Log	Metric label	Protected evidence
workload class	yes	bounded	yes
capability/result	yes	bounded	yes
cert_ref	protected log	no	yes
subject/serial	no	no	only if approved
operator/resource ID	safe reference	no	protected
private key/token	never	never	never

CORRELATION WITHOUT LEAKAGE

Decision references join workload, human and owner policy evidence; raw credentials are neither required nor useful for that join.

METRICS**Security and availability have bounded service indicators**

Metrics separate authentication refusal, authorization denial and dependency unavailability. Certificate expiry is reported as coarse time-to-expiry buckets per workload class, avoiding raw serial or subject labels.

METRIC SET 45-AW

Metric	Bounded dimensions
workload_authentication_total	service, result_class
workload_authorization_total	destination, capability, result
mtls_handshake_failures_total	listener, failure_class
workload_certificate_expiry	workload_class, bucket
broker_external_auth_total	workload_class, result
security_dependency_duration	dependency_class, outcome

DO NOT CREATE AN ORACLE

Public dashboards aggregate bounded classes; they do not enumerate certificate identities or sensitive resource scope.

READINESS

Each workload exposes safe trust readiness

Readiness checks prove that the current certificate is accessible, has sufficient remaining lifetime, matches the expected identity, chains to the configured trust profile and can establish bounded destination/broker probes where required. Public responses expose only ready/not-ready classes.

READINESS CHECKS 45-AX

Check	Not-ready when
certificate source	missing key or identity mismatch
remaining lifetime	below rotation safety window
trust/catalogue	digest mismatch or stale revision
destination TLS	server identity/handshake invalid
RabbitMQ	EXTERNAL/vhost connection refused
policy	required destination capability absent

PROBE SAFELY

A readiness probe never mutates Domain state, publishes a business fact or exposes certificate diagnostics publicly.

EVIDENCE SCHEMA

Every workload decision has a value-safe record

The evidence record is immutable, schema-versioned and validates closed result codes. It captures observation time, caller reference, destination, capability, authentication method, trust/policy identities, correlation and zero-effect probe reference when denied.

EVIDENCE ASSET 45-E - DECISION RECORD

```
public sealed record WorkloadDecisionEvidenceV1(  
    string EvidenceId,  
    DateTimeOffset ObservedAtUtc,  
    string CallerReference,  
    string Destination,  
    string Capability,  
    string AuthenticationMethod,  
    string ResultCode,  
    string CertificateReference,  
    string TrustRevision,  
    string PolicyRevision,  
    string CorrelationReference,  
    string? ZeroEffectProbeReference);
```

NO FABRICATED EVIDENCE

The manuscript specifies this record. A runtime claim remains PENDING until a real execution artifact validates against the schema.

EVIDENCE MANIFEST

A fail-closed manifest separates specification from execution

The chapter manifest lists every claim, test selector, expected non-zero discovery count, artifact digest, environment identity, execution time and result. PENDING is valid editorial truth; PASSED without an existing, fresh, matching artifact is rejected.

EVIDENCE STATES 45-AY

State	Meaning	Allowed book claim
SPECIFIED	design and test contract written	mechanism specified
PENDING	runtime result absent	not implemented/proved
PASSED	fresh matching artifact validated	bounded environment result
FAILED	test or gate failed	claim prohibited
STALE	artifact no longer matches revision	claim prohibited

NO ZERO-TEST GREEN

The validator requires discovered > 0, executed = discovered, all passed, and matching build/configuration identities.

EVIDENCE VALIDATOR

Runtime proof is accepted only when identity and freshness match

The validator compares the evidence schema, repository commit, container digest, trust manifest, policy digest, test discovery and timestamps. It rejects manually edited summaries and results from a different security configuration.

EVIDENCE ASSET 45-F - FAIL-CLOSED VALIDATOR

```
public void Validate(WorkloadEvidenceManifest manifest, ExpectedEvidence expected)
{
    DemandEqual("nexus1.workload-evidence/1", manifest.Schema);
    DemandEqual(expected.Commit, manifest.RepositoryCommit);
    DemandEqual(expected.ImageDigest, manifest.ImageDigest);
    DemandEqual(expected.TrustDigest, manifest.TrustManifestDigest);
    DemandEqual(expected.PolicyDigest, manifest.PolicyDigest);
    Demand(manifest.Discovered > 0, "zero-discovery");
    DemandEqual(manifest.Discovered, manifest.Executed);
    DemandEqual(manifest.Executed, manifest.Passed);
    Demand(manifest.CompletedAtUtc >= expected.NotBeforeUtc, "stale-evidence");
}
```

CLAIM SCOPE

A passing Phase-0 Compose artifact proves only the named environment, build and scenarios. It is not a general production security certification.

DEPLOYMENT SEQUENCE

Trust is introduced before callers depend on it

Deployment follows an order that preserves refusal safety. Receivers learn trust and policy first, then callers receive credentials, then typed clients switch to mTLS, then old/shared paths are removed and tested as negative cases.

DEPLOYMENT STAGES 45-AZ

Order	Deliverable	Gate
1	identity/capability catalogues	digests reviewed
2	receiver trust + direct listener	negative cert matrix
3	broker EXTERNAL + permissions	real broker matrix
4	caller certificate source	identity/profile readiness
5	typed HTTP/AMQP clients	positive + wrong-destination tests
6	BFF pending gate replacement	user-token-only negative test
7	remove password/shared/bypass paths	absence inventory

SAFE ROLLOUT

A receiver may temporarily reject a not-yet-enabled caller. Rollout never temporarily accepts unauthenticated traffic.

ROLLBACK

Rollback may reduce availability but never remove authentication

A failed rollout returns to the last trusted certificate generation and policy revision while retaining mTLS. It does not re-enable plain HTTP, password fallback, shared broker credentials or user-token-only owner calls.

ROLLBACK ACTIONS 45-BA

Failure	Safe rollback	Forbidden rollback
new caller cert rejected	restore old active generation during overlap	disable client cert requirement
policy regression	restore signed/reviewed revision	allow all internal
broker mapping error	restore exact prior binding	shared PLAIN user
rotation outage	extend controlled overlap	accept expired certificate
proxy insertion	restore direct path	trust arbitrary cert header

AVAILABILITY IS EXPENDABLE; AUTHORITY IS NOT

The safe degraded state is not-ready or unavailable, not insecurely reachable.

INCIDENT RESPONSE

Credential compromise closes sessions and preserves evidence

The incident runbook identifies the affected workload, disables its catalogue binding and destination capabilities, removes broker permissions, closes live connections, rotates the credential and verifies old-certificate rejection. Human sessions are treated separately unless evidence links them.

INCIDENT ACTIONS 45-BB

Incident	Immediate action	Recovery proof
workload key suspected	disable binding + isolate service	old cert rejected
root trust compromised	stop affected boundary + replace hierarchy	new root/digest only
broker user over-permitted	reduce permissions + close sessions	foreign operations refused
BFF compromised	disable BFF workload + revoke sessions as scoped	two-principal tests pass
policy corrupted	fail closed + restore revision	digest and matrix match

DO NOT ERASE HISTORY

Evidence records remain append-only with safe references. Credential removal never deletes the audit trail needed to reconstruct the incident.

OPERATIONAL OWNERSHIP

Every trust artifact has one accountable owner

The Phase-0 catalogue names ownership for service identity, destination policy, broker permissions, trust roots, certificate source, evidence retention and incident response. A service team cannot silently change another owner's policy.

OWNERSHIP 45-BC

Artifact	Owner	Review trigger
workload identity catalogue	platform/security	new/retired service role
destination capabilities	receiving service	endpoint/resource change
broker permissions	topology owner + service	route/queue change
trust roots/profile	security/platform	lifecycle or incident
private-key source	deployment owner	Chapter 47 control change
evidence manifest	delivery pipeline	every build/deployment

TWO-PARTY REVIEW

Changes that combine identity recognition and new authority require review by both trust and destination owners.

CAPACITY AND ABUSE

Handshake and validation work have explicit limits

TLS handshakes, chain building, revocation checks and authorization resolution can be abused. Listeners enforce connection limits, handshake timeouts and bounded caches while keeping denial fail-closed. Metrics avoid attacker-controlled high-cardinality labels.

ABUSE CONTROLS 45-BD

Surface	Control
TLS handshake	connection/rate/time limits
certificate size/chain	profile and depth limits
revocation	bounded cache + freshness + bulkhead
policy lookup	immutable local snapshot
broker reconnect	backoff + circuit + jitter
denial telemetry	sampling with guaranteed counters

NO CACHE-CREATED TRUST

A cache key includes identity, trust revision and policy revision; stale positive decisions are not reused across configuration changes.

RESIDUAL RISKS

Known limitations remain first-class deliverables

The Phase-0 mechanism authenticates named service roles but does not attest a particular binary, node or process instance. A stolen private key can impersonate the role until containment completes. Certificate lifecycle, production CA design and advanced workload attestation remain outside this chapter's implemented claim.

RESIDUAL-RISK REGISTER 45-BE

Risk	Current treatment	Next owner
role-level identity	dedicated cert + constrained policy	future workload platform
private-key theft	least authority + incident disable	Chapter 47
rotation/session lag	overlap/drain evidence	Chapter 47
claim over-forwarding	minimum workload claims	Chapter 46
evidence incompleteness	PENDING manifest	Chapter 48
production maturity	explicitly not claimed	security program

HONEST LIMIT

mTLS is a mechanism. Production identity maturity also requires issuance governance, protected keys, rotation, revocation, deployment isolation, monitoring and tested incident recovery.

REJECTED ALTERNATIVES

Convenient shortcuts fail the trust model

The decision record keeps rejected designs visible so they do not return through implementation convenience. A future alternative must satisfy the same proof obligations and receive its own ADR.

REJECTED CHOICES 45-BF

Choice	Why rejected for Phase-0
shared service token	no caller separation; replay blast radius
forward user access token	human identity confused with workload trust
internal network allow	location is not identity or authorization
single wildcard certificate	all services impersonate each other
CN-only HTTP identity	weak/ambiguous profile and fallback behavior
self-signed allow-list only	poor chain/lifecycle structure for this demonstrator
service mesh claim	not deployed or evidenced

ALTERNATIVES ARE NOT UNIVERSALLY BAD

The rejection is scoped to this Phase-0 architecture, project goals and evidence model, not a universal protocol verdict.

DEFINITION OF DONE

Completion means refusal paths and limits are executable

The chapter is complete as a specification when identity grammar, profile, host composition, local policy, BFF dual-principal gate, broker mapping, permissions, failure semantics, tests, fault laboratory and evidence schema agree. Runtime status remains PENDING until artifacts exist.

COMPLETION CHECKLIST 45-BG

Area	Required artifact
decision	ADR-045 + canonical digest
HTTP	direct-mTLS host + certificate/capability tests
BFF	valid-human/missing-workload zero-effect test
RabbitMQ	EXTERNAL deployment + real permission matrix
rotation	old/new/drain/retire laboratory
operations	readiness, rollback, incident runbook
evidence	fail-closed manifest and validator

STATUS

SPECIFIED | RUNTIME PENDING. Production security maturity is NOT CLAIMED.

IMPLEMENTATION SEQUENCE

Build authentication, refusal and local authorization in that order

The sequence keeps a partial implementation from creating a permissive boundary. Each stage adds a gate and preserves all previous negative tests.

IMPLEMENTATION PLAN 45-BH

Order	Deliverable
1	ADR, identity grammar, catalogue and certificate profile
2	ephemeral CA, parser and chain corpus
3	direct-mTLS listener and certificate handler
4	destination capability policy and route inventory
5	typed outgoing mTLS clients and rotation overlap
6	real BFF workload gate and dual-principal suite
7	RabbitMQ EXTERNAL identities and least permissions
8	fault, recovery, evidence and deployment gates

FIRST RELEASE BLOCKER

Before any positive owner call is accepted, the user-token-only and wrong-workload tests must prove zero effects.

CHAPTER 46 HANDOFF

The next chapter minimizes context and maps policy locally

Chapter 45 authenticates the calling workload and supplies a minimal typed principal. Chapter 46 decides which bounded identity context may cross each boundary, maps external claims to local policy inputs and prevents any one service from becoming the authorization database for all others.

HANDOFF 45-BI

Chapter 45 leaves	Chapter 46 must provide
canonical WorkloadId	minimal forwarding contract
certificate authentication method	claim allow-list and normalization
destination capability input	external-to-local policy mapping
separate human/workload principals	delegation context minimization
local destination ownership	policy ownership and cache boundaries
safe decision reference	cross-boundary evidence mapping

PROTECTED HANDOFF

Chapter 46 may minimize and map context; it may not turn authenticated claims into a global authorization database or weaken destination ownership.

CHAPTER 47 HANDOFF

Credential lifecycle remains an explicit unfinished obligation

The mechanism assumes a certificate source and trust material with known identity, freshness and revision. Chapter 47 must implement secure loading, storage, file/secret permissions, redaction, issuance, overlap, rotation, revocation, recovery and environment-specific configuration.

LIFECYCLE OBLIGATIONS 45-BJ

Required from Chapter 47	Chapter 45 consumer
protected private-key custody	HTTP/AMQP client adapters
server and client certificate issuance	Kestrel and typed clients
trust-root distribution	custom chain validation/broker
atomic reload/generation	handler and connection rotation
expiry/revocation alerting	readiness and incident runbook
secret-free evidence	manifest validator

NO HIDDEN COMPLETION

Until Chapter 47 artifacts exist, certificate lifecycle and production key protection remain PENDING.

CHAPTER 48 HANDOFF

Security evidence must exercise the real distributed seams

Chapter 48 collects the HTTP, broker, browser, policy, rotation, fault and zero-effect suites into retained audit evidence. It validates provenance and freshness rather than accepting narrative summaries.

EVIDENCE OBLIGATIONS 45-BK

Claim	Required execution
user token is not workload trust	real mTLS owner negative test
wrong service denied	valid foreign cert + zero effects
broker least privilege	real EXTERNAL permission matrix
rotation completes	new accepted; old reconnect rejected
faults fail closed	trust/revocation/key/broker laboratory
no production overclaim	evidence claim validator

EVIDENCE IS PART OF THE ARCHITECTURE

A security boundary without reproducible negative and recovery evidence remains an intention, not a demonstrated runtime property.

OFFICIAL TECHNICAL ANCHORS

Platform and protocol behavior is grounded in maintained sources

ASP.NET Core certificate authentication:

<https://learn.microsoft.com/en-us/aspnet/core/security/authentication/certauth?view=aspnetcore-8.0>

HttpClientFactory guidance: <https://learn.microsoft.com/en-us/dotnet/core/extensions/httpclient-factory>

HttpClientHandler client certificates:

<https://learn.microsoft.com/en-us/dotnet/api/system.net.http.httpclienthandler.clientcertificates?view=net-8.0>

RabbitMQ TLS: <https://www.rabbitmq.com/docs/ssl>

RabbitMQ access control: <https://www.rabbitmq.com/docs/access-control>

RabbitMQ AMQP extensions: <https://www.rabbitmq.com/docs/extensions>

RabbitMQ publishers and connections: <https://www.rabbitmq.com/docs/publishers> and

<https://www.rabbitmq.com/docs/connections>

X.509 certificate and CRL profile: <https://www.rfc-editor.org/rfc/rfc5280>

Zero Trust Architecture: <https://csrc.nist.gov/pubs/sp/800/207/final>

SPIFFE overview for explicit future comparison, not implemented conformance: <https://spiffe.io/docs/latest/>

READING RULE

These anchors constrain platform behavior and terminology. They do not prove that the NEXUS-1 runtime has executed the specified controls.

WHAT THIS CHAPTER HAS PROVED

A complete specification with bounded runtime claims

Chapter 45 specifies ADR-045, the workload identity grammar and certificate profile, custom-root direct mTLS, strict URI-SAN mapping, separate identity and capability catalogues, destination authorization, typed outgoing clients, the real BFF workload gate, dual-principal construction, RabbitMQ EXTERNAL mapping and least permissions, failure semantics, zero-effect tests, certificate and policy corpora, broker component tests, rotation and fault laboratories, safe telemetry, evidence manifests, deployment, rollback and incident procedures.

CLAIM STATUS 45-BL

Claim	Status
mechanism and refusal path specified	SPECIFIED
compile-oriented implementation assets written	SPECIFIED
runtime HTTP and broker evidence	PENDING
certificate lifecycle and custody	PENDING - Chapter 47
complete audit evidence	PENDING - Chapter 48
production security maturity	NOT CLAIMED

CENTRAL PROOF

A user token alone remains insufficient. Only an authenticated, locally authorized workload - plus human context where required - can reach an owner effect.

CHAPTER CHECKPOINT

Review the trust boundary before moving on

- What exactly does an accepted client certificate prove, and what does it not prove?
- Why is the URI SAN authoritative for HTTP while common name is rejected as a fallback?
- Why are identity recognition and destination capability stored separately?
- Which test proves that a valid user token is not service trust?
- Why can a valid RootCause certificate still receive 403 at AlarmManagement?
- What does RabbitMQ EXTERNAL authenticate, and what semantic checks remain in the consumer?
- Why must broker users, vhosts and resource permissions remain service-specific?
- How does handler/session reuse affect certificate rotation and emergency removal?
- Which negative tests prove zero Domain, database, outbox and broker effects?
- Which lifecycle and audit claims remain PENDING for Chapters 47 and 48?

NEXT - CHAPTER 46

Claims Minimization and Policy Mapping will forward only required identity context, translate external facts into local policy inputs and preserve each service as the authority for its own authorization decisions.

PART VIII - SECURITY AND TRUST ACROSS SERVICES

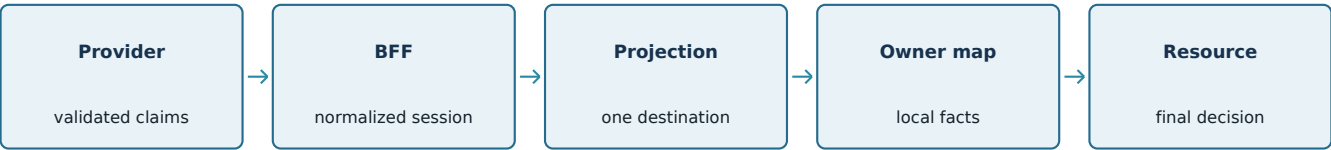
46. Claims Minimization and Policy Mapping

External identity claims stop at the BFF, every downstream context is reduced to one destination and one intent, and each owner service maps those facts to its own policy and current resource without a central authorization database.

CHAPTER DECISION

Keep exact provider claims inside one validated identity adapter. Store a normalized server-side session for edge policy, derive an audience-pseudonymous subject reference, project only one requested intent and scope, and let the receiving owner map the approved facts locally. Raw tokens, cookies, claim bags, global role lists and precomputed allow decisions never cross the boundary.

PROOF PATH 46-A - BREADTH DECREASES WHILE AUTHORITY STAYS LOCAL



GUARANTEE LEDGER 46-A

Boundary	Carries	Allowed claim	Still forbidden
IdP -> BFF	validated claims	authenticated subject facts	owner permission
BFF session	normalized edge state	edge eligibility	portable authority
BFF -> owner	minimal context	decision input	raw token/claim bag
owner	local policy + resource	one allow/deny	global authorization DB

CHAPTER BOUNDARY

Minimize first; map locally; decide against the resource

Chapter 46 completes the claims seam left by Chapters 44 and 45. External OpenID Connect claims are admitted and normalized only at the BFF identity boundary. The BFF stores a server-side capability snapshot for its own edge decisions, then projects a separate minimal context for one destination and one intent. The owner maps that context to local policy and current resource state.

DECISION LEDGER 46-1

Input	Local meaning	Forbidden inference
external claims	authenticated facts at the BFF	portable owner permission
edge snapshot	BFF-local early eligibility	final owner allow
delegated context	minimal decision input for one owner	raw identity bag
owner policy	resource-specific allow or deny	central/global decision
evidence	bounded decision references	credential or PII archive

NON-NEGOTIABLE INVARIANT

No service receives a raw browser token, cookie or general claim bag, and no caller may forward a precomputed allowed=true decision.

DECISION RECORD

ADR-046 - Context is destination-minimal and authorization remains owner-local

The decision chooses exact provider profiles, issuer-plus-subject identity, unmapped claim names at the protocol edge, server-side normalized sessions, per-destination forwarding profiles, audience-pseudonymous subject references and immutable local owner policies. It rejects a central authorization database and any transitive edge allow.

DECISION LEDGER 46-2

Input	Local meaning	Forbidden inference
claim namespace	exact external names	framework alias guesses
stable subject	case-sensitive iss + sub	email/display name
session	normalized server-owned facts	raw token persistence
forwarding	one audience + one intent	all roles/groups/scopes
owner decision	local mapping + resource	BFF allow inheritance
policy storage	per-owner versioned snapshot	shared authorization DB

DECISION IDENTITY

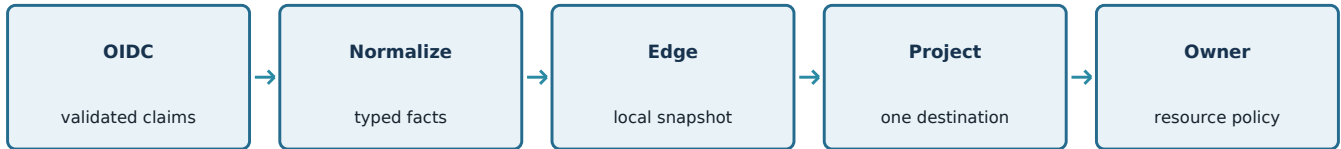
ADR-046 canonical SHA-256: 7ba53263d364b03da55a8836903a64ee87d4d6af0eef46283b3808cbeb7fe29a

TRUST FLOW

Identity data loses breadth at every boundary

Each step emits a smaller, more local type. The external claim bag never crosses the identity adapter. The edge snapshot never crosses the BFF. The delegated context contains only facts required by one owner operation, and the owner decision never travels back as reusable authority.

PROOF PATH 46-A - EXTERNAL IDENTITY TO OWNER DECISION



PROOF OBLIGATIONS 46-A

Transition	Must remove	Must preserve
token -> subject	signature bytes, unused claims	issuer, subject, auth evidence
subject -> session	provider syntax	normalized identity revision
session -> owner	global capability/scope sets	one requested intent/scope
context -> decision	external-policy authority	decision/evidence references

MONOTONIC MINIMIZATION

A downstream representation may be equally narrow or narrower. It may never reconstruct discarded external claims from logs, caches or directory lookups.

DECISION OWNERSHIP

Authentication facts, edge eligibility and owner permission have different owners

The design prevents convenience from moving business authorization into the identity provider or BFF. The provider proves an authentication event. The BFF decides whether its own route is eligible. The receiving service decides whether the exact operation is allowed on the exact resource now.

DECISION LEDGER 46-3

Input	Local meaning	Forbidden inference
identity provider	authentication event and issued claims	NEXUS-1 resource decision
BFF identity adapter	provider-specific normalization	owner capability semantics
BFF edge policy	browser route eligibility	owner final permission
owner Edge	context admission	Domain state mutation
owner Application	resource policy and operation	external token validation
Audit	decision evidence	authorization oracle

NO TRANSITIVE ALLOW

An allow at the provider or BFF is only an input to the next boundary. The owner can still deny, hide, conflict or require fresher authentication.

VOCABULARY

A claim is an asserted fact, not an instruction

ASP.NET Core guidance describes a claim as a name-value statement about a subject, not what the subject may do. NEXUS-1 preserves that distinction in type names: external claims become normalized facts; policy evaluation produces a local authorization decision.

DECISION LEDGER 46-4

Input	Local meaning	Forbidden inference
ExternalClaim	issuer-asserted name/value	owner allow
NormalizedIdentityFact	validated provider-neutral fact	resource scope
ExternalEntitlementRef	opaque workforce entitlement input	local capability
RequestedIntent	operation being attempted	granted authority
OwnerPolicyFact	owner-local mapped meaning	global role
AuthorizationDecision	one resource-time result	reusable credential

LANGUAGE DISCIPLINE

Types named RoleClaim, IsInternal or GrantedCapability at the external boundary are rejected because they collapse fact, mapping and decision.

THREAT BOUNDARY

Over-forwarding creates authority, privacy and drift failures

A broad claim bag allows downstream services to invent incompatible mappings, exposes identity data they do not need and encourages a central mapping service. A precomputed capability list hides policy ownership and remains valid after the destination or resource has changed.

DECISION LEDGER 46-5

Input	Local meaning	Forbidden inference
raw token forwarding	credential replay and audience confusion	tokens stay in BFF
all groups/roles	data exposure and accidental mapping	destination allow-list
email as key	rename/reassignment ambiguity	issuer + subject
edge allowed=true	transitive authorization	requested intent only
central policy lookup	runtime coupling and global blast radius	local snapshots
stale mapping cache	removed privilege survives	revision/expiry/fail closed

THREAT RESULT

Every protected boundary must be safe when a claim is missing, duplicated, mistyped, newly introduced, removed or semantically redefined.

WHAT MINIMIZATION MEANS

Minimal is defined per decision, not per organization

A claim may be legitimate and still unnecessary for a particular destination. Minimization is an explicit profile that answers which facts are required for this operation, why each is needed, how long it survives and whether it may appear in a response, log or event.

DECISION LEDGER 46-6

Input	Local meaning	Forbidden inference
subject reference	join local grants/evidence	one audience only
authentication time	freshness policy	bounded timestamp
assurance class	privileged-operation gate	mapped profile value
entitlement reference	local policy input	destination allow-list
requested scope	load target resource	one coordinate
display name	UI display	never sent to owner policy

COUNTERFACTUAL TEST

If removing a field cannot change admission, mapping, authorization, evidence or required display behavior, the field does not belong in that contract.

BOUNDARY INVENTORY

Each boundary has its own allowed identity surface

The inventory makes data movement reviewable. It distinguishes token claims, server-side session facts, delegated owner context, pure workload calls, public events, UI summaries and security evidence.

DECISION LEDGER 46-7

Input	Local meaning	Forbidden inference
IdP -> BFF	validated OIDC claims required by provider profile	no owner resources
BFF session	normalized subject, auth and edge snapshot	no raw token
BFF -> owner	destination context	no cookie, display data or broad sets
service -> service	workload principal; human only when delegated	no synthetic user
event	domain/public provenance	no authorization claim bag
Angular	safe session summary	no tokens, groups or owner grants
evidence	references/revisions/result	no credential

INVENTORY GATE

A new field or destination requires a reason, owner, retention class and negative leak test before it can enter the profile.

PROVIDER TRUST**Token validation completes before claim interpretation**

Claim mapping operates only on an authentication result from the configured OpenID Connect scheme. Issuer, audience, signature, time, nonce, correlation and code-flow protections belong to Chapter 44's authentication boundary. A JSON object that merely contains iss and sub is not an authenticated subject.

DECISION LEDGER 46-8

Input	Local meaning	Forbidden inference
signature	accepted configured keys/algorithms	claim truth by itself
issuer	exact configured identifier	domain suffix match
audience	BFF client identifier	owner service audience
time	token valid at validation clock	session forever
nonce/correlation	login response bound to request	authorization grant
scheme	OIDC authentication identity	workload identity

ORDERING RULE

No provider-specific mapping, subject projection or capability resolution runs on an unvalidated token or user-supplied claim payload.

EXACT CLAIM NAMESPACE**Disable implicit inbound remapping**

Phase-0 keeps the issuer's exact claim names so the provider profile can validate them deterministically. ASP.NET Core's per-scheme MapInboundClaims switch is disabled; display NameClaimType is configured deliberately and role membership is not used for owner authorization.

EXECUTABLE ASSET 46-A - EXPLICIT OIDC CLAIM NAMESPACE

```
services.AddAuthentication()
    .AddOpenIdConnect("nexus-oidc", options =>
    {
        options.Authority = identity.Authority;
        options.ClientId = identity.ClientId;
        options.ResponseType = OpenIdConnectResponseType.Code;
        options.UsePkce = true;
        options.RequireHttpsMetadata = true;
        options.MapInboundClaims = false;
        options.SaveTokens = false;
        options.TokenValidationParameters.NameClaimType = "name";
        options.TokenValidationParameters.RoleClaimType = "nexus:unused-role";
    });
```

NO ROLE SHORTCUT

The deliberately unused RoleClaimType prevents IsInRole from becoming the owner authorization path. Local policies consume typed mapping results.

REQUESTED SCOPES

The BFF asks the provider only for required identity data

The OIDC client starts from openid and adds only scopes required by the Phase-0 UI and mapping profile. Offline access is absent. UserInfo is disabled unless a named profile requires claims unavailable in the ID token and its additional network/privacy cost is accepted.

DECISION LEDGER 46-9

Input	Local meaning	Forbidden inference
openid	authentication protocol	always required
profile	bounded display name if UI needs it	never owner authority
email	not requested for authorization	not stable identity
groups/entitlements	provider-specific minimum	only if mapping needs them
offline_access	not requested	no refresh token in browser/session
UserInfo	explicit profile-only exception	not automatic enrichment

SCOPE IS NOT PERMISSION

Requesting an OIDC scope controls what the BFF may receive. It does not grant a NEXUS-1 owner operation.

EXTERNAL SUBJECT

Identity is the exact issuer-plus-subject pair

OpenID Connect specifies that iss and sub together are the stable unique identifier an RP can rely on. Both are case-sensitive. NEXUS-1 stores the canonical issuer URI and exact subject string; it never keys identity by email, name, group or tenant display label.

EXECUTABLE ASSET 46-B - STABLE EXTERNAL SUBJECT

```
public readonly record struct ExternalSubjectKey(Uri Issuer, string Subject)
{
    public static ExternalSubjectKey Create(string issuer, string subject)
    {
        ArgumentException.ThrowIfNullOrWhiteSpace(issuer);
        ArgumentException.ThrowIfNullOrWhiteSpace(subject);

        var uri = new Uri(issuer, UriKind.Absolute);
        if (uri.Scheme != Uri.UriSchemeHttps || uri.Query.Length != 0 ||
            uri.Fragment.Length != 0 || subject.Length > 255)
            throw ExternalIdentityRejected.InvalidSubjectKey();

        return new(uri, subject); // preserve case exactly
    }
}
```

NO NORMALIZATION GUESS

The adapter does not lowercase, trim, Unicode-normalize or decode sub. Any provider-specific canonicalization must happen at the issuer and be covered by its profile evidence.

DISPLAY CLAIMS

Email and name are presentation data, never identity keys

Display data changes, can be absent and may be reassigned. The BFF may retain a bounded display name for the Angular session summary, but it does not use email or name to join entitlements, derive the owner subject reference or authorize resources.

DECISION LEDGER 46-10

Input	Local meaning	Forbidden inference
sub changes?	new external subject	same person by matching email
email changes	display/contact update	new authorization identity
email reused	unrelated subject possible	inherit previous grants
name missing	fallback safe display label	authentication failure
name contains markup	encode for UI	trusted HTML
locale claim	optional display preference	policy input

IDENTITY TEST

Changing name and email while preserving validated iss+sub must not change any edge or owner authorization decision.

PROVIDER PROFILE

Every issuer has an explicit claim contract

A provider profile defines exact claim names, JSON shapes, required/optional status, allowed values, maximum counts and the mapping version. The BFF refuses startup when two enabled profiles claim the same issuer or when a policy references an unknown claim.

CONFIGURATION ASSET 46-A - PROVIDER PROFILE

```
public sealed record ExternalClaimProfile(  
    string ProfileId,  
    Uri Issuer,  
    string SubjectClaim,  
    string AuthenticationTimeClaim,  
    string AssuranceClaim,  
    string EntitlementsClaim,  
    ClaimCardinality EntitlementsCardinality,  
    IReadOnlySet<string> RequiredClaims,  
    IReadOnlySet<string> ForwardingForbiddenClaims,  
    string Revision,  
    string CanonicalDigest);
```

CONFIGURATION IS EXECUTABLE

Profile validation fails closed before the login route becomes ready. It is not an informal list in documentation.

CLAIM CARDINALITY

Scalar, array, duplicate and null are different inputs

The strict reader validates the provider profile's expected JSON shape. A scalar group value is not silently wrapped into an array, duplicate singleton claims are not resolved by first-wins, and null is not the empty set.

DECISION LEDGER 46-11

Input	Local meaning	Forbidden inference
one required scalar	exactly one non-empty string	first/last wins
optional scalar absent	None	empty string
array	bounded distinct strings	CSV parsing
duplicate singleton	ambiguous; reject mapping	pick one
null	invalid when required	missing/empty equivalence
object/number	type mismatch	ToString conversion

SHAPE IS MEANING

A provider change from array to scalar is drift and must fail the mapping contract rather than change permissions silently.

STRICT CLAIM READER

Mapping reads exact types and bounded collections

The reader hides ClaimsPrincipal quirks behind a small adapter. It verifies issuer, authentication scheme and cardinality, caps collection size and returns immutable values. Policy code never calls FindFirst for an authorization-critical claim.

EXECUTABLE ASSET 46-C - STRICT CLAIM ACCESS

```
public interface IExternalClaimReader
{
    string RequireSingleString(string claimType, int maximumLength);
    Instant RequireSingleInstant(string claimType);
    IReadOnlySet<string> ReadDistinctStringSet(
        string claimType, int maximumCount, int maximumItemLength);
}

public sealed record AuthenticatedExternalSubject(
    ExternalSubjectKey Key,
    Instant AuthenticatedAt,
    AssuranceClass Assurance,
    IReadOnlySet<ExternalEntitlementRef> Entitlements,
    string ProviderProfileId,
    string ProviderProfileRevision);
```

NO CLAIMSPRINCIPAL LEAKAGE

Only the identity adapter sees the framework principal. All downstream code receives the closed authenticated subject type.

STRING SEMANTICS

Case, whitespace and Unicode rules are claim-specific

The adapter never applies blanket Trim, lowercase or Unicode normalization. The profile defines whether a value is an opaque identifier, a URI, a registered code or display text. Authorization identifiers use ordinal exact comparison unless their issuing contract states otherwise.

DECISION LEDGER 46-12

Input	Local meaning	Forbidden inference
iss/sub	case-sensitive opaque/protocol values	case-folding
entitlement code	profile-defined ordinal code	culture comparison
URI	absolute shape + exact registered issuer	suffix match
display name	encoded presentation	policy key
whitespace	invalid for identifiers	automatic trim
Unicode confusable	reject outside profile alphabet	visual equality

ADVERSARIAL CORPUS

Tests include leading/trailing spaces, Turkish-I casing, composed/decomposed Unicode, look-alike characters and percent-encoded variants.

UNKNOWN CLAIMS

Unknown input grants nothing and is retained only as drift metadata

An identity provider may add unrelated claims without breaking authentication. The adapter projects only the allow-list. Unknown claim names are counted and compared to the profile for drift, but never forwarded or mapped. Unknown values inside an authorization-critical entitlement claim grant nothing and may fail privileged mapping by policy.

DECISION LEDGER 46-13

Input	Local meaning	Forbidden inference
unknown claim name	ignore + drift observation	forward by default
known claim, unknown value	no mapping	generic role
required claim missing	mapping failure	empty/default grant
known claim wrong type	mapping failure	coerce
forbidden claim present	profile violation	persist
extra display claim	ignore	owner context

COMPATIBILITY WITHOUT AUTHORITY

Adding an irrelevant claim should not change decisions; changing a required claim's shape or meaning must not pass silently.

EXTERNAL CLAIM BAG

Raw claims stop inside the identity adapter

The adapter converts the framework principal into an authenticated external subject and then discards the bag. It records only safe schema observations and the provider/profile revision. No generic dictionary is stored in the session or accepted by policy APIs.

EXECUTABLE ASSET 46-D - BOUNDARY PROJECTOR

```
public sealed class ExternalSubjectProjector(
    IExternalClaimReaderFactory readers,
    IExternalClaimProfileStore profiles,
    IClaimDriftRecorder drift)
{
    public AuthenticatedExternalSubject Project(ClaimsPrincipal principal)
    {
        var profile = profiles.RequireFor(principal);
        var reader = readers.CreateValidated(principal, profile);
        drift.Record(reader.CompareShape(profile));

        return new AuthenticatedExternalSubject(
            ExternalSubjectKey.Create(profile.Issuer.AbsoluteUri,
                reader.RequireSingleString(profile.SubjectClaim, 255)),
            reader.RequireSingleInstant(profile.AuthenticationTimeClaim),
            profile.MapAssurance(reader),
            profile.MapEntitlements(reader),
            profile.ProfileId,
            profile.Revision);
    }
}
```

LIFETIME BOUNDARY

ClaimsPrincipal and tokens are out of scope immediately after projection. Route handlers, sessions and owners cannot query the original bag later.

TOKEN RETENTION

Authentication tokens are not session claims

Chapter 44's server-side session stores a revocable reference and normalized identity state. Chapter 46 makes the exclusion executable: access_token, id_token, refresh_token, authorization_code and raw cookie values are forbidden fields in session, cache, telemetry and owner contracts.

DECISION LEDGER 46-14

Input	Local meaning	Forbidden inference
ID token	login validation input	session serialization
access token	provider resource only if explicitly required	owner API credential
refresh token	not requested/stored in Phase-0	long-lived session
cookie	opaque browser session reference	downstream header
authorization code	one-time callback input	audit/log
token hash	not authorization identity	casual correlation label

LEAK SCANNER

Serialized sessions and outbound requests are scanned for token-shaped fields and known claim-bag keys, not only literal token values.

SESSION PROJECTION

The server-side session stores normalized facts and edge-local policy

A successful login creates a minimal session state containing the external subject key, display projection, authentication evidence, identity revision and an edge capability snapshot. Provider syntax and owner-specific resource decisions remain outside it.

EXECUTABLE ASSET 46-E - NORMALIZED SERVER SESSION

```
public sealed record OperatorSessionStateV2(  
    SessionReference Session,  
    ExternalSubjectKey ExternalSubject,  
    SafeDisplayName DisplayName,  
    Instant AuthenticatedAt,  
    AssuranceClass Assurance,  
    EdgeCapabilitySnapshot EdgeCapabilities,  
    long IdentityRevision,  
    string ProviderProfileRevision,  
    string EdgePolicyRevision,  
    Instant ExpiresAt,  
    bool Revoked);
```

BFF-LOCAL ONLY

OperatorSessionStateV2 is never serialized to Angular or an owner request. Each boundary receives a smaller projection.

DISPLAY PROJECTION

Angular still receives a safe summary, not authorization material

The UI contract from Chapter 44 remains separate. It can expose authenticated state, encoded display name and coarse feature visibility. It does not expose external issuer, subject, groups, entitlements, policy rules, site/tenant grant sets or owner decision reasons.

DECISION LEDGER 46-15

Input	Local meaning	Forbidden inference
authenticated	render session state	owner authorization
displayName	encoded UI label	identity key
expiresAt	session UX	owner context lifetime
feature visibility	navigation hint	protected endpoint allow
site picker options	bounded UX projection	resource permission
denial detail	safe generic message	policy enumeration

UI IS ADVISORY

Hiding a button is not an authorization control. Showing a button is not evidence that the owner will allow the operation.

CLAIMS TRANSFORMATION

Framework transformations are idempotent and side-effect free

ASP.NET Core may invoke `IClaimsTransformation` multiple times. If the BFF uses it for safe display normalization, the transformation must add a marker once, perform no remote policy lookup and never grant owner capabilities. Security-critical projection remains an explicit login/session step.

EXECUTABLE ASSET 46-F - IDEMPOTENT TRANSFORMATION

```
public sealed class SafeDisplayClaimsTransformation : IClaimsTransformation
{
    public Task<ClaimsPrincipal> TransformAsync(ClaimsPrincipal principal)
    {
        if (principal.HasClaim("nexus:display-projected", "1"))
            return Task.FromResult(principal);

        var identity = new ClaimsIdentity("nexus-display");
        identity.AddClaim(new Claim("nexus:display-name",
            SafeDisplayName.From(principal).Value));
        identity.AddClaim(new Claim("nexus:display-projected", "1"));
        principal.AddIdentity(identity);
        return Task.FromResult(principal);
    }
}
```

NO HIDDEN POLICY CALL

A claims transformation cannot become a synchronous authorization database client. Repeated execution must produce the same bounded principal.

CAPABILITY RESOLVER SEAM

Chapter 44's resolver now has a deterministic implementation

The resolver maps an authenticated external subject to BFF-local capabilities and coarse server-owned scope. It consumes one immutable mapping snapshot, returns a revision and validity bound, and never calls owner databases.

EXECUTABLE ASSET 46-G - EDGE RESOLVER

```
public sealed class OperatorCapabilityResolver(
    IEdgeMappingSnapshotSource snapshots,
    IClock clock) : IOperatorCapabilityResolver
{
    public Task<CapabilitySnapshot> ResolveAsync(
        AuthenticatedExternalSubject subject,
        CancellationToken cancellationToken)
    {
        var snapshot = snapshots.Current();
        var mapped = snapshot.Map(subject);
        if (!mapped.IsAccepted || mapped.ValidUntil <= clock.GetCurrentInstant())
            throw EdgeMappingUnavailable.From(mapped.SafeCode);
        return Task.FromResult(mapped.CapabilitySnapshot!);
    }
}
```

BOUNDED RESPONSIBILITY

The result permits or filters BFF routes. It is not a portable authorization verdict and contains no owner resource truth.

CLOSED MAPPING OUTCOME

Mapping distinguishes absence, ambiguity, denial and unavailability

A closed result family prevents exceptions and empty collections from acquiring accidental meaning. The adapter can distinguish an authenticated subject with no mapped edge capability from a malformed profile or unavailable policy snapshot.

DECISION LEDGER 46-16

Input	Local meaning	Forbidden inference
Mapped	validated snapshot with revision/expiry	owner allow
NoEntitlement	authenticated but edge-ineligible	anonymous
Ambiguous	duplicate/conflicting critical values	pick strongest
ProfileMismatch	shape or issuer contract changed	best effort
PolicyStale	snapshot outside freshness	reuse indefinitely
Unavailable	mapping dependency not ready	empty allow

FAIL-CLOSED TRANSPORT

Only Mapped creates a protected BFF session. All other outcomes produce bounded refusal or unavailability and zero business effects.

EXTERNAL ENTITLEMENTS

Opaque provider values are inputs, not NEXUS-1 capabilities

An entitlement value such as a directory group object ID is retained as an opaque ExternalEntitlementRef. The BFF mapping snapshot may translate it to an edge capability. Owners that need the fact receive only their approved subset and map it independently to local policy facts.

DECISION LEDGER 46-17

Input	Local meaning	Forbidden inference
external group ID	opaque entitlement reference	human-readable role
group display name	not used	stable identifier
edge mapping	entitlement -> BFF route capability	owner grant
owner mapping	approved ref -> owner policy fact	global permission
unknown ref	no mapped meaning	default operator
removed ref	new snapshot removes eligibility	cached forever

NO SEMANTIC LAUNDERING

Renaming an external group to Administrator does not give it authority. Only a reviewed versioned mapping creates local meaning.

EDGE MAPPING SNAPSHOT

The BFF policy is immutable for one decision

The edge snapshot contains only BFF route capabilities and coarse scopes derived from reviewed entitlement mappings. It has a canonical digest, effective interval and source revision. Every login/mapping decision records that identity.

EXECUTABLE ASSET 46-H - IMMUTABLE EDGE POLICY

```
public sealed record EdgeMappingSnapshot(  
    string Revision,  
    string CanonicalSha256,  
    Instant EffectiveFrom,  
    Instant ValidUntil,  
    IReadOnlyDictionary<ExternalEntitlementRef,  
        IReadOnlySet<EdgeCapability>> CapabilityMap,  
    IReadOnlyDictionary<ExternalEntitlementRef,  
        IReadOnlySet<ScopeCoordinate>> ScopeMap)  
{  
    public EdgeMappingResult Map(AuthenticatedExternalSubject subject) =>  
        EdgeMapper.MapExact(this, subject);  
}
```

NO MUTABLE DICTIONARY EXPOSURE

The loaded snapshot is validated and frozen before publication. A request never observes a half-reloaded policy.

SCOPE MAPPING

External scope is intersected, never expanded

Provider entitlements may identify organizational scope, but the BFF converts them into typed site or tenant coordinates through reviewed maps. A browser-selected site can narrow that set. The owner still compares the single requested coordinate with its loaded resource.

DECISION LEDGER 46-18

Input	Local meaning	Forbidden inference
external scope ref	input to edge map	owner resource coordinate
edge site set	BFF early filter	final grant
browser site	requested target	membership proof
delegated scope	one requested coordinate	all authorized sites
owner resource site	authoritative local coordinate	client assertion
intersection empty	deny before owner call	fallback wider scope

NARROWING LAW

requested scope must be in the current edge snapshot and must still match the owner's current resource and local grant.

POLICY CACHE

Caching preserves revision, expiry and revocation semantics

The BFF caches normalized mapping results by external subject key, identity revision and edge policy revision. It never keys on email or raw token. Privileged operations require current mapping and authentication freshness rather than accepting a stale positive cache entry.

DECISION LEDGER 46-19

Input	Local meaning	Forbidden inference
cache key	iss+sub + revisions	email/name
cache value	minimal edge snapshot	raw claims/token
TTL	no later than policy/identity/session expiry	sliding forever
revocation	invalidates session/cache	wait for TTL
policy change	new revision misses old key	mutate in place
lookup failure	bounded unavailable	reuse stale privileged allow

FRESHNESS BELONGS TO POLICY

A cache library's expiration behavior cannot decide how long an authorization fact remains acceptable.

DRIFT EVIDENCE

Provider changes are detected before they change authority

A value-free claim-shape manifest records the issuer profile, required claim presence, JSON type/cardinality classes, unknown-name count, mapped-value count and canonical profile digest. It excludes claim values and subject identifiers.

DECISION LEDGER 46-20

Input	Local meaning	Forbidden inference
claim added	drift observation; no forwarding	automatic mapping
claim removed	required-profile failure if needed	empty grant
type changed	mapping failure	coercion
new entitlement value	unknown-value evidence	generic role
profile revision changed	revalidation required	silent cache reuse
issuer metadata changed	Chapter 44 readiness gate	trust old/new blindly

VALUE-FREE EVIDENCE

Drift diagnostics describe schema and mapping coordinates, not the human's actual claims.

DESTINATION PROFILES

Every owner receives a different minimum

The BFF keeps a forwarding profile per destination and operation class. AlarmManagement may need one site coordinate and an operator entitlementment reference; Reporting may need a tenant and read intent; a pure workflow call needs no human context at all.

DECISION LEDGER 46-21

Input	Local meaning	Forbidden inference
AlarmManagement command	subject ref, auth time, assurance, one site, needed refs	tenant list/display name
RootCause review	subject ref, one case scope, reviewer refs	all edge capabilities
Compliance approve	subject ref, assurance, approval ref, separation data	email/groups
Reporting read	subject ref, one tenant/report scope	command entitlements
Repair execute	subject ref, freshness, approval/request refs	broad owner roles
Workflow automation	no human context	synthetic subject

DIFFERENT CONTRACTS BY REASON

Minimization is not one universal internal JWT. It is an explicit typed projection for the receiving boundary.

CHAPTER 44 REFINEMENT

The broad OperatorContextV1 becomes BFF-internal state

Chapter 44 intentionally used a broad working shape containing audience, site, tenant and capability sets. Chapter 46 keeps that information in the server-side snapshot but deprecates forwarding it. Owner requests use a new one-audience, one-intent DelegatedOperatorContextV1.

DECISION LEDGER 46-22

Input	Local meaning	Forbidden inference
Audiences set	keep in BFF policy	forward all
Capabilities set	keep for edge filtering	owner permission
Sites/Tenants sets	keep server-side	send entire membership
Issuer raw value	BFF identity key	send to every owner
SubjectReference	derive per audience	globally correlatable ID
single intent/scope	new delegated context	broad reusable token

COMPATIBLE CORRECTION

TrustedEdgeCall still contains workload and human principals. Its human member now uses the destination-minimal contract rather than the earlier broad projection.

DELEGATED CONTEXT

One immutable context describes one attempted owner operation

The context is a request-bound data contract carried over Chapter 45 direct mTLS. It contains a pairwise subject reference, one destination, one requested intent, one scope coordinate, required assurance/freshness facts, a minimal entitlement subset and policy/evidence references.

EXECUTABLE ASSET 46-I - DESTINATION-MINIMAL CONTEXT

```
public sealed record DelegatedOperatorContextV1(
    ContextId ContextId,
    AudienceSubjectReference Subject,
    OwnerService Audience,
    RequestedIntent Intent,
    ScopeCoordinate Scope,
    AssuranceClass Assurance,
    Instant AuthenticatedAt,
    IReadOnlySet<ExternalEntitlementRef> EntitlementRefs,
    long IdentityRevision,
    string EdgePolicyRevision,
    string ProjectionProfileRevision,
    Instant IssuedAt,
    Instant ExpiresAt,
    string EdgeDecisionReference);
```

NO GRANTED FLAG

The contract deliberately has no Allowed, Roles, Capabilities, IsAdmin, IsInternal or OwnerResourceIds field.

CONTEXT FIELD BUDGET

Every forwarded field has one destination-owned reason

The field ledger prevents convenient additions. The owner documents how each field affects admission, policy, resource lookup or protected evidence. Display-only or speculative values are rejected from the contract.

DECISION LEDGER 46-23

Input	Local meaning	Forbidden inference
ContextId	replay/evidence key	global session ID
Subject	join owner-local grant/reference	raw iss/sub
Audience	destination binding	audience set
Intent	requested operation	granted capability
Scope	one target coordinate	membership collection
Assurance/auth time	freshness/privilege gate	raw amr/acr bag
EntitlementRefs	approved local mapping inputs	all groups
revisions/decision ref	evidence/drift	policy contents

BUDGET ENFORCEMENT

A reflection-based contract test compares actual fields with the approved ledger and fails on any unreviewed addition.

PAIRWISE OWNER REFERENCE

Owners receive a non-reversible audience-specific subject reference

The BFF derives a deterministic reference from the validated external subject and owner audience using a protected audience key. Different owners receive different values, reducing cross-service correlation. Key custody and rotation are Chapter 47 work; Phase-0 tests use explicit test keys.

EXECUTABLE ASSET 46-J - PAIRWISE REFERENCE PORT

```
public interface IAudienceSubjectReferenceDeriver
{
    AudienceSubjectReference Derive(
        ExternalSubjectKey subject,
        OwnerService audience,
        ReferenceKeyRevision keyRevision);
}

public sealed record AudienceSubjectReference(
    string Value,
    OwnerService Audience,
    ReferenceKeyRevision KeyRevision);
```

BOUNDED CLAIM

This is a NEXUS-1 audience-pseudonymous reference inspired by correlation-minimizing identity design. The book does not claim OpenID Provider pairwise-sub conformance.

REFERENCE DERIVATION

Canonical bytes and key revision make pseudonymization testable

The adapter canonicalizes the issuer URI according to the registered profile, preserves the exact subject string, includes the target audience and versioned domain separator, and uses an approved keyed construction. Plain SHA-256 of iss+sub is rejected because it is enumerable and cross-service correlatable.

EXECUTABLE ASSET 46-K - DERIVATION ADAPTER

```
public AudienceSubjectReference Derive(
    ExternalSubjectKey subject,
    OwnerService audience,
    ReferenceKeyRevision revision)
{
    var key = keys.Require(revision, audience);
    var input = Encoding.UTF8.GetBytes(
        $"nexus1:operator-ref:v1\n{subject.Issuer.AbsoluteUri}\n" +
        $"{subject.Subject}\n{audience.Value}");
    var digest = HMACSHA256.HashData(key.Bytes.Span, input);
    return new(
        "opref:v1:" + WebEncoders.Base64UrlEncode(digest),
        audience,
        revision);
}
```

SECRET LIFECYCLE DEFERRED

Chapter 46 fixes the contract and test vectors. Chapter 47 must protect keys, rotate revisions and define overlap without exposing subject mappings.

PROJECTION PROFILE

A profile chooses the minimum entitlement subset

The destination profile declares the intent family, scope type, maximum context lifetime, minimum assurance and external entitlement references the owner knows how to map. Projection is an intersection with the current edge snapshot, never a union with caller input.

CONFIGURATION ASSET 46-B - DESTINATION PROJECTION

```
public sealed record DestinationProjectionProfile(  
    OwnerService Audience,  
    IReadOnlySet<RequestedIntent> Intents,  
    ScopeKind ScopeKind,  
    AssuranceClass MinimumAssurance,  
    Duration MaximumContextLifetime,  
    IReadOnlySet<ExternalEntitlementRef> AllowedEntitlementRefs,  
    string Revision,  
    string CanonicalSha256);
```

NO WILDCARD DESTINATION

Profiles are exact. An unknown audience, intent or scope kind fails projection rather than using a default internal profile.

CONTEXT FACTORY

Projection narrows the session to one request

Only trusted BFF boundary code can create the context. It verifies session freshness and revocation, BFF edge eligibility, requested scope intersection and destination profile before deriving the audience reference and expiry.

EXECUTABLE ASSET 46-L - MINIMAL CONTEXT FACTORY

```
public DelegatedOperatorContextV1 Create(  
    OperatorSessionStateV2 session,  
    OwnerOperation operation,  
    ScopeCoordinate requestedScope,  
    Instant now)  
{  
    session.DemandCurrent(now);  
    var profile = profiles.Require(operation.Audience, operation.Intent);  
    session.EdgeCapabilities.Demand(operation.EdgeCapability, requestedScope);  
  
    return new DelegatedOperatorContextV1(  
        ContextId.New(), references.Derive(session.ExternalSubject,  
            operation.Audience, keys.CurrentRevision),  
        operation.Audience, operation.Intent, requestedScope,  
        session.Assurance, session.AuthenticatedAt,  
        session.EdgeCapabilities.Entitlements.Intersect(  
            profile.AllowedEntitlementRefs).ToFrozenSet(),  
        session.IdentityRevision, session.EdgePolicyRevision,  
        profile.Revision, now,  
        Min(session.ExpiresAt, now + profile.MaximumContextLifetime),  
        decisions.CurrentReference);  
}
```

PROJECTION CANNOT WIDEN

The factory can only select from server-side session facts and fixed operation metadata. Browser headers and request bodies cannot add entitlements, assurance or audience.

CONTEXT WIRE CONTRACT

The JSON grammar rejects extension by accident

The owner uses a strict versioned reader with duplicate-property detection, maximum body size, exact enum/code values and rejection of unknown security-sensitive fields. A forward-compatible extension requires a new projection profile and contract test, not permissive object binding.

DECISION LEDGER 46-24

Input	Local meaning	Forbidden inference
contextSchema	exact version 1	best-effort newer version
properties	approved closed set	over-posted roles
strings	bounded canonical identifiers	trim/coerce
entitlementRefs	bounded distinct allow-listed set	arbitrary array
timestamps	UTC integer/ISO profile	local time
duplicate property	reject	last value wins
body size	small fixed maximum	unbounded claims payload

CLOSED SECURITY GRAMMAR

Permissive JSON evolution is inappropriate for authorization inputs. Compatibility is explicit and versioned.

TRANSPORT BINDING

The context is accepted only from the Chapter 45 BFF workload

Direct mTLS authenticates the caller workload. The owner verifies that the certificate-backed WorkloadId is NEXUS-1 BFF, the context audience is local, the operation metadata matches the context intent and the projection profile is known. A context copied to another caller or destination fails.

DECISION LEDGER 46-25

Input	Local meaning	Forbidden inference
mTLS caller	BFF workload identity	human subject
context audience	exact local owner	routing hint
intent	matches endpoint metadata	caller-selected capability
scope	matches route/resource coordinate	membership proof
profile revision	recognized for overlap window	any old profile
correlation	evidence continuity	authorization fact

TWO INDEPENDENT PRINCIPALS

A valid minimal human context without the BFF certificate still fails, and a valid BFF certificate without required human context still fails.

TEMPORAL ADMISSION

Issued, expiry, authentication time and policy time are separate

The owner evaluates all times against one injected decision clock. Context expiry limits transport reuse; authentication time enforces human freshness; policy revisions have their own validity; resource preconditions guard state changes.

DECISION LEDGER 46-26

Input	Local meaning	Forbidden inference
IssuedAt	not unreasonably future	authentication time
ExpiresAt	now before expiry	session revocation state
AuthenticatedAt	within operation freshness	context issuance
projection revision	accepted deployment overlap	owner policy revision
owner policy	current local snapshot	edge policy
resource version	current precondition	identity revision

NO FRESH BOOLEAN

The BFF does not forward `IsFresh=true`. The owner computes freshness from authenticated time, current time and operation policy.

REPLAY BOUNDARY

A short context is still not a reusable permission token

Privileged mutations bind `ContextId` to request idempotency, destination, intent and resource. The owner records consumed context/request coordinates in its local transaction where required. Reads may accept repeated context within the short lifetime but still reauthorize current resources.

DECISION LEDGER 46-27

Input	Local meaning	Forbidden inference
read retry	reauthorize current resource	cached allow
idempotent command retry	same request key and meaning	new effect
<code>ContextId</code> + different body	conflict/replay reject	second command
<code>ContextId</code> + different owner	audience reject	portable context
expired context	reject	refresh at owner
revoked session	BFF stops projection; short residual window	owner introspection service

BOUNDED RESIDUAL

Without a central online session check, owner contexts have a deliberately short residual lifetime. The bound and privileged-operation strategy are explicit evidence.

PROVIDER CLAIM CORPUS**Tests vary one external claim defect at a time**

A corpus builds authenticated principals for the configured provider profile and mutates claim presence, type, cardinality, case and value. It exercises the real strict reader and projector, not a mocked MappingSucceeded flag.

DECISION LEDGER 46-28

Input	Local meaning	Forbidden inference
valid exact iss+sub	project subject	continue to edge mapping
duplicate sub	ambiguous reject	first wins
email only	missing-sub reject	email identity
groups scalar vs array	profile mismatch	coerce
unknown entitlement	no mapped capability	generic role
extra irrelevant claim	same decision + drift	forward
future/missing auth_time	freshness mapping failure	now default

AUTHENTICATED FIXTURE

The corpus begins after token validation; separate Chapter 44 tests continue to prove issuer, signature, audience and nonce failures.

MAPPING TABLE TESTS**External values map only through the reviewed revision**

Table-driven tests enumerate provider profile, entitlement set, requested edge capability, scope and expected result. Unknown coordinates deny. The expected policy revision and digest are asserted with every result.

EXECUTABLE TEST 46-A - EDGE MAPPING MATRIX

```
[Theory]
[MemberData(nameof(EdgeMappingCases.All), MemberType = typeof(EdgeMappingCases))]
public void Edge_mapping_is_exact(EdgeMappingCase test)
{
    var decision = snapshot.Map(test.Subject);

    decision.IsAccepted.Should().Be(test.ExpectedAccepted);
    decision.SafeCode.Should().Be(test.ExpectedCode);
    decision.PolicyRevision.Should().Be("edge-claims/9");
    decision.PolicySha256.Should().Be(Expected.EdgePolicySha256);
}

[Fact]
public void Unknown_entitlement_grants_nothing() =>
    snapshot.Map(Subjects.With("ext:unknown")).Capabilities.Should().BeEmpty();
```

NO ACCIDENTAL OR

A test with one valid and one malformed critical claim follows the profile's explicit rule; the valid value does not hide ambiguity.

NON-INTERFERENCE PROPERTY

Irrelevant claims cannot change authorization

Property-based tests generate arbitrary claim names and values outside the provider allow-list, append them to a valid authenticated principal and require the normalized subject, edge snapshot, delegated context and owner decision to remain identical except for drift metadata.

DECISION LEDGER 46-29

Input	Local meaning	Forbidden inference
extra display claim	no decision change	new capability
unknown role-like claim	no decision change	admin interpretation
nested JSON object	ignored/rejected by profile	flattened claims
large unknown array	size/profile guard	resource exhaustion
claim order	canonical result unchanged	first-wins meaning
duplicate irrelevant claim	drift only	singleton override

SECURITY METAMORPHIC TEST

Adding information that the policy does not declare relevant must not increase authority.

TRANSFORMATION IDEMPOTENCY

Repeated framework transformation produces no duplicate meaning

The test invokes the registered `IClaimsTransformation` several times and verifies one safe display identity, no added role/capability claims and no network or policy-store calls. This follows the platform warning that transformation may run multiple times.

EXECUTABLE TEST 46-B - IDEMPOTENT CLAIMS TRANSFORMATION

```
[Fact]
public async Task Display_transformation_is_idempotent_and_non_authorizing()
{
    var principal = ProviderPrincipals.Valid();

    await transformation.TransformAsync(principal);
    await transformation.TransformAsync(principal);
    await transformation.TransformAsync(principal);

    principal.FindAll("nexus:display-projected").Should().ContainSingle();
    principal.FindAll("nexus:display-name").Should().ContainSingle();
    principal.Claims.Should().NotContain(c =>
        c.Type.Contains("role", StringComparison.OrdinalIgnoreCase) ||
        c.Type.Contains("capability", StringComparison.OrdinalIgnoreCase));
    remoteProbe.Calls.Should().Be(0);
}
```

FRAMEWORK SEAM ONLY

The test protects against duplicate claims and against turning a frequently invoked framework hook into a central authorization lookup.

SESSION LEAK TESTS

Serialized server sessions contain only the approved schema

The session serializer is tested with a fully populated provider principal containing tokens, profile claims and many entitlements. The stored document must match `OperatorSessionStateV2` exactly and fail a forbidden-name/value scanner.

DECISION LEDGER 46-30

Input	Local meaning	Forbidden inference
access_token/id_token	absent	string or nested property
refresh_token/code	absent	value/hash
email/phone/address	absent	unused PII
raw groups claim	absent	provider bag
normalized edge snapshot	present and bounded	owner policy
revisions/expiry	present	secret material

SCHEMA ALLOW-LIST

The test passes only when every serialized property is approved; a blacklist alone cannot catch a newly named sensitive field.

FORWARDING LEAK TEST

Outbound owner requests are allow-list exact

A production typed client is given a broad server-side session and a destination profile. A recording handler captures the request after serialization. The test asserts the exact JSON property set, entitlement subset, body size and absence of cookies, Authorization headers and provider claims.

EXECUTABLE TEST 46-C - MINIMAL OUTBOUND CONTRACT

```
[Fact]
public async Task Alarm_owner_request_contains_only_profile_fields()
{
    var session = Sessions.BroadAuthenticatedOperator();
    await client.AcknowledgeAsync(session, AlarmId.Parse("alarm-42"), SiteId.Parse("ath-17"));

    var sent = recorder.SingleRequest();
    sent.Headers.Contains("Authorization").Should().BeFalse();
    sent.Headers.Contains("Cookie").Should().BeFalse();
    sent.Json.PropertyNames.Should().BeEquivalentTo(
        DestinationSchemas.AlarmAcknowledgeV1, o => o.WithStrictOrdering());
    sent.Json.ToString().Should().NotContainAny(
        "email", "groups", "roles", "access_token", "id_token", "refresh_token");
}
```

PRODUCTION SERIALIZER

The test uses the same typed client and JSON options as runtime, catching naming policies, extension data and accidental framework serialization.

CONTRACT CORPUS

Owners reject over-posted and under-specified contexts

The delegated-context corpus exercises the actual strict JSON reader with duplicate properties, unknown security fields, invalid enum values, excessive collections, wrong audience, scope mismatch and unsupported versions. Every rejection occurs before local mapping and resource loading where possible.

DECISION LEDGER 46-31

Input	Local meaning	Forbidden inference
extra roles field	unknown-field reject	ignore
allowed=true	unknown-field reject	short-circuit
duplicate audience	duplicate-property reject	last wins
two scopes	shape reject	choose one
too many entitlements	budget reject	truncate
unknown intent/version	contract reject	default policy
valid minimal context	continue	automatic allow

ZERO WORK ON MALFORMED INPUT

Malformed contexts do not invoke owner mapping, resource repositories, Domain handlers or persistence.

PAIRWISE-REFERENCE VECTORS

Reference derivation is deterministic and audience-separated

Fixed test keys and canonical inputs produce retained vectors. The same iss+sub and audience reproduce the reference; different owners or key revisions differ; email/name changes do not matter; raw iss+sub never appear in the output.

EXECUTABLE TEST 46-D - PAIRWISE PROPERTIES

```
[Fact]
public void Reference_is_stable_only_within_audience_and_revision()
{
    var subject = Subjects.External("https://id.phase0.example", "00u-123");

    var alarm1 = deriver.Derive(subject, Owners.AlarmManagement, Keys.R7);
    var alarm2 = deriver.Derive(subject, Owners.AlarmManagement, Keys.R7);
    var root = deriver.Derive(subject, Owners.RootCause, Keys.R7);
    var rotated = deriver.Derive(subject, Owners.AlarmManagement, Keys.R8);

    alarm1.Should().Be(alarm2);
    root.Value.Should().NotBe(alarm1.Value);
    rotated.Value.Should().NotBe(alarm1.Value);
    alarm1.Value.Should().NotContain("00u-123");
}
```

LIFECYCLE CONTRACT

Chapter 47 must define how owner assignments bridge approved reference-key overlap without creating a global reverse-lookup service.

PROJECTION MATRIX**Every destination receives only its approved subset**

A single broad BFF session is projected to each destination and the resulting field/entitlement sets are compared. The test proves that a Compliance approval reference never reaches Reporting and a Reporting reader reference never reaches AlarmManagement commands.

DECISION LEDGER 46-32

Input	Local meaning	Forbidden inference
AlarmManagement	alarm operator refs + one site	compliance/report/repair refs
RootCause	reviewer refs + case scope	alarm command refs
Compliance	approver refs + plan scope	report reader refs
Reporting	reader refs + tenant/report scope	mutation refs
Repair	request/approve profile-specific refs	all owner roles
Workflow pure call	no human context	synthetic ref

CROSS-DESTINATION NEGATIVE

The test fails on any superset forwarding even if the receiving owner would ignore the extra data.

OWNER POLICY MATRIX**Mapped fact, scope, resource and time vary independently**

Table-driven component tests cross entitlement mapping, owner assignment, requested scope, resource scope/state, assurance, authentication age and policy revision. Only the exact conjunction permits the Application handler.

DECISION LEDGER 46-33

Input	Local meaning	Forbidden inference
all valid	allow and invoke once	commit not yet guaranteed
unknown entitlement	deny	fallback operator
mapping valid, assignment absent	deny	edge allow
scope mismatch	deny/hide	route value grant
stale authentication	deny/re-auth required	context fresh flag
resource terminal	conflict/deny	identity overrides state
policy stale	unavailable/deny	reuse indefinitely

ONE POSITIVE CELL PER RULE SET

The suite asserts safe code, policy identity, resource version and zero effects for every negative row.

BFF-OWNER DISAGREEMENT

An edge allow never overrides the owner

The integration test deliberately configures the BFF to allow navigation and projection while the owner lacks the local assignment or denies the current resource state. The request reaches the owner through real mTLS and ends with a bounded denial and zero business effects.

EXECUTABLE TEST 46-E - OWNER REMAINS FINAL

```
[Fact]
public async Task Edge_allow_does_not_override_owner_denial()
{
    await using var runtime = await Runtime.StartAsync(
        EdgePolicies.AllowAlarmCommand,
        AlarmPolicies.SubjectHasNoLocalAssignment);

    var response = await runtime.Bff.AcknowledgeAlarmAsync(
        Operators.ValidSession, AlarmId.Parse("alarm-42"));

    response.StatusCode.Should().Be(HttpStatusCode.Forbidden);
    runtime.EdgeDecisions.Last().IsAllowed.Should().BeTrue();
    runtime.OwnerDecisions.Last().IsAllowed.Should().BeFalse();
    (await runtime.BusinessEffects.DeltaAsync()).Should().BeEmpty();
}
```

EXPECTED DEFENSE

The mismatch is not automatically an error. It proves that edge eligibility and owner resource authorization are independent decisions.

AUDIENCE AND INTENT TESTS

A valid context cannot move to another endpoint

Component tests copy an accepted context across owners, endpoints, methods and scope coordinates. The workload remains the valid BFF, so each failure proves application-level audience and intent binding rather than TLS rejection.

DECISION LEDGER 46-34

Input	Local meaning	Forbidden inference
Alarm context -> RootCause	wrong-audience deny	reinterpret
read context -> command	wrong-intent deny	capability widening
site A context -> site B resource	scope deny	membership reuse
approval context -> execution	wrong-intent deny	privilege step-up
old profile revision	overlap policy or deny	accept forever
unknown owner route	404/deny	default internal policy

REPLAY IS COORDINATE-BOUND

ContextId, audience, intent, scope and operation are one authorization coordinate; none may be substituted independently.

TEMPORAL TESTS

Time boundaries are deterministic and fail closed

An injected clock tests exact instants around IssuedAt, ExpiresAt, AuthenticatedAt, policy validity and assignment validity. No test relies on wall-clock sleeps, and the decision records the single observation time used for all comparisons.

DECISION LEDGER 46-35

Input	Local meaning	Forbidden inference
now < issuedAt-skew	reject future context	wait and accept
now == expiresAt	expired reject	inclusive expiry
auth age == maximum	profile-defined boundary	arbitrary skew
policy validUntil reached	stale/unavailable	last positive
assignment effectiveFrom future	deny	eventual allow
assignment expired	deny	session grant

ONE CLOCK

BFF and owner evidence record their clocks; environment tests bound acceptable skew without letting skew extend authorization lifetime silently.

REVOCATION AND RESIDUAL WINDOW

Session revocation stops new contexts; owners bound old ones

The BFF revokes the server-side session and cannot project new contexts. Contexts already issued remain acceptable only until their short expiry and operation replay rules. Privileged flows may require an online or one-time confirmation owned by the relevant boundary, but ordinary owners do not call a global session database.

DECISION LEDGER 46-36

Input	Local meaning	Forbidden inference
session revoked before projection	no owner request	cached context creation
context issued before revocation	bounded residual lifetime	instant global guarantee
privileged approval	shorter context/one-time record	ordinary TTL
owner receives expired	deny	BFF refresh header
revocation service unavailable	BFF fail closed for new context	stale allow
incident	disable workload/session/policy as scoped	one global kill fiction

HONEST TRADEOFF

Avoiding a central online authorization database creates a measurable residual revocation window; the book states and tests that bound.

NO CENTRAL DEPENDENCY TEST

Owner production composition has no authorization-service client

Architecture and descriptor tests inspect owner Application and Domain dependencies plus production HTTP clients. They allow local policy/configuration sources and owner repositories, but reject a generic AuthorizationService, directory client or shared policy DbContext in the request path.

EXECUTABLE TEST 46-F - LOCAL AUTHORIZATION ARCHITECTURE

```
[Fact]
public void Owners_do_not_depend_on_a_central_authorization_database()
{
    OwnerAssemblies.ApplicationAndDomain()
        .ShouldNotReference("Nexus.Authorization.ServiceClient")
        .And.ShouldNotReference("Nexus.Identity.DirectoryClient")
        .And.ShouldNotReference("Nexus.SharedAuthorizationDb");

    ProductionServices.Descriptors()
        .ShouldNotContainServiceNamed("IAuthorizationOracle")
        .And.ShouldNotContainDbContextNamed("AuthorizationDbContext");
}
```

ZERO DISCOVERY GUARD

The architecture test first proves it discovered every owner assembly and production composition root.

ENDPOINT INVENTORY

Every delegated route names a projection and owner policy

A production-host inventory joins BFF route metadata, destination projection profile, owner endpoint intent, workload capability and owner operation policy. Missing or orphaned coordinates fail the build.

DECISION LEDGER 46-37

Input	Local meaning	Forbidden inference
BFF route	edge capability + destination/intent	generic proxy
projection profile	exact field/entitlement budget	default context
owner endpoint	audience + intent + workload requirement	bare authenticated route
owner policy	operation/resource requirements	claim-only allow
negative tests	wrong audience/intent/scope	happy path only
documentation	decision owner/evidence ID	unowned route

BIDIRECTIONAL INVENTORY

Every delegated BFF route resolves to exactly one owner contract, and every owner operator route has an approved BFF projection path or explicit non-BFF caller.

ZERO-EFFECT DENIALS

Claim and policy failures stop before business mutation

Negative tests snapshot Application invocations, owner database changes, outbox/inbox rows, broker publications, process transitions and repair actions. Only bounded security evidence may change after a denied mapping or authorization decision.

DECISION LEDGER 46-38

Input	Local meaning	Forbidden inference
claim profile failure	no session or owner call	drift evidence only
edge mapping deny	no owner call	edge denial evidence
context admission deny	no resource/Application call	owner security evidence
owner mapping deny	no Domain/commit	decision evidence
resource policy deny	no mutation/outbox	safe audit decision
policy unavailable	no fallback effect	readiness/incident signal

NOT STATUS-ONLY

A 401, 403 or 503 passes only when all business-effect probes remain unchanged.

CLAIM INJECTION FAULT

Caller-controlled JSON and headers cannot add identity facts

Fault tests add roles, subject, assurance, site, tenant and entitlement-shaped fields to browser requests, query strings, headers and command bodies. The BFF ignores or rejects them and constructs projection solely from its server-side session and operation metadata.

DECISION LEDGER 46-39

Input	Local meaning	Forbidden inference
X-User-Role	reject/ignore	edge mapping input
X-Subject	reject/ignore	subject reference
assurance=high	reject/ignore	step-up evidence
sitelds=[all]	intersect one requested scope	membership
entitlements	unknown field reject	forwarding set
allowed=true	unknown field reject	owner short-circuit

SERVER-OWNED PROJECTION

The browser may request an operation and target. It cannot supply any fact that increases authority.

POLICY DRIFT FAULT**Edge and owner revisions can diverge safely**

The fault laboratory deploys new edge mapping before owner mapping, new projection profiles before owner readers and new owner rules before BFF profiles. Each ordering either remains compatible within a declared overlap or fails closed without widening authority.

DECISION LEDGER 46-40

Input	Local meaning	Forbidden inference
edge new, owner old	old approved subset or deny	send new broad refs
projection new, reader old	version reject	ignore fields
owner new, edge old	owner may deny	require BFF to know owner rule
entitlement removed	new decisions lose mapping	cached positive
reference key overlap	both explicit revisions	global reverse lookup
unknown digest	readiness/deployment fail	best effort

SAFE CHOREOGRAPHY

Policy deployment order affects availability, not whether an unreviewed entitlement becomes authorized.

DEPENDENCY FAULT LABORATORY**Identity and policy dependencies fail without broadening context**

Fault injection interrupts provider metadata, session store, mapping source, reference-key source, owner policy reload and assignment projection. Each failure has a bounded timeout, readiness consequence, denial/unavailability outcome and zero-effect assertion.

DECISION LEDGER 46-41

Input	Local meaning	Forbidden inference
provider metadata stale	new login unavailable	trust old issuer silently
session store unavailable	no projection	stateless token forwarding
edge map unavailable	no protected session/context	empty allow
reference key unavailable	no owner call	raw subject fallback
owner policy expired	owner unavailable/deny	trust edge decision
assignment projection stale	operation-class policy	directory hot call

NO EMERGENCY OVER-SHARING

A dependency outage never causes the BFF to forward the original token or entire claim bag as a fallback.

VALUE-FREE MANIFEST

Environments compare security mapping without exporting identity data

The environment manifest records enabled provider profiles, allowed claim-name digests, edge/projection/owner policy digests, reference-key revision, context schema and maximum budgets. It contains no subjects, claim values, entitlements, tokens or keys.

EVIDENCE ASSET 46-B - CLAIMS MAPPING MANIFEST

```
{
  "schema": "nexus1.claims-policy-manifest/1",
  "environment": "phase0",
  "externalClaimNamespace": "unmapped-exact",
  "providerProfileDigest": "sha256:<digest>",
  "edgePolicyDigest": "sha256:<digest>",
  "projectionProfileDigest": "sha256:<digest>",
  "ownerPolicyDigests": { "alarm-management": "sha256:<digest>" },
  "contextSchema": "nexus1.delegated-operator-context/1",
  "referenceKeyRevision": "operator-ref/7",
  "rawTokensForwarded": false,
  "centralAuthorizationDatabase": false,
  "productionSecurityClaimed": false
}
```

COMPARABLE CONFIGURATION

CI can prove that two environments run the same mapping contracts without reading any human identity or secret material.

DECISION EVIDENCE

Every projection and owner decision retains bounded provenance

The evidence schema joins the external profile, edge mapping, projection profile, workload decision and owner policy while keeping identities pseudonymous and destination-scoped. Denials reference the zero-effect probe where applicable.

EVIDENCE ASSET 46-C - MAPPING DECISION RECORD

```
public sealed record ClaimsPolicyDecisionEvidenceV1(
  string EvidenceId,
  Instant ObservedAt,
  string Stage,
  string? AudienceSubjectReference,
  string Destination,
  string Intent,
  string ResultCode,
  ClaimsPolicyIdentity PolicyIdentity,
  string? ContextReference,
  string WorkloadDecisionReference,
  string CorrelationReference,
  string? ZeroEffectProbeReference);
```

NO CLAIMS DOSSIER

Evidence proves which profiles and policies produced a decision. It does not preserve the original claim bag as a convenience archive.

EVIDENCE VALIDATOR

Passing requires non-zero fresh artifacts from the same configuration

The Chapter 46 validator checks schema, commit, image, provider/edge/projection/owner digests, context-contract digest, test discovery, execution counts, freshness and forbidden-field scan results. A summary marked PASSED without artifacts is invalid.

DECISION LEDGER 46-42

Input	Local meaning	Forbidden inference
schema/build	exact expected identities	near match
policy digests	all participating revisions	one combined label
tests discovered	> 0 per required suite	zero green
tests executed/passed	equal counts	skipped negatives
leak scan	zero forbidden fields	warnings accepted
completedAt	within evidence window	stale reuse
production claim	false	inferred maturity

EDITORIAL HONESTY

The book specifies the evidence. Runtime status remains PENDING until the repository retains matching artifacts.

DEPLOYMENT SEQUENCE

Contracts and owner denial paths arrive before new mappings

Rollout first deploys owner readers and deny-safe policies, then BFF projection profiles, then edge mappings and provider profile changes. Each stage has compatibility and negative tests, and no stage forwards a broader fallback.

DECISION LEDGER 46-43

Input	Local meaning	Forbidden inference
1	claim/context schemas and forbidden-field tests	contract corpus green
2	owner local maps and denial paths	owner matrix green
3	projection profiles	subset/leak tests green
4	pairwise references/assignments	vector/reconciliation green
5	edge maps/provider profile	mapping corpus green
6	remove broad V1 forwarding	absence inventory green
7	evidence/operations	environment manifest match

AVAILABILITY-SAFE ORDER

Temporary denial is acceptable during rollout. Temporary acceptance of broad claims or edge-only permission is not.

ROLLBACK

Rollback restores a complete prior mapping set

A safe rollback restores compatible provider, edge, projection and owner revisions as a reviewed set. It may force re-login or make routes unavailable. It never re-enables raw token forwarding, broad OperatorContextV1, unknown-field binding or central authorization fallback.

DECISION LEDGER 46-44

Input	Local meaning	Forbidden inference
provider shape regression	restore prior profile/re-login	coerce new shape
projection incompatibility	restore prior exact schema	send both broad forms
owner policy fault	restore signed/digested snapshot	trust BFF allow
reference-key issue	controlled overlap revision	raw subject
assignment lag	deny/degrade	directory hot path
evidence failure	stop promotion	mark passed manually

NO INSECURE ROLLBACK

A previous minimal, locally authorized configuration is recoverable; an older permissive shortcut is not a rollback target.

INCIDENT RESPONSE

Claim or mapping compromise closes the smallest authority surface

Runbooks distinguish compromised external identity, provider profile, edge map, projection profile, owner policy, pairwise-reference key and local assignment. Response revokes or disables only the affected coordinate where possible, preserves evidence and proves old contexts no longer create effects.

DECISION LEDGER 46-45

Input	Local meaning	Forbidden inference
subject compromise	revoke BFF sessions	new context blocked
provider drift/compromise	disable profile/new login	validated metadata restored
edge map overgrant	disable revision/revoke affected sessions	negative mapping tests
owner map overgrant	restore local policy	resource matrix denies
reference key compromise	rotate audience revisions	old/new linkage controlled
assignment corruption	tombstone/reconcile local projection	old grant denied

NO GLOBAL GUESS

Because policy is owner-local, incident response uses manifest evidence to identify affected destinations rather than assuming one universal role map.

CAPACITY AND ABUSE

Claim parsing and mapping have explicit budgets

Attackers can inflate claims, group arrays, context bodies, mapping misses and denial telemetry. The profile caps values and collection sizes, mapping is local and bounded, caches use safe keys, and metrics avoid attacker-controlled labels.

DECISION LEDGER 46-46

Input	Local meaning	Forbidden inference
token/claim count	provider/profile limits	unbounded projection
entitlement count	strict maximum	truncate and allow
string/body size	per-field/context budgets	large log
mapping work	immutable dictionaries/sets	remote N+1 lookups
unknown claims	bounded counts/sampling	claim names as labels
policy reload	single-flight validation	request-thread compile

OVERLOAD FAILS CLOSED

Resource exhaustion can deny or make the security dependency unavailable; it cannot skip mapping or forward raw material.

RESIDUAL RISKS

Minimization reduces blast radius but does not remove trust

The BFF still sees provider claims and constructs delegated contexts. A compromised BFF can lie within its allowed protocol, although owners limit consequences with workload identity, local mappings and resources. Policy administration, pairwise-key custody and assignment sources remain security-critical.

DECISION LEDGER 46-47

Input	Local meaning	Forbidden inference
BFF compromise	owner-local checks + narrow profiles	fabricated bounded context
provider mis-issuance	exact profile + local mapping	false external facts
owner policy error	tests/digest/review	wrong local decision
reference-key compromise	audience keys + rotation	subject correlation
revocation residual	short context lifetime	instant global revoke
production maturity	not claimed	Chapter 47/48 evidence pending

BOUNDED CONSEQUENCE

The architecture makes trust dependencies explicit and local enough to test; it does not make them disappear.

REJECTED ALTERNATIVES

Convenient claim designs recreate the exact coupling we are removing

ADR-046 retains rejected choices so later implementation pressure cannot reintroduce them without a new evidence-backed decision.

DECISION LEDGER 46-48

Input	Local meaning	Forbidden inference
forward access token	audience/credential exposure	BFF session + minimal context
internal super-JWT	portable global authority	destination contract
send all groups	over-sharing and local guesswork	approved subset
email as subject	unstable/reassignable	iss+sub then pairwise ref
central authorization service	runtime/data coupling	local snapshots/projections
shared policy package	hidden global semantics	shared grammar only
trust edge allow	transitive permission	owner resource decision

SCOPED REJECTION

These choices are rejected for this Phase-0 architecture and proof model; a future alternative must satisfy the same boundary obligations.

DEFINITION OF DONE

Completion is a traceable minimum, not a large claim catalogue

Chapter 46 is complete as a specification when provider profiles, normalized sessions, destination projections, pairwise references, local owner maps, resource decisions, no-central-dependency gates, leak scans, negative matrices, fault recovery and evidence identities agree.

DECISION LEDGER 46-49

Input	Local meaning	Forbidden inference
identity adapter	strict claim corpus	generic claim dictionary
BFF session	minimal normalized schema	token/claim persistence
forwarding	one destination/intent/scope	broad V1 context
owner	local map + resource policy	edge allow
architecture	no central auth DB/client	hidden shared policy
evidence	fresh non-zero artifacts	narrative pass
maturity	SPECIFIED RUNTIME PENDING	production claim

STATUS

The mapping and test assets are compile-oriented specification. Repository execution and production security maturity remain unclaimed.

IMPLEMENTATION SEQUENCE

Build the narrow types and denials before positive mappings

The order prevents a working login from becoming a broad permission path. Each stage adds one mapping surface and keeps prior leak and zero-effect tests green.

DECISION LEDGER 46-50

Input	Local meaning	Forbidden inference
1	ADR, boundary/field/ownership ledgers	reviewed digests
2	strict provider profile and claim corpus	no mapping yet
3	normalized session and token/claim leak scans	server-side only
4	edge mapping snapshot and closed outcomes	edge scope only
5	destination context + pairwise reference	subset tests
6	owner maps, assignments and resource policies	owner matrix
7	architecture/fault/deployment evidence	fail-closed validator

FIRST RELEASE BLOCKER

Before adding a positive entitlement mapping, unknown and injected claims must demonstrably grant nothing.

CHAPTER 47 HANDOFF

Secrets and configuration must make these profiles operable

Chapter 46 depends on protected OIDC client configuration, pairwise-reference keys, atomic policy loading, environment manifests and safe reload. Chapter 47 must implement custody, injection, rotation, file/secret permissions, redaction and startup failure on invalid security configuration.

DECISION LEDGER 46-51

Input	Local meaning	Forbidden inference
OIDC client secret/config	secure source + rotation	source/image
reference derivation keys	audience-specific custody/overlap	plain config
policy documents	integrity/digest/atomic reload	partial mutation
trust/certificate config	Chapter 45 lifecycle	manual drift
environment manifest	value-free comparison	secret export
startup validation	fail on mismatch	run permissively

PROTECTED HANDOFF

Chapter 47 may change how configuration and keys are supplied. It may not widen the Chapter 46 contracts or centralize owner policy.

CHAPTER 48 HANDOFF

Audit evidence must prove absence as well as success

Chapter 48 must execute and retain the provider corpus, non-interference tests, session/request leak scans, projection subset matrix, owner disagreement, architecture gates, zero-effect denials, drift/fault laboratory and environment manifest validation.

DECISION LEDGER 46-52

Input	Local meaning	Forbidden inference
claim minimization	forbidden-field and exact-schema artifacts	code review only
local mapping	owner policy matrices	one happy path
no central DB	dependency/composition inventory	architectural assertion
owner final authority	edge-allow/owner-deny test	duplicate policies assumed
fault safety	stale/unavailable zero-effect evidence	status only
maturity	bounded Phase-0 dossier	production certification

EVIDENCE OBLIGATION

A value is proven absent only by inspecting real serialized sessions, requests, responses, logs and retained artifacts.

OWNER ADMISSION

The owner reconstructs typed inputs, never a generic principal

Owner Edge validates the BFF workload, strict delegated-context grammar, audience, intent, scope and time. It then constructs an OwnerAuthorizationInput. Application code does not receive ClaimsPrincipal, HTTP headers, entitlement arrays or provider claim names.

DECISION LEDGER 46-53

Input	Local meaning	Forbidden inference
WorkloadPrincipal	authenticated BFF caller	human permission
DelegatedOperatorContext	bounded human decision input	raw external identity
endpoint metadata	fixed operation/audience	caller-selected policy
route coordinate	requested resource key	access proof
decision clock	single observation time	client timestamp
OwnerAuthorizationInput	typed Application input	framework principal

BOUNDARY TYPE RULE

Domain and Application assemblies have no reference to System.Security.Claims, OpenID Connect packages or HTTP header types.

OWNER AUTHORIZATION INPUT

The local policy receives facts, operation and resource separately

The input keeps authenticated caller, delegated subject, requested intent, local operation, target coordinate, assurance and external entitlement references distinct. The loaded owner resource is supplied separately so policy cannot treat a route value as authoritative state.

EXECUTABLE ASSET 46-M - OWNER DECISION INPUT

```
public sealed record OwnerAuthorizationInput(  
    WorkloadId Caller,  
    AudienceSubjectReference Subject,  
    RequestedIntent RequestedIntent,  
    OwnerOperation Operation,  
    ScopeCoordinate RequestedScope,  
    AssuranceClass Assurance,  
    Instant AuthenticatedAt,  
    IReadOnlySet<ExternalEntitlementRef> EntitlementRefs,  
    long IdentityRevision,  
    string EdgePolicyRevision,  
    string ProjectionProfileRevision,  
    ContextId ContextId,  
    Instant ObservedAt);
```

NO FINAL PERMISSION FIELD

The type carries no IsAllowed, Role, Capability or OwnerGrant. Those meanings exist only after local mapping and resource evaluation.

LOCAL MAPPING

Each owner translates external references into its own policy facts

The same external entitlement reference may have no meaning at one owner, read eligibility at another and approval-candidate status at Compliance. Mapping tables live with the receiving service and produce typed local facts, not global roles.

DECISION LEDGER 46-54

Input	Local meaning	Forbidden inference
ext:control-room-operator	AlarmManagement.OperatorCandidate	RootCause reviewer
ext:root-cause-reviewer	RootCause.ReviewerCandidate	Alarm command
ext:compliance-approver	Compliance.ApproverCandidate	repair executor
ext:report-reader	Reporting.ReaderCandidate	owner mutation
ext:repair-requester	Repair.RequesterCandidate	repair approval
unknown reference	no local fact	generic authenticated role

MEANING IS LOCAL

A shared external code does not imply shared authorization semantics. Every owner mapping has its own revision, tests and review.

OWNER MAPPING SNAPSHOT

Local policy is immutable and versioned

The owner loads a validated snapshot containing only its recognized entitlement-to-fact mappings, operation requirements, assurance/freshness limits and scope rules. It publishes the snapshot atomically and identifies every decision by canonical digest.

EXECUTABLE ASSET 46-N - OWNER POLICY SNAPSHOT

```
public sealed record OwnerPolicySnapshot(  
    OwnerService Owner,  
    string Revision,  
    string CanonicalSha256,  
    Instant EffectiveFrom,  
    Instant ValidUntil,  
    IReadOnlyDictionary<ExternalEntitlementRef,  
        IReadOnlySet<OwnerPolicyFact>> FactMappings,  
    IReadOnlyDictionary<OwnerOperation,  
        OwnerOperationPolicy> Operations)  
{  
    public OwnerAuthorizationDecision Evaluate(  
        OwnerAuthorizationInput input,  
        OwnerAuthorizationResource resource) =>  
        OwnerPolicyEvaluator.Evaluate(this, input, resource);  
}
```

ATOMIC VIEW

One decision observes one complete policy revision. Hot reload cannot mix old mappings with new operation rules.

POLICY FACT GRAMMAR

Local facts are narrow predicates, not reusable roles

OwnerPolicyFact names one locally meaningful attribute such as AlarmOperatorCandidate, SiteAssignmentMatched or IndependentApprover. It is evaluated with the current resource and operation; it is never serialized back to the BFF or shared as a universal role.

DECISION LEDGER 46-55

Input	Local meaning	Forbidden inference
AlarmOperatorCandidate	input eligibility	may acknowledge every alarm
SiteAssignmentMatched	subject assignment matches resource site	global site membership
CurrentShift	local schedule fact	identity provider truth
IndependentApprover	not requester for this plan	permanent approver role
CaseReviewerCandidate	review policy input	can close any case
ReportReaderCandidate	report policy input	database access

PREDICATE OVER ROLE

The owner evaluates facts for one decision. It does not mint a portable role claim that another owner could consume.

RESOURCE AUTHORIZATION

Current owner state completes the decision

Policy-based endpoint admission can reject obvious callers, but resource authorization occurs after the owner loads a minimal authorization projection. The projection includes only state needed for the decision and is fetched without exposing broader resource data.

DECISION LEDGER 46-56

Input	Local meaning	Forbidden inference
alarm	site, status, assignment, version	full history/payload
root-cause case	site, review state, owner, version	all evidence before allow
compliance plan	requester, approver, risk, expiry	secret comments
report	tenant/site, classification, readiness	report contents
repair plan	owner, requester, approval, status, version	replacement body
missing resource	bounded 404 path	existence disclosure

LOAD TO DECIDE, NOT TO LEAK

The authorization projection is not returned to the caller until policy permits the operation.

OWNER EVALUATOR

Mapping and resource requirements are explicit conjunctions

The evaluator maps the forwarded entitlement subset to local facts, verifies audience and operation, computes assurance freshness, checks local assignments and resource state, then returns a closed decision. Unknown or missing coordinates deny.

EXECUTABLE ASSET 46-O - LOCAL EVALUATION

```
public OwnerAuthorizationDecision Evaluate(  
    OwnerPolicySnapshot policy,  
    OwnerAuthorizationInput input,  
    OwnerAuthorizationResource resource)  
{  
    DemandEqual(policy.Owner, resource.Owner, "owner");  
    var operation = policy.Operations.Require(input.Operation);  
    var facts = policy.MapExact(input.EntitlementRefs);  
  
    return OwnerDecisionBuilder.Start(policy, input)  
        .RequireIntent(operation, input.RequestedIntent)  
        .RequireAssurance(operation, input.Assurance)  
        .RequireFreshness(operation, input.AuthenticatedAt, input.ObservedAt)  
        .RequireFacts(operation, facts)  
        .RequireScope(input.RequestedScope, resource.Scope)  
        .RequireLocalAssignment(input.Subject, resource)  
        .RequireResourceState(operation, resource)  
        .Finish();  
}
```

CONJUNCTIVE GATES

Mapping an entitlement to a local fact is necessary only where policy says so; it is never sufficient without the remaining resource and state requirements.

LOCAL ASSIGNMENT PROJECTION

Owners keep only authorization data relevant to their resources

Some decisions require an owner-local assignment such as subject-to-site or reviewer-to-case-class. That projection belongs to the owner database and is updated through explicit administrative or event-driven contracts. It does not become a copy of the whole workforce directory.

DECISION LEDGER 46-57

Input	Local meaning	Forbidden inference
subject reference	audience-specific join key	raw external subject
assignment scope	owner resource coordinate	all organization memberships
effective interval	local time bound	session lifetime
source revision	projection provenance	global authority
tombstone	explicit removal	delete history
reconciliation	detect missing/stale owner rows	online directory dependency

SELECTIVE DUPLICATION

Local authorization projections duplicate only facts required for local decisions and preserve independent availability and ownership.

NO CENTRAL AUTHORIZATION DATABASE

A shared policy service would recreate a distributed monolith

The Phase-0 runtime has no synchronous AuthorizationService client in every request path and no shared policy database. Centralized identity administration may publish governed facts, but services own their mapping, resource authorization projection, policy revision and decision evidence.

DECISION LEDGER 46-58

Input	Local meaning	Forbidden inference
central auth HTTP call	rejected hot-path dependency	single source of permission truth
shared authorization DB	rejected data coupling	cross-service joins
shared policy library	types/evaluator helpers only	policy data/decisions
identity events/config	source facts with provenance	owner final allow
owner projection	local bounded authorization data	enterprise directory copy
owner policy	local semantic authority	global role catalogue

AVAILABILITY CONSEQUENCE

An owner can continue with its validated local snapshot when a central administration surface is unavailable, within explicit freshness limits.

SHARED-CODE BOUNDARY**Shared packages define grammar, never policy meaning**

A small contracts package may contain versioned context DTOs, identifiers and validation primitives. It must not contain entitlement-to-capability maps, owner operation lists, resource predicates or a default allow handler.

DECISION LEDGER 46-59

Input	Local meaning	Forbidden inference
DelegatedOperatorContextV1	shared wire grammar	owner mapping
AudienceSubjectReference	shared value type	grant lookup
AssuranceClass	shared bounded vocabulary	required level per operation
ExternalEntitlementRef	opaque identifier type	meaning
validation helpers	syntax/size/time primitives	policy outcome
owner policy data	owner assembly/config	shared package

ARCHITECTURE GATE

A dependency test fails if the shared contracts assembly references an owner Domain/Application assembly or contains policy map resources.

POLICY OWNERSHIP MATRIX**Every mapping change has one accountable reviewer**

The identity team owns provider claim profiles; the BFF owns its route mappings and projection profiles; each receiving team owns local mappings and resource rules. Changes that cross two ownership domains require both reviews and matching digests.

DECISION LEDGER 46-60

Input	Local meaning	Forbidden inference
provider profile	identity/BFF owner	issuer or shape change
edge entitlement map	BFF owner	navigation/edge capability change
destination projection	BFF + owner	field/entitlement forwarding change
owner fact mapping	receiving service	local semantic change
resource rule	receiving service	Domain/operation change
reference derivation	security/platform	key/profile rotation

NO UNILATERAL AUTHORITY EXPANSION

A provider or BFF administrator cannot grant a new owner action by changing one mapping alone.

POLICY IDENTITY

Canonical serialization makes drift visible

Provider, edge, projection and owner policies have separate canonical digests. Decision evidence records all relevant identities. Environment equivalence compares the digests without exporting claim values, subjects, keys or policy secrets.

EVIDENCE ASSET 46-A - MAPPING IDENTITY

```
public sealed record ClaimsPolicyIdentity(  
    string ProviderProfileRevision,  
    string ProviderProfileSha256,  
    string EdgePolicyRevision,  
    string EdgePolicySha256,  
    string ProjectionProfileRevision,  
    string ProjectionProfileSha256,  
    string OwnerPolicyRevision,  
    string OwnerPolicySha256,  
    string ReferenceKeyRevision);
```

INDEPENDENT REVISIONS

The evidence never compresses all security configuration into one version string that hides which mapping changed.

POLICY RELOAD

New snapshots are validated before atomic publication

A reload parses the complete policy, validates identifiers and cross-references, canonicalizes it, verifies expected digest and runs a built-in denial corpus. Only then may it replace the current immutable snapshot. Failure retains the last valid snapshot within its allowed lifetime and marks readiness degraded.

DECISION LEDGER 46-61

Input	Local meaning	Forbidden inference
parse failure	reject candidate	partial publish
unknown entitlement	reject referenced mapping	ignore authority rule
unknown operation	reject candidate	generic fallback
digest mismatch	reject candidate	recompute and accept
denial corpus fails	reject candidate	deploy anyway
old snapshot expires	not ready/fail closed	indefinite stale allow

RELOAD IS A TRANSACTION

Readers observe either the previous complete policy or the new complete policy, never a mixture.

STALENESS**Different operations tolerate different policy age**

A read-only operator view may tolerate a bounded policy snapshot age, while repair approval or execution requires the current revision. The owner policy names maximum age and unavailable behavior; infrastructure caches cannot silently choose it.

DECISION LEDGER 46-62

Input	Local meaning	Forbidden inference
ordinary read	bounded last-valid snapshot	unbounded stale
operator mutation	current or short bound	cache default
compliance approval	current revision	stale entitlement
repair execute	current + fresh human auth	offline allow
pure workload automation	current workload policy	human mapping cache
expired snapshot	503/deny before effects	anonymous fallback

POLICY STATES UNAVAILABLE EXPLICITLY

The caller receives a bounded unavailability/denial contract; uncertainty never becomes an allow.

BFF DENY, OWNER DENY**The two authorization surfaces are intentionally non-equivalent**

The BFF rejects obvious missing edge capabilities or out-of-snapshot scopes to save work. The owner may still deny because its local mapping, assignment, resource state, separation-of-duty rule, operation freshness or current policy differs.

DECISION LEDGER 46-63

Input	Local meaning	Forbidden inference
BFF deny	owner not called	final business denial evidence at owner
BFF allow + owner allow	operation may proceed	effect already committed
BFF allow + owner deny	expected safe outcome	policy inconsistency bug by default
BFF stale	edge unavailable	call owner with broad context
owner stale	owner unavailable/deny	trust edge allow
resource changed	owner conflict/deny	replay previous allow

DEFENSE, NOT DUPLICATION

The BFF optimizes and protects its boundary; the owner enforces business truth. Their policies overlap only where necessary.

ALARM MANAGEMENT MAPPING

Acknowledging an alarm requires local site and state meaning

The BFF forwards one AlarmAcknowledge intent, one site scope, freshness facts and only entitlement references allowed by AlarmManagement's profile. AlarmManagement maps those references locally, verifies the subject's local assignment, loads alarm authorization state and applies status/version rules.

DECISION LEDGER 46-64

Input	Local meaning	Forbidden inference
requested intent	AlarmAcknowledge	generic OperatorCommand
external ref	AlarmOperatorCandidate input	final grant
scope	one SiteId	all session sites
local assignment	subject assigned to site/shift	IdP group membership
resource	alarm at site and acknowledgeable	route ID truth
precondition	current version	identity revision

OWNER RULE

Even a correctly mapped operator cannot acknowledge an alarm outside the local assignment or in a terminal state.

ROOTCAUSE MAPPING

Review authority is scoped to one case and decision stage

RootCause receives a pairwise subject reference, RootCauseReview intent, one case/site scope and only reviewer entitlement references. It maps them to reviewer-candidate facts and checks local case assignment, evidence state, current reviewer separation and version.

DECISION LEDGER 46-65

Input	Local meaning	Forbidden inference
review entitlement	candidate fact	verdict permission
case scope	target case/site	portfolio membership
assignment	current local reviewer assignment	external group truth
evidence state	review precondition	identity claim
separation	reviewer not prohibited actor	permanent role
version	optimistic precondition	context freshness

DOMAIN MEANING REMAINS LOCAL

The identity provider cannot know whether a RootCause case is ready for review or whether this subject is separated from the initiating action.

COMPLIANCE MAPPING

Approval depends on identity, separation and plan state

Compliance maps an approved entitlement reference to ApproverCandidate, but final authorization also requires current assurance/freshness, owner-local assignment, a distinct requester, an unexpired plan, correct risk class and current version.

DECISION LEDGER 46-66

Input	Local meaning	Forbidden inference
approver ref	candidate input	approval outcome
pairwise subject	compare with requester reference	raw human identity
assurance/auth time	privileged freshness	provider role
plan risk	local approval threshold	edge capability
plan expiry/state	current resource rule	context expiry
approval evidence	one decision reference	portable permission

SEPARATION SURVIVES MINIMIZATION

The owner receives the minimum stable reference needed to prove requester and approver are distinct without receiving names or emails.

REPORTING MAPPING

Read eligibility does not expose broad report membership

Reporting receives one report/read intent, one tenant or site coordinate and reader entitlement references. It maps locally, checks the report classification and projection readiness, and returns only the authorized delayed-truth view.

DECISION LEDGER 46-67

Input	Local meaning	Forbidden inference
read ref	Reporting.ReaderCandidate	all report access
scope	one tenant/site	membership list
classification	local resource rule	IdP claim
projection readiness	data availability	authorization
query filter	narrow requested view	grant widening
cache	vary by pairwise subject/scope/policy	shared private response

DELAYED TRUTH, CURRENT AUTHORIZATION

A report may be eventually consistent while its authorization decision still uses the current local policy and resource classification.

REPAIRCONTROL MAPPING

Request, approval and execution receive different contexts

Repair operations use separate intents and entitlement subsets. The requester context cannot approve; the approver reference must be distinct; execution requires the authenticated RepairControl workload from Chapter 45 plus current plan approval, owner capability and fresh human evidence where policy requires it.

DECISION LEDGER 46-68

Input	Local meaning	Forbidden inference
RepairRequest	requester candidate + scope	approval
RepairApprove	approver candidate + freshness	execution
RepairExecute	RepairControl workload + approved plan	BFF direct mutation
subject reference	separation comparison	display identity
plan ID/version	server-owned resource	replacement plan body
owner port	destination-local repair policy	global repair admin

NO SUPER-REPAIR ROLE

No external group or shared claim grants request, approve and execute authority together.

PURE WORKLOAD CALLS

Machine operations carry no invented human context

Workflow transitions, scheduled reconciliation and message consumers are authorized by workload identity and owner policy. They do not receive a synthetic operator reference or copied BFF entitlement set. If a process was originally human-initiated, immutable business/audit references preserve that history without turning it into current human authority.

DECISION LEDGER 46-69

Input	Local meaning	Forbidden inference
workflow timer	Workflow workload + local operation	operator context
message consumer	broker/workload provenance + contract	human claims
scheduled repair scan	RepairControl workload	repair execution grant
human-originated process	initiator evidence reference	live authorization
public fact	domain initiator reference if meaningful	raw subject/groups
owner command	current workload/local policy	historic edge allow

NO SYNTHETIC IDENTITY

Absence of a human is a valid typed state, not a reason to fabricate service-user or system-admin claims.

EVENTS AND CLAIMS

Authorization context does not become integration-event payload

Public facts describe domain meaning and causal lineage. They do not broadcast operator claim bags, group membership, assurance methods or local capabilities. When accountability is part of the event meaning, the producer emits a bounded initiator reference and decision/evidence reference.

DECISION LEDGER 46-70

Input	Local meaning	Forbidden inference
initiatedBy	owner-approved safe subject reference	iss/sub/email
authorizationDecisionId	evidence join	allow rules
workload producer	envelope provenance	human identity
capabilities/groups	excluded	consumer policy input
resource scope	domain payload only if meaningful	authorization set
audit detail	separate protected evidence	public event

HISTORICAL FACT, NOT FUTURE GRANT

A consumer may record who initiated an earlier action; it must not authorize a new action from that historical reference.

AUDIT EVIDENCE

Decision evidence joins revisions without storing claim bags

The security record contains pairwise subject reference, workload reference, destination, intent, scope class, result code, provider/edge/projection/owner policy identities, context and correlation references. Actual external claims and raw policy documents remain excluded.

DECISION LEDGER 46-71

Input	Local meaning	Forbidden inference
subject ref	audience-specific protected reference	raw iss/sub
workload ref	Chapter 45 caller evidence	certificate
policy identities	revision/digest	mapping contents
result code	closed safe class	sensitive explanation
context/request ref	replay/correlation	cookie/token
zero-effect ref	negative proof	business payload

EVIDENCE MINIMIZATION

Auditability requires stable references and configuration identity, not credential retention or indiscriminate PII.

TELEMETRY

Metrics use bounded classes; traces use protected references

Metrics report provider profile, destination, intent class, mapping outcome, owner result and policy revision with bounded cardinality. Pairwise subject, scope and context references are excluded from labels and appear only in protected traces/evidence when required.

DECISION LEDGER 46-72

Input	Local meaning	Forbidden inference
mapping_total	profile, outcome class	subject/entitlement value
projection_total	destination, intent, result	scope ID
owner_auth_total	owner, operation, result	resource ID
policy_staleness	owner/profile class	raw config
unknown_claim_total	provider profile, claim-class	claim value
context_size	destination/profile histogram	serialized body

NO IDENTITY ORACLE

Public health and metrics never reveal whether a particular human, group, site or owner grant exists.

PRIVACY AND CORRELATION

Cross-service correlation uses request evidence, not one global human ID

Trace and correlation IDs connect operations. Audience-specific subject references support local grants and protected evidence without allowing every service to correlate the same person automatically. Authorized incident workflows can reconcile references through controlled evidence, not ordinary request code.

DECISION LEDGER 46-73

Input	Local meaning	Forbidden inference
correlationId	causal request chain	human identity
contextId	delegated-context instance	session ID
decisionId	authorization evidence	credential
pairwise subject	one owner namespace	global subject
session reference	BFF local	owner/log label
incident resolution	controlled audited workflow	general lookup API

CORRELATION BY PURPOSE

Operational causality and human identity are separate coordinates and are joined only where a named purpose requires it.

CONTEXT SIZE BUDGET

Small contracts make over-forwarding measurable

Each destination profile declares maximum serialized bytes, entitlement count, string length and lifetime. The BFF records size-class metrics and rejects projection when the required context cannot fit the approved budget rather than silently adding a broad token.

DECISION LEDGER 46-74

Input	Local meaning	Forbidden inference
body bytes	small destination maximum	generic JWT size
entitlement refs	0..profile maximum	all groups
scope coordinates	exactly one	membership collection
subject reference	fixed scheme/length	issuer+sub
lifetime	operation profile maximum	session expiry only
unknown fields	zero	forward-compatible bag

BUDGET FAILURE

If an owner truly requires more data, the profile and contract change through review. A size exception cannot bypass minimization.

CONTEXT RESPONSES

Owners never echo identity context

Owner API responses return business contracts and safe problem details. They do not echo delegated context, subject references, entitlement sets, policy digests or detailed denial coordinates to the browser. The BFF translates responses without enriching them from its claim bag.

DECISION LEDGER 46-75

Input	Local meaning	Forbidden inference
success response	business DTO + correlation	identity context
401/403	safe problem type	which entitlement missing
404	bounded hidden-resource behavior	existence evidence
409/412	state/precondition reference	policy rule
503	dependency class	trust/config details
UI message	safe operator guidance	raw owner reason

ONE-WAY MINIMIZATION

A response cannot become a side channel that reconstructs the claims or policy removed from the request.

OFFICIAL TECHNICAL ANCHORS

The design follows maintained platform and protocol guidance

ASP.NET Core claim mapping and transformations:

<https://learn.microsoft.com/en-us/aspnet/core/security/authentication/claims?view=aspnetcore-8.0>

ASP.NET Core policy-based authorization:

<https://learn.microsoft.com/en-us/aspnet/core/security/authorization/policies?view=aspnetcore-8.0>

ASP.NET Core resource-based authorization:

<https://learn.microsoft.com/en-us/aspnet/core/security/authorization/resource-based?view=aspnetcore-8.0>

OpenID Connect Core 1.0 (issuer, subject, claims and pairwise identifiers):

https://openid.net/specs/openid-connect-core-1_0.html

OAuth 2.0 Security Best Current Practice: <https://www.rfc-editor.org/rfc/rfc9700>

JWT Best Current Practices: <https://www.rfc-editor.org/rfc/rfc8725>

JWT Profile for OAuth 2.0 Access Tokens: <https://www.rfc-editor.org/rfc/rfc9068>

OAuth 2.0 Authorization Framework: <https://www.rfc-editor.org/rfc/rfc6749>

NIST Zero Trust Architecture: <https://csrc.nist.gov/pubs/sp/800/207/final>

READING RULE

The sources constrain terminology and platform behavior. They do not prove that the NEXUS-1 repository has executed the specified mappings or tests.

WHAT THIS CHAPTER HAS PROVED

A complete mapping specification with deliberately local authority

Chapter 46 specifies ADR-046, exact provider claim profiles, issuer-plus-subject identity, strict type/cardinality rules, server-side normalized sessions, idempotent safe transformation, BFF edge mapping, destination-minimal contexts, audience-pseudonymous references, owner-local mapping and assignments, resource authorization, event/audit/telemetry minimization, no-central-database architecture, adversarial tests, fault laboratories, evidence manifests, rollout and incident procedures.

DECISION LEDGER 46-76

Claim	Status	Scope
mapping architecture	SPECIFIED	provider -> BFF -> owner boundaries
compile-oriented C# assets	SPECIFIED	typed seams and tests
runtime claim/leak evidence	PENDING	Chapter 48
key/config lifecycle	PENDING	Chapter 47
environment execution	PENDING	repository artifacts
production security maturity	NOT CLAIMED	outside Phase-0 proof

CENTRAL PROOF

Only required facts cross a boundary, and only the resource owner can convert them into an allow decision for its current state.

CHAPTER CHECKPOINT

Review what was removed before adding more identity data

- Why are iss and sub used together, and why are email and display name rejected as identity keys?
- Why is MapInboundClaims disabled for the provider profile?
- Which data remains inside the BFF session and never reaches an owner?
- How does DelegatedOperatorContextV1 narrow the broad Chapter 44 working context?
- Why does the forwarded contract contain requested intent but no allowed or capabilities field?
- How do audience-specific subject references reduce cross-service correlation?
- What local data must an owner combine with external entitlement references?
- Why can the BFF allow a route while the owner safely denies the resource?
- Which architecture test prevents a central authorization service or shared database?
- How do non-interference and leak tests prove claims minimization?
- Which key, configuration and audit obligations remain for Chapters 47 and 48?

NEXT - CHAPTER 47

Secrets, Certificates, and Configuration Handling will protect and inject the keys and credentials used by Chapters 44-46, rotate certificates and policy configuration safely, redact sensitive values and fail startup when security configuration is invalid.

PART VIII - SECURITY AND TRUST ACROSS SERVICES

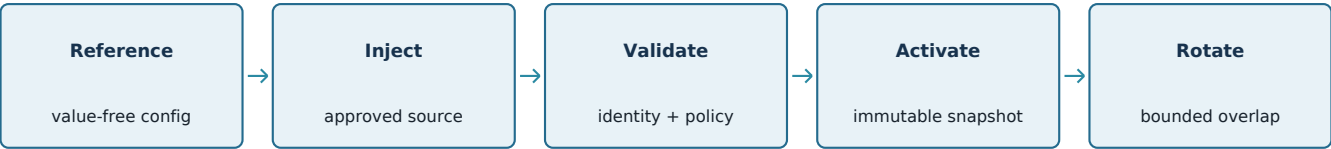
47. Secrets, Certificates, and Configuration Handling

Security material stays outside source and images, reaches only its consuming process, becomes active only as a complete validated snapshot, and rotates through bounded overlap without leaking values or weakening the trust decisions of Chapters 44-46.

CHAPTER DECISION

Store safe references and public policy in configuration; inject development values with .NET Secret Manager and Phase-0 runtime values through per-service read-only files; validate permissions, certificates, keys, policy compatibility and lifetime before atomic activation; fail startup when no valid security snapshot exists; retain only value-free evidence.

PROOF PATH 47-A - PROTECTED MATERIAL BECOMES BOUNDED RUNTIME STATE



GUARANTEE LEDGER 47-A

Boundary	Carries	Allowed claim	Still forbidden
source/image	references + schema	no material embedded	secret value/private key
deployment -> process	exact service grant	material available	cross-service bundle
candidate -> current	complete snapshot	validated activation	partial reload
rotation -> evidence	revision + result	bounded lifecycle	value/hash export

ORDINARY DEVELOPER PROBLEM

A value that starts the service can also compromise it

A developer places an OIDC client secret, database password and PEM key path in appsettings.json because configuration binding is convenient. The service starts, tests pass and the image is pushed. The same convenience has now copied credential material into source history, build context, image layers, diagnostics and every environment that reuses the file.

DECISION LEDGER 47-1

Input	Required handling	Forbidden shortcut
appsettings secret	replace with an external reference	commit the value
environment variable	treat as visible process state	call it encrypted
mounted file	grant only to one service	mount one shared directory
certificate path	validate key, chain, use and lifetime	file exists means valid
reload	validate a complete candidate	mutate options field by field

PLAIN-LANGUAGE PRINCIPLE

Configuration says what to use. A secret source supplies protected material. A certificate proves an identity. None of those is interchangeable, and none is safe merely because the application can read it.

CHAPTER BOUNDARY

References may travel; protected values stay at the consuming boundary

Chapter 47 makes the security profiles from Chapters 44-46 operable without widening them. Source, image and ordinary configuration contain stable references, revisions and non-secret policy. Each process resolves only the material it owns, validates a complete snapshot and exposes only value-free evidence.

DECISION LEDGER 47-2

Input	Required handling	Forbidden shortcut
source/image	schema, references, safe defaults	secret value/private key
deployment	per-service grants and mounts	global secret bundle
process	minimum resolved working set	enumerable vault
runtime	immutable validated snapshot	mutable IConfiguration reads
evidence	identity, revision, status, time	value or reversible digest

BOUNDARY INVARIANT

A component receives a secret or private key only when its own runtime responsibility requires that exact material.

DECISION RECORD

ADR-047 - Resolve outside the image; validate before activation

The Phase-0 decision uses .NET Secret Manager for local development, explicit test material for isolated tests and per-service read-only files for the reproducible Compose runtime. Configuration stores references only. Certificates, pairwise keys and policy documents rotate through staged immutable snapshots. No complete production secret-management platform is claimed.

DECISION LEDGER 47-3

Input	Required handling	Forbidden shortcut
development	user secrets or isolated fixtures	production credentials
Compose	read-only service-specific files	secret environment values
startup	resolve and validate all required material	partial service
reload	stage, validate, atomically swap	bind live mutable object
maturity	Phase-0 custody and rotation proof	HSM/vault certification

DECISION IDENTITY

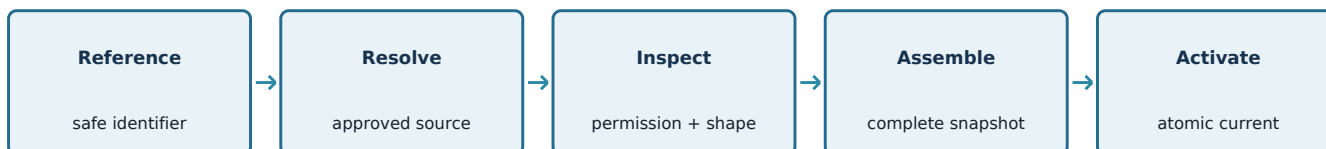
ADR-047 canonical SHA-256: ceb8a8221075d8509eb72e488bdf5560f21b6c3e84524ce56e360d33e7d6b880

TRUST FLOW

Material crosses five gates before use

A stable reference is admitted from configuration, resolved through one approved source, checked for permission and shape, assembled with related material, and activated only after cross-field validation. Runtime code consumes the immutable snapshot rather than rereading files on every request.

PROOF PATH 47-A - REFERENCE TO ACTIVE MATERIAL



PROOF OBLIGATIONS 47-A

Gate	Must prove	Must not expose
reference	known scheme and owner	material
resolve	exact source and bounded read	directory listing
inspect	permission, type, lifetime	log value
assemble	cross-material compatibility	partial snapshot
activate	one revision becomes current	mixed revision

ACTIVATION RULE

A readable value is only a candidate. It becomes runtime material after the whole security snapshot validates.

THREAT LEDGER**Disclosure is only one failure mode**

Security material can be disclosed, replaced, rolled back, expired, broadened, partially reloaded or made unavailable. Chapter 47 therefore treats integrity, freshness, ownership and recoverability as first-class alongside confidentiality.

DECISION LEDGER 47-4

Input	Required handling	Forbidden shortcut
disclosure	revoke/rotate and retain incident evidence	delete the log only
substitution	identity and chain checks	trust filename
rollback	monotonic accepted revision	activate older copy
partial reload	all-or-nothing snapshot	mixed old/new
unavailability	bounded fail-closed state	use empty/default
overgrant	per-service access	shared secret directory

SECURITY PROPERTY

Confidential material must remain secret, but public configuration and certificates still require integrity, ownership and lifecycle proof.

MATERIAL TAXONOMY**Config, secret, key, certificate and policy are different assets**

A certificate is normally public while its private key is secret. A policy document may be non-secret yet authorization-critical. A secret string may authenticate a client but cannot establish a trust chain. Typed categories prevent one generic configuration bag from erasing those differences.

DECISION LEDGER 47-5

Input	Required handling	Forbidden shortcut
operational config	ports, timeouts, feature choices	credential value
secret	password/token/client secret	public policy
symmetric key	HMAC/reference derivation	ordinary string
certificate	public identity and validity	private-key custody
trust bundle	approved roots/intermediates	endpoint key
policy document	reviewed decision mapping	secret container

TYPED HANDLING

The loader, validator, rotation rule, evidence shape and failure response are chosen by material kind, not by a generic key-value API.

OWNERSHIP INVENTORY

Every material coordinate has one issuer, custodian and consumer

The inventory distinguishes who creates a value, who grants access, which process consumes it, which revision is current and who may rotate or revoke it. No service may infer ownership from a shared path or naming convention.

DECISION LEDGER 47-6

Input	Required handling	Forbidden shortcut
OIDC client secret	identity admin -> BFF	owner services
BFF pairwise key	security admin -> BFF	reverse lookup
workload private key	Phase-0 CA -> one workload	peer workload
trust bundle	security admin -> validator	all host roots
owner policy	owner admin -> owner service	central policy service
database password	database admin -> one service	cross-database access

NO AMBIENT AUTHORITY

Access to one mounted secret does not imply permission to list, resolve or rotate any other material.

REFERENCE GRAMMAR

A reference is narrow enough to validate before resolution

NEXUS-1 uses typed references that name a material kind, approved source, logical identifier and expected revision policy. Raw values, inline data URIs, command substitutions and arbitrary absolute paths are rejected by the production profile.

DECISION LEDGER 47-7

Input	Required handling	Forbidden shortcut
file:/run/secrets/name	approved root + exact file	../ traversal
user-secret:Section:Key	Development only	production mode
test-fixture:name	isolated test host	deployed process
revision	exact/current/minimum rule	latest string guess
kind	secret/cert/key/policy	automatic coercion

REFERENCE IS NOT SECRECY

A reference may be logged only when it contains no credential, subject identifier or provider access token and its disclosure does not reveal a protected value.

TYPED OPTIONS

Configuration binds references, never protected values

The options graph contains paths and logical revisions. The validator rejects inline values and cross-field inconsistencies before any business endpoint is mapped.

EXECUTABLE ASSET 47-A - REFERENCE-ONLY OPTIONS

```
public sealed record SecurityMaterialOptions
{
    public const string Section = "SecurityMaterial";
    public required MaterialRef OidcClientSecret { get; init; }
    public required CertificateRef ServerCertificate { get; init; }
    public required CertificateRef WorkloadClientCertificate { get; init; }
    public required MaterialRef PairwiseReferenceKeyRing { get; init; }
    public required MaterialRef OwnerPolicyDocument { get; init; }
}

public sealed record MaterialRef(string Scheme, string Identifier);
public sealed record CertificateRef(
    MaterialRef CertificatePem,
    MaterialRef PrivateKeyPem,
    string ExpectedDnsName,
    string Revision);
```

NO VALUE PROPERTY

The production options type has no Password, Secret, PrivateKey or Token value field. Resolution occurs behind a material-specific port.

STARTUP REGISTRATION

Options validation is necessary but not sufficient

Binding validates the reference graph. A hosted startup gate then resolves files, validates permissions, certificates, keys and policy compatibility, and publishes the first complete snapshot. ValidateOnStart alone cannot prove external material exists or is usable.

EXECUTABLE ASSET 47-B - TWO-STAGE STARTUP VALIDATION

```
services.AddOptions<SecurityMaterialOptions>()
    .BindConfiguration(SecurityMaterialOptions.Section)
    .ValidateDataAnnotations()
    .ValidateOnStart();

services.AddSingleton<IValidateOptions<SecurityMaterialOptions>,
    SecurityMaterialReferenceValidator>();
services.AddSingleton<SecurityMaterialSnapshotStore>();
services.AddSingleton<ISecurityMaterialLoader, SecurityMaterialLoader>();
services.AddHostedService<SecurityMaterialInitializer>();
```

TWO DISTINCT PROOFS

Reference validation proves configuration shape. Material initialization proves the referenced security set can actually be activated.

CONFIGURATION PRECEDENCE

Later providers may select references, not smuggle values

ASP.NET Core provider precedence is explicit and value-free in the evidence manifest. Development user secrets can replace development references. Deployment environment variables may select mounted-file references. Command-line secret values and secret-bearing appsettings files are forbidden.

DECISION LEDGER 47-8

Input	Required handling	Forbidden shortcut
appsettings.json	safe defaults and reference schema	secret value
environment appsettings	safe environment references	password
user secrets	Development protected values	production source
environment variables	reference/path selection	production secret value
command line	non-secret diagnostics only	credential argument

PRECEDENCE EVIDENCE

The manifest records provider order and winning key names, never winning values.

ENVIRONMENT BOUNDARY

The environment name is not a security control

ASPNETCORE_ENVIRONMENT selects behavior but does not authenticate the host or protect a file. Development-only sources are admitted only when the host profile, build manifest and explicit startup policy all agree. A process started as Development in a deployed image fails.

DECISION LEDGER 47-9

Input	Required handling	Forbidden shortcut
Development	local identity + user-secret source	production material
Test	ephemeral fixture identities	developer credentials
Staging	deployment mounts + staging issuer	production reuse
Production	approved external source profile	user secrets
unknown name	startup failure	Production guess

NO MAGIC STRING

An environment label changes allowed sources only after a validated deployment profile admits it.

DEVELOPMENT SECRET INJECTION

Secret Manager keeps values out of the project, not out of the machine

For local development, dotnet user-secrets supplies the BFF client secret and test-only passwords outside the repository. The store is a development convenience, is not encrypted as a production vault and must never contain staging or production credentials.

DECISION LEDGER 47-10

Input	Required handling	Forbidden shortcut
UserSecretsId	stable project-local identifier	shared production ID
dotnet user-secrets set	developer-owned local value	script with literal value
repository	reference key names only	secrets.json
test	dedicated disposable value	real provider secret
rotation	replace local value/restart	production lifecycle claim

BOUNDED CLAIM

Secret Manager reduces accidental source-control disclosure during development. It does not prove production custody, access review, HSM use or automatic rotation.

DEVELOPMENT COMMANDS

Examples reveal key names but never example credentials

Documentation uses shell placeholders and verifies only presence. A committed script may name the configuration coordinate, but the operator supplies the value interactively or through a protected local channel.

DEVELOPMENT ASSET 47-A - USER-SECRET COORDINATES

```
dotnet user-secrets init --project src/Nexus1.Bff
dotnet user-secrets set \
  "SecurityMaterial:OidcClientSecretValue" "<interactive-value>" \
  --project src/Nexus1.Bff

dotnet user-secrets list --project src/Nexus1.Bff
dotnet user-secrets remove \
  "SecurityMaterial:OidcClientSecretValue" \
  --project src/Nexus1.Bff
```

PUBLICATION RULE

The book and repository show coordinates and commands. They never contain a plausible secret, private key block, certificate password or recoverable digest.

DEVELOPMENT SEPARATION

Each developer and automated test receives a different identity

Shared development credentials turn one accidental disclosure into a team-wide incident and make revocation ambiguous. Phase-0 uses separate local identities where the external provider permits them and disposable test material for automated runs.

DECISION LEDGER 47-11

Input	Required handling	Forbidden shortcut
developer BFF client	personal/local registration	shared team secret
component tests	ephemeral generated CA and leaves	developer certificate
contract tests	synthetic non-secret vectors	live credential
demo runtime	dedicated Phase-0 material	production identity
revocation	one identity coordinate	rotate everyone

IDENTITY BEFORE SECRET

A different string under the same overprivileged shared principal is not meaningful isolation.

COMPOSE INJECTION

Secrets are mounted as files and granted per service

The reproducible Docker Compose runtime references external secret files and grants each secret only to the service that consumes it. Docker Compose exposes a granted secret under `/run/secrets`. The design does not place protected values in image ENV instructions, `compose.yaml`, `.env` or command arguments.

DECISION LEDGER 47-12

Input	Required handling	Forbidden shortcut
top-level secret	external file source/reference	inline value
service grant	explicit one-service list	global mount
container path	<code>/run/secrets/exact-name</code>	host secret directory
mount	read-only and bounded	writable replacement
image	expects a reference	contains fallback secret

COMPOSE LIMITATION

A file mount is an injection mechanism, not a complete secret-management platform. Host-file custody, rotation authority and access evidence remain explicit obligations.

COMPOSE CONTRACT

The BFF receives only its own OIDC and pairwise material

The service definition grants exact secret names and passes only reference paths as non-secret environment configuration. Owner services do not receive the BFF OIDC client secret or pairwise derivation key.

DEPLOYMENT ASSET 47-A - PER-SERVICE COMPOSE GRANTS

```
services:
  nexus-bff:
    image: nexus1/bff:${NEXUS_IMAGE_TAG:?required}
    secrets:
      - bff_oidc_client_secret
      - bff_pairwise_keyring
    environment:
      SecurityMaterial__OidcClientSecret__Identifier: file:/run/secrets/bff_oidc_client_secret
      SecurityMaterial__PairwiseReferenceKeyRing__Identifier: file:/run/secrets/bff_pairwise_keyring

secrets:
  bff_oidc_client_secret:
    file: ${NEXUS_SECRET_ROOT:?required}/bff/oidc-client-secret
  bff_pairwise_keyring:
    file: ${NEXUS_SECRET_ROOT:?required}/bff/pairwise-keyring.json
```

REFERENCE-ONLY ENVIRONMENT

Environment variables contain file references. The secret values remain outside the process environment and image metadata.

FILE ADMISSION

Resolve beneath one approved root without following a path escape

A file resolver canonicalizes the approved root and requested path, rejects traversal and symbolic-link surprises according to the host profile, bounds size, opens for read only and requires a regular file. Directory enumeration is not part of the runtime interface.

DECISION LEDGER 47-13

Input	Required handling	Forbidden shortcut
root	configured absolute approved directory	current directory
relative name	strict safe grammar	../ or separator injection
canonical path	must remain under root	prefix string only
file type	regular file	directory/device/socket
size	material-specific maximum	read unbounded
open	read-only/no write	create missing file

NO FALLBACK SEARCH

If the exact reference is unavailable, the resolver fails. It does not search home directories, certificate stores or neighboring mounts.

BOUNDED RESOLVER

A secret file is copied once into a bounded buffer

The resolver reads exact bytes with a maximum length and rejects empty or oversized values. The returned object owns the buffer and clears it when disposed. Framework integrations that require strings are treated as narrower exceptions.

EXECUTABLE ASSET 47-C - BOUNDED FILE SECRET

```
public async ValueTask<SecretBuffer> ReadAsync(
    FileSecretRef reference,
    CancellationToken cancellationToken)
{
    var path = paths.ResolveUnderApprovedRoot(reference.RelativeName);
    permissions.DemandPrivateRegularFile(path);

    await using var stream = new FileStream(path, FileMode.Open,
        FileAccess.Read, FileShare.Read, 4096, FileOptions.Asynchronous);
    if (stream.Length is <= 0 or > 16_384)
        throw new SecurityMaterialException("secret-size-invalid");

    var bytes = GC.AllocateUninitializedArray<byte>(
        checked((int)stream.Length));
    await stream.ReadExactlyAsync(bytes, cancellationToken);
    return new SecretBuffer(bytes);
}
```

BOUNDED LIFETIME

Disposal reduces residual copies; it cannot erase values copied into immutable strings, framework internals, process dumps or external libraries.

SECRET BUFFER

Owned bytes are cleared after the integration boundary

The wrapper exposes a read-only span only while alive and uses `CryptographicOperations.ZeroMemory` during disposal. Callers cannot retain the backing array through the public contract.

EXECUTABLE ASSET 47-D - CLEARABLE SECRET BUFFER

```
public sealed class SecretBuffer(byte[] bytes) : IDisposableable
{
    private byte[]? _bytes = bytes;

    public ReadOnlySpan<byte> Span => _bytes is { } value
        ? value
        : throw new ObjectDisposedException(nameof(SecretBuffer));

    public void Dispose()
    {
        var value = Interlocked.Exchange(ref _bytes, null);
        if (value is not null)
            CryptographicOperations.ZeroMemory(value);
    }
}
```

HONEST LIMIT

Memory clearing is defense in depth. It does not turn a general-purpose managed process into a hardened secret enclave.

FILE PERMISSIONS

Readable by the service is not the same as safely owned

On Unix Phase-0 hosts, the startup gate verifies that private-key and secret files are not writable or readable by other users beyond the approved service group policy. Windows development uses a separate ACL-aware profile. Unsupported permission semantics fail rather than being silently ignored.

DECISION LEDGER 47-14

Input	Required handling	Forbidden shortcut
owner	expected service UID	root-owned but world-readable
group	explicit approved service group	shared application group
other bits	none for secret/private key	read permission
write bits	custodian only	service replaces material
directory	no untrusted traversal/write	safe file in unsafe dir
platform	named validation profile	skip unknown OS

PERMISSION FAILURE

A service that cannot prove the expected permission model does not activate the material.

UNIX PERMISSION GATE

Mode checks are explicit and material-specific

The Phase-0 Unix profile permits owner read and an optional approved group read, but rejects every other read/write/execute bit for secret files. Deployment ownership is verified separately from mode bits.

EXECUTABLE ASSET 47-E - UNIX MODE VALIDATOR

```
public static void DemandSecretMode(string path, bool allowGroupRead)
{
    if (OperatingSystem.IsWindows())
        throw new PlatformNotSupportedException("Use the Windows ACL profile.");

    var mode = File.GetUnixFileMode(path);
    var allowed = UnixFileMode.UserRead |
        (allowGroupRead ? UnixFileMode.GroupRead : 0);
    var forbidden = mode & ~allowed;

    if ((mode & UnixFileMode.UserRead) == 0 || forbidden != 0)
        throw new SecurityMaterialException("secret-permission-invalid");
}
```

NO CHMOD REPAIR

The application reports the defect and exits. It does not silently change deployment permissions or ownership.

ENVIRONMENT VARIABLES

Process environment is observable state, not a vault

Environment variables are acceptable for safe references and some constrained development uses, but production secret values are avoided in the Phase-0 Compose profile. Process inspection, crash collection, child processes and diagnostics can expose them.

DECISION LEDGER 47-15

Input	Required handling	Forbidden shortcut
reference path	allowed non-secret coordinate	inline PEM
feature flag	safe bounded value	authorization override
connection secret	file reference	full connection string
diagnostic dump	key names only	environment values
child process	clean allow-listed environment	inherit all

PROVIDER WARNING

Configuration precedence is not confidentiality. A later provider can still leak a value if the schema allows it.

FRAMEWORK STRING BOUNDARY

Some .NET integrations require immutable strings

OIDC and database libraries may require a string client secret or connection string. The adapter creates that string as late as possible, never logs it, avoids caching additional copies and scopes its lifetime to the framework configuration. The evidence record marks this residual limitation.

DECISION LEDGER 47-16

Input	Required handling	Forbidden shortcut
byte buffer	preferred internal representation	global string
UTF-8 decode	late adapter boundary	startup dump
framework options	minimum required property	whole secret object
logging	event ID and reference only	options serialization
disposal	dispose owner objects where supported	guaranteed erasure

NO FALSE CLAIM

Managed-code discipline reduces unnecessary copies; the chapter does not claim deterministic erasure of every framework-created string.

OIDC CLIENT MATERIAL

The browser login secret belongs only to the BFF

The BFF resolves its client authentication material after validating the exact issuer and client profile. Owner services never receive it. Rotation supports the provider's configured overlap or a coordinated restart; the book does not assume every provider accepts two simultaneous client secrets.

DECISION LEDGER 47-17

Input	Required handling	Forbidden shortcut
client ID	non-secret exact provider identifier	owner identity
client secret	BFF-only protected material	browser setting
redirect URI	exact reviewed configuration	wildcard
issuer metadata	TLS validated provider profile	secret source
rotation	provider capability-specific plan	universal overlap

NO BROWSER SECRET

Anything delivered to Angular is public to the browser. A confidential OIDC client credential remains server-side in the BFF.

CERTIFICATE INVENTORY

Public identity, private key and trust roots rotate independently

Chapter 45 established workload identity. Chapter 47 supplies its material: server leaves, client leaves, private keys, intermediate chains and trust roots. The snapshot records their relationship without treating a thumbprint as the complete identity policy.

DECISION LEDGER 47-18

Input	Required handling	Forbidden shortcut
server leaf	DNS/SAN + server EKU	client identity
client leaf	workload SAN + client EKU	human identity
private key	matches one leaf	trust anchor
intermediate	build approved chain	root by position
root	explicit trust anchor	leaf allow-list
revision	deployment lifecycle coordinate	certificate truth

IDENTITY RULE

Chain trust, lifetime, key usage, SAN mapping and local workload policy are evaluated together. A matching thumbprint alone is insufficient.

PEM LOADING

Certificate and private key are loaded as one candidate

The loader uses the platform PEM API, requires a private key for endpoint identity and rejects malformed or mismatched key material. It does not copy PEM text into IConfiguration.

EXECUTABLE ASSET 47-F - PEM IDENTITY LOADER

```
public X509Certificate2 LoadIdentity(CertificateRef reference)
{
    var certificatePath = paths.Resolve(reference.CertificatePem);
    var privateKeyPath = paths.Resolve(reference.PrivateKeyPem);
    permissions.DemandPublicCertificate(certificatePath);
    permissions.DemandPrivateRegularFile(privateKeyPath);

    var certificate = X509Certificate2.CreateFromPemFile(
        certificatePath, privateKeyPath);
    if (!certificate.HasPrivateKey)
    {
        certificate.Dispose();
        throw new SecurityMaterialException("certificate-key-missing");
    }
    return certificate;
}
```

API BOUNDARY

CreateFromPemFile verifies that the private key matches the certificate public key; policy validation still must prove identity, use, chain and time.

CERTIFICATE POLICY

A cryptographically valid certificate can still be the wrong identity

Validation crosses chain trust, time, key strength, signature policy, EKU, SAN, expected workload mapping and revocation profile. The startup gate validates local certificates, while peers repeat the applicable checks during TLS authentication.

DECISION LEDGER 47-19

Input	Required handling	Forbidden shortcut
chain	approved roots/intermediates	any system root
validity	now within NotBefore/NotAfter	startup only forever
EKU	server or client as required	any-purpose
SAN	exact DNS/workload grammar	subject display name
algorithm/key	approved profile	platform accepts
revocation	explicit mode and availability rule	disabled silently

POLICY IDENTITY

The certificate result records the policy revision used to validate it, not only the certificate thumbprint.

CUSTOM TRUST

Phase-0 trust roots are explicit inputs

The component validator builds a chain against the exact Phase-0 root set. CustomTrustStore is effective only with CustomRootTrust. Intermediates belong in ExtraStore and the chosen revocation behavior is recorded as a limitation.

EXECUTABLE ASSET 47-G - EXPLICIT CUSTOM-ROOT CHAIN

```
public static bool BuildsTo(
    X509Certificate2 leaf,
    X509Certificate2 root,
    IEnumerable<X509Certificate2> intermediates)
{
    using var chain = new X509Chain();
    chain.ChainPolicy.TrustMode = X509ChainTrustMode.CustomRootTrust;
    chain.ChainPolicy.CustomTrustStore.Add(root);
    chain.ChainPolicy.ExtraStore.AddRange(
        new X509Certificate2Collection(intermediates.ToArray()));
    chain.ChainPolicy.RevocationMode = X509RevocationMode.NoCheck;
    chain.ChainPolicy.VerificationTime = DateTime.UtcNow;
    return chain.Build(leaf);
}
```

PHASE-0 LIMITATION

NoCheck is an explicit offline demonstrator choice, not production guidance. Chapter evidence must show the selected revocation profile rather than hiding it.

CERTIFICATE TIME

Validity, rotation readiness and connection lifetime are separate clocks

A leaf may be valid now but too close to expiry for safe activation. Rotation readiness requires a minimum remaining lifetime. Existing TLS connections can outlive a certificate swap, so tests distinguish new handshakes from already established channels.

DECISION LEDGER 47-20

Input	Required handling	Forbidden shortcut
NotBefore	not future beyond skew	truncate to date
NotAfter	minimum activation horizon	valid for one second
rotation due	renew-before threshold	wait for expiry
connection	new handshake selects current	old socket changes identity
clock	injected UTC source in tests	local wall-time guess

HANDSHAKE BOUNDARY

Certificate rotation affects new TLS handshakes. Draining or recycling long-lived connections is a separate operational step.

SERVER SELECTION

Kestrel reads the current immutable certificate snapshot

The selector returns the active certificate for each new handshake. The snapshot store owns disposal and retains replaced material until the bounded drain window has passed. Request code never reads PEM files.

EXECUTABLE ASSET 47-H - SERVER CERTIFICATE SELECTOR

```
builder.WebHost.ConfigureKestrel((context, kestrel) =>
{
    var endpoint = context.Configuration.GetSection("Endpoints:Https");
    var port = endpoint.GetValue<int>("Port");

    kestrel.ListenAnyIP(port, listen => listen.UseHttps(https =>
    {
        https.ServerCertificateSelector = (_, _) =>
            certificateStore.Current.ServerCertificate;
    }));
});
```

OWNERSHIP WARNING

A selector makes activation possible. It does not define issuance, overlap, trust distribution, connection draining or rollback by itself.

OUTBOUND IDENTITY

Client-certificate rotation also depends on connection pooling

An outbound handler can select current material for a new TLS connection, but an existing pooled connection continues under its established session. Rotation tests therefore force new handshakes and prove the old certificate is rejected only after the planned overlap and drain.

DECISION LEDGER 47-21

Input	Required handling	Forbidden shortcut
new connection	select current client leaf	random eligible cert
pooled connection	bounded lifetime/drain plan	instant identity change
server trust	current approved roots	accept any certificate
overlap	old and new peer acceptance	dual write
retirement	old rejected after drain	delete first

TRANSPORT EVIDENCE

A rotation test must observe certificate identity on real new handshakes; inspecting the snapshot object is not enough.

ROTATION PROTOCOL

Stage, verify, activate, drain and retire are distinct states

Rotation is a distributed compatibility change. The new material is staged without use, validated locally, admitted by peers, activated for new work, observed during a bounded drain and only then used to retire the old material.

DECISION LEDGER 47-22

Input	Required handling	Forbidden shortcut
stage	new revision present but inactive	overwrite current
verify	local checks + peer compatibility	file exists
activate	single current write revision	random selection
drain	old sessions/messages bounded	immediate revoke
retire	old no longer accepted	keep forever
abort	candidate removed; current unchanged	mixed rollback

SINGLE-WRITE RULE

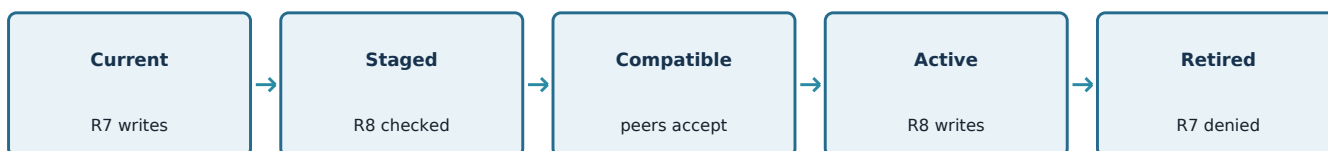
During normal overlap, new work uses one current revision. Dual acceptance exists only where compatibility requires it.

ROTATION STATE MACHINE

Activation advances only after peer readiness

The state machine is monotonic for one rotation attempt. An abort returns to the previous active revision; it does not partially retain a new key, policy or trust root. A later retry receives a new attempt identity.

STATE MACHINE 47-A - BOUNDED OVERLAP



ROTATION EVIDENCE 47-A

Transition	Positive proof	Negative proof
Current -> Staged	candidate loads	not selected
Staged -> Compatible	all required peers ready	unready peer blocks
Compatible -> Active	new handshake/write R8	no new R7
Active -> Retired	drain elapsed	R7 rejected
any -> Abort	R7 remains complete	no mixed snapshot

NO TIME-ONLY PROMOTION

Elapsed time can satisfy a drain condition, but it cannot substitute for missing peer-readiness or new-revision-use evidence.

TRUST-BUNDLE ROTATION

Roots expand before leaves change and contract after drain

A CA or trust-root change requires a different order from a leaf renewal. Validators first accept both old and new approved chains, then issuers activate new leaves, then evidence proves old leaves have drained, and only then is the old root removed.

DECISION LEDGER 47-23

Input	Required handling	Forbidden shortcut
trust expand	old + new roots accepted	new only first
leaf stage	new chain complete	unknown intermediate
leaf activate	new handshakes use new chain	old-only peer
drain	old leaf usage reaches zero/bound	assume
trust contract	old root removed	keep global root

ASYMMETRIC ORDER

Adding trust before changing identity preserves availability. Removing old trust before the last old identity drains creates a distributed outage.

EMERGENCY REVOCATION

Compromise can override graceful overlap

A suspected private-key compromise may require immediate trust removal and connection termination. The runbook accepts availability loss, records the exact blast radius and never restores a compromised credential merely to make readiness green.

DECISION LEDGER 47-24

Input	Required handling	Forbidden shortcut
suspected key theft	disable revision and drain/kill	graceful overlap
wrong SAN issued	reject identity policy	trust chain only
root compromise	replace trust set and all descendants	leaf-only rotate
unknown exposure	bound affected consumers	global guess
recovery	fresh material + negative old test	manual status pass

SECURITY OVER AVAILABILITY

Emergency rotation may make the service unavailable; it may not keep known-compromised material active as a hidden fallback.

PAIRWISE-KEY CUSTODY

The BFF owns derivation keys; owners receive only references

Chapter 46 derives audience-specific operator references. Chapter 47 keeps the HMAC key ring inside the BFF, separates keys by revision and audience policy, and prevents any owner, event or evidence record from reconstructing issuer-plus-subject mappings.

DECISION LEDGER 47-25

Input	Required handling	Forbidden shortcut
key bytes	BFF-only protected buffer	owner service
revision	public lifecycle identifier	key digest
audience	canonical derivation input	shared output namespace
output ref	owner-local pseudonymous ID	reversible token
evidence	revision and safe reference	iss/sub/key

NO REVERSE SERVICE

Rotation compatibility is implemented through explicit owner assignment bridges, not a central API that translates every old and new human reference.

KEY-RING SNAPSHOT

One revision writes while a bounded set remains readable

The key ring is immutable. Current selects the sole derivation revision. Accepted revisions support bounded verification or assignment migration. Key bytes never appear in ToString, JSON or evidence models.

EXECUTABLE ASSET 47-I - IMMUTABLE KEY-RING CONTRACT

```
public sealed class PairwiseKeyRing : IDisposable
{
    private readonly FrozenDictionary<KeyRevision, SecretBuffer> _keys;
    public KeyRevision Current { get; }
    public FrozenSet<KeyRevision> Accepted { get; }

    public ReadOnlySpan<byte> CurrentKey => _keys[Current].Span;
    public ReadOnlySpan<byte> Require(KeyRevision revision) =>
        Accepted.Contains(revision)
            ? _keys[revision].Span
            : throw new SecurityMaterialException("key-revision-rejected");

    public void Dispose()
    {
        foreach (var key in _keys.Values) key.Dispose();
    }
}
```

ONE-WAY EVIDENCE

The manifest records Current and Accepted revision IDs. It never records a raw hash of a potentially low-entropy secret or key file.

PAIRWISE ROTATION

New references write once; old assignments bridge explicitly

When R8 activates, the BFF derives only R8 references. Each owner receives an audited assignment migration that links its local R7 and R8 references for approved subjects. The overlap expires and R7 contexts are rejected after migration evidence completes.

DECISION LEDGER 47-26

Input	Required handling	Forbidden shortcut
stage R8	fixed vectors and custody checks	emit R8
bridge	owner-local R7/R8 assignment relation	global map
activate	new contexts use R8 only	random revision
overlap	owner admits bounded R7 contexts	new R7 grant
retire	R7 context and key rejected	retain key forever

LOCAL CONTINUITY

The rotation preserves owner-local grants without making the BFF or a new service the authorization database.

POLICY MATERIAL

Authorization policy is not secret, but it is security-critical

Provider, edge, projection and owner policies are versioned documents with strict schemas and canonical digests. They can be reviewed and compared without confidentiality, but unauthorized modification or partial reload can grant authority.

DECISION LEDGER 47-27

Input	Required handling	Forbidden shortcut
schema	closed and versioned	unknown field ignored
canonical bytes	deterministic digest	pretty-print identity
revision	monotonic policy coordinate	timestamp only
signature/source	approved administrative path	writable app path
reload	complete cross-policy snapshot	one document at a time

INTEGRITY BEFORE SECRECY

A public policy file can still be the most dangerous mutable input in the service.

CANONICAL POLICY IDENTITY

Policy evidence is computed from strict canonical content

The loader parses with an exact schema, normalizes only defined semantic forms and serializes canonical UTF-8 before SHA-256. A digest is an identity and drift signal, not proof that the policy is authorized or correct.

EXECUTABLE ASSET 47-J - CANONICAL POLICY IDENTITY

```
public static ConfigurationIdentity Identify<T>(
    T policy,
    string schema,
    string revision)
{
    var bytes = JsonSerializer.SerializeToUtf8Bytes(
        policy, CanonicalJson.Options);
    var digest = SHA256.HashData(bytes);
    return new ConfigurationIdentity(
        schema,
        revision,
        Convert.ToHexString(digest).ToLowerInvariant(),
        bytes.Length);
}
```

DIGEST BOUNDARY

A digest proves byte identity after canonicalization. Administrative authorization, semantic safety and freshness require separate evidence.

SNAPSHOT COMPATIBILITY

Cross-field rules validate the security set as one unit

A server certificate, trust bundle, workload map, OIDC profile, projection policy and pairwise key ring may each be valid alone yet incompatible together. The candidate validator checks the exact relationships used by runtime before activation.

DECISION LEDGER 47-28

Input	Required handling	Forbidden shortcut
certificate SAN	matches workload identity map	certificate subject guess
trust roots	validate current and staged leaves	unused root set
policy revision	known context contract revision	latest independently
key current	present in accepted set	missing write key
overlap deadline	covers deployment/drain bound	unbounded dual read
environment	allowed material sources	Development provider

COMPLETE CANDIDATE

Activation accepts one compatible graph of identities, policies, keys and lifetimes. It never combines the newest value from each source independently.

ATOMIC ACTIVATION

Readers observe either the old snapshot or the new snapshot

A single-flight reload builds an isolated candidate, validates it, then swaps one reference. The store delays disposal of the retired snapshot until active readers and connection-drain obligations are bounded.

EXECUTABLE ASSET 47-K - ATOMIC SNAPSHOT STORE

```
public sealed class SecurityMaterialSnapshotStore
{
    private SecurityMaterialSnapshot? _current;

    public SecurityMaterialSnapshot Current =>
        Volatile.Read(ref _current) ??
        throw new SecurityMaterialUnavailableException("not-initialized");

    public SecurityMaterialSnapshot Activate(
        SecurityMaterialSnapshot validatedCandidate)
    {
        validatedCandidate.DemandValid();
        return Interlocked.Exchange(ref _current, validatedCandidate)
            ?? SecurityMaterialSnapshot.Empty;
    }
}
```

DISPOSAL IS SEPARATE

Interlocked.Exchange makes visibility atomic; safe retirement still requires ownership, reader leases and protocol-specific drain logic.

RELOAD SEMANTICS

A bad candidate cannot corrupt a good current snapshot

Runtime reload failures are classified. A malformed optional future candidate is rejected while the last-known-good snapshot continues. A revoked current credential, expired certificate or explicitly unsafe current policy makes the dependency unavailable and readiness fails closed.

DECISION LEDGER 47-29

Input	Required handling	Forbidden shortcut
candidate malformed	reject candidate; keep current	partial merge
candidate incompatible	report blocked rotation	activate subset
current near expiry	readiness warning/failure by threshold	ignore
current expired/revoked	stop protected operation	last-known-good forever
source unavailable	bounded retry; current policy rule	empty default

LAST-KNOWN-GOOD LIMIT

Continuing a valid current snapshot is safe only within its explicit validity, revocation and staleness policy.

STARTUP STATE MACHINE

The process becomes ready only after material activation

Liveness remains process-local. Startup resolves and validates required material within a bound, activates the first snapshot, then permits readiness. Business endpoints may be mapped earlier but receive no traffic until readiness succeeds.

DECISION LEDGER 47-30

Input	Required handling	Forbidden shortcut
Created	no material claim	ready
Resolving	bounded external/file reads	serve protected work
Validating	complete candidate checks	partial activation
Ready	current snapshot active	expose values
Failed	stable safe code and exit/not-ready	permissive default

FAIL-CLOSED STARTUP

Missing, malformed, over-permitted, expired or incompatible required security material prevents readiness and, where configured, terminates startup.

BOUNDED INITIALIZER

Initialization has one deadline and one published result

The initializer creates a linked timeout, loads the complete candidate and activates only after validation. Failure records a safe code and rethrows so the host cannot appear healthy with no security snapshot.

EXECUTABLE ASSET 47-L - BOUNDED SECURITY INITIALIZER

```
public async Task StartAsync(CancellationTokentoken)
{
    using var timeout = CancellationTokenSource.CreateLinkedTokenSource(
        cancellationTokentoken);
    timeout.CancelAfter(options.StartupTimeout);

    try
    {
        var candidate = await loader.LoadAsync(timeout.Token);
        validator.DemandValid(candidate, clock.GetCurrentInstant());
        store.Activate(candidate);
        state.MarkReady(candidate.ManifestIdentity);
    }
    catch (Exception ex)
    {
        state.MarkFailed(SecurityFailureClassifier.SafeCode(ex));
        throw;
    }
}
```

SAFE DIAGNOSTICS

The exception reaching logs is classified and sanitized. Raw paths, values, PEM content and provider responses are not emitted.

STARTUP FAILURE MATRIX

Different defects converge on one safe operational result

Tests vary one defect at a time and assert that no protected endpoint, worker or publisher performs business work. Operators receive a bounded code that identifies the remediation class without revealing material.

DECISION LEDGER 47-31

Input	Required handling	Forbidden shortcut
reference missing	security-material-reference-missing	path/value
permission broad	security-material-permission-invalid	mode dump
PEM/key mismatch	certificate-key-mismatch	PEM
chain/SAN/EKU invalid	certificate-policy-invalid	subject details
policy/key incompatibility	security-snapshot-incompatible	secret digest
deadline exceeded	security-material-timeout	continue empty

ZERO-EFFECT ASSERTION

Every startup-failure row proves zero listener admission, zero broker consumption and zero business database mutation.

HEALTH ENDPOINTS

Health reports state, not material

Readiness reports whether a valid current snapshot exists and whether its bounded expiry/staleness policy is satisfied. It exposes a safe status and manifest reference, not filenames, subjects, certificate chains, secret references or policy bodies.

DECISION LEDGER 47-32

Input	Required handling	Forbidden shortcut
liveness	process loop responsive	secret-source call
startup	initialization completed/failed	value
readiness	current snapshot usable	certificate details
rotation	safe phase and attempt ID	key revision inventory
degraded	safe dependency code	path/provider error

NO ORACLE

A public health response cannot be used to enumerate configured issuers, certificates, paths, owner grants or secret-source identifiers.

LOGGING BOUNDARY

Redaction starts with event design, not string replacement

Security events use fixed names and allow-listed scalar fields. The logger receives safe references and classifications, never arbitrary exception Data, configuration objects, request bodies, certificate PEM or environment dictionaries.

DECISION LEDGER 47-33

Input	Required handling	Forbidden shortcut
event name	stable bounded identifier	exception message template
material kind	small enum	path
revision	public lifecycle ID	secret hash
result	safe code	provider payload
correlation	rotation/startup attempt	human subject
exception	classified type only	ToString dump

ALLOW-LIST TELEMETRY

A redactor is defense in depth. The primary control is never handing sensitive values to the logging API.

SAFE EVENT CONTRACT

Structured logging accepts a bounded evidence record

A source-generated logging method fixes event ID, field names and cardinality. Call sites cannot attach the material object or configuration tree.

EXECUTABLE ASSET 47-M - VALUE-FREE MATERIAL EVENT

```
public static partial class SecurityMaterialLog
{
    [LoggerMessage(
        EventId = 4701,
        Level = LogLevel.Information,
        Message = "Security material {Kind} revision {Revision} result {ResultCode}")]
    public static partial void MaterialResult(
        ILogger logger,
        SecurityMaterialKind kind,
        string revision,
        string resultCode);
}
```

CARDINALITY RULE

Kind, revision and result code come from bounded validated sets. Paths, thumbprints, subjects and raw exception messages are excluded from ordinary metrics/logs.

PROBLEM DETAILS

Clients learn the action class, not the security topology

Startup normally prevents traffic, while runtime material faults produce safe 503 or authorization outcomes through the owning boundary. Problem details include a stable type, status, correlation and operator action class only.

DECISION LEDGER 47-34

Input	Required handling	Forbidden shortcut
type	stable public URI/code	internal exception type
title	dependency unavailable	which secret failed
correlation	incident lookup	material reference
retry	bounded when meaningful	rotation deadline
extensions	allow-listed safe fields	config dump

SAME EXTERNAL SHAPE

Missing file, invalid certificate and secret-source outage can share a safe external response while retained internal evidence preserves the distinct cause.

REPOSITORY SCANNING

The build fails on secret-like content and private-key markers

A deterministic scanner inspects tracked source, configuration, fixtures and generated publication artifacts. It rejects private-key PEM blocks, known token formats, forbidden key names with values and approved test-vector violations. Findings are locations and rule IDs, not copied secret values.

DECISION LEDGER 47-35

Input	Required handling	Forbidden shortcut
BEGIN PRIVATE KEY	always reject outside fixture generator	redact later
password/client secret	schema-aware value rule	name-only match
token pattern	entropy/format plus context	print match
test material	generated at run time	committed reusable key
allow-list	narrow path/rule/expiry	global ignore

HISTORY WARNING

Removing a value from the current tree does not remove it from source history. A confirmed leak starts revocation and history-remediation procedures.

IMAGE SCANNING

The final image is inspected, not only the Dockerfile

BuildKit secret mounts avoid copying build credentials into layers, but the produced image still requires inspection. The gate exports the final filesystem and metadata, searches layers and history, and proves that runtime secret paths are absent until deployment.

DECISION LEDGER 47-36

Input	Required handling	Forbidden shortcut
Dockerfile ENV/ARG	no runtime secret	build-time substitution
layer filesystem	no secret/private key	deleted later layer
image history	safe commands	literal credential
labels	build identity only	config snapshot
runtime path	absent in image	placeholder secret file

LAYER PERMANENCE

Deleting a secret in a later image layer does not erase it from an earlier layer.

RUNTIME LEAK SCANNING

Logs, traces, responses and retained artifacts are scanned after faults

The fault laboratory injects malformed material and provider errors, then scans stdout, structured logs, traces, problem bodies, health responses and evidence artifacts for forbidden field names and exact canary values.

DECISION LEDGER 47-37

Input	Required handling	Forbidden shortcut
canary secret	must appear only in source fixture	log/trace
private-key fragment	zero output occurrences	exception
PEM path	zero public occurrence	health response
OIDC value	zero process artifact occurrence	options dump
policy body	digest only in evidence	telemetry payload

ABSENCE EVIDENCE

The suite proves absence against known canaries and forbidden schemas; it does not claim universal detection of every possible sensitive value.

CRASH AND DIAGNOSTIC ARTIFACTS

Dumps, backups and support bundles inherit the material threat

Managed memory, mounted files and framework strings may appear in crash dumps or host snapshots. Phase-0 disables automatic publication, restricts capture access, records retention and treats any exported diagnostic bundle as sensitive until scanned.

DECISION LEDGER 47-38

Input	Required handling	Forbidden shortcut
process dump	restricted incident artifact	attach to public issue
container snapshot	forbidden with live mounts	debug image
backup	encrypted/restricted owner process	copy secret directory
support bundle	allow-listed files + scan	collect all
retention	short named period	indefinite archive

RESIDUAL EXPOSURE

Keeping secrets out of source and logs does not guarantee they are absent from memory or privileged host diagnostics.

ADMINISTRATIVE BOUNDARY

Application APIs do not create, reveal or rotate raw material

The Phase-0 services expose no endpoint that returns secrets, uploads private keys, edits trust roots or writes policy files. Deployment and administrative procedures prepare material outside the application process, then trigger or observe a bounded reload.

DECISION LEDGER 47-39

Input	Required handling	Forbidden shortcut
GET config	safe manifest only	values
POST secret	absent	runtime secret admin
certificate upload	deployment path	public API
policy update	reviewed artifact pipeline	request body to disk
rotation trigger	attempt reference only	material payload

SEPARATION OF DUTIES

The service consumes approved material. It does not become the credential issuer, certificate authority and policy administration plane.

LEAST PRIVILEGE

Runtime credentials express one service responsibility

Database principals, broker credentials, OIDC client identity and workload certificates are issued per service and environment. A successful compromise should not unlock another service database, queue publisher, secret path or owner policy.

DECISION LEDGER 47-40

Input	Required handling	Forbidden shortcut
database	one service database permissions	shared dbo
broker	exact vhost/exchange/queue rights	administrator
OIDC	BFF confidential client only	owner client secret
mTLS	one workload identity/SAN	wildcard client
filesystem	exact mounted files	secret-root read
policy	owner-local read	cross-owner write

CREDENTIAL MATRIX

Every credential is tested against one allowed action and at least one forbidden neighboring action.

ENVIRONMENT EQUIVALENCE

Equivalent environments share policy identity, not secret values

Development, test and deployment environments can be structurally equivalent while using different identities and credentials. Comparison checks schemas, source kinds, policy digests, certificate policy, expected SANs and rotation rules without exporting material values.

DECISION LEDGER 47-41

Input	Required handling	Forbidden shortcut
schema versions	equal required set	same secret
source kind	allowed per environment profile	same path
policy digest	equal intended behavior	policy body export
certificate policy	equal trust requirements	same leaf
material revision	compatible deployment coordinate	raw key compare
permissions	meets platform profile	same mode number

NO SECRET COMPARISON

Environment equivalence never requires moving a credential, private key or its reversible fingerprint into a shared report.

VALUE-FREE MANIFEST

Evidence names material identities without serializing material

The manifest records the build, environment profile, safe source class, public policy identities, certificate public metadata and validation results. Secret values and low-entropy hashes are impossible to add through the type.

EVIDENCE ASSET 47-A - SECURITY MATERIAL MANIFEST

```
public sealed record SecurityMaterialManifest(  
    string Schema,  
    string BuildCommit,  
    string EnvironmentProfile,  
    IReadOnlyList<MaterialEvidence> Materials,  
    IReadOnlyList<PolicyEvidence> Policies,  
    string SnapshotIdentity,  
    Instant ValidatedAt,  
    EvidenceVerdict Verdict);  
  
public sealed record MaterialEvidence(  
    SecurityMaterialKind Kind,  
    string PublicRevision,  
    MaterialSourceKind SourceKind,  
    string SafeResultCode);
```

TYPE-LEVEL EXCLUSION

There is no Value, RawHash, Pem, Password, Token, Subject or AbsolutePath member in the retained manifest grammar.

EVIDENCE VALIDATION

A PASSED label without fresh artifacts is rejected

The fail-closed validator checks schema, build commit, environment profile, non-zero test discovery, execution counts, freshness, snapshot identity, policy digests, rotation/fault results and forbidden-field scans. It rejects manual edits and stale copied evidence.

DECISION LEDGER 47-42

Input	Required handling	Forbidden shortcut
build	matches image/commit	unknown
tests	required suites discovered and passed	zero green
freshness	within named run window	old report
snapshot	matches runtime observation	planned revision
leak scan	all required surfaces scanned	repository only
verdict	derived by validator	manual PASSED

CURRENT HONESTY

Until repository and deployment runs retain these artifacts, Chapter 47 remains SPECIFIED | RUNTIME PENDING.

FAULT LABORATORY

One defect is injected at each material gate

The laboratory mutates references, permissions, contents, chain, SAN, ECU, time, policy compatibility, source availability and rotation ordering. Each experiment asserts the controller action and absence of unintended business effects, not only an exception type.

DECISION LEDGER 47-43

Input	Required handling	Forbidden shortcut
reference defect	startup reject	no source search
permission defect	startup reject	no chmod
content defect	candidate reject	current unchanged
identity defect	TLS/auth reject	no fallback trust
rotation order defect	promotion blocked	no mixed revision
source outage	bounded degraded/fail state	no empty default

CONTROLLER EVIDENCE

The important result is what the host, reload controller, TLS peer and business boundary did after the defect.

ATOMIC RELOAD TEST

Concurrent readers never observe a mixed security set

The test blocks candidate loading between fields while many readers capture the current snapshot. Every observation is entirely R7 or entirely R8, and an invalid R8 leaves R7 current.

EXECUTABLE TEST 47-A - SNAPSHOT ATOMICITY

```
[Fact]
public async Task Readers_see_only_complete_old_or_new_snapshots()
{
    store.Activate(Snapshots.Valid("R7"));
    loader.Stage(Snapshots.Valid("R8"), pauseBeforeCompletion: true);

    var observations = Enumerable.Range(0, 200)
        .Select(_ => Task.Run(() => store.Current.Identity))
        .ToArray();
    loader.Release();
    await reloader.ReloadAsync(CancellationTokens.None);

    (await Task.WhenAll(observations)).Should()
        .OnlyContain(x => x is "R7" or "R8");
    store.Current.Identity.Should().Be("R8");
}
```

NO MIXED ASSERTION

The full snapshot identity includes certificate, trust, key-ring and policy identities; checking one revision string alone is insufficient.

CERTIFICATE ROTATION TEST

Real handshakes prove overlap and retirement

The harness starts two workloads with an ephemeral CA, forces new connections and observes the authenticated workload identity. It activates R8, proves R7 and R8 acceptance during overlap, drains, removes R7 trust and proves a fresh R7 handshake fails.

EXECUTABLE TEST 47-B - BOUNDED MTLS OVERLAP

```
[Fact]
public async Task Old_leaf_is_rejected_after_new_leaf_drain()
{
    await runtime.Trust.AcceptAsync(Certificates.R7, Certificates.R8);
    await runtime.Clients.ActivateAsync(Certificates.R8);

    (await runtime.NewHandshakeAsync(Certificates.R8)).Should().BeAccepted();
    (await runtime.NewHandshakeAsync(Certificates.R7)).Should().BeAccepted();

    await runtime.Connections.DrainAsync();
    await runtime.Trust.RemoveAsync(Certificates.R7);

    (await runtime.NewHandshakeAsync(Certificates.R7)).Should().BeRejected();
}
```

TRANSPORT PROOF

The test opens new TLS connections. Rechecking a cached HttpClient response or in-memory certificate object would not prove retirement.

ROTATION FAILURE MATRIX

Every unsafe ordering is executable

Table-driven deployment tests cover certificate, trust-root, HMAC key, OIDC secret and policy rotation. Each row fixes required preconditions, accepted overlap, sole write revision, retirement oracle and rollback target.

DECISION LEDGER 47-44

Input	Required handling	Forbidden shortcut
new leaf before trust	connection failure; block promotion	disable validation
old root removed early	peer outage; restore complete trust set	partial root add
new HMAC write before bridge	owner deny; block activation	global lookup
OIDC old secret revoked first	login failure; provider plan	owner secret
policy partial update	candidate reject; current stays	mixed maps

EXPECTED FAILURE

A safe test may intentionally produce denial or outage. It passes only when the failure is bounded, observable and recoverable without a permissive bypass.

CRASH WINDOWS

Rotation state and material activation can disagree after a crash

The controller can crash after staging, after peer readiness, after local activation or after retirement evidence. Recovery reconstructs state from durable attempt evidence and runtime observation rather than assuming the last command completed.

DECISION LEDGER 47-45

Input	Required handling	Forbidden shortcut
after stage	candidate inactive	promote on restart
after compatibility	revalidate peers/current	trust stale flag
after activate	observe sole write revision	write both
during drain	resume bounded observation	restart timer blindly
after retire	negative old-revision probe	delete evidence

UNKNOWN OUTCOME RULE

A command response lost during rotation is resolved through material identity and observed behavior in a fresh controller context.

CONFIGURATION DRIFT

The runtime snapshot, deployment manifest and expected policy must agree

Drift detection compares value-free identities from the deployed files, current runtime snapshot and release manifest. It distinguishes an expected staged candidate from an unauthorized current change and never repairs drift by copying values from another environment.

DECISION LEDGER 47-46

Input	Required handling	Forbidden shortcut
expected=current	healthy	copy secret
expected=staged	rotation in named phase	mark active
runtime unknown	not ready/incident	accept newest
file differs/current same	pending unauthorized candidate	auto reload
policy digest differs	block promotion	manual pass

REPAIR BOUNDARY

Automation may restore a reviewed reference or policy artifact. It does not invent credentials, trust roots or authorization mappings.

COMPONENT-TEST HOST**Tests enter through the production composition path**

A provider-backed host receives ephemeral mounted files with explicit modes, generated certificate chains and strict policy documents. Tests use the same options binding, resolver, initializer, Kestrel selector, health endpoints and evidence writer as the runtime.

DECISION LEDGER 47-47

Input	Required handling	Forbidden shortcut
filesystem	temporary explicit material root	mock resolver
certificates	generated CA/leaf chain	static boolean
clock	controlled injected instant	edit certificate dates
host	production registrations	test-only startup
effects	real endpoint/worker probes	exception-only assert

PROVIDER-BACKED CLAIM

Pure unit tests protect parsers and policies; filesystem, TLS, host-start and reload claims require component or deployment tests.

NEGATIVE CAPABILITY MATRIX**A credential fails outside its exact responsibility**

The suite proves that each service credential can perform its intended action and cannot access a neighboring database, queue, secret file, owner endpoint or policy path. A positive connectivity test alone cannot prove least privilege.

DECISION LEDGER 47-48

Input	Required handling	Forbidden shortcut
Alarm DB principal	AlarmManagementDb only	RootCauseDb
RootCause broker principal	consume flood/publish case	publish audit fact
BFF client secret	OIDC token endpoint	owner service auth
BFF workload cert	approved owner routes	database
owner secret mount	own exact files	BFF secret
policy reader	own policy read	policy write

NEIGHBOR DENIAL

For every allowed coordinate, at least one structurally similar but unauthorized coordinate is exercised and retained.

INCIDENT RESPONSE**A suspected leak creates a rotation and evidence case**

The runbook identifies material kind, revision, consumers, privileges, exposure window and dependent snapshots. It revokes or disables the smallest affected coordinate, rotates dependents, scans artifacts and proves old material no longer authenticates or authorizes.

DECISION LEDGER 47-49

Input	Required handling	Forbidden shortcut
detect	safe finding ID and affected revision	copy value
contain	disable/grant removal	delete evidence
rotate	new identity and compatibility plan	same principal forever
recover	new positive + old negative probes	readiness only
review	source, access and retention gap	blame filename

OLD-MATERIAL ORACLE

Recovery is incomplete until a real use of the retired credential, key or certificate fails where it previously succeeded.

ROLLBACK**Rollback restores a complete known-safe snapshot**

A failed candidate is discarded without changing current state. A post-activation rollback may restore the prior complete snapshot only while its credentials, certificates, trust and policies remain valid and uncompromised. A permissive or expired snapshot is not a rollback target.

DECISION LEDGER 47-50

Input	Required handling	Forbidden shortcut
candidate validation fail	keep current; remove candidate	partial copy
peer incompatibility	restore complete prior identity/trust	disable mTLS
policy regression	restore reviewed prior policy set	empty policy
key compromise	no rollback to compromised key	availability shortcut
prior expired	forward recovery required	time override

NO INSECURE ROLLBACK

Recoverability never means reactivating known-compromised, expired, overbroad or structurally invalid material.

CAPACITY AND ABUSE

Material loading and reload work have explicit budgets

Attackers or operator mistakes can inflate files, certificate chains, policy documents, reload frequency and error telemetry. The resolver caps size and count, reload is single-flight, parsers are strict and labels remain bounded.

DECISION LEDGER 47-51

Input	Required handling	Forbidden shortcut
secret bytes	small material-specific maximum	unbounded read
certificate chain	maximum depth/count	host trust enumeration
policy JSON	body/depth/collection limits	large extension data
reload events	debounce/single-flight	parallel builds
failure telemetry	bounded codes/sampling	path/value labels

OVERLOAD BEHAVIOR

Resource pressure can delay or reject a candidate; it cannot activate a partially parsed or unvalidated security set.

RESIDUAL RISKS

Phase-0 custody is explicit but not production-grade assurance

The host can read mounted material, privileged diagnostics can inspect process memory, Compose relies on host-file custody and offline revocation is limited. No HSM, managed identity, automated production vault, organizational key ceremony or certification is claimed.

DECISION LEDGER 47-52

Input	Required handling	Forbidden shortcut
host compromise	all process-readable material exposed	container isolation proof
memory inspection	framework strings may remain	deterministic erase
host secret files	external operational custody	vault equivalence
offline revocation	bounded demonstrator profile	production PKI
admin compromise	review/evidence limits	perfect issuance

BOUNDED CONCLUSION

The chapter makes material movement, validation and rotation testable. It does not transform the Phase-0 demonstrator into a certified secret or PKI platform.

REJECTED ALTERNATIVES

Convenient material handling recreates global trust

ADR-047 records rejected shortcuts so later deployment pressure cannot reintroduce them silently. A future mechanism may differ, but it must satisfy the same custody, activation, rotation, redaction and evidence obligations.

DECISION LEDGER 47-53

Input	Required handling	Forbidden shortcut
secrets in appsettings	source/image disclosure	external reference
all secrets as ENV	process/diagnostic exposure	file or approved provider
shared secret volume	cross-service blast radius	per-service grants
thumbprint allow-list	lifecycle/identity ambiguity	chain + SAN + policy
reload mutable options	mixed revision	immutable snapshot
central secret API in app	new runtime authority	deployment/provider boundary

MECHANISM-NEUTRAL RULE

Vaults, orchestrators and HSMs can improve custody, but none removes the need for strict references, typed validation, bounded overlap and negative evidence.

DEFINITION OF DONE

Completion is the absence of unsafe material paths

Chapter 47 is complete as a specification when source and image scans, reference-only options, development injection, per-service mounts, permission gates, certificate and key validation, atomic reload, rotation tests, leak scans and value-free evidence agree.

DECISION LEDGER 47-54

Input	Required handling	Forbidden shortcut
source/image	no protected values	review only
startup	required snapshot activated or fail	permissive default
runtime	one immutable current snapshot	mixed revision
rotation	old/new/retirement oracles	time-only success
telemetry	forbidden-field/canary scans	redactor assumed
maturity	SPECIFIED RUNTIME PENDING	production claim

STATUS

The reference code and tests are compile-oriented specification. Repository execution and production security maturity remain unclaimed.

IMPLEMENTATION SEQUENCE

Build denial and observability before live rotation

The order prevents a readable file from becoming active material and prevents a rotation endpoint from existing before safe rollback and evidence. Each stage keeps prior repository, image, leak and zero-effect gates green.

DECISION LEDGER 47-55

Input	Required handling	Forbidden shortcut
1	ADR, taxonomy, ownership and source profiles	no material use
2	reference-only options and strict validators	no resolver fallback
3	development and Compose injection	per-service grants
4	certificate/key/policy candidate loaders	startup validation
5	immutable snapshot and safe health/logging	atomic activation
6	rotation, fault and leak laboratories	retained evidence

FIRST RELEASE BLOCKER

Before any live reload is enabled, an invalid candidate must demonstrably leave the complete current snapshot unchanged and leak no material.

CHAPTER 48 HANDOFF

Security evidence must attack the complete trust path

Chapter 48 must combine the negative authorization matrices from Chapters 44 and 46, workload-origin checks from Chapter 45 and material/tamper/fault evidence from Chapter 47 into one bounded dossier. It must prove denied effects and value absence, not merely list enabled mechanisms.

DECISION LEDGER 47-56

Input	Required handling	Forbidden shortcut
user boundary	expired/missing/insufficient identities	happy login
workload origin	wrong/missing certificate	network reachability
claims	over-post/non-interference/leak	schema review
material	tamper/rotation/startup faults	file exists
audit	fresh artifacts and zero effects	manual checklist

EVIDENCE HANDOFF

Chapter 48 may aggregate safe evidence references. It may not collect credentials, private keys, raw claims or policy bodies into an audit archive.

OFFICIAL TECHNICAL ANCHORS

The implementation shape is tied to primary documentation

The chapter uses the official .NET and Docker contracts for development secret storage, configuration providers, options validation, PEM loading, certificate authentication, custom trust and Compose secret mounts. Local policy choices remain explicitly NEXUS-1 decisions.

DECISION LEDGER 47-57

Input	Required handling	Forbidden shortcut
ASP.NET Core app secrets	https://learn.microsoft.com/aspnet/core/security/app-secrets	production vault claim
ASP.NET Core configuration	https://learn.microsoft.com/aspnet/core/fundamentals/configuration	provider is secret
.NET options validation	https://learn.microsoft.com/dotnet/core/extensions/options	external material proof
PEM certificate loading	https://learn.microsoft.com/dotnet/api/system.security.cryptography.x509certificates.x509certificate2.createfrompemfile	identity policy
certificate authentication	https://learn.microsoft.com/aspnet/core/security/authentication/certauth	Phase-0 maturity
Docker Compose secrets	https://docs.docker.com/compose/how-to-use-secrets/	host custody proof

SOURCE DISCIPLINE

Official API documentation establishes platform behavior. The book's custody, rotation and evidence rules remain architecture decisions that must be tested in the retained runtime.

EVIDENCE RECORD

The chapter closes with a machine-checkable pending verdict

The retained record identifies the decision digest, required suites and expected deployment observations. In this manuscript it remains pending because no repository run, Compose fault laboratory or security dossier has been supplied to this writing process.

DECISION LEDGER 47-58

Input	Required handling	Forbidden shortcut
ADR-047	ceb8a8221075d8509eb72e48...	specified
source/image scans	required	PENDING
startup/fault suites	required non-zero	PENDING
TLS/key/policy rotation	required real boundaries	PENDING
leak scans	logs/traces/responses/artifacts	PENDING
runtime maturity	Phase-0 bounded claim	PENDING

VERDICT

SPECIFIED | RUNTIME PENDING. Nothing in this chapter claims that production credentials, certificates or rotation infrastructure already exist.

CHAPTER CHECKPOINT

Can the runtime lose material without losing its security boundary?

Review the chapter by answering the questions without consulting a value-bearing configuration file.

- Which configuration fields are references, and which component resolves each material kind?
- Why is Secret Manager acceptable for development but not a production custody claim?
- Which service can read the BFF OIDC secret and pairwise-reference key ring?
- What proves a certificate is the right workload identity rather than merely cryptographically valid?
- Why do certificate rotation tests require fresh TLS handshakes and connection draining?
- How does single-write, bounded dual-read rotation preserve pairwise owner assignments?
- When may last-known-good continue, and when must readiness fail closed?
- Which telemetry and evidence types make protected values structurally impossible to serialize?
- What negative test proves every credential fails at a neighboring responsibility?
- Which remaining claims belong to Chapter 48 rather than this chapter?

NEXT - CHAPTER 48

Security Tests and Audit Evidence will execute negative authorization, message-origin, tamper, leak, fault and zero-effect suites and assemble a bounded Phase-0 security evidence dossier.

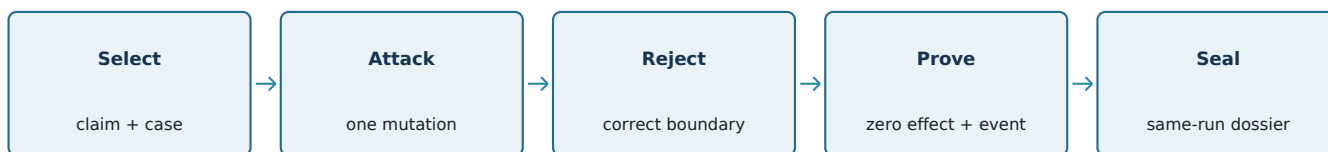
PART VIII - SECURITY AND TRUST ACROSS SERVICES

48. Security Tests and Audit Evidence

The security architecture earns a bounded Phase-0 claim only when adversarial attempts are rejected, protected effects remain absent, security events stay value-free, and every result joins one fresh, tamper-evident campaign dossier.

CHAPTER DECISION

Drive one versioned claim-and-case catalogue across policy, host, component, deployment and fault layers; require decision, zero-effect, event and artifact joins; validate exact discovery, same-run identity, freshness, integrity and leak absence; reject flaky, incomplete, stale or mixed evidence; state every Phase-0 residual explicitly.

PROOF PATH 48-A - ADVERSARIAL ATTEMPT BECOMES BOUNDED EVIDENCE**GUARANTEE LEDGER 48-A**

Boundary	Carries	Allowed claim	Still forbidden
catalogue -> runner	ClaimId + Caseld	exact discovery	unnamed coverage
attack -> decision	one mutation	attributable rejection	generic failure
decision -> probes	correlation + RunId	zero protected effect	response-only proof
artifacts -> dossier	hash + same-run joins	bounded Phase-0 evidence	audit/certification claim

ORDINARY DEVELOPER PROBLEM

A green security test can prove only the fake that produced it

A test host replaces authentication with a handler that accepts any claims, replaces the database with a mock and asserts only HTTP 403. The suite is green, yet it never exercised token validation, resource lookup, policy mapping, transaction boundaries, broker permissions or the security-event path. The result proves that a fabricated host returned one status code.

DECISION LEDGER 48-1

Input	Required handling	Forbidden shortcut
mock principal	label the bypass and bound its use	call it authentication proof
403 response	also prove zero business effect	stop at status code
fake store	use only for policy-unit semantics	claim rollback
old TRX	bind to one fresh run identity	reuse yesterday's pass
manual screenshot	retain machine-readable evidence	treat as dossier

PLAIN-LANGUAGE PRINCIPLE

Security is not proved by a rejection alone. The rejected attempt must cause no protected effect, produce the intended bounded event, and leave fresh evidence from the same run.

CHAPTER BOUNDARY

This chapter tests decisions; it does not invent a second security architecture

Chapters 43-47 defined assets, trust boundaries, operator and workload identities, owner-local policy and protected material. Chapter 48 converts those decisions into adversarial cases. A test may introduce fixtures and probes, but it may not add an authorization service, alternate message signature or production security platform merely to make the test pass.

DECISION LEDGER 48-2

Input	Required handling	Forbidden shortcut
Chapter 43	asset and boundary catalogue	generic OWASP checklist only
Chapter 44	operator edge and resource authorization	role-only proof
Chapter 45	workload identity and destination capability	network-is-trusted assumption
Chapter 46	claim minimization and owner mapping	central raw-subject map
Chapter 47	material lifecycle and safe evidence	secret-bearing test output

SCOPE INVARIANT

Every Chapter 48 case names the earlier decision it challenges and the observable effect it must leave absent or present.

DECISION RECORD

ADR-048 - One adversarial catalogue, one run, one fail-closed dossier

The campaign catalogue assigns a stable identifier to each security claim, boundary, attack mutation, expected decision, zero-effect oracle and evidence set. Unit, host, component and deployment suites may execute separately, but promotion accepts only a dossier whose required artifacts share one RunId, one build identity and one environment manifest.

DECISION LEDGER 48-3

Input	Required handling	Forbidden shortcut
catalogue	versioned claims and cases	test-name discovery
execution	real boundary at required layer	one all-mock suite
decision	expected deny/reject/unavailable	any non-2xx
effect	database/outbox/broker/audit oracle	assume no write
dossier	same run + digest + freshness	folder of mixed artifacts

DECISION IDENTITY

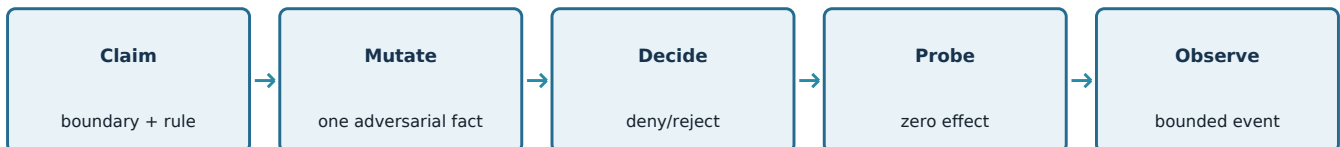
ADR-048 canonical SHA-256: 789f3da25bf52b77386c670de14cab2bd257cc509d3cf58fe7751be58ea68157

PROOF PATH

An attack becomes evidence only after six joins

The campaign starts with an architectural claim, applies one controlled adversarial mutation, observes the security decision, queries protected-effect probes, captures a bounded security event and seals the result into the same-run dossier.

PROOF PATH 48-A - CLAIM TO RETAINED EVIDENCE



PROOF OBLIGATIONS 48-A

Join	Must match	Failure meaning
claim -> case	catalogue identifiers	uncovered promise
case -> run	one run identity	stale result
decision -> effect	same correlation	partial mutation
event -> decision	reason + policy revision	unexplained denial
artifact -> manifest	length + SHA-256	tampered/missing file

EVIDENCE RULE

A PASS without its zero-effect and event joins is incomplete, even when the response code is correct.

PROOF VOCABULARY**Prevent, detect, contain and recover are different claims**

A boundary may prevent an unauthorized request, detect a malformed message, contain a poison delivery, or recover after policy restoration. The catalogue states which property is under test. Detecting an attack is not evidence that the protected effect was prevented; successful recovery is not evidence that the initial boundary held.

DECISION LEDGER 48-4

Input	Required handling	Forbidden shortcut
prevent	forbidden effect absent	event exists
detect	classified signal present	no effect assumed
contain	scope does not widen	process stayed alive
recover	bounded return to readiness	manual restart happened
attribute	bounded actor/workload reference	raw identity export

CLAIM DISCIPLINE

Each case has one primary property and may record secondary observations; it never upgrades detection into prevention.

SECURITY CLAIM CATALOGUE**The catalogue is source data, not prose hidden in test names**

A typed record ties each claim to its owner, trust boundary, protected effect, required test layers and Phase-0 limitation. The dossier validator can therefore reject missing cases, zero discovery and claims supported only by the wrong test surface.

DECISION LEDGER 48-5

Input	Required handling	Forbidden shortcut
claimId	stable SEC-###	method name
decisionRef	ADR/chapter rule	free-form memory
boundaryId	edge/workload/broker/store	service name only
effectOracle	queryable protected consequence	response-only
requiredLayers	unit/host/component/deployment	fastest layer only

CATALOGUE GATE

Every completed security promise has at least one required adversarial case; every case references one existing promise.

TYPED CATALOGUE

Claims and cases carry machine-checkable obligations

The record separates expected decision from expected effect and requires an event class. A case can therefore fail because authorization was wrong, because a hidden write occurred, or because the evidence path lied.

EXECUTABLE ASSET 48-A - CLAIM AND ADVERSARIAL CASE

```
public sealed record SecurityClaim(  
    string ClaimId,  
    string DecisionRef,  
    string BoundaryId,  
    string ProtectedEffect,  
    IReadOnlySet<TestLayer> RequiredLayers,  
    string Phase0Limit);  
  
public sealed record AdversarialCase(  
    string CaseId,  
    string ClaimId,  
    string Mutation,  
    ExpectedDecision Decision,  
    string EffectOracle,  
    string EventClass);  
  
public enum TestLayer { PolicyUnit, Host, Component, Deployment, Fault }  
public enum ExpectedDecision { Challenge, Deny, Reject, Unavailable }
```

NO AMBIGUOUS PASS

A test result must name ClaimId and CaseId. Display names and source filenames are useful diagnostics, not evidence identity.

COVERAGE MATRIX

Each trust boundary needs both positive and negative coordinates

One successful call proves only that an admitted coordinate works. The matrix also covers neighboring responsibilities: wrong actor, workload, audience, issuer, capability, resource, owner, policy revision, credential and transport. The expected result is deny or reject with no protected effect.

DECISION LEDGER 48-6

Input	Required handling	Forbidden shortcut
operator -> BFF	session + anti-forgery + route	cookie means allowed
BFF -> owner API	audience + workload + capability	internal network
producer -> broker	principal + exchange + user-id	Appld string
broker -> consumer	contract + origin evidence	header alone
service -> database	owner credential + schema	shared server login

NEIGHBOR TEST

A credential that succeeds at its intended coordinate must fail at every adjacent coordinate included in the deployment inventory.

TEST LAYERS

The cheapest layer proves semantics; the real layer proves the mechanism

Pure policy tests enumerate decision logic quickly. Host tests exercise routing and middleware. Component tests add the real database or broker. Deployment tests prove TLS, credentials, ports and process isolation. Fault tests disturb dependencies and lifecycle. No layer silently substitutes for another.

DECISION LEDGER 48-7

Input	Required handling	Forbidden shortcut
policy unit	rule permutations	framework pipeline
test host	authn/authz middleware and endpoint metadata	real TLS
component	provider transaction or broker ACL	full deployment
deployment	real processes, sockets, identities	production equivalence
fault	failure response and recovery	penetration test

SURFACE RULE

Mock authentication may inject a carefully described principal for authorization semantics. It cannot prove token or certificate validation.

CAMPAIGN IDENTITY

One RunId binds every observation to one execution

The orchestrator creates a random RunId before any suite starts and passes it as non-secret metadata. Every test result, zero-effect snapshot, security event export, environment manifest and artifact record repeats it. A copied artifact from another run fails the join even when its hash is valid.

DECISION LEDGER 48-8

Input	Required handling	Forbidden shortcut
RunId	random opaque identifier	timestamp only
build	commit + artifact/image digest	branch name
environment	value-free manifest digest	machine nickname
catalogue	canonical content digest	latest on disk
time window	trusted runner start/end	event local time only

SAME-RUN INVARIANT

The dossier contains no required artifact whose RunId, build identity, catalogue digest or environment digest differs from the campaign header.

CAMPAIGN CONTEXT

Run identity is immutable and injected before host creation

The test infrastructure does not let individual cases invent their own build or environment labels. Those values originate once in the orchestrator and are exposed read-only to hosts, probes and evidence writers.

EXECUTABLE ASSET 48-B - IMMUTABLE CAMPAIGN CONTEXT

```
public sealed record SecurityRunContext(  
    Guid RunId,  
    string CommitSha,  
    string BuildDigest,  
    string EnvironmentDigest,  
    string CatalogueDigest,  
    DateTimeOffset StartedAtUtc);  
  
public interface ISecurityRunContext  
{  
    SecurityRunContext Current { get; }  
}  
  
public sealed class FixedSecurityRunContext(SecurityRunContext current)  
    : ISecurityRunContext  
{  
    public SecurityRunContext Current { get; } = current;  
}
```

FIXTURE RULE

Tests may read the current run context; they cannot mutate it or replace a digest after execution begins.

TEST IDENTITIES

Synthetic principals are explicit coordinates, not convenient superusers

The identity fixture catalogue defines operators, workloads, issuers, audiences and pairwise references with no production values. Each coordinate is either allowed for one exact action or intentionally wrong in one dimension. An Admin test identity that bypasses all policies would erase the boundary under test.

DECISION LEDGER 48-9

Input	Required handling	Forbidden shortcut
operator.allowed	one capability + one resource	all roles
operator.wrongOwner	same role, neighboring resource	anonymous denial only
bff.allowed	one audience + intent	wildcard scope
service.wrongCert	valid chain, wrong SAN/EKU	self-signed garbage only
broker.wrongPublisher	valid credential, wrong exchange	invalid password only

ADVERSARIAL PRECISION

Prefer a valid-but-wrong identity over a malformed credential; the former reaches the decision boundary that matters.

PRINCIPAL CONSTRUCTION

Authorization-unit principals declare provenance and revision

A small principal factory prevents test methods from assembling arbitrary claims. It emits only the minimized claim set of Chapter 46 and labels the synthetic authentication type so evidence cannot confuse it with a validated token.

EXECUTABLE ASSET 48-C - BOUNDED SYNTHETIC PRINCIPAL

```
public static class TestPrincipal
{
    public static ClaimsPrincipal Operator(
        string pairwiseSubject,
        string intent,
        string policyRevision) => new(
            new ClaimsIdentity(
                [
                    new("sub", pairwiseSubject),
                    new("nexus:intent", intent),
                    new("nexus:policy_rev", policyRevision),
                    new("iss", "urn:nexus1:test-issuer"),
                    new("aud", "urn:nexus1:test-bff")
                ], authenticationType: "SyntheticAuthorizationFixture"));
}
```

BOUNDED USE

This factory supports policy-unit and selected host tests. Token-validation claims require a real test issuer and the production authentication handler.

AUTHENTICATION SUBSTITUTION

A test handler is useful only when the bypass is visible

WebApplicationFactory can replace authentication for endpoint authorization tests. The factory name, case metadata and dossier label must say SYNTHETIC_AUTH. Token signature, issuer, audience, expiry and scheme-selection cases run in a separate host that keeps the production handler.

DECISION LEDGER 48-10

Input	Required handling	Forbidden shortcut
synthetic handler	authorization semantics	token proof
real handler + test issuer	token validation	production IdP proof
real TLS socket	certificate handshake	TestServer proof
real broker	credentials and ACL	fake channel proof
real provider	transaction/effect oracle	in-memory list

EVIDENCE LABEL

The dossier records the authentication mode for every host case and rejects a synthetic result for a claim that requires validated credentials.

TEST HOST

Synthetic authentication carries one scenario header under test-only code

The handler resolves a predeclared scenario from a fixture catalogue. Arbitrary claims are not accepted from the request. The registration exists only in the test assembly and the production composition root has no reference to it.

EXECUTABLE ASSET 48-D - SCENARIO-BOUND TEST HANDLER

```
public sealed class ScenarioAuthHandler(
    IOptionsMonitor<AuthenticationSchemeOptions> options,
    ILoggerFactory logger,
    UriEncoder encoder,
    IScenarioPrincipalCatalogue catalogue)
    : AuthenticationHandler<AuthenticationSchemeOptions>(options, logger, encoder)
{
    protected override Task<AuthenticateResult> HandleAuthenticateAsync()
    {
        var id = Request.Headers["X-Nexus-Test-Scenario"].SingleOrDefault();
        if (id is null || !catalogue.TryResolve(id, out var principal))
            return Task.FromResult(AuthenticateResult.NoResult());

        var ticket = new AuthenticationTicket(principal!, Scheme.Name);
        return Task.FromResult(AuthenticateResult.Success(ticket));
    }
}
```

NO CLAIM INJECTION

The request selects an immutable fixture identifier. It cannot submit role, subject, owner or policy claims directly.

HTTP DECISION MATRIX

Challenge, deny and unavailable have different meanings

The endpoint campaign distinguishes absent/invalid authentication from authenticated-but-forbidden access and from an unavailable authorization dependency. It asserts problem type, cache headers and absence of sensitive detail in addition to the status code.

DECISION LEDGER 48-11

Input	Required handling	Forbidden shortcut
no credential	401 challenge	403
invalid credential	401 stable problem	exception detail
valid, insufficient	403 deny	404 everywhere
policy unavailable	503 fail closed	403 policy decision
resource concealed	documented 404 rule	inconsistent guessing

DECISION SEMANTICS

Tests assert the documented boundary response, not merely any failure-shaped status.

NEGATIVE EDGE TEST

A forbidden operator must leave one denial and zero effects

The host test captures state before the request, invokes the production endpoint pipeline with a bounded synthetic principal, then joins the response, owner-store probe, outbox probe and security-event sink by Caseld and RunId.

EXECUTABLE TEST 48-A - DECISION PLUS ZERO-EFFECT JOIN

```
[Fact]
public async Task Wrong_owner_is_forbidden_without_effect()
{
    var before = await probes.CaptureAsync(CaseId, ct);
    using var request = Scenarios.Command("operator.wrongOwner", CaseId);

    using var response = await client.SendAsync(request, ct);
    var after = await probes.CaptureAsync(CaseId, ct);

    response.StatusCode.Should().Be(HttpStatusCode.Forbidden);
    after.BusinessRows.Should().Be(before.BusinessRows);
    after.OutboxRows.Should().Be(before.OutboxRows);
    events.SingleFor(CaseId).ReasonClass.Should().Be("resource-owner-mismatch");
}
```

FOUR-PART ASSERTION

The case fails if the response, protected state, public-fact queue or security event disagrees with the expected denial.

TOKEN MUTATION CORPUS

Invalid tokens vary one trust coordinate at a time

A controlled test issuer creates short-lived tokens for the production JWT bearer handler. Cases alter signature, issuer, audience, time window, token type, intent, policy revision or pairwise subject independently. Random garbage is retained only as a parser-hardening case.

DECISION LEDGER 48-12

Input	Required handling	Forbidden shortcut
signature	unknown signing key	corrupt all bytes
issuer	valid signature, wrong issuer	missing token
audience	neighbor service audience	wildcard accepted
time	expired/not-yet-valid boundaries	long-lived token only
claims	missing/duplicate/oversized	happy-path subset

MUTATION RULE

Each case preserves every coordinate except the one named by Mutation, so a PASS identifies the boundary that rejected it.

ISSUER AND AUDIENCE

A correctly signed token can still belong to the wrong service

The most important negative token cases use valid signatures from known test issuers. One changes the issuer, another changes only the audience to a neighboring service. Both must be challenged before owner policy executes and must leave no resource lookup or business effect.

DECISION LEDGER 48-13

Input	Required handling	Forbidden shortcut
wrong issuer	401 before authorization	map issuer to default
wrong audience	401 at destination	shared internal audience
multiple audiences	only documented set	any match without test
missing typ	follow explicit token policy	accept implicit
neighbor token	fail at every adjacent API	network location trust

PRE-POLICY ORACLE

A token-validation rejection emits its own event class and proves the application handler was not invoked.

TIME-BOUND TOKENS

Expiry tests need an injected clock and boundary values

Tests cover just-before, exact-boundary and just-after values using the handler's configured clock-skew policy. Sleeping makes tests slow and nondeterministic; changing the wall clock contaminates the run. The evidence records the effective policy revision and test instant, never the token.

DECISION LEDGER 48-14

Input	Required handling	Forbidden shortcut
exp - epsilon	admit if policy allows	sleep
exp boundary	document inclusive/exclusive rule	guess
exp + skew	challenge	silent refresh
nbf future	challenge	treat as absent
runner clock	fixed authoritative instant	host local drift

TIME EVIDENCE

The case retains timestamps, skew policy and result; it never retains the bearer token or a reversible token digest.

CLAIM MINIMIZATION

Extra claims must not create authority

A negative corpus adds plausible role, group, email, tenant and administrator claims that are not part of the owner policy contract. The decision must remain identical. Duplicate authorization-critical claims, unknown versions and oversized values are rejected rather than resolved by first/last wins.

DECISION LEDGER 48-15

Input	Required handling	Forbidden shortcut
extra role	ignored unless mapped contractually	implicit elevation
email/name	not used in policy	resource identity
duplicate intent	reject ambiguity	first wins
unknown claim version	deny/unavailable	best effort parse
oversized value	bounded rejection	log entire value

MONOTONIC AUTHORITY

Adding an unrecognized claim never changes DENY to ALLOW.

PAIRWISE IDENTITY

A pairwise subject is valid only inside one owner namespace

The same synthetic person receives different pairwise references for AlarmManagement and RootCause. Cross-owner substitution must fail even when issuer, audience and capability are otherwise valid. Tests also prove that logs and public events never reveal the source subject used by the BFF bridge.

DECISION LEDGER 48-16

Input	Required handling	Forbidden shortcut
Alarm pairwise ref	Alarm owner lookup only	RootCause lookup
RootCause pairwise ref	RootCause namespace	global join
unknown revision	fail closed	derive with newest key
disabled mapping	deny affected coordinate	delete all identity
telemetry	bounded pseudonymous reference	raw subject

UNLINKABILITY TEST

Exports from two owners contain no shared human identifier or stable cross-owner pseudonym.

RESOURCE SCOPE CONFLICTS

Route, header, body and token cannot vote on ownership

The owner service derives the authoritative resource from local state. Adversarial cases submit conflicting alarm IDs, tenant-like headers, body owner fields and pairwise references. Any client-supplied scope that disagrees with local truth is rejected and cannot redirect the command.

DECISION LEDGER 48-17

Input	Required handling	Forbidden shortcut
route id	lookup local aggregate	trust header owner
body owner	ignore/reject as contract dictates	overwrite local owner
scope header	diagnostic only if allowed	authorization fact
token subject	map through owner-local table	resource key
missing aggregate	document concealment response	create on command

NO CONFUSED DEPUTY

The BFF may forward a requested coordinate; the owner service remains responsible for deriving and authorizing the target.

POLICY REVISION TESTS

Stale, unknown and partially loaded policy must fail closed

The campaign runs with current policy, a token naming an older incompatible revision, a future unknown revision, a corrupted candidate and an unavailable policy source. Atomic reload tests prove the current snapshot remains coherent while readiness communicates when its bounded lifetime expires.

DECISION LEDGER 48-18

Input	Required handling	Forbidden shortcut
current revision	evaluate normally	latest string
stale incompatible	deny/unavailable	reuse positive cache
future unknown	deny/unavailable	assume compatible
corrupt candidate	reject; current intact	partial swap
current expired	readiness fail closed	serve forever

NO FALLBACK ALLOW

Policy failure can reduce availability; it cannot widen authority.

ENDPOINT INVENTORY

Every mapped endpoint must appear in the security matrix

Reflection over endpoint data sources captures route pattern, methods, authorization metadata, anonymous markers and owning service. The gate rejects an unclassified endpoint, a protected route with no policy, a duplicate route with different metadata and a zero-endpoint discovery run.

DECISION LEDGER 48-19

Input	Required handling	Forbidden shortcut
route + method	stable endpoint identity	display name only
authorization metadata	one documented profile	implicit default
AllowAnonymous	explicit approved exception	forgotten attribute
health endpoints	narrow response + network profile	public internals
discovery count	> 0 and expected set	empty pass

INVENTORY GATE

The matrix and production endpoint inventory are equal sets after approved infrastructure exceptions.

ENDPOINT GATE

Unknown or ambiguous endpoint metadata fails the build

The inventory test asks the production composition root for `EndpointDataSource` entries. It compares a canonical key against the reviewed security catalogue and rejects any drift before adversarial cases run.

EXECUTABLE TEST 48-B - ENDPOINT COVERAGE SET EQUALITY

```
[Fact]
public void Every_endpoint_has_one_security_profile()
{
    var actual = endpointSources.SelectMany(x => x.Endpoints)
        .OfType<RouteEndpoint>()
        .Select(EndpointSecurityDescriptor.From)
        .ToArray();

    actual.Should().NotBeEmpty();
    actual.Select(x => x.Key).Should().OnlyHaveUniqueItems();
    actual.Select(x => x.Key).Should().BeEquivalentTo(catalogue.EndpointKeys);
    actual.Should().OnlyContain(x => x.Profile is not SecurityProfile.Unknown);
}
```

PRODUCTION PATH

The test discovers the same endpoint graph mapped by the service host; a separately constructed route list is not evidence.

ZERO-EFFECT MODEL

Protected effects extend beyond database rows

A denied command may still enqueue an outbox record, publish a message, update a cache, acquire a lease, increment a domain sequence or write an audit success. The oracle catalogue declares every protected effect for the case and captures them before and after by correlation-safe probes.

DECISION LEDGER 48-20

Input	Required handling	Forbidden shortcut
owner state	row/version/domain digest unchanged	row count only
outbox	no pending public fact	publisher stopped
broker	no routed message	queue empty eventually
cache/lease	no authority-bearing state	ignore side effect
audit	denial event only, no success	no log assertion

EFFECT COMPLETENESS

A case is complete only when its claim's protected-effect set is fully covered by probes.

EFFECT PROBES

Snapshots are typed, bounded and correlated

The probe does not export business payloads. It returns counts, versions and content digests scoped to the case's synthetic resource and correlation. The same interface supports before/after comparison and dossier serialization.

EXECUTABLE ASSET 48-E - ZERO-EFFECT SNAPSHOT

```
public sealed record ProtectedEffectSnapshot(  
    string CaseId,  
    long AggregateVersion,  
    int BusinessRows,  
    int OutboxRows,  
    int RoutedMessages,  
    int SuccessAuditRows,  
    string ValueFreeStateDigest);  
  
public interface IProtectedEffectProbe  
{  
    Task<ProtectedEffectSnapshot> CaptureAsync(  
        string caseId,  
        CancellationToken cancellationToken);  
}
```

NO PAYLOAD EXPORT

The digest is computed over an approved value-free projection. A state hash is not permission to serialize protected business or identity data.

DATABASE ZERO EFFECT

A denied request must not reach SaveChanges

The strongest host oracle combines database state with an interceptor that records SaveChanges attempts. Unchanged rows alone could hide a write followed by compensation or rollback. A forbidden request should cause neither a committed change nor an attempted business unit of work.

DECISION LEDGER 48-21

Input	Required handling	Forbidden shortcut
row/version	unchanged provider-backed state	in-memory entity
SaveChanges attempt	zero for pre-handler denial	rollback equals no attempt
transaction	none opened when policy denies early	empty commit
concurrency token	unchanged	refresh hides write
audit denial	separate bounded path	business transaction

LAYERED ORACLE

Provider state proves no commit; interception proves the protected handler did not attempt a write.

OUTBOX AND BROKER ZERO EFFECT

No unauthorized public fact may escape a denied command

The case observes owner outbox rows, dispatcher attempts, publisher confirms and target queue deliveries. A queue-depth snapshot alone is racy; a bounded probe filters the synthetic correlation and waits past the documented dispatch window before declaring absence.

DECISION LEDGER 48-22

Input	Required handling	Forbidden shortcut
outbox row	absent for correlation	undispatched accepted
dispatcher	no attempt	publish failed
confirm	no sequence/correlation	queue empty
target delivery	no matching message	global depth
wait budget	bounded > dispatch SLA for lab	arbitrary sleep

ABSENCE NEEDS A WINDOW

The dossier records the observation deadline used to prove that no matching message appeared.

DENIED-ACTION EVENT

A denial is observable without revealing the attempted value

The security event records when, where, bounded actor/workload reference, action class, target class, outcome, stable reason, policy identity, correlation and zero-effect reference. It excludes bearer tokens, cookies, secret material, raw human subjects, request bodies and exception dumps.

DECISION LEDGER 48-23

Input	Required handling	Forbidden shortcut
who	pairwise actor or workload reference	raw subject/email
what	action + target class	request body
why	stable reason class	stack trace
which policy	revision + digest	policy contents
proof join	RunId/CasId/correlation	credential hash

MINIMUM SUFFICIENT EVENT

The event supports reconstruction of the decision without becoming a second store of identities, secrets or domain payloads.

SECURITY EVENT CONTRACT

The schema makes unsafe fields difficult to express

The event uses bounded reference types and controlled enums rather than an arbitrary property bag. Extended diagnostics remain a separate protected artifact and are not part of the default event stream.

EXECUTABLE ASSET 48-F - VALUE-FREE SECURITY EVENT

```
public sealed record SecurityEventV1(  
    Guid EventId,  
    Guid RunId,  
    string CaseId,  
    DateTimeOffset OccurredAtUtc,  
    ServiceId Service,  
    SecurityEventClass EventClass,  
    ActionClass Action,  
    TargetClass Target,  
    DecisionOutcome Outcome,  
    ReasonClass Reason,  
    PolicyIdentity Policy,  
    string CorrelationId,  
    string ZeroEffectRef);  
  
public enum DecisionOutcome { Challenged, Denied, Rejected, Unavailable }
```

NO EXTENSION DICTIONARY

Arbitrary key-value fields are forbidden in the public event because they bypass classification, length limits and leak review.

EVENT HYGIENE

Control characters and unbounded values are attack inputs

Every external string is normalized to an allow-listed identifier grammar before structured logging. CR/LF, delimiters, oversized values and invalid Unicode are rejected or replaced with stable classes. The original hostile value is not echoed into logs, problems or dossier files.

DECISION LEDGER 48-24

Input	Required handling	Forbidden shortcut
CR/LF injection	reject/sanitize before event	raw interpolation
oversized id	bounded reason	truncate silently
invalid Unicode	stable invalid-input class	logger exception
unknown reason	fallback enum	attacker text
exception	error class + internal ref	ToString()

INJECTION TEST

A hostile identifier cannot create a second apparent event line, field or JSON object in any retained sink.

EVENT RELIABILITY

Logging failure must not turn DENY into ALLOW

A diagnostic security-event sink can fail after a decision. The request remains denied and the failure raises a bounded health/metric signal. For high-risk admitted mutations whose audit intent is a business invariant, the owner writes that intent atomically with its state or fails the command. The two paths are not conflated.

DECISION LEDGER 48-25

Input	Required handling	Forbidden shortcut
denial event sink fails	deny remains; signal degradation	allow request
security logger blocks	bounded enqueue/drop policy	unbounded wait
admin mutation audit fails	transaction fails if invariant requires	commit silently
collector unavailable	buffer within explicit limit	memory grows
disk full	alert + bounded behavior	crash loop blindly

TWO GUARANTEES

Decision safety and evidence availability are separate. The failure policy states which one may degrade and which must fail the operation.

EVENT CAPTURE TEST

The test sink observes typed events, not formatted log text

A bounded in-memory sink is registered only in the test host alongside production event creation. Assertions inspect the structured contract. Separate serialization tests prove the real sink's JSON shape and leak rules.

EXECUTABLE ASSET 48-G - BOUNDED EVENT CAPTURE

```
public sealed class SecurityEventCapture : ISecurityEventSink
{
    private readonly Channel<SecurityEventV1> _events =
        Channel.CreateBounded<SecurityEventV1>(256);

    public ValueTask WriteAsync(
        SecurityEventV1 value,
        CancellationToken cancellationToken) =>
        _events.Writer.WriteAsync(value, cancellationToken);

    public async Task<SecurityEventV1> SingleAsync(
        string caseId,
        TimeSpan timeout,
        CancellationToken cancellationToken) =>
        await EventAwaiter.SingleAsync(_events.Reader, caseId, timeout, cancellationToken);
}
```

TEST-ONLY CAPACITY

The bounded sink fails loudly on overflow. Dropping events to keep a security test green is forbidden.

LEAK CANARIES

Known synthetic values probe every output surface

The campaign injects unmistakable non-production canaries into tokens, cookies, secrets, headers and payloads, exercises failures, then scans logs, traces, metrics labels, Problem Details, test results, dumps and dossier files. It also searches encoded forms where the output path could transform data.

DECISION LEDGER 48-26

Input	Required handling	Forbidden shortcut
raw canary	must be absent	grep logs only
URL/base64 form	scan relevant encodings	plain text only
token fragments	header/payload/signature absent	whole token only
private key marker	absent everywhere	path scan
connection string	coordinate names only	password-only scan

SYNTHETIC ONLY

Canaries are generated for the run and are never copied from real credentials, identities or operational data.

LEAK SCANNER

The scanner works on bytes and refuses unreadable artifacts

Artifacts are scanned before sealing. A binary file that cannot be classified is not automatically safe; it is rejected or routed through a format-specific extractor. The result records only canary identifiers and artifact-relative paths.

EXECUTABLE ASSET 48-H - BYTE-LEVEL CANARY SCAN

```
public static async Task AssertNoCanariesAsync(
    IEnumerable<string> paths,
    IReadOnlyList<ReadOnlyMemory<byte>> canaries,
    CancellationToken cancellationToken)
{
    foreach (var path in paths)
    {
        var bytes = await File.ReadAllBytesAsync(path, cancellationToken);
        foreach (var canary in canaries)
            if (bytes.AsSpan().IndexOf(canary.Span) >= 0)
                throw new SecurityEvidenceException(
                    $"Protected canary found in {Path.GetFileName(path)}");
    }
}
```

NO VALUE IN EXCEPTION

The scanner reports the canary class and artifact; it never prints the matched bytes.

DIAGNOSTIC ARTIFACTS

Crash and hang dumps can contain the secrets that tests protect

Test-runner blame features are useful, but process dumps may capture tokens, keys and decrypted material. The normal security campaign retains sequence/timeout diagnostics without memory dumps. A dump-enabled diagnostic rerun is an explicitly protected incident workflow and is never merged automatically into the dossier.

DECISION LEDGER 48-27

Input	Required handling	Forbidden shortcut
hang timeout	bounded termination + sequence	infinite wait
memory dump	disabled by default	ordinary artifact
diagnostic rerun	separate access and retention	publish with TRX
console log	redacted structured output	verbose token debug
failure attachment	allow-listed formats	collect directory

CUSTODY RULE

More diagnostics are not automatically more evidence; some artifacts widen the security boundary and require separate custody.

WORKLOAD IDENTITY MATRIX**A valid certificate must fail at the wrong destination**

Deployment tests issue synthetic certificates from the Phase-0 test CA with controlled SAN, EKU, issuer and lifetime. Cases cover unknown root, wrong SAN, wrong EKU, expired/not-yet-valid certificates and a valid AlarmManagement credential presented to RootCause. Tests use fresh TLS handshakes.

DECISION LEDGER 48-28

Input	Required handling	Forbidden shortcut
unknown root	handshake fails	HTTP 403
wrong SAN	identity mapping rejects	chain-only success
wrong EKU	client-auth validation fails	any certificate
expired/not yet valid	time validation fails	cached success
neighbor workload	destination authorization denies	internal trust

TWO-STAGE PROOF

TLS proves certificate acceptance; destination policy separately proves the authenticated workload may perform the requested capability.

FRESH-HANDSHAKE TEST**Connection reuse cannot hide certificate rejection**

Each certificate case creates a new handler and connection pool. The test records whether failure occurred during TLS authentication or after HTTP authorization; those are different boundary outcomes.

EXECUTABLE TEST 48-C - ISOLATED TLS ATTEMPT

```
public static async Task<HttpResponseMessage> SendWithCertificateAsync(
    Uri endpoint,
    X509Certificate2 clientCertificate,
    CancellationToken cancellationToken)
{
    using var handler = new SocketsHttpHandler();
    handler.SslOptions.ClientCertificates = new X509CertificateCollection
    {
        clientCertificate
    };
    handler.PooledConnectionLifetime = TimeSpan.Zero;
    using var client = new HttpClient(handler) { BaseAddress = endpoint };
    return await client.GetAsync("/internal/security-probe", cancellationToken);
}
```

LIFECYCLE ORACLE

A new certificate case never reuses a TLS session authenticated by a previous case.

CERTIFICATE ROTATION EVIDENCE**Overlap and retirement are observed on new connections**

The campaign establishes connections with old and new workload certificates across the documented overlap. After retirement, fresh old-certificate handshakes fail while fresh new-certificate handshakes succeed. Existing pooled connections are drained within the configured bound.

DECISION LEDGER 48-29

Input	Required handling	Forbidden shortcut
before activation	old succeeds; new rejected/unused	both forever
overlap	old + new succeed if policy says	unbounded overlap
after retirement	old fresh handshake fails	existing socket only
new current	fresh handshake succeeds	cached connection
drain	old sessions gone by deadline	restart evidence

TEMPORAL DOSSIER

Rotation proof includes phase timestamps, certificate revision identifiers and fresh-handshake results, never private keys or certificate passwords.

BROKER ORIGIN MODEL**Publisher-declared Appld is not an authenticated origin**

RabbitMQ authenticates a connection, authorizes resources and validates the AMQP User ID property when set. The NEXUS-1 consumer requires the expected exchange/routing key, validated UserId, envelope producer and contract type to agree. Appld remains descriptive and cannot establish origin by itself.

DECISION LEDGER 48-30

Input	Required handling	Forbidden shortcut
connection principal	broker authentication	network address
exchange permission	producer-specific write ACL	shared publisher
UserId property	set and broker-validated	Appld
envelope producer	matches expected contract owner	self-asserted only
routing key/type	catalogued combination	free-form string

ORIGIN CLAIM

Phase-0 origin evidence is the agreement of broker-enforced identity/ACL and application contract metadata; it is not end-to-end message non-repudiation.

MESSAGE ORIGIN GATE

The consumer validates transport evidence before deserialization effects

The delivery adapter receives broker metadata separately from the body. It rejects an unexpected publisher, exchange, routing key or contract owner before invoking the application consumer and records a bounded origin-rejection event.

EXECUTABLE ASSET 48-I - ORIGIN EVIDENCE VALIDATOR

```
public sealed record BrokerOriginEvidence(  
    string AuthenticatedUserId,  
    string Exchange,  
    string RoutingKey);  
  
public sealed class MessageOriginValidator(IOriginCatalogue catalogue)  
{  
    public OriginDecision Validate(  
        BrokerOriginEvidence broker,  
        PublicEnvelopeHeader envelope)  
    {  
        var expected = catalogue.RequiredFor(envelope.ContractName);  
        return broker.AuthenticatedUserId == expected.UserId  
            && broker.Exchange == expected.Exchange  
            && broker.RoutingKey == expected.RoutingKey  
            && envelope.Producer == expected.Producer  
            ? OriginDecision.Accept  
            : OriginDecision.Reject;  
    }  
}
```

BEFORE BODY EFFECTS

Origin rejection occurs before domain materialization, inbox insertion or acknowledgement as successfully processed.

BROKER NEGATIVE CASES

Valid credentials probe resource isolation more deeply than bad passwords

The deployment laboratory connects as each real Phase-0 service principal. It attempts to publish to its own exchange, a neighboring exchange, a management surface and an undeclared route. It also sets matching and mismatching UserId properties and records broker-level outcomes.

DECISION LEDGER 48-31

Input	Required handling	Forbidden shortcut
own exchange	publish + confirm succeeds	admin credential
neighbor exchange	ACCESS_REFUSED	consumer rejects later
wrong UserId	broker validation rejects	trust envelope producer
management API	no access	shared monitoring user
undeclared route	unroutable/denied as topology says	silent loss

ACL PROOF

The broker test uses the same imported definitions and per-service credentials as the retained Compose environment.

ENVELOPE TAMPER CORPUS**Headers, payload and routing are mutated independently**

Tamper cases change contract name/version, message ID, occurred-at value, producer, content type, routing key and JSON body. The expected outcome distinguishes origin rejection, contract rejection, poison handling and domain abstention. Every path proves no unauthorized business effect and an explicit acknowledgment decision.

DECISION LEDGER 48-32

Input	Required handling	Forbidden shortcut
producer changed	origin reject	accept envelope
contract/version changed	contract reject	best-effort bind
message id replay	inbox idempotency	second effect
body schema changed	poison/quarantine policy	domain defaults
routing mismatch	origin/topology reject	header wins

ONE MUTATION

Each specimen starts from a canonical valid envelope and alters one field or byte range so the failing gate is attributable.

TAMPER LIMITATION**Schema validity does not prove payload authenticity**

A privileged broker operator or producer credential holder could replace a schema-valid payload and matching application headers. The Phase-0 broker identity/ACL model does not provide end-to-end signatures. The campaign therefore proves the controls it has and records this residual instead of claiming cryptographic non-repudiation.

DECISION LEDGER 48-33

Input	Required handling	Forbidden shortcut
schema-invalid body	detected by contract gate	authenticity claim
wrong producer metadata	detected by origin gate	payload integrity
wrong publisher credential	blocked by broker ACL	compromised credential
privileged substitution	residual risk	HMAC invented in test
future control	decision record before mechanism	silent signature

HONEST BOUNDARY

Chapter 48 does not add message signing. A future requirement for end-to-end authenticity must define key ownership, canonicalization, rotation and replay semantics first.

TAMPER COMPONENT TEST

A rejected delivery records no inbox success or domain effect

The test publishes a valid baseline and a one-field tampered specimen through the real broker. It joins broker delivery, consumer disposition, inbox state, owner state and security event under one CaseId.

EXECUTABLE TEST 48-D - TAMPERED ORIGIN IS REJECTED

```
[Fact]
public async Task Producer_header_mismatch_has_zero_effect()
{
    var specimen = corpus.Valid("AlarmFloodDetected.v1")
        .MutateHeader("producer", "RootCause");

    await broker.PublishAsync(specimen, publisher: "alarm-management", ct);
    var result = await observer.WaitAsync(specimen.CaseId, ct);

    result.Disposition.Should().Be(DeliveryDisposition.Rejected);
    result.InboxAccepted.Should().BeFalse();
    result.DomainEffects.Should().Be(0);
    result.SecurityEvent.ReasonClass.Should().Be("origin-mismatch");
}
```

REAL SEAM

The specimen crosses the retained broker topology; a direct consumer method call is a separate contract-unit test.

REPLAY AND DUPLICATION

Security rejection and idempotency must compose

An unauthorized message is not inserted as a successful inbox record merely to suppress repeats. The rejection/quarantine policy prevents hot loops without marking a protected effect complete. A valid duplicate is acknowledged through the established inbox semantics and produces one effect.

DECISION LEDGER 48-34

Input	Required handling	Forbidden shortcut
invalid origin repeat	bounded reject/quarantine	success inbox
valid duplicate	one inbox/effect	second domain action
same id, changed body	tamper/collision reject	first wins silently
replayed old policy	freshness rule	cached allow
redelivery	same decision/effect semantics	skip event

IDENTITY PLUS CONTENT

The inbox records the approved message identity and canonical content digest needed to detect same-ID/different-body specimens.

ADMINISTRATIVE BOUNDARY

Security configuration paths receive their own negative campaign

Health endpoints, evidence exports, policy reload, certificate refresh, repair execution and broker management are higher-risk surfaces. Tests cover unauthenticated, ordinary operator, wrong workload, stale approval and direct-network attempts, plus event and zero-effect assertions.

DECISION LEDGER 48-35

Input	Required handling	Forbidden shortcut
policy reload	owner admin + reviewed revision	ordinary operator
certificate refresh	deployment controller boundary	HTTP public route
evidence export	auditor/read-only profile	business writer
repair execute	approved plan + capability	repair preview
broker management	separate admin principal	service credential

NEGATIVE CAPABILITY

Every administrative credential succeeds only at its named surface and fails at all ordinary service and neighboring admin surfaces.

SECURITY EVENT ACCESS

The evidence reader is not a runtime administrator

The dossier and security-event store have separate read and write responsibilities. Service processes append bounded events; the campaign collector reads only the run partition; the auditor reads sealed reports; none of those identities can alter business state, policy or broker topology.

DECISION LEDGER 48-36

Input	Required handling	Forbidden shortcut
service writer	append own events	read/export all
campaign collector	read RunId partition	delete or mutate
auditor	read sealed dossier	execute repair
runtime admin	operate deployment	rewrite evidence
application operator	no evidence backend	download raw logs

SEPARATION TEST

Each identity is exercised against allowed and forbidden operations in the retained deployment.

FAULT INJECTION

Evidence infrastructure failures are part of the security campaign

The laboratory removes event-sink connectivity, fills its bounded queue, expires policy, makes the owner store unavailable, drops broker permission and corrupts a candidate security snapshot. Each fault has a documented fail-closed or degraded mode and a recovery oracle.

DECISION LEDGER 48-37

Input	Required handling	Forbidden shortcut
event sink down	deny stays deny + degradation signal	allow
policy expired	readiness fail closed	last allow forever
owner store down	503 no scope guess	404/allow
broker permission removed	publish failure/outbox retained	drop fact
snapshot corrupt	current stays or startup fails	partial activate

FAULT OWNERSHIP

The test names which controller detects, contains and recovers the fault; a later manual fix is not a passing recovery result.

LOGGING-FAILURE TEST

A broken event sink cannot authorize the request

The fault host replaces only the event transport with a deterministic failing sink. The production decision and effect paths remain intact. The case proves response, zero effect and degradation signal, while marking event delivery unavailable rather than fabricating it.

EXECUTABLE TEST 48-E - DENIAL SURVIVES EVENT-SINK FAILURE

```
[Fact]
public async Task Event_sink_failure_never_changes_deny_to_allow()
{
    failingSink.FailWith(new IOException("synthetic-sink-unavailable"));
    var before = await probes.CaptureAsync(CaseId, ct);

    using var response = await client.SendAsync(
        Scenarios.Command("operator.wrongOwner", CaseId), ct);
    var after = await probes.CaptureAsync(CaseId, ct);

    response.StatusCode.Should().Be(HttpStatusCode.Forbidden);
    after.Should().BeEquivalentTo(before);
    health.SecurityEventDelivery.Should().Be(Degraded);
}
```

HONEST RESULT

The dossier records that the decision held and event delivery failed. It does not claim a complete denial-event record for this case.

RECOVERY EVIDENCE

Recovery means bounded restoration without widening access

After the injected fault is removed, the controller must validate fresh state, restore readiness and pass both an allowed coordinate and its nearest denied neighbor. Recovery is incomplete if only the happy path returns or if stale positive authorization survives.

DECISION LEDGER 48-38

Input	Required handling	Forbidden shortcut
fault removed	controller observes new state	sleep and assume
readiness	returns within budget	process alive
allowed probe	intended coordinate succeeds	health only
denied neighbor	still zero effect	not rerun
evidence	fault + action + recovery timeline	final screenshot

BOUNDARY AFTER RECOVERY

The same negative coordinate that failed closed during the fault must remain denied after service restoration.

DETERMINISM

Adversarial cases replay from immutable specimen descriptors

Each token, certificate, message and request mutation is generated from a versioned descriptor with a fixed clock and random seed recorded when randomness affects non-secret fixture data. Retries create new execution attempts but preserve Caseld and specimen identity.

DECISION LEDGER 48-39

Input	Required handling	Forbidden shortcut
specimen descriptor	canonical JSON + digest	binary only
clock	fixed per case	wall-clock sleep
randomness	seed/fixture revision	unlogged fuzz seed
retry	attempt number	new case identity
environment	same manifest	developer machine drift

REPLAY BOUNDARY

Deterministic fixtures reproduce the case input; external process scheduling and network timing remain bounded observations, not promised bit-for-bit replay.

FLAKY SECURITY TESTS

A pass after retry is not evidence of a stable boundary

Automatic reruns may help diagnose infrastructure noise, but the dossier reports every attempt. A required case that fails once and passes later is FLAKY and fails promotion until the cause is classified and the campaign reruns cleanly from a fresh environment.

DECISION LEDGER 48-40

Input	Required handling	Forbidden shortcut
pass first attempt	eligible	hide timing
fail then pass	FLAKY; fail gate	last result wins
timeout	INCONCLUSIVE; fail gate	skip
environment fault	invalid run or explicit fault case	ignore
quarantine	claim remains unproved	remove from required set

NO GREEN BY EXHAUSTION

Repeating a nondeterministic security test until it passes produces confidence theater, not evidence.

TEST DISCOVERY

Zero discovered tests is a hard failure

The orchestrator compares discovered Caselds with the catalogue before execution and executed Caselds afterward. Filters, runner changes and renamed assemblies cannot silently turn an empty security suite into success. Duplicate Caselds and unregistered tests also fail.

DECISION LEDGER 48-41

Input	Required handling	Forbidden shortcut
discovered count	equals required catalogue subset	exit code only
executed count	no required missing	skipped as pass
duplicate Caseld	reject	merge results
unknown test	register or reject	untracked evidence
filter	retained in run manifest	local default

ENUMERATION GATE

A test runner success code is insufficient until catalogue-to-discovery equality is proved.

CASE RESULT

The result shape distinguishes failure from absence of evidence

PASS, FAIL, FLAKY, NOT_RUN and INCONCLUSIVE are explicit. Only PASS satisfies a required claim. The record names every retained artifact and oracle; it does not embed large logs or sensitive diagnostics.

EXECUTABLE ASSET 48-J - SECURITY CASE RESULT

```
public sealed record SecurityCaseResult(  
    Guid RunId,  
    string ClaimId,  
    string CaseId,  
    int Attempt,  
    SecurityCaseStatus Status,  
    string DecisionObserved,  
    string ZeroEffectRef,  
    string SecurityEventRef,  
    IReadOnlyList<string> ArtifactPaths,  
    TimeSpan Duration);  
  
public enum SecurityCaseStatus  
{  
    Pass, Fail, Flaky, NotRun, Inconclusive  
}
```

FAIL-CLOSED STATUS

Unknown status values and required cases without a result are dossier failures.

ARTIFACT TAXONOMY

Evidence is a graph of small typed artifacts

The dossier includes the campaign header, catalogue, environment manifest, test results, effect snapshots, bounded event export, topology/permission inventory, leak-scan result and timeline. Raw secrets, tokens, private keys, production dumps and unrestricted logs are never dossier members.

DECISION LEDGER 48-42

Input	Required handling	Forbidden shortcut
campaign.json	run/build/time/toolchain	credential
catalogue.json	claims/cases/digest	test source copy
environment.json	value-free identities/revisions	config values
results/	typed case records + TRX	mixed prior runs
manifest.json	path/length/SHA-256/media type	absolute secret path

MINIMUM DOSSIER

Every file has a declared purpose, producer, schema version, RunId and content digest.

ARTIFACT RECORD

Content addressing detects missing and changed files

The manifest hashes bytes after redaction and leak scanning. Relative paths are normalized, duplicates are rejected and the manifest excludes itself from its own entry set. A later detached signature or CI attestation can bind this manifest without changing artifact schemas.

EXECUTABLE ASSET 48-K - ARTIFACT DESCRIPTOR

```
public sealed record EvidenceArtifact(
    string RelativePath,
    string MediaType,
    long Length,
    string Sha256,
    Guid RunId,
    string Producer,
    string SchemaVersion);

public static EvidenceArtifact Describe(
    string root, string path, Guid runId, string producer, string schema)
{
    var relative = Path.GetRelativePath(root, path).Replace('\\', '/');
    using var stream = File.OpenRead(path);
    return new(relative, MediaType.For(path), stream.Length,
        Convert.ToHexString(SHA256.HashData(stream)).ToLowerInvariant(),
        runId, producer, schema);
}
```

BOUNDED INTEGRITY

A SHA-256 manifest detects change when the trusted manifest is retained independently. By itself it does not prove who created the evidence or prevent a privileged rewrite.

CANONICAL SERIALIZATION

Evidence digests require one byte representation

Typed objects are serialized with fixed property order, UTF-8 without BOM, invariant timestamps, normalized line endings and ordinal sorting of set-like collections. Floating-point values are avoided in security decisions. Schema and canonicalization versions are part of every digest domain.

DECISION LEDGER 48-43

Input	Required handling	Forbidden shortcut
properties	explicit order/source generator	reflection order assumption
sets	ordinal sorted arrays	hash-set enumeration
timestamps	UTC round-trip format	local culture
text	UTF-8 + LF + normalization rule	platform default
domain prefix	schema/canonical version	same hash context

DIGEST MEANING

The dossier hashes canonical artifact bytes; it never claims arbitrary JSON objects are inherently canonical.

MANIFEST INTEGRITY TEST**Deletion, insertion and mutation must all fail validation**

The validator is tested by changing one artifact byte, deleting a required file, adding an unlisted file under the sealed root, altering RunId and replacing a result with a valid result from another run. Each mutation must fail with a stable reason before claim evaluation.

DECISION LEDGER 48-44

Input	Required handling	Forbidden shortcut
byte changed	digest mismatch	parse and accept
file deleted	missing artifact	skip optional guess
extra file	unsealed artifact failure	ignore
RunId changed	same-run failure	hash-only pass
old PASS copied	freshness/run failure	best available

TAMPER ORACLE

The test mutates a copy of the completed dossier; it never damages the retained source artifacts.

SAME-RUN DOSSIER**Freshness is a join, not a filename convention**

All required artifacts repeat the campaign RunId and fall within the trusted runner interval plus documented collection grace. Build, catalogue, policy and environment digests must match the header. File modification time is diagnostic only and cannot establish provenance.

DECISION LEDGER 48-45

Input	Required handling	Forbidden shortcut
RunId	equal everywhere	folder name
time	within runner interval/grace	mtime
build digest	equal campaign header	same version label
policy/environment	equal active manifests	similar profile
test catalogue	exact digest	same count

NO ARTIFACT COLLAGE

A dossier assembled from individually valid artifacts from different runs is invalid.

DOSSIER VALIDATOR

Validation fails before producing an acceptance verdict

The validator checks structure, identity, hashes, freshness, required case coverage, status, effect/event joins and leak-scan success. It returns detailed reasons but never a partial PASS.

EXECUTABLE ASSET 48-L - FAIL-CLOSED DOSSIER GATE

```
public DossierVerdict Validate(SecurityDossier dossier)
{
    Require(dossier.Manifest.Artifacts.Count > 0, "zero-artifacts");
    Require(AllHashesMatch(dossier), "artifact-integrity");
    Require(AllSameRun(dossier), "mixed-run");
    Require(IsFresh(dossier), "stale-run");
    Require(CatalogueCoverageIsExact(dossier), "coverage-gap");
    Require(dossier.Results.All(x => x.Status == SecurityCaseStatus.Pass),
        "non-pass-case");
    Require(AllZeroEffectsJoined(dossier), "missing-effect-proof");
    Require(AllRequiredEventsJoined(dossier), "missing-event-proof");
    Require(dossier.LeakScan.Passed, "leak-scan-failed");
    return DossierVerdict.Accepted(dossier.RunId);
}
```

NO WARNING VERDICT

A required failure, flaky case, stale artifact or missing join yields REJECTED, not ACCEPTED_WITH_WARNINGS.

TOOLCHAIN MANIFEST

The runner is part of the evidence-producing system

The dossier records SDK version, test platform, framework packages, OS, architecture, container runtime, broker/provider versions, command, filters and timeouts. Runner migration can change discovery and result formats, so it is a reviewed evidence change rather than an invisible optimization.

DECISION LEDGER 48-46

Input	Required handling	Forbidden shortcut
SDK	exact version/global.json digest	net10 label
runner	VSTest or MTP + version	dotnet test generic
packages	lock/artifact digest	latest restore
dependencies	image/provider digests	service tags
command	argument vector + filter	shell history

REPRODUCTION BOUNDARY

The manifest supplies enough non-secret identity to repeat the campaign; it does not claim identical scheduling or external infrastructure behavior.

CAMPAIGN COMMAND

Test execution writes into a fresh run directory

The Phase-0 example pins the VSTest path explicitly for the retained environment, emits TRX, enforces a bounded hang timeout without automatic memory dumps and passes the RunId as non-secret metadata. A different runner requires a separate command contract.

EXECUTION ASSET 48-A - BOUNDED VSTEST CAMPAIGN

```
dotnet test NEXUS1.sln \
  --configuration Release \
  --no-restore \
  --results-directory "artifacts/security/$RUN_ID/test-results" \
  --logger "trx;LogFilePrefix=security" \
  --filter "Category=SecurityCampaign" \
  --blame-hang \
  --blame-hang-timeout 5m \
  --blame-hang-dump-type none
```

NO LITERAL VALUES

RUN_ID is a generated non-secret identifier. Credentials and protected material remain in the Chapter 47 injection paths and never appear in the command line.

ENVIRONMENT MANIFEST

The laboratory proves the environment it actually ran

The value-free manifest lists process identities, image digests, exposed and internal ports, database principals, broker vhost/permissions, certificate/policy revisions, active contract versions and probe endpoints. It excludes secret values and certificate private material.

DECISION LEDGER 48-47

Input	Required handling	Forbidden shortcut
services	name/image digest/process identity	container ID only
network	declared reachable edges	full host inventory
databases	owner/principal/schema revision	connection string
broker	vhost/topology/permission digest	password
security	issuer/trust/policy revision	token/key

EQUIVALENCE CLAIM

The manifest proves the retained Phase-0 Compose profile, not staging or production equivalence.

COMPOSE CAMPAIGN

The deployment suite uses normal process and socket boundaries

The campaign starts the same service images and composition profiles intended for the Phase-0 demonstrator, waits for bounded readiness, runs inventory and negative probes from a dedicated test controller, then destroys only the isolated run resources after artifacts are sealed.

DECISION LEDGER 48-48

Input	Required handling	Forbidden shortcut
startup	fresh project/run namespace	shared developer stack
readiness	all required dependencies validated	fixed sleep
controller	no service credentials beyond probes	admin superuser
data	synthetic run partition	existing demo data
cleanup	after seal + recovery capture	delete evidence

ISOLATION RULE

A campaign cannot pass by observing state created by a previous run or by sharing credentials with an interactive developer environment.

EVIDENCE TIMELINE

Causality requires ordered controller observations

The campaign timeline records orchestrator actions, dependency state, request/delivery correlation, decision event, effect snapshot and recovery transition. Service-local timestamps are retained with clock-source identity and uncertainty; the controller's monotonic sequence orders the test narrative.

DECISION LEDGER 48-49

Input	Required handling	Forbidden shortcut
sequence	controller-assigned monotonic number	wall clock sort
occurredAt	service observation time	global order claim
receivedAt	collector time	event time replacement
correlation	case/request/message refs	payload join
clock quality	source/offset/uncertainty	perfect sync claim

BOUNDED CAUSATION

The timeline proves the controlled campaign sequence; Chapter 49 will generalize correlation and causation across ordinary runtime traffic.

ACCEPTANCE REPORT

The report starts from claims, not test framework totals

The human-readable report summarizes each security claim, required layers, adversarial cases, outcome, zero-effect/evidence joins, runtime cost and residual risk. Test counts are supporting data. A thousand passing unit cases cannot hide one missing deployment boundary.

DECISION LEDGER 48-50

Input	Required handling	Forbidden shortcut
claim summary	proved/not proved/limited	green percentage
case table	all required statuses	failed only
effect proof	oracle references	response codes
runtime cost	latency/storage/availability impact	benefit only
residual	owner + next decision	generic risk

READER CONTRACT

The report can be understood without opening raw TRX, while every conclusion links back to machine-checkable artifacts.

REPORT MODEL

A claim verdict preserves limits and residual risk

The model prevents a report generator from translating every PASS into an unqualified security statement. The proved property, environment and expiry are explicit, and residual risks remain first-class.

EXECUTABLE ASSET 48-M - BOUNDED CLAIM VERDICT

```
public sealed record SecurityClaimVerdict(  
    string ClaimId,  
    ClaimStatus Status,  
    string PropertyProved,  
    string EnvironmentIdentity,  
    DateTimeOffset ValidatedAtUtc,  
    DateTimeOffset EvidenceExpiresAtUtc,  
    IReadOnlyList<string> CaseIds,  
    IReadOnlyList<string> ResidualRisks,  
    IReadOnlyList<string> ArtifactRefs);  
  
public enum ClaimStatus { ProvedInPhase0, NotProved, Limited }
```

NO TIMELESS PROOF

A verdict expires when build, policy, environment, catalogue or time-bound security material changes.

EVIDENCE EXPIRY**A security dossier is invalidated by relevant change**

A new service build, endpoint, contract, policy revision, trust bundle, certificate profile, broker permission, database migration, test catalogue or environment topology invalidates affected claims. The impact mapper selects the minimum required rerun but may not retain an old PASS across an unclassified change.

DECISION LEDGER 48-51

Input	Required handling	Forbidden shortcut
code/image change	affected service/boundaries rerun	same version string
policy change	authorization matrix rerun	syntax test only
certificate/trust	fresh handshake/rotation rerun	file digest
broker definition	origin/ACL/topology rerun	connection test
unknown change	full security campaign	assume unrelated

EVIDENCE IS PER ARTIFACT

The book rejects the phrase 'security tests passed' unless the exact retained build and environment are named.

AUDIT ACCESS AND RETENTION**Retention is an architecture decision, not a test default**

Phase-0 retains only synthetic, value-free evidence for a documented bounded period. Access, export, deletion and restore actions are themselves security events. Production legal, regulatory, privacy and records-management requirements remain outside the demonstrator and require organizational decisions.

DECISION LEDGER 48-52

Input	Required handling	Forbidden shortcut
retention period	Phase-0 documented value	forever
read access	auditor profile + event	shared link
export	sealed manifest subset	raw log archive
deletion	authorized expiry + tombstone	silent rm
legal hold	outside Phase-0	pretend implemented

PRIVACY BOUNDARY

The dossier is designed to minimize protected data; that does not establish legal compliance or a production retention schedule.

PERFORMANCE AND CAPACITY

Security evidence has a runtime and storage cost

Negative campaigns consume service capacity, broker traffic, database probes and evidence storage. Production-like load is not mixed with destructive security cases. The report records case duration, event volume, artifact bytes and probe overhead so protection does not become an unbounded denial-of-service mechanism.

DECISION LEDGER 48-53

Input	Required handling	Forbidden shortcut
event volume	bounded per case/class	one event per byte
artifact size	allow-list + quotas	collect everything
probe latency	budget and separate credentials	request hot path query
campaign load	isolated environment	production attack
failure storm	sampling without losing required classes	disable logging

COST EVIDENCE

The dossier records evidence cost; it does not claim production capacity from a Phase-0 campaign.

RESIDUAL RISK REGISTER

The campaign documents what its mechanisms cannot prove

The security report keeps unresolved risks beside proved claims. Important Phase-0 residuals include privileged broker substitution without end-to-end signatures, local digest manifests without independent signing, synthetic identity issuance, single-host Compose isolation and absence of organizational monitoring and incident operations.

DECISION LEDGER 48-54

Input	Required handling	Forbidden shortcut
broker privilege	record; future authenticity decision	claim non-repudiation
manifest rewrite	independent retention/signing future	hash equals signature
test issuer	token handler proof only	production IdP proof
single host	Compose boundary only	cluster segmentation
operations	procedure design pending	SOC/SIEM claim

RESIDUALS ARE OUTPUTS

A complete campaign may pass while explicitly retaining risks outside its architecture and environment.

OUTSIDE PHASE-0

The dossier is not a certification, audit opinion or penetration report

This chapter does not claim independent penetration testing, red-team coverage, vulnerability-free dependencies, production PKI/HSM custody, SIEM operation, legal compliance, safety qualification, plant connectivity, multi-region resilience or adversary simulation against a real facility.

DECISION LEDGER 48-55

Input	Required handling	Forbidden shortcut
penetration/red team	external scoped engagement	unit test label
independent audit	auditor criteria/opinion	self-generated report
production PKI/HSM	operational platform evidence	PEM fixtures
SIEM/SOC	people/process/tool operation	JSON logs
safety/plant	outside software demonstrator	nuclear terminology

STANDING BOUNDARY

The retained result is a Phase-0 engineering evidence dossier for the educational NEXUS-1 runtime.

REJECTED ALTERNATIVES

Convenient evidence shortcuts weaken the claim

The chapter rejects status-only authorization tests, all-mock end-to-end suites, screenshots as primary evidence, shared admin credentials, copied prior results, mutable audit spreadsheets, secret-bearing dumps, flaky-pass reruns and one aggregate 'security score'.

DECISION LEDGER 48-56

Input	Required handling	Forbidden shortcut
403-only	add effect/event evidence	ship
all mocks	assign correct test layer	call deployment test
screenshots	machine-readable source + optional view	primary proof
prior PASS	fresh same-run execution	copy
security score	claim-by-claim verdict	average gaps away

ENGINEERING PREFERENCE

A smaller set of attributable boundary proofs is stronger than a large unjoined test count.

DEFINITION OF DONE

Chapter 48 is implemented only when the dossier can reject itself

Implementation requires the claim catalogue, endpoint/identity/permission inventories, negative HTTP and message cases, zero-effect probes, typed security events, leak scan, fault/recovery cases, artifact manifest, same-run validator and bounded report. Every gate has at least one mutation test proving it fails closed.

DECISION LEDGER 48-57

Input	Required handling	Forbidden shortcut
catalogue	exact coverage + mutation test	document only
negative cases	real required seams	unit only
zero effects	all declared probes	status only
dossier	hash/run/freshness/leak gates	zip folder
verdict	limits + residuals + expiry	green banner

COMPLETION RULE

A dossier generator that accepts a deleted, stale, tampered, mixed-run or incomplete artifact set is itself a failed security control.

IMPLEMENTATION SEQUENCE

Build the oracle before multiplying attack cases

Begin with the claim/effect catalogue and run identity, then implement probes and typed events, then endpoint and credential inventories, then negative HTTP/broker/message tests, then fault recovery, leak scanning and dossier sealing. Otherwise the team creates many responses without knowing whether protected effects occurred.

DECISION LEDGER 48-58

Input	Required handling	Forbidden shortcut
1	claims + protected effects	test methods
2	RunId + probes + events	report UI
3	inventories + exact coverage	manual checklist
4	adversarial suites by layer	one mega-test
5	seal/validate/report	copy artifacts

ORDER MATTERS

A trustworthy oracle makes each later adversarial specimen evidence-producing rather than merely failure-producing.

OFFICIAL TECHNICAL ANCHORS

Platform behavior is tied to primary documentation

The implementation uses official ASP.NET Core integration-test and policy contracts, .NET test-runner and cryptographic APIs, RabbitMQ authenticated User ID and access-control behavior, OWASP logging guidance and NIST audit-control terminology. NEXUS-1 decisions remain explicitly local architecture choices.

DECISION LEDGER 48-59

Anchor	What it supports	What it cannot prove
ASP.NET Core integration tests	WebApplicationFactory/TestServer surface	production equivalence
policy authorization	requirements/handlers/resources	NEXUS policy design
dotnet test	runner/result/blame behavior	evidence acceptance
RabbitMQ validated User ID	broker-validated publisher property	end-to-end signature
RabbitMQ access control	vhost/resource permissions	payload authenticity
OWASP logging guidance	event content, hygiene and verification	certification
NIST SP 800-53	audit-control terminology	implemented baseline

SOURCE DISCIPLINE

Primary documentation establishes platform semantics. The catalogue, zero-effect rule, same-run join and Phase-0 verdict are NEXUS-1 engineering decisions.

EVIDENCE RECORD

The chapter closes with specified gates and no fabricated run

The manuscript now defines executable shapes, required suites, failure oracles and dossier validation. No repository test run, Compose campaign, broker permission export, leak scan or independently retained manifest has been supplied to this writing process. All runtime verdicts therefore remain pending.

DECISION LEDGER 48-60

Input	Required handling	Current evidence
ADR-048	789f3da25bf52b77386c670d...	SPECIFIED
catalogue/inventories	required artifacts	PENDING
HTTP/workload/broker cases	required non-zero run	PENDING
zero-effect/event joins	required for every claim	PENDING
same-run dossier	fresh sealed artifact set	PENDING

VERDICT

SPECIFIED | RUNTIME PENDING. Nothing in this chapter claims that the Phase-0 security campaign, external audit, penetration test or production security platform already exists.

CHAPTER CHECKPOINT

Can every denial prove both the boundary and the absence of effect?

Review the chapter by following one Caseld from claim catalogue to the sealed report without consulting a secret-bearing artifact.

- Which earlier security decision does the case challenge, and at which real boundary?
- Why is a 401, 403 or rejected delivery insufficient without a protected-effect oracle?
- Which tests may use synthetic authentication, and which must retain the production handler?
- How does the endpoint inventory prevent zero-discovery and unclassified-route passes?
- What proves a valid credential fails at a neighboring service, exchange or database?
- Which RabbitMQ property is broker-validated, and why is Appld not origin proof?
- What can the Phase-0 tamper campaign detect, and what remains outside its mechanism?
- How are denial events useful without exposing subjects, tokens, secrets or payloads?
- Why does a fail-then-pass security test fail the promotion gate?
- What makes the dossier same-run, value-free, fresh and self-rejecting?

NEXT - CHAPTER 49

Correlation and Causation Across Every Boundary will carry stable execution context through HTTP, messages, stores and evidence while distinguishing observed sequence from causal proof.

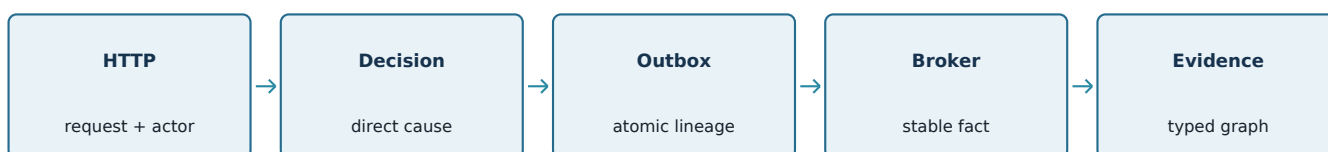
PART IX - OBSERVABILITY, RESILIENCE, AND FAILURE EVIDENCE

49. Correlation and Causation Across Every Boundary

A distributed case becomes explainable only when business lineage survives every handoff while diagnostic trace context remains explicitly non-authoritative.

CHAPTER DECISION

Carry immutable MessageId, workflow-scoped CorrelationId and direct CausationId as business evidence; carry W3C trace context as attempt-local diagnostics; carry only a bounded owner-specific actor reference. Capture lineage at the business commit, preserve it through retries, and reconstruct causes from typed edges rather than timestamps.

PROOF PATH 49-A - ONE CASE ACROSS FIVE BOUNDARIES**GUARANTEE LEDGER 49-A**

Boundary	Carries	Allowed claim	Still forbidden
HTTP -> command	request, correlation, actor ref	admitted internal context	caller-authoritative identity
commit -> outbox	message, correlation, cause	durable producer assertion	broker delivery
delivery -> worker	stable fact + new attempt	attributable processing	exactly-once transport
records -> graph	typed nodes and edges	bounded causal account	global total order

ORDINARY DEVELOPER PROBLEM

One string called CorrelationId destroys four meanings

A request ID is copied into trace-id, messageId, idempotency key and audit actor. Retries then look like new business facts, fan-in is flattened into one fictional parent, and a sampled-out trace makes the transaction appear causeless. Search becomes easy only because every identifier has lost its contract.

DECISION LEDGER 49-1

Shortcut	Failure	Required correction
one ID everywhere	deduplication, grouping and diagnostics collide	typed identities
timestamp implies cause	clock skew manufactures history	explicit causal edge
Activity.Current at dispatch	ambient context belongs to another attempt	outbox snapshot
subject in baggage	identity leaks across trust boundaries	pairwise actor reference
one parent at fan-in	other accepted causes disappear	primary cause + edge set

PLAIN-LANGUAGE PRINCIPLE

Correlation helps find related work. Causation says what directly produced a new fact. Trace context explains one execution attempt. None may silently stand in for another.

CHAPTER BOUNDARY

Semantics now; telemetry products later

Chapter 49 defines identities, propagation, trust rules, persistence and evidence reconstruction. Chapter 50 will define the stable structured-log schema. Chapter 51 will configure ActivitySource, OpenTelemetry SDKs, sampling and exporters. This chapter uses diagnostic context only where required to prove that business truth does not depend on it.

SCOPE INVARIANT

No collector, backend or dashboard is claimed. All listings are compile-oriented reference assets until repository execution evidence is retained.

DECISION RECORD

ADR-049 - Three ledgers, joined but never merged

NEXUS-1 records business lineage, diagnostic execution context and security attribution as separate typed ledgers. A retained evidence record may join them by explicit references. It may not derive business cause from a span parent, authorization from baggage, or uniqueness from correlation.

DECISION LEDGER 49-2

Ledger	Stable fields	Owns	Does not own
business lineage	MessageId, CorrelationId, CausationId	fact identity and direct cause	execution attempt
diagnostic	TraceId, SpanId, AttemptId	observed path and duration	business truth
security	ActorRef, WorkloadRef, PolicyRev	bounded attribution	raw credential
workflow	WorkflowId, transition	owner-local progression	global transaction

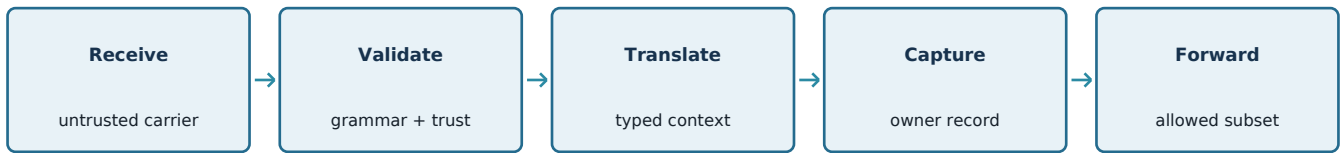
DECISION IDENTITY

ADR-049 canonical SHA-256 is computed from the retained decision text by the evidence build; the manuscript does not invent a runtime digest.

PROOF PATH

Each handoff performs validation, translation and capture

PROOF PATH 49-B - BOUNDARY DISCIPLINE



PROOF OBLIGATIONS 49-A

Join	Must match	Failure meaning
request -> command	internal request + actor mapping	unattributed decision
decision -> message	CausationId names accepted cause	orphan fact
message -> receipt	same MessageId and fingerprint	identity mutation
attempt -> trace	observed attempt reference	diagnostic gap only
transition -> evidence	typed causal edge	unexplained state

IDENTITY VOCABULARY

Seven identifiers answer seven different questions

IDENTITY MATRIX 49-A

Identifier	Question	Lifetime	May change on retry?
MessageId	which public fact?	fact lifetime	no
CorrelationId	which business conversation?	workflow lifetime	no
CausationId	which direct accepted cause?	new fact edge	no
IdempotencyKey	which client command replay?	command window	no
RequestId	which edge admission?	HTTP attempt	yes
TraceId / SpanId	which diagnostic path/operation?	trace / span	yes
AttemptId	which local execution?	one attempt	yes

NON-EQUIVALENCE RULE

Equal textual values do not make identifiers semantically equal. Code, schemas and evidence use distinct types and column names even when an ingress adapter initially derives one value from another.

MESSAGE IDENTITY

MessageId survives dispatch, redelivery and replay

The V1 envelope fixed in Chapter 20 names one producer-created public fact. The outbox stores its exact bytes. Publisher recovery, broker redelivery and consumer retry preserve MessageId, correlation, causation, content fingerprint and payload. A new business fact receives a new MessageId.

MESSAGE MUTATIONS 49-B

Situation	MessageId	AttemptId	Diagnostic span
publisher retry	same	new	new or linked
broker redelivery	same	new	new consumer attempt
consumer replay	same	new	new or absent
derived public fact	new	new	new producer span
dead-letter evidence	original retained	new transfer record	new diagnostic attempt

CORRELATION SEMANTICS

Correlation groups a case; it does not order it

The first trusted NEXUS-1 edge creates or admits a bounded CorrelationId. Downstream public facts in the same business conversation preserve it so operators can locate a distributed case. Separate work may share a correlation; concurrent facts may have no total order; a correlation never authorizes an action.

DECISION LEDGER 49-3

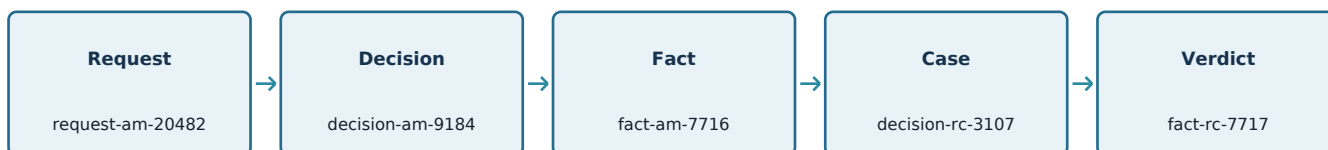
Claim	Allowed?	Reason
find all retained facts for one case	yes	search/grouping contract
choose the first fact by CorrelationId	no	identifier has no order
deduplicate by CorrelationId	no	many facts share it
authorize because correlation is trusted	no	not an identity credential
infer direct cause from shared correlation	no	requires CausationId/edge

CAUSATION SEMANTICS

Causation names the immediate accepted reason

A producer sets CausationId to the request, command, decision or input message that directly caused the new fact under a documented policy. Each hop advances the causal chain. Pointing every downstream message back to the original HTTP request erases the decisions that actually transformed the workflow.

PROOF PATH 49-C - DIRECT CAUSES ADVANCE



CAUSAL ASSERTION

A received CausationId is the producer's signed-by-boundary assertion about direct cause. The consumer records provenance; it does not pretend to verify a global graph that no single database owns.

TYPED IDENTITY KERNEL

Make illegal substitution visible to the compiler

EXECUTABLE ASSET 49-A - BOUNDED IDENTIFIERS

```
public readonly record struct MessageId(string Value);
public readonly record struct CorrelationId(string Value);
public readonly record struct CausationId(string Value);
public readonly record struct RequestId(string Value);
public readonly record struct AttemptId(Guid Value);
public readonly record struct PairwiseActorRef(string Value);

public static class LineageGrammar
{
    private static readonly Regex Token = new(
        "^[a-z0-9][a-z0-9._:-]{7,127}$",
        RegexOptions.CultureInvariant | RegexOptions.NonBacktracking);

    public static T Parse<T>(string? raw, Func<string, T> create)
    {
        var value = raw?.Trim();
        if (value is null || !Token.IsMatch(value))
            throw new LineageRejected("lineage-token-invalid");
        return create(value);
    }
}
```

GRAMMAR DECISIONS 49-C

Property	Rule	Why
alphabet	lowercase bounded token	stable comparison and safe evidence
length	8..128 characters	abuse and storage bound
normalization	trim then exact	no hidden aliases
construction	factory only	no raw string substitution
logging	field name + value	meaning travels with data

REFERENCE-ASSET STATUS

The listing is compile-oriented. Exact package, target-framework and executed-test evidence remain pending until the repository checkpoint exists.

EXECUTION CONTEXT

The application receives an explicit immutable snapshot

EXECUTABLE ASSET 49-B - CONTEXT SPLIT BY AUTHORITY

```
public sealed record BusinessLineage(  
    CorrelationId CorrelationId,  
    CausationId CausationId,  
    RequestId? RequestId);  
  
public sealed record DiagnosticContext(  
    string? TraceId,  
    string? ParentSpanId,  
    bool Remote,  
    bool Sampled);  
  
public sealed record SecurityAttribution(  
    PairwiseActorRef? Actor,  
    string? WorkloadRef,  
    string PolicyRevision);  
  
public sealed record ExecutionContextSnapshot(  
    BusinessLineage Business,  
    DiagnosticContext Diagnostic,  
    SecurityAttribution Security,  
    AttemptId AttemptId,  
    DateTimeOffset AdmittedAtUtc);
```

SNAPSHOT INVARIANTS 49-D

Field group	Required?	Missing behavior
business lineage	yes for state-changing path	fail before Domain
diagnostic	no	new diagnostic root or no span
actor	operator command only	deny protected operation
workload	service delivery only	reject untrusted producer path
attempt	always local	generate before handler

NO AMBIENT AUTHORITY

Activity.Current and AsyncLocal may assist diagnostics, but the Application handler receives the snapshot explicitly. Business decisions never depend on ambient trace state.

CONTEXT FACTORY

Creation policy is deterministic and owner-local

EXECUTABLE ASSET 49-C - TRUSTED INTERNAL LINEAGE

```
public sealed class EdgeLineageFactory(IIdSource ids, IClock clock)
{
    public ExecutionContextSnapshot Create(
        AdmittedHttpRequest request,
        PairwiseActorRef actor,
        string policyRevision)
    {
        var requestId = new RequestId(ids.NewToken("req"));
        var correlation = request.AcceptedCorrelationId
            ?? new CorrelationId(ids.NewToken("corr"));

        return new(
            new(correlation, new CausationId(requestId.Value), requestId),
            request.TraceContext.ToDiagnosticSnapshot(),
            new(actor, null, policyRevision),
            new AttemptId(ids.NewGuid()),
            clock.UtcNow);
    }
}
```

FACTORY POLICY 49-E

Input	Trust decision	Output
external request ID	never accepted as internal authority	new RequestId
correlation header	grammar + route allow-list	admit or replace
traceparent	W3C parse only	remote diagnostic context or none
credential claims	Chapter 46 owner mapping	pairwise actor ref
clock	owner-local observation	AdmittedAtUtc

CREATION RULE

When diagnostic context is invalid or absent, the edge may start a new trace. It must not invent a business cause: the admitted internal request remains the explicit cause.

HTTP CARRIER CONTRACT

Headers are untrusted bytes until the edge admits them

HTTP CARRIER 49-F

Header	Purpose	Admission	Never used for
traceparent	portable trace position	W3C parse; 1 value	identity/authorization
tracestate	vendor trace state	bounded; only with valid parent	business fields
baggage	optional diagnostics	deny-by-default allow-list	subject/role/secret
x-correlation-id	business search hint	route policy + grammar	dedup/order
idempotency-key	command replay	endpoint policy	message identity

TRUST-BOUNDARY RULE

Duplicate, oversized or malformed propagation headers are rejected or ignored according to one explicit route policy. They are never concatenated into a value that changes meaning.

SAFE RESPONSE CONTRACT

Return locators, not credentials or internal topology

The BFF may return a safe internal request reference and admitted correlation reference in response headers or Problem Details extensions. It does not echo bearer tokens, raw subject claims, tracestate, baggage, database IDs that reveal topology, or a claim that a trace was retained.

RESPONSE EXPOSURE 49-G

Value	Expose?	Reason
RequestId	yes	support locator for one edge attempt
CorrelationId	yes when route allows	distributed case locator
TraceId	optional	diagnostic locator, not guarantee
CausationId	no by default	internal lineage detail
ActorRef / WorkloadRef	no	security attribution

HTTP ADMISSION MIDDLEWARE

Parse diagnostics without granting them authority

EXECUTABLE ASSET 49-D - REMOTE TRACE ADMISSION

```
public static RemoteTraceAdmission AdmitTrace(IHeaderDictionary headers)
{
    var parents = headers["traceparent"];
    var states = headers["tracestate"];
    if (parents.Count > 1 || states.Count > 1)
        return RemoteTraceAdmission.Invalid("trace-header-duplicate");

    var parent = parents.SingleOrDefault();
    var state = states.SingleOrDefault();
    if (parent is null)
        return state is null
            ? RemoteTraceAdmission.Absent()
            : RemoteTraceAdmission.Invalid("tracestate-without-parent");

    if (parent.Length > 55 || (state?.Length ?? 0) > 512)
        return RemoteTraceAdmission.Invalid("trace-header-oversized");

    return ActivityContext.TryParse(parent, state, isRemote: true, out var parsed)
        ? RemoteTraceAdmission.Accepted(parsed)
        : RemoteTraceAdmission.Invalid("traceparent-invalid");
}
```

FAILURE POLICY 49-H

Route class	Invalid trace	Business request
public read	discard + bounded event	continue with new diagnostic root
operator command	discard + bounded event	continue if auth and business context valid
internal control	reject by contract	no Domain call
health	discard	no business lineage required

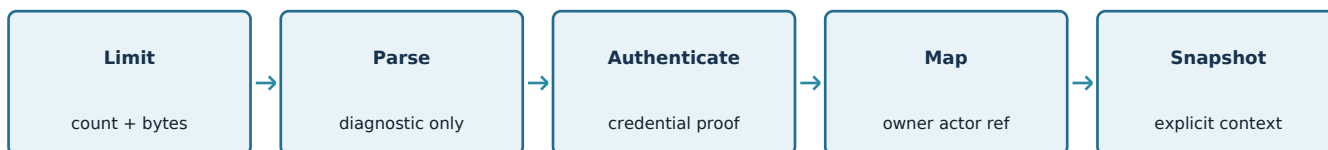
STANDARDS ALIGNMENT

W3C Trace Context requires invalid traceparent values to be ignored and tracestate without traceparent to be discarded. NEXUS-1 adds endpoint-specific evidence and abuse limits without treating trace validity as authorization.

HTTP MIDDLEWARE PIPELINE

Admission order prevents forged context from reaching Domain

PROOF PATH 49-D - EDGE ORDER



EXECUTABLE ASSET 49-E - SCOPED SNAPSHOT FEATURE

```
public async Task InvokeAsync(HttpContext http, RequestDelegate next)
{
    var carrier = propagation.Admit(http.Request.Headers, routePolicy.For(http));
    var principal = await authenticator.AuthenticateAsync(http);
    var attribution = policyMap.Map(principal, http.Request);

    var snapshot = lineage.CreateHttp(carrier, attribution);
    http.Features.Set<IExecutionContextFeature>(
        new ExecutionContextFeature(snapshot));

    using var scope = diagnostics.Push(snapshot); // diagnostic only
    await next(http);
}
```

ORDERING PROOF 49-I

If reversed	Failure
mapping before authentication	unproved claims become attribution
snapshot before limits	oversized headers allocate state
Domain before snapshot	decision lacks cause
ambient scope after next	downstream sees stale context
response before handler	locator exists without outcome

HTTP CONTRACT TESTS

Malformed diagnostics must not mutate business meaning

VERIFICATION ASSET 49-A - TRACE REJECTION IS BOUNDED

```
[Theory]
[InlineData("00-00000000000000000000000000000000-1111111111111111-01")]
[InlineData("00-11111111111111111111111111111111-0000000000000000-01")]
[InlineData("ff-11111111111111111111111111111111-1111111111111111-01")]
public async Task Invalid_trace_never_supplies_business_cause(string traceparent)
{
    using var request = AuthorizedCommand();
    request.Headers.TryAddWithoutValidation("traceparent", traceparent);

    var response = await client.SendAsync(request);

    response.StatusCode.Should().Be(HttpStatusCode.Accepted);
    committed.Single().CausationId.Should().Be(expectedInternalRequestId);
    events.Should().ContainSingle(e => e.Code == "traceparent-invalid");
}
```

REQUIRED EDGE CASES 49-J

Mutation	Expected business effect	Expected evidence
duplicate traceparent	route policy unchanged	duplicate classified
tracestate alone	no remote parent	invalid pair recorded
oversized baggage	no actor/role mapping	carrier rejected
forged correlation	new internal correlation	replacement recorded
sampling flag off	lineage still committed	trace may be absent

TEST HONESTY

The example proves route policy and business-lineage independence. It does not prove an exporter retained a trace; that belongs to Chapter 51.

USER CONTEXT MINIMIZATION

Attribution crosses a boundary only after owner mapping

Chapter 46 forbids a universal raw-subject map. The BFF validates the credential, maps claims into an owner-specific pairwise actor reference and passes only the bounded reference and policy revision required by the command. Internal services do not reconstruct roles from trace baggage.

EXECUTABLE ASSET 49-F - BOUNDED ATTRIBUTION

```
public sealed record CommandAttribution(  
    PairwiseActorRef Actor,  
    string PolicyRevision,  
    string AuthenticationClass);  
  
public CommandAttribution Map(ClaimsPrincipal principal, Resource resource)  
{  
    authorizer.Demand(principal, resource, "alarm.acknowledge");  
    return new(  
        pairwise.Map(principal, owner: "alarm-management"),  
        policy.CurrentRevision,  
        principal.FindFirstValue("amr") ?? "unknown");  
}
```

FORBIDDEN PROPAGATION 49-K

Candidate	Disposition	Reason
bearer token	never forward	credential replay
raw subject/email	never in lineage	linkability and privacy
role list	never in baggage	owner policy differs
pairwise actor ref	only required command evidence	bounded attribution
policy revision	record with decision	reconstruct authorization semantics

SECURITY INVARIANT

Correlation, causation, trace and baggage are attacker-controlled carrier data at ingress. Only authentication and owner-local authorization create security attribution.

APPLICATION BOUNDARY

Commands carry context beside intent, not inside Domain values

EXECUTABLE ASSET 49-G - COMMAND PLUS CONTEXT

```
public sealed record AcknowledgeAlarm(  
    AlarmId AlarmId,  
    string Reason,  
    IdempotencyKey IdempotencyKey);  
  
public sealed record CommandInvocation<TCommand>(  
    TCommand Command,  
    ExecutionContextSnapshot Context);  
  
public async Task<Outcome> Handle(  
    CommandInvocation<AcknowledgeAlarm> invocation,  
    CancellationToken cancellationToken)  
{  
    var decision = aggregate.Acknowledge(invocation.Command, clock.UtcNow);  
    await unitOfWork.CommitAsync(  
        decision,  
        invocation.Context.Business,  
        invocation.Context.Security,  
        cancellationToken);  
    return Outcome.Accepted(decision.DecisionId);  
}
```

RESPONSIBILITY SPLIT 49-L

Surface	Reads	May decide
Edge	headers + credential	admission and actor mapping
Application	command + snapshot	orchestration and commit
Domain	typed command + state	business transition
Infrastructure	decision + lineage	atomic persistence shape
Diagnostics	snapshot copy	observation only

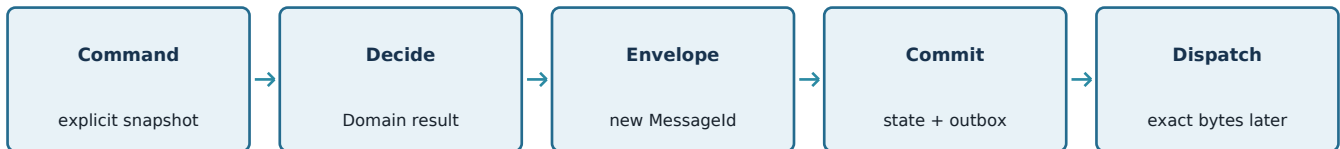
DOMAIN PURITY

The aggregate never reads `Activity.Current`, HTTP headers, `ClaimsPrincipal` or broker properties. It returns a decision that Infrastructure can join to explicit lineage.

ATOMIC LINEAGE CAPTURE

The producer freezes cause at the same commit as truth

PROOF PATH 49-E - PRODUCER TRANSACTION



TRANSACTION SET 49-M

Row	Identity	Lineage role
AlarmTransition	DecisionId	owner-local cause
OperatorAttribution	DecisionId + ActorRef	security evidence
OutboxMessage	MessageId	public fact + correlation + cause
EvidenceJoin	DecisionId + AttemptId	commit observation
IdempotencyReceipt	IdempotencyKey	command replay result

ENVELOPE COMPATIBILITY

Chapter 20 V1 remains unchanged

The public envelope still contains `EnvelopeVersion`, `MessageId`, `EventType`, `SchemaVersion`, `OccurredAtUtc`, `Producer`, `CorrelationId`, `CausationId`, content coordinates, fingerprint and payload. Trace context remains transport/diagnostic metadata outside the business fingerprint. This avoids making sampling or vendor state part of public fact identity.

V1 PRESERVATION 49-N

Value	Stored in envelope?	Fingerprint?	Retry behavior
<code>MessageId</code>	yes	yes	stable
<code>CorrelationId</code>	yes	yes	stable
<code>CausationId</code>	yes	yes	stable
<code>traceparent/tracestate</code>	no	no	attempt-specific
<code>ActorRef</code>	payload only if contract requires	then yes	business-specific

OUTBOX ROW

Persist exact business lineage before the transaction closes

EXECUTABLE ASSET 49-H - IMMUTABLE OUTBOX SNAPSHOT

```
public sealed class OutboxMessageRow
{
    public required Guid Id { get; init; }
    public required string MessageId { get; init; }
    public required string CorrelationId { get; init; }
    public required string CausationId { get; init; }
    public required byte[] EnvelopeBytes { get; init; }
    public required byte[] BodySha256 { get; init; }
    public required DateTimeOffset OccurredAtUtc { get; init; }
    public string? ProducerTraceId { get; init; } // diagnostic hint only
    public string? ProducerSpanId { get; init; }
    public DateTimeOffset? PublishedAtUtc { get; set; }
    public int DispatchAttempts { get; set; }
}
```

PERSISTENCE CONSTRAINTS 49-O

Constraint	Effect
unique <code>MessageId</code>	one producer fact intent
non-empty correlation/cause	no orphan state-changing fact
bytes + SHA-256	dispatch identity does not drift
trace columns nullable	sampling cannot block commit
dispatch fields mutable only	recovery does not rewrite fact

COMMIT INVARIANT

If business state exists, its outbox lineage exists in the same local transaction. If the transaction rolls back, neither state nor lineage may remain.

EF CORE MAPPING

Database constraints repeat the application invariant

EXECUTABLE ASSET 49-I - OUTBOX LINEAGE MAPPING

```
internal sealed class OutboxMessageMap : IEntityTypeConfiguration<OutboxMessageRow>
{
    public void Configure(EntityTypeBuilder<OutboxMessageRow> b)
    {
        b.ToTable("OutboxMessage", "messaging");
        b.HasKey(x => x.Id);
        b.HasIndex(x => x.MessageId).IsUnique();
        b.Property(x => x.MessageId).HasMaxLength(128).IsRequired();
        b.Property(x => x.CorrelationId).HasMaxLength(128).IsRequired();
        b.Property(x => x.CausationId).HasMaxLength(128).IsRequired();
        b.Property(x => x.EnvelopeBytes).IsRequired();
        b.Property(x => x.BodySha256).HasMaxLength(32).IsFixedLength();
        b.Property(x => x.ProducerTraceId).HasMaxLength(32);
        b.Property(x => x.ProducerSpanId).HasMaxLength(16);
        b.HasCheckConstraint("CK_Outbox_Lineage",
            "LEN([CorrelationId]) >= 8 AND LEN([CausationId]) >= 8");
    }
}
```

PROVIDER PROOF 49-P

Test	Oracle
duplicate MessageId	real provider unique violation classified
short lineage token	check constraint rejects
rollback after outbox add	state and row both absent
materialization	all identifiers round-trip exactly
nullable trace	business commit succeeds without sampling

PROVIDER HONESTY

An in-memory provider cannot prove relational uniqueness, checks, rollback or provider error classification. Those claims require the actual database provider used by the Phase-0 runtime.

DISPATCHER ATTEMPTS

The dispatcher creates diagnostics; it never recreates business lineage

EXECUTABLE ASSET 49-J - STABLE PUBLISH INPUT

```
public async Task DispatchAsync(OutboxMessageRow row, CancellationToken ct)
{
    var attempt = AttemptId.New();
    using var activity = dispatchDiagnostics.Start(
        row.MessageId,
        producerTraceId: row.ProducerTraceId,
        producerSpanId: row.ProducerSpanId,
        attempt);

    var properties = new PublishProperties
    {
        MessageId = row.MessageId,
        CorrelationId = row.CorrelationId,
        Type = envelopeCoordinates.ReadType(row.EnvelopeBytes),
        Headers = traceCarrier.Inject(activity?.Context)
    };

    await publisher.PublishConfirmedAsync(properties, row.EnvelopeBytes, ct);
    await outbox.MarkPublishedAsync(row.Id, attempt, clock.UtcNow, ct);
}
```

DISPATCH RETRY 49-Q

Value	First attempt	Retry
envelope bytes	stored	same stored bytes
MessageId / lineage	stored	same stored values
AttemptId	new	new
trace span	new or absent	new or linked
confirm evidence	attempt-specific	attempt-specific

AMBIENT-CONTEXT PROHIBITION

A background dispatcher must not read the HTTP request's Activity.Current hours later. Optional producer trace references are copied into the row at commit and used only as diagnostic links.

AMQP CARRIER

Business coordinates stay in envelope and properties; trace stays in headers

BROKER MAPPING 49-R

Meaning	AMQP location	Stability
public fact identity	message-id + envelope MessageId	same on retry
business grouping	correlation-id + envelope	same on retry
direct cause	envelope CausationId	same on retry
contract coordinate	type + routing key + envelope	same on retry
diagnostic parent	traceparent/tracestate headers	attempt-specific
delivery attempt	delivery tag/redelivered + local AttemptId	new

EXECUTABLE ASSET 49-K - BOUNDED AMQP TRACE CARRIER

```
public static IReadOnlyDictionary<string, object?> Inject(ActivityContext? context)
{
    if (context is null || context.Value == default)
        return FrozenDictionary<string, object?>.Empty;

    var parent = $"00-{context.Value.TraceId}-{context.Value.SpanId}-" +
        (context.Value.TraceFlags.HasFlag(ActivityTraceFlags.Recorded) ? "01" : "00");
    return new Dictionary<string, object?>(StringComparer.Ordinal)
    {
        ["traceparent"] = Encoding.ASCII.GetBytes(parent)
    };
}
```

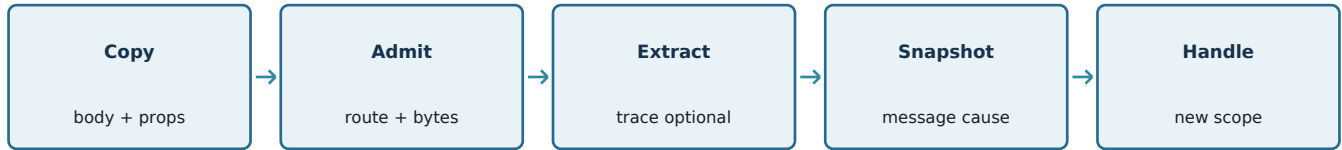
CARRIER LIMIT

Only allow-listed ASCII headers with bounded count and byte length enter or leave the broker adapter. Baggage is absent from the default NEXUS-1 message carrier.

CONSUMER ADMISSION

Validate the fact before starting owner-local work

PROOF PATH 49-F - DELIVERY TO EXPLICIT CONTEXT



EXECUTABLE ASSET 49-L - MESSAGE-DERIVED BUSINESS LINEAGE

```
public ExecutionContextSnapshot CreateConsumer(  
    AcceptedEnvelope envelope,  
    WorkloadIdentity producer,  
    ActivityContext? remoteTrace)  
{  
    return new(  
        new(  
            new CorrelationId(envelope.CorrelationId),  
            new CausationId(envelope.MessageId),  
            RequestId: null),  
        DiagnosticContext.From(remoteTrace),  
        new SecurityAttribution(  
            Actor: null,  
            WorkloadRef: producer.PairwiseReference,  
            PolicyRevision: producer.PolicyRevision),  
        AttemptId.New(),  
        clock.UtcNow);  
}
```

CAUSE TRANSLATION 49-S

Field	Consumer meaning
envelope CausationId	producer's asserted direct cause of received fact
envelope MessageId	direct cause of this consumer invocation
new local decision ID	cause of any derived fact
remote trace parent	diagnostic relation only

PER-DELIVERY ISOLATION

One scope and one context per work item

EXECUTABLE ASSET 49-M - BOUNDED WORKER SCOPE

```
public async Task ProcessAsync(RawDelivery raw, CancellationToken stop)
{
    await using var scope = scopeFactory.CreateAsyncScope();
    var admitted = scope.ServiceProvider
        .GetRequiredService<IMessageAdmission>().Accept(raw);
    var snapshot = scope.ServiceProvider
        .GetRequiredService<IConsumerContextFactory>().Create(admitted);

    using var diagnosticScope = diagnostics.Push(snapshot);
    try
    {
        var outcome = await handler.HandleAsync(admitted, snapshot, stop);
        await settlement.ApplyAsync(raw.DeliveryTag, outcome, stop);
    }
    finally
    {
        diagnosticScope.AssertRestored();
    }
}
```

ISOLATION HAZARDS 49-T

Hazard	Required gate
singleton mutable context	forbid registration
AsyncLocal not restored	post-item assertion
fire-and-forget task	structured await or owned queue
shared DbContext	one scope per item/batch policy
cancellation skips cleanup	finally restores diagnostic scope

SCOPE RULE

A batch may share a broker channel, but each item owns its application scope, AttemptId and diagnostic context. No item inherits the previous item's business lineage.

REDELIVERY SEMANTICS

A repeated delivery is a new attempt at the same fact

REDELIVERY LEDGER 49-U

Observation	Stable	New	Decision
handler fails before commit	MessageId + lineage	AttemptId/span	retry under budget
commit succeeds; ack lost	receipt + fact	delivery attempt	matching duplicate then ack
unknown commit	input identity	resolution attempt	do not ack optimistically
poison contract	raw body digest	terminal evidence ID	reject/dead-letter
retry exhausted	original identity	exhaustion record	terminal route

VERIFICATION ASSET 49-B - STABLE BUSINESS IDENTITY

```
[Fact]
public async Task Redelivery_changes_attempt_not_business_lineage()
{
    await harness.DeliverAsync(fixture, redelivered: false, failAfterCommit: true);
    await harness.DeliverAsync(fixture, redelivered: true, failAfterCommit: false);

    receipts.Should().ContainSingle(x => x.MessageId == fixture.MessageId);
    effects.Should().ContainSingle();
    attempts.Select(x => x.AttemptId).Should().OnlyHaveUniqueItems();
    attempts.Select(x => x.CorrelationId).Distinct().Should().Equal(fixture.CorrelationId);
    attempts.Select(x => x.CausationId).Distinct().Should().Equal(fixture.MessageId);
}
```

EXACTLY-ONCE BOUNDARY

Stable lineage plus inbox idempotency yields one owned effect under the tested key. It does not turn RabbitMQ into exactly-once transport.

DEAD-LETTER AND REPUBLISH

Terminal movement must not rewrite the original cause

Dead-lettering is a transport consequence, not a new business fact. NEXUS-1 retains the original MessageId, envelope fingerprint and lineage in terminal evidence. If an operator later republishes, the repair command and the new transport attempt receive their own identities and explicitly reference the original message.

TERMINAL IDENTITY 49-V

Artifact	Identity	Required references
original fact	original MessageId	correlation + causation
dead-letter observation	DeadLetterRecordId	original MessageId + reason
repair authorization	RepairCommandId	actor + policy + original
republish attempt	AttemptId	repair command + original
replacement fact, if policy creates one	new MessageId	RepairCommandId as cause

NO COMMA-SEPARATED CAUSES

V1 has one CausationId and keeps that promise

A transfer adapter never encodes multiple identifiers into the singular V1 field. A repair that truly creates a new public fact selects its immediate repair command as primary cause and records original-message provenance in a typed evidence relation or a versioned payload field defined by that contract.

SCHEMA DISCIPLINE

If public consumers require multiple causes, publish a new compatible envelope or event version. Do not smuggle an unparseable list into CausationId.

FAN-OUT

Every child gets a new identity and the same direct parent

PROOF PATH 49-G - VERDICT FAN-OUT



FAN-OUT COORDINATES 49-W

Fact	MessageId	CorrelationId	CausationId
RootCauseVerdictIssued	fact-rc-7717	request-am-20482	decision-rc-3107
AuditObservationRecorded	fact-au-2201	request-am-20482	fact-rc-7717
ComplianceEvaluationQueued	fact-co-3301	request-am-20482	fact-rc-7717
ReportingProjectionAdvanced	fact-rp-4401	request-am-20482	fact-rc-7717

EXECUTABLE ASSET 49-N - CHILD LINEAGE

```
public BusinessLineage ForChild(
    BusinessLineage parent,
    MessageId directParentMessage) =>
    new(
        parent.CorrelationId,
        new CausationId(directParentMessage.Value),
        RequestId: null);
```

FAN-OUT INVARIANT

Siblings share correlation and direct cause but never MessageId. Their processing order is unspecified unless an owner-local stream contract says otherwise.

FAN-IN

Several accepted causes require links, not a fictional parent

A Compliance decision may require a verdict, an operator attestation and a policy snapshot. The V1 public envelope still needs one immediate CausationId. The owner selects a documented primary trigger and persists every accepted contributor as a typed causal edge. Diagnostic spans use links for the other inputs when tracing exists.

EXECUTABLE ASSET 49-O - PRIMARY CAUSE PLUS CONTRIBUTORS

```
public sealed record FanInDecision(
    DecisionId DecisionId,
    CausationId PrimaryCause,
    IReadOnlyList<CausalContributor> Contributors);

public sealed record CausalContributor(
    CausalNodeId NodeId,
    ContributorKind Kind,
    byte[] EvidenceSha256);

public static FanInDecision Decide(
    MessageId trigger,
    IReadOnlyList<CausalContributor> contributors) =>
    new(DecisionId.New(), new(trigger.Value),
        contributors.OrderBy(x => x.NodeId.Value, StringComparer.Ordinal).ToArray());
```

FAN-IN POLICY 49-X

Rule	Reason
primary cause is documented	V1 singular field remains truthful
all contributors persisted	no accepted evidence disappears
contributors sorted canonically	stable digest and replay
trace links optional	sampling is not business evidence
no timestamp-selected parent	arrival time is not cause

CAUSAL EVIDENCE MODEL

Typed nodes and edges preserve what was actually asserted

EXECUTABLE ASSET 49-P - CAUSAL GRAPH VOCABULARY

```
public enum CausalNodeKind
{
    HttpRequest, DomainDecision, PublicMessage, ConsumerAttempt,
    StateTransition, RepairAction, EvidenceArtifact
}

public enum CausalEdgeKind
{
    Causes, CorrelatesWith, ObservedBy, Retries,
    Compensates, Repairs, DerivedFrom
}

public sealed record CausalEdge(
    CausalNodeId From,
    CausalNodeId To,
    CausalEdgeKind Kind,
    string AssertingOwner,
    DateTimeOffset RecordedAtUtc,
    byte[] EvidenceSha256);
```

EDGE SEMANTICS 49-Y

Edge	Meaning	May imply cause?
Causes	direct accepted production relation	yes
CorrelatesWith	same business conversation	no
ObservedBy	artifact observed a node	no
Retries	new attempt for same work	attempt relation only
Compensates	new historical action addresses prior effect	not undo
Repairs	authorized reconciliation action	explicit repair
DerivedFrom	projection/evidence transformation	data lineage

VOCABULARY RULE

A graph query may traverse different edge kinds, but it must never relabel CorrelatesWith or ObservedBy as Causes.

GRAPH INVARIANTS

Direct causes form a DAG inside the retained evidence window

GRAPH GATE 49-Z

Invariant	Rejected condition	Meaning
node identity unique	duplicate kind/id mismatch	ambiguous artifact
Causes acyclic	path returns to source	impossible lineage
cause precedes created fact logically	self-edge	invalid production
owner asserted	empty owner	unattributed relation
evidence bound	missing digest	unverifiable edge
correlation consistent	child changes without bridge	broken case chain

EXECUTABLE ASSET 49-Q - CYCLE REJECTION

```

public static void DemandAcyclicCauses(IReadOnlyCollection<CausalEdge> edges)
{
    var graph = edges.Where(x => x.Kind == CausalEdgeKind.Causes)
        .GroupBy(x => x.From)
        .ToDictionary(g => g.Key, g => g.Select(x => x.To).ToArray());
    var visiting = new HashSet<CausalNodeId>();
    var visited = new HashSet<CausalNodeId>();

    foreach (var node in graph.Keys)
        Visit(node, graph, visiting, visited);
}

private static void Visit(CausalNodeId node, IDictionary<CausalNodeId,
    CausalNodeId[]> graph, HashSet<CausalNodeId> visiting,
    HashSet<CausalNodeId> visited)
{
    if (visited.Contains(node)) return;
    if (!visiting.Add(node)) throw new EvidenceRejected("causal-cycle");
    foreach (var child in graph.GetValueOrDefault(node, []))
        Visit(child, graph, visiting, visited);
    visiting.Remove(node); visited.Add(node);
}

```

TIME AND ORDERING

Four clocks record observations; none creates global truth

CLOCK VOCABULARY 49-AA

Time	Owner	Meaning	Not evidence of
OccurredAtUtc	fact producer	business occurrence assertion	consumer arrival
ReceivedAtUtc	consumer edge	delivery observation	producer order
RecordedAtUtc	evidence store	local durable record	direct cause
CompletedAtUtc	attempt owner	local end observation	global completion
MonotonicElapsed	one process	attempt duration	wall-clock timestamp

Two services can observe concurrent facts in opposite wall-clock order. NTP can reduce clock error but does not turn timestamps into causal proof. Where an owner needs strict order, it uses its own stream position, row version or transition sequence, then relates that order to explicit causal edges.

PARTIAL ORDER

Happens-before is smaller than a total timeline

ORDERING CLAIMS 49-AB

Evidence	Allowed claim
A Causes B	A is an accepted direct predecessor of B
same owner stream positions 12 then 13	owner-local order
A and B share correlation	related only
A timestamp earlier than B	observed clock comparison only
no path between A and B	possibly concurrent/unknown

COMPLEXITY BOUNDARY

Vector clocks are not introduced for the current slice. If a future invariant requires concurrent version comparison, that requirement must justify a versioned protocol rather than a hidden timestamp convention.

EVIDENCE TIMESTAMPS

Record producer, transport and store observations separately

EXECUTABLE ASSET 49-R - ATTEMPT OBSERVATION

```
public sealed record ProcessingObservation(  
    AttemptId AttemptId,  
    MessageId MessageId,  
    CorrelationId CorrelationId,  
    CausationId CausationId,  
    DateTimeOffset ProducerOccurredAtUtc,  
    DateTimeOffset ReceivedAtUtc,  
    DateTimeOffset RecordedAtUtc,  
    TimeSpan MonotonicElapsed,  
    string? TraceId,  
    string OutcomeCode);
```

INTERPRETATION 49-AC

Comparison	Permitted interpretation
Received - Occurred	rough cross-clock latency indicator only
Recorded - Received	local processing/commit interval with caveats
MonotonicElapsed	attempt duration inside one process
two Occurred values	producer assertions, not global order
edge plus timestamps	cause from edge; times enrich observation

NEGATIVE DURATION

A negative cross-service wall-clock interval is retained and classified as clock uncertainty; it is never clamped to zero or used to reverse a causal edge.

CLOCK FAULT TEST

Reordered clocks must not rewrite lineage

VERIFICATION ASSET 49-C - SKEW DOES NOT ALTER CAUSES

```
[Fact]
public void Clock_skew_changes_display_not_causal_graph()
{
    var graph = fixture.WithProducerClock("+00:03:00")
                        .WithConsumerClock("-00:02:00")
                        .Build();

    graph.Edge(message, decision).Kind.Should().Be(CausalEdgeKind.Causes);
    graph.RenderedLatency.Should().BeClassifiedAs("clock-uncertain");
    graph.TopologicalOrder().Should().PlaceBefore(message, decision);
}
```

DATABASE EVIDENCE

Persist lineage at boundaries, not on every domain table

STORAGE CATALOGUE 49-AD

Store	Required lineage	Reason
outbox	message + correlation + cause	producer public intent
inbox receipt	input message + fingerprint + correlation	dedup and provenance
transition journal	decision + direct cause	state explanation
attempt journal	AttemptId + optional trace	execution evidence
repair journal	repair command + target	authorized intervention
ordinary domain row	only if domain meaning requires	avoid observability pollution

EXECUTABLE ASSET 49-S - OWNER-LOCAL EVIDENCE JOIN

```
CREATE TABLE evidence.CausalEdge (
    EdgeId uniqueidentifier NOT NULL PRIMARY KEY,
    FromKind varchar(32) NOT NULL,
    FromId varchar(128) NOT NULL,
    ToKind varchar(32) NOT NULL,
    ToId varchar(128) NOT NULL,
    EdgeKind varchar(24) NOT NULL,
    CorrelationId varchar(128) NOT NULL,
    AssertingOwner varchar(64) NOT NULL,
    RecordedAtUtc datetimeoffset(7) NOT NULL,
    EvidenceSha256 binary(32) NOT NULL,
    CONSTRAINT UQ_CausalEdge UNIQUE (FromKind, FromId, ToKind, ToId, EdgeKind)
);
```

OWNERSHIP RULE

Each service stores only its assertions and observations. Cross-service reconstruction joins exported evidence by typed public identifiers; it creates no cross-database foreign key.

INBOX AND TRANSITION JOINS

One transaction explains what input produced what local effect

EXECUTABLE ASSET 49-T - CONSUMER COMMIT SET

```
await using var tx = await db.Database.BeginTransactionAsync(ct);

var receipt = InboxReceipt.Accept(admitted.MessageId, admitted.Fingerprint,
    admitted.CorrelationId, admitted.EnvelopeCausationId, context.AttemptId);
db.Inbox.Add(receipt);

var decision = caseAggregate.Apply(admitted.Payload, clock.UtcNow);
db.Transitions.Add(TransitionRow.From(decision, admitted.MessageId));
db.CausalEdges.Add(CausalEdgeRow.Causes(
    PublicMessage(admitted.MessageId), DomainDecision(decision.Id), receipt.Digest));

var child = envelopeFactory.Create(decision, context.Business.CorrelationId,
    new CausationId(decision.Id.Value));
db.Outbox.Add(child.Row);

await db.SaveChangesAsync(ct);
await tx.CommitAsync(ct);
```

ATOMIC ORACLES 49-AE

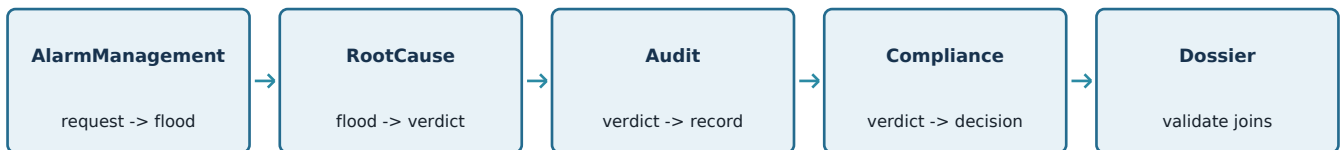
Fault point	Required state
after receipt add	no receipt/transition/outbox
after transition add	no partial effect
after causal edge add	no orphan edge
after outbox add before commit	none durable
after commit before ack	all durable; redelivery deduplicates

LOCAL ATOMICITY

The transaction proves one consumer-owned effect and its next public intent. It does not prove downstream delivery or a cross-service transaction.

CROSS-SERVICE RECONSTRUCTION

Evidence joins by public identity, never by shared tables

PROOF PATH 49-H - RETAINED CASE RECONSTRUCTION**EXPORT RECORD 49-AF**

Field	Purpose	Sensitivity
NodeKind + NodeId	typed identity	bounded
EdgeKind + endpoints	asserted relationship	bounded
CorrelationId	case grouping	internal
AssertingOwner	provenance	public service name
EvidenceSha256	artifact integrity join	non-secret
ActorRef	only when required	restricted/pairwise

RECONSTRUCTION LIMITS

Absence can mean missing evidence, not absent cause

A dossier validator distinguishes complete, partial, conflicting and unverifiable graphs. It never fills a gap by sorting timestamps or guessing from shared correlation. A missing owner artifact remains an explicit residual with the affected claims downgraded.

GRAPH VERDICT 49-AG

Status	Meaning
complete	all required nodes, edges and digests join
partial	known gap is bounded and named
conflicting	owners assert incompatible direct cause
unverifiable	artifact missing, stale or digest mismatch

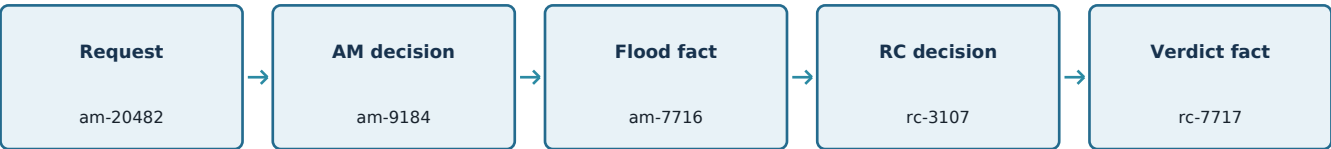
END-TO-END LINEAGE SPECIMEN

One request becomes five owner assertions

CASE SPECIMEN 49-AH

Owner	Node	Correlation	Direct cause
BFF	request-am-20482	request-am-20482	external admission
AlarmManagement	decision-am-9184	request-am-20482	request-am-20482
AlarmManagement	fact-am-7716	request-am-20482	decision-am-9184
RootCause	decision-rc-3107	request-am-20482	fact-am-7716
RootCause	fact-rc-7717	request-am-20482	decision-rc-3107
Compliance	decision-co-4410	request-am-20482	fact-rc-7717

PROOF PATH 49-I - DIRECT CAUSAL CHAIN



SPECIMEN LIMIT

These identifiers are deterministic manuscript fixtures. They demonstrate relationships; they are not presented as captured production records.

DIAGNOSTIC OVERLAY

One business chain can contain several trace forests

The HTTP request may be sampled, the delayed dispatcher may start a new trace linked to the producer span, and a redelivered consumer may be unsampled. The causal chain remains identical because it is stored in business evidence. Diagnostic graphs enrich attempts but do not own the chain.

POSSIBLE TRACE VIEW 49-AI

Attempt	Trace relation	Business relation
HTTP command	remote parent or new root	request causes decision
outbox dispatch	link to producer context	same fact intent
first consume	parent/link from carrier	message causes decision
redelivery	new span or no span	same message; retry attempt

FAILURE TAXONOMY

Propagation failures have different business consequences

FAILURE MATRIX 49-AJ

Failure	Business action	Evidence action	Retry?
missing required correlation	fail before Domain	lineage-missing	no until fixed
invalid traceparent	route policy; business unchanged	diagnostic-invalid	not for this alone
missing trace context	continue	new root/none	no
cause target unknown locally	record producer assertion	provenance-unverified	policy-specific
outbox lineage mismatch	rollback	integrity-failure	terminal
consumer context leak	stop worker batch	isolation-failure	after recovery

FAIL-CLOSED BOUNDARY

Missing diagnostic context degrades diagnostics. Missing required business lineage prevents a state-changing fact. The two conditions are never handled by the same fallback.

UNKNOWN REMOTE CAUSES

Record provenance without pretending to own remote truth

A consumer can validate the received envelope, producer path, fingerprint and internal consistency. It usually cannot query the producer's private decision store. Therefore it records the envelope CausationId as a producer assertion and the received MessageId as the immediate cause of its own local invocation.

VERIFICATION LEVELS 49-AK

Level	What is proved
syntax	identifier grammar and envelope consistency
transport provenance	approved workload and route delivered bytes
producer assertion	envelope states one direct cause
producer-local proof	producer dossier joins cause to fact
cross-service proof	fresh exported dossiers join without conflict

POLICY CATALOGUE

Every boundary names what it propagates and what it strips

EXECUTABLE ASSET 49-U - PROPAGATION POLICY

```
public sealed record PropagationPolicy(  
    string BoundaryId,  
    bool RequireCorrelation,  
    bool RequireCausation,  
    bool AcceptRemoteTrace,  
    bool RestartTrace,  
    IReadOnlySet<string> AllowedBaggageKeys,  
    int MaxHeaderCount,  
    int MaxHeaderBytes,  
    InvalidCarrierAction InvalidAction);  
  
public enum InvalidCarrierAction  
{  
    RejectRequest, StripAndContinue, QuarantineMessage  
}
```

NEXUS-1 POLICY ENTRIES 49-AL

Boundary	Business lineage	Trace	Baggage
public HTTP -> BFF	admit/create correlation	validate; may restart	strip
BFF -> internal API	required request/correlation	forward/new child	empty
outbox -> broker	required envelope lineage	optional headers	empty
broker -> consumer	required + validated	optional remote	deny
repair tool -> service	repair command + target	new trace	empty

CONFIGURATION IDENTITY

The policy catalogue is versioned configuration. Evidence records its digest without recording secrets or raw header values.

PRIVACY AND ABUSE

Baggage is a propagation surface, not a convenient claims bag

The W3C Baggage specification warns that arbitrary entries can carry sensitive information and cross boundaries. NEXUS-1 therefore permits no baggage on broker paths and uses a deny-by-default allow-list on HTTP. User subjects, emails, roles, tokens, tenant authorization and secrets are prohibited.

PRIVACY RULES 49-AM

Risk	Control	Evidence
cross-boundary linkability	pairwise actor refs	mapping policy revision
header amplification	count + byte limits	rejection code only
credential leakage	never place token/claims in carrier	value-free leak scan
vendor state exposure	strip/restart at trust boundary	policy decision
identifier injection	grammar + typed factory	invalid-class count

VERIFICATION ASSET 49-D - PROHIBITED BAGGAGE

```
[Theory]
[InlineData("user.email=operator@example.test")]
[InlineData("role=plant-admin")]
[InlineData("authorization=Bearer%20secret")]
[InlineData("tenant=facility-17")]
public async Task Sensitive_baggage_never_crosses_internal_boundary(string baggage)
{
    using var request = AuthorizedCommand();
    request.Headers.TryAddWithoutValidation("baggage", baggage);
    await client.SendAsync(request);

    downstreamHeaders.Should().NotContainKey("baggage");
    retainedArtifacts.Should().PassLeakScan();
}
```

STANDARDS RULE

Trace identifiers must not contain personally identifiable or sensitive information. NEXUS-1 uses random/opaque diagnostic IDs and independently minimizes business attribution.

SAMPLING INDEPENDENCE

No sampled span is required to explain a committed fact

VERIFICATION ASSET 49-E - DIAGNOSTICS CAN DISAPPEAR

```
[Fact]
public async Task Sampling_off_preserves_business_lineage()
{
    using var listener = new ActivityListener
    {
        ShouldListenTo = _ => true,
        Sample = static (ref ActivityCreationOptions<ActivityContext> _) =>
            ActivitySamplingResult.None
    };
    ActivitySource.AddActivityListener(listener);

    await slice.ExecuteAsync(fixture.Command);

    outbox.Single().MessageId.Should().Be(fixture.ExpectedMessageId);
    outbox.Single().CorrelationId.Should().Be(fixture.CorrelationId);
    outbox.Single().CausationId.Should().Be(fixture.ExpectedDecisionId);
    outbox.Single().ProducerTraceId.Should().BeNull();
}
```

SAMPLING OUTCOMES 49-AN

Diagnostic state	Business requirement
recorded	same exact lineage
propagated but not recorded	same exact lineage
invalid and restarted	same internal request cause
absent end-to-end	same durable graph
exporter unavailable	commit continues; telemetry fault bounded

OPERATIONAL RULE

Telemetry backpressure must not enter the business transaction. Export is bounded and lossy by design; business lineage is local durable evidence.

DIAGNOSTIC FAN-IN

Use links for multiple inputs; never fabricate one remote parent

OpenTelemetry messaging conventions allow batch or multi-message processing to represent input contexts as links. NEXUS-1 applies the same semantic discipline: the processing attempt may link to each valid remote ActivityContext, while the business decision stores its documented primary cause plus all causal contributors.

EXECUTABLE ASSET 49-V - LINKS PREPARED EXPLICITLY

```
public static ActivityLink[] DiagnosticLinks(
    IEnumerable<AcceptedInput> inputs) =>
    inputs.Where(x => x.RemoteTraceContext is not null)
        .Select(x => new ActivityLink(
            x.RemoteTraceContext!.Value,
            new ActivityTagsCollection
            {
                ["messaging.message.id"] = x.MessageId.Value
            })
        ).ToArray();
```

PARENT VERSUS LINK 49-AO

Situation	Diagnostic relation	Business relation
one synchronous HTTP caller	remote parent	internal request admission
one consumed message	parent or link by convention	message causes invocation
batch of independent messages	one link per input	one decision per item
true fan-in decision	links to inputs	primary cause + contributors
redelivery	new attempt, optional link	Retries edge to prior attempt

CHAPTER BOUNDARY

Exact span names, kinds, attributes and exporter configuration are deferred to Chapter 51. The semantic prohibition against false parents is fixed here.

WORKER LEAKAGE TESTS

Two consecutive items must remain causally disjoint

VERIFICATION ASSET 49-F - NO AMBIENT CARRY-OVER

```
[Fact]
public async Task Worker_restores_context_between_items()
{
    var first = Fixture.Message("fact-am-7716", "corr-a");
    var second = Fixture.Message("fact-am-8827", "corr-b");

    await worker.ProcessAsync(first, CancellationToken.None);
    Activity.Current.Should().BeNull();
    ContextAccessor.Current.Should().BeNull();

    await worker.ProcessAsync(second, CancellationToken.None);

    attempts.Single(x => x.MessageId == first.MessageId)
        .CorrelationId.Should().Be(first.CorrelationId);
    attempts.Single(x => x.MessageId == second.MessageId)
        .CorrelationId.Should().Be(second.CorrelationId);
}
```

ISOLATION CAMPAIGN 49-AP

Fault	Oracle
handler throws	scope disposed and context restored
cancellation during commit	no leaked context; commit classified
parallel deliveries	unique AttemptId and correct correlation
poison body	no typed context reaches Domain
batch shutdown	bounded drain and final restoration

CONCURRENCY GATE

Assertions inspect the retained attempt journal, not only Activity.Current on the test thread. Async scheduling can otherwise hide cross-item leakage.

COMPONENT-TEST MATRIX

Provider, broker and host prove different seams

TEST LAYERS 49-AQ

Layer	Real dependency	Primary claim	Forbidden substitute
Domain	none	decision selects cause	trace fixture
Application	ports	explicit snapshot/order	ambient context
database component	real provider	atomic lineage/constraints	in-memory DB
broker component	RabbitMQ	headers/redelivery/settlement	direct handler call
host	production composition	middleware + DI isolation	alternate Program
deployment	container network	cross-process propagation	same-process shortcut

CLAIM CATALOGUE

Every promise names one executable oracle

CLAIMS 49-AR

ClaimId	Promise	Minimum evidence
LIN-001	business IDs stable across retry	broker + receipt journal
LIN-002	outbox captures cause atomically	provider rollback test
LIN-003	diagnostics cannot authorize	host adversarial test
LIN-004	fan-in retains contributors	graph materialization test
LIN-005	worker context does not leak	parallel fault test
LIN-006	sampling does not erase lineage	no-listener slice
LIN-007	dossier rejects causal ambiguity	validator negative corpus

DISCOVERY GATE

A test run with zero discovered LIN cases is a failure, not an empty success.

UNIT AND ARCHITECTURE TESTS

Static gates keep context out of forbidden layers

VERIFICATION ASSET 49-G - DEPENDENCY PROHIBITION

```
[Fact]
public void Domain_does_not_reference_diagnostic_or_transport_types()
{
    Types.InAssembly(DomainAssembly)
        .ShouldNot().HaveDependencyOnAny(
            "System.Diagnostics.Activity",
            "Microsoft.AspNetCore.Http",
            "RabbitMQ.Client",
            "OpenTelemetry")
        .GetResult().ShouldBeSuccessful();
}

[Fact]
public void Lineage_identifiers_are_not_interchangeable()
{
    typeof(MessageId).Should().NotBe(typeof(CorrelationId));
    typeof(CorrelationId).Should().NotBe(typeof(CausationId));
}
```

STATIC GATES 49-AS

Gate	Failure caught
Domain dependency	ambient diagnostic authority
typed identifiers	string substitution
outbox required properties	nullable business lineage
no baggage keys outside catalogue	privacy drift
no singleton context accessor	cross-request leakage
contract snapshot unchanged	silent envelope evolution

BUILD-TIME LIMIT

Architecture tests prove references and declarations. Only host/component tests prove runtime scope, propagation and rollback behavior.

PROVIDER-BACKED TESTS

Commit, rollback and materialization preserve exact lineage

VERIFICATION ASSET 49-H - ATOMIC ROLLBACK

```
[Fact]
public async Task Failed_commit_leaves_no_state_or_lineage()
{
    await using var lease = await SqlProviderLease.StartAsync();
    await using var db = lease.CreateContext();
    var sut = new AlarmCommitter(db, fixture.EnvelopeFactory);

    await Assert.ThrowsAsync<InjectedCommitFailure>(() =>
        sut.CommitAsync(fixture.Decision, fixture.Lineage,
            failAfter: CommitStep.OutboxAdded, CancellationToken.None));

    (await db.AlarmTransitions.CountAsync()).Should().Be(0);
    (await db.OutboxMessages.CountAsync()).Should().Be(0);
    (await db.CausalEdges.CountAsync()).Should().Be(0);
}
```

PROVIDER CORPUS 49-AT

Case	Required assertion
commit success	state, outbox and edge join by decision
constraint failure	provider code classified, no partial state
duplicate message	one receipt and one effect
concurrent duplicate	loser resolves matching durable receipt
trace null	business columns materialize exactly
digest mismatch	terminal integrity classification

EVIDENCE STATUS

These are reference tests until the exact provider container, migration identity and retained TRX/JUnit artifacts are available.

BROKER-BACKED TESTS

Observe headers, confirms, redelivery and settlement on the real path

VERIFICATION ASSET 49-I - REDELIVERY CAMPAIGN

```
[Fact]
public async Task Commit_then_connection_loss_preserves_lineage()
{
    await using var broker = await RabbitMqLease.StartAsync();
    await using var database = await SqlProviderLease.StartAsync();
    var scenario = await SliceHarness.StartAsync(broker, database);

    scenario.FailAfterConsumerCommitBeforeAckOnce();
    await scenario.PublishAsync(fixture.EnvelopeBytes);
    await scenario.WaitForMatchingReceiptAsync(fixture.MessageId);
    await scenario.WaitForRedeliveryResolutionAsync(fixture.MessageId);

    scenario.Effects.Should().ContainSingle();
    scenario.Attempts.Should().HaveCount(2);
    scenario.Attempts.Should().OnlyContain(x =>
        x.CorrelationId == fixture.CorrelationId &&
        x.CausationId == fixture.MessageId);
}
```

BROKER ORACLES 49-AU

Observation	Evidence source
exact body stable	published/consumed SHA-256
trace carrier bounded	captured AMQP headers
new delivery attempt	attempt journal + redelivered flag
one business effect	provider query
ack after durable resolution	settlement probe
queue converges	broker management observation

SETTLEMENT CLAIM

A returned BasicAck call is an observation, not proof that the network delivered the frame. The durable receipt makes later redelivery safe.

END-TO-END CAUSAL TEST

Reconstruct the slice from independent owner artifacts

VERIFICATION ASSET 49-J - DOSSIER RECONSTRUCTION

```
[Fact]
public async Task One_case_reconstructs_without_timestamp_inference()
{
    var run = await deployment.ExecuteCaseAsync("LIN-E2E-001");
    var artifacts = await run.ExportOwnerEvidenceAsync();
    var graph = CausalDossier.Build(artifacts);

    graph.DemandEdge(request, alarmDecision, Causes);
    graph.DemandEdge(alarmDecision, floodFact, Causes);
    graph.DemandEdge(floodFact, rootCauseDecision, Causes);
    graph.DemandEdge(rootCauseDecision, verdictFact, Causes);
    graph.DemandEdge(verdictFact, complianceDecision, Causes);
    graph.DemandAcyclicCauses();
    graph.DemandNoTimestampDerivedEdges();
    graph.DemandOneCorrelation(fixture.CorrelationId);
}
```

FAULT VARIANTS 49-AV

Variant	Graph expectation
HTTP trace absent	same causal graph; no HTTP trace node required
publisher retry	Retries attempt edge; one public fact
consumer redelivery	two attempts; one local decision
fan-out concurrent	siblings unordered; same parent
clock skew	timestamps differ; edges unchanged
owner artifact missing	PARTIAL, never guessed

CROSS-PROCESS REQUIREMENT

The deployment case must traverse the real BFF, service hosts, RabbitMQ and service-owned databases. An in-process direct handler call cannot satisfy LIN-E2E-001.

EVIDENCE MANIFEST

One fresh dossier binds claims, artifacts and policy

EXECUTABLE ASSET 49-W - LINEAGE DOSSIER

```
public sealed record LineageEvidenceManifest(  
    string RunId,  
    string BuildIdentity,  
    string EnvironmentDigest,  
    string PropagationPolicyDigest,  
    DateTimeOffset StartedAtUtc,  
    DateTimeOffset CompletedAtUtc,  
    IReadOnlyList<ClaimResult> Claims,  
    IReadOnlyList<ArtifactDigest> Artifacts,  
    GraphVerdict Graph,  
    EvidenceStatus Status);  
  
public enum EvidenceStatus  
{  
    SpecifiedRuntimePending, Passed, Failed, Partial, Unverifiable  
}
```

REQUIRED ARTIFACTS 49-AW

Artifact	Joins
test results	ClaimId + Caseld + RunId
owner evidence exports	node/edge + digest + build
outbox/inbox probes	MessageId + transaction observation
broker attempt journal	MessageId + AttemptId
leak scan	artifact set + prohibited patterns
graph report	all causal joins + residuals

SAME-RUN RULE

Promotion rejects a dossier assembled from different RunIds, builds, environments or propagation-policy digests even when every individual artifact says PASS.

FAIL-CLOSED VALIDATOR

Ambiguous or stale lineage cannot become a green claim

EXECUTABLE ASSET 49-X - DOSSIER GATE

```
public static void Validate(LineageEvidenceManifest manifest, DateTimeOffset now)
{
    DemandExactClaimDiscovery(manifest.Claims, RequiredClaims.All);
    DemandSameRun(manifest.RunId, manifest.Claims, manifest.Artifacts);
    DemandFresh(manifest.CompletedAtUtc, now, TimeSpan.FromHours(24));
    DemandArtifactDigests(manifest.Artifacts);
    DemandNoLeaks(manifest.Artifacts);
    DemandNoMissingRequiredEdges(manifest.Graph);
    DemandNoConflictingCauses(manifest.Graph);
    DemandAcyclicCauses(manifest.Graph.Edges);

    if (manifest.Status != EvidenceStatus.Passed)
        throw new PromotionRejected($"lineage-status:{manifest.Status}");
}
```

NEGATIVE CORPUS 49-AX

Mutation	Required rejection
zero discovered tests	claim-discovery-empty
duplicate direct cause	cause-ambiguous
missing producer artifact	graph-partial
causal cycle	causal-cycle
timestamp-derived edge	edge-provenance-invalid
raw subject in artifact	evidence-leak
stale prior run	evidence-stale

CURRENT TRUTH

The chapter supplies the validator contract and negative corpus. It does not claim that a fresh runtime dossier already passes.

STANDARDS AND PLATFORM ALIGNMENT

Use standards for diagnostics; keep business meaning explicit

PINNED SOURCE SET 49-AY

Source	Used for	NEXUS-1 decision
W3C Trace Context	traceparent/tracestate grammar and processing	diagnostic only; validate at ingress
W3C Baggage	propagation/privacy risks	deny by default; no identity
System.Diagnostics ActivityContext	parse remote diagnostic context	optional snapshot
System.Diagnostics ActivityLink	multiple diagnostic predecessors	fan-in links
OpenTelemetry messaging conventions	producer/consumer/batch relations	implemented in Chapter 51

Primary references: <https://www.w3.org/TR/trace-context/> ; <https://www.w3.org/TR/baggage/> ;
<https://learn.microsoft.com/dotnet/api/system.diagnostics.activitycontext.tryparse> ;
<https://learn.microsoft.com/dotnet/api/system.diagnostics.activitylink> ;
<https://opentelemetry.io/docs/specs/semconv/messaging/messaging-spans/> . Accessed 2026-08-05.

VERSION BOUNDARY

The manuscript pins semantic decisions, not a production package graph. Package versions, exporter behavior and executed interoperability tests remain repository evidence for Chapter 51.

IMPLEMENTATION SEQUENCE

Build business lineage before adding exporters

SEQUENCE 49-AZ

Step	Deliverable	Exit gate
1	typed IDs + grammar	substitution tests fail
2	HTTP/broker carrier policies	malformed corpus passes
3	atomic outbox/inbox joins	provider fault tests pass
4	per-attempt worker isolation	parallel leakage tests pass
5	causal graph + dossier	negative validator corpus passes
6	diagnostic instrumentation	Chapter 51 evidence

COMPLETION LEDGER

What Chapter 49 establishes and what remains unproved

COMPLETED DESIGN 49-BA

Area	Established contract
identity	MessageId, correlation, causation, request and attempt are distinct
HTTP	untrusted carrier admission and minimized attribution
producer	lineage captured atomically with outbox bytes
broker	stable business IDs and attempt-local diagnostics
consumer	one explicit context and scope per item
fan-out/fan-in	truthful direct cause plus contributor edges
evidence	typed causal DAG and fail-closed dossier

OPEN RESIDUALS 49-BB

Residual	Consequence
no executed repository checkpoint	listings remain compile-oriented
no retained provider/broker campaign	runtime propagation unproved
no production collector/exporter	trace availability unproved
no global clock guarantee	timestamp order remains bounded
no global causal consistency theorem	dossier proves retained slice only
no cross-service transaction	local commits remain independently recoverable

EVIDENCE VERDICT

SPECIFIED | RUNTIME PENDING. The chapter defines compile-oriented assets, test obligations and evidence gates. No production deployment, collector or fresh execution dossier is claimed.

HANDOFF TO CHAPTER 50

Structured Logs That Preserve Domain Meaning

Chapter 50 will serialize the bounded identities and decision vocabulary defined here into a stable structured-log contract. It must preserve domain meaning, cardinality budgets, redaction and evidence joins without dumping payloads, credentials or arbitrary baggage.

NEXT INVARIANT

A log event is an observation with schema and provenance. It is never a substitute for the business transaction, public fact or causal edge it describes.

PART IX - OBSERVABILITY, RESILIENCE, AND FAILURE EVIDENCE

50. Structured Logs That Preserve Domain Meaning

Logs become trustworthy evidence only when every event has a stable schema, bounded provenance and a meaning that survives formatters, retries and incident pressure.

CHAPTER DECISION

Publish one versioned, provider-neutral event catalogue. Emit only typed allow-listed fields; keep business outcome and reason codes stable; join Chapter 49 lineage explicitly; and make payloads, credentials, arbitrary baggage and unbounded exception text structurally impossible at the call site.

PROOF PATH 50-A - OBSERVATION WITHOUT SEMANTIC LOSS**GUARANTEE LEDGER 50-A**

Boundary	Preserves	Allowed claim	Still forbidden
domain -> event	outcome + reason	meaning observed	log proves commit
context -> event	typed lineage refs	records can be joined	trace grants authority
event -> provider	name, ID, fields	provider-neutral schema	formatter defines contract
sink -> evidence	accepted records + loss state	bounded dossier	perfect retention

CHAPTER EXIT CRITERIA 50-B

Question	Required answer
Can an event be queried without parsing prose?	yes - name, outcome and reason are typed
Can forbidden data reach a provider parameter?	no - catalogue signatures omit it
Can a sink failure change business truth?	no - failure is isolated and recorded as evidence loss
Can one case be reconstructed?	yes - logs join explicit lineage to durable owner records

ORDINARY DEVELOPER PROBLEM

A readable sentence is not a durable contract

A handler logs 'alarm done' together with a serialized command and an exception. The sentence looks useful in a terminal, but it cannot distinguish accepted, duplicate, deferred or rejected work. The payload leaks data, the exception changes between providers, and a retry creates a second apparently successful story.

FAILURE LEDGER 50-1

Shortcut	Operational failure	Required correction
interpolated prose	fields disappear into one string	named template fields
serialize the object	payload and cardinality escape	typed scalar allow-list
log 'success' before commit	observation outruns truth	emit after authoritative outcome
exception.ToString()	secrets and unstable stacks leak	safe error code + fingerprint
dictionary enrichment	schema grows without review	catalogue-owned field set

PLAIN-LANGUAGE PRINCIPLE

The message is for a person. The event name, outcome, reason and typed fields are for the system that must explain the case months later.

CHAPTER BOUNDARY

Logs observe truth; they do not manufacture it

Chapter 49 defined business lineage and diagnostic context. This chapter defines the structured observation contract. Chapter 51 will create spans and export traces; Chapter 52 will define metrics and dashboards. A log may reference those ledgers, but it never replaces a transaction row, public fact, inbox receipt, authorization record or causal edge.

NO-SUCCESS-BY-LOG

Provider state, durable receipt and broker settlement remain authoritative. A success event can report an accepted outcome; it cannot prove durability by its own existence.

DECISION RECORD

ADR-050 - One logical event catalogue, many transports

NEXUS-1 owns a versioned logical log schema independent of console, OpenTelemetry, cloud or file providers. Application code emits catalogue methods. Adapters translate the same event into provider state. A provider may add resource metadata, but it may not rename domain fields or reinterpret outcome codes.

DECISION LEDGER 50-2

Concern	Owner	Stable surface	Not an owner
domain meaning	service catalogue	event/outcome/reason	formatter
lineage	Chapter 49 contract	typed locator fields	ambient scope
wire representation	logging adapter	canonical JSON profile	business layer
storage/retention	platform policy	access + loss evidence	call site

DECISION IDENTITY

ADR-050 is retained with its canonical text and digest by the evidence build. This manuscript does not invent a runtime digest.

EVENT OR FACT

A log record is an observation with provenance

SEMANTIC SEPARATION 50-A

Record	Authoritative for	May reference	Cannot prove
domain state	owner truth	decision ID	external delivery
public fact	producer assertion	message lineage	consumer commit
audit intent	required attribution	policy revision	log retention
structured log	what code observed	all safe locators	business durability
trace span	diagnostic timing	trace/span IDs	business cause

NAMING GRAMMAR

EventName identifies one versioned event structure

The canonical grammar is lower-case dotted tokens: subject.action.outcome.vN. A name changes when field meaning or required structure changes, not when prose changes. Names are finite catalogue constants; user input, exception text and tenant values can never become event names.

EXECUTABLE ASSET 50-A - EVENT-NAME GRAMMAR

```
public static partial class NexusLog
{
    private static readonly Regex EventNameGrammar = new(
        "^[a-z][a-z0-9]*(\\.[a-z][a-z0-9]*){2,5}\\.[v][1-9][0-9]*$",
        RegexOptions.CultureInvariant | RegexOptions.NonBacktracking);

    public static void ValidateName(string eventName)
    {
        if (eventName.Length > 96 || !EventNameGrammar.IsMatch(eventName))
            throw new InvalidOperationException("log-event-name-invalid");
    }
}
```

NAMING DECISIONS 50-B

Part	Meaning	Example
subject	bounded business or runtime noun	alarm / outbox
action	completed transition or observation	committed / dispatch
outcome	semantic result when needed	confirmed / deferred
version	schema compatibility boundary	v1

OPENTELEMETRY ALIGNMENT

EventName is treated as the event class and therefore identifies its structure. Human prose remains a non-authoritative Body/message.

EVENT IDENTITY

EventId is a reviewed allocation, not a random integer

ALLOCATION LEDGER 50-C

Range	Owner	Examples
5000-5099	Edge and application admission	HTTP admitted/rejected
5100-5199	AlarmManagement	alarm commit/state observation
5200-5299	RootCauseAnalysis	case/verdict observation
5300-5399	Messaging	dispatch/consume/retry/dead-letter
5400-5499	Repair and reconciliation	mismatch/action outcome

EXECUTABLE ASSET 50-B - REVIEWED IDENTITIES

```
public static class LogEventIds
{
    public const int HttpCommandAdmitted = 5001;
    public const int AlarmFloodCommitted = 5101;
    public const int RootCauseCaseOpened = 5201;
    public const int RootCauseVerdictIssued = 5202;
    public const int OutboxDispatchConfirmed = 5301;
    public const int MessageConsumeCompleted = 5311;
    public const int MessageRetryScheduled = 5321;
    public const int MessageDeadLettered = 5331;
    public const int ReconciliationMismatch = 5401;
    public const int RepairActionCompleted = 5411;
}
```

UNIQUENESS GATE

The build reflects over generated logging methods and rejects duplicate EventId or EventName allocations across the solution. A duplicate is a release-blocking schema collision.

LOGICAL RECORD

Top-level fields carry universal meaning

EXECUTABLE ASSET 50-C - PROVIDER-NEUTRAL LOG RECORD

```
public sealed record NexusLogRecordV1(
    DateTimeOffset TimestampUtc,
    DateTimeOffset ObservedTimestampUtc,
    int EventId,
    string EventName,
    string SchemaVersion,
    string SeverityText,
    int SeverityNumber,
    ServiceResource Resource,
    LogOperation Operation,
    SafeLineage? Lineage,
    SafeDiagnosticContext? Diagnostic,
    SafeSecurityAttribution? Security,
    ErrorDescriptor? Error);

public sealed record LogOperation(
    string Name, string OutcomeCode, string? ReasonCode, long? DurationMs);
```

FIELD OWNERSHIP 50-D

Field	Source	Invariant
TimestampUtc	origin clock	when event occurred
ObservedTimestampUtc	adapter/collector	when record observed
EventId/EventName	catalogue	stable and unique pair
Resource	deployment manifest	not user supplied
Operation	event method	bounded code set
Lineage/Diagnostic/Security	typed snapshot	separate ledgers

TIMESTAMP HONESTY

Origin and observation time remain distinct. Ordering across services is reconstructed from causal edges and durable state, not clock comparison alone.

TYPED ATTRIBUTES

No arbitrary object graph reaches a log provider

EXECUTABLE ASSET 50-D - CLOSED SCALAR VOCABULARY

```
public abstract record SafeLogValue
{
    private SafeLogValue() { }
    public sealed record Text(string Value) : SafeLogValue;
    public sealed record Integer(long Value) : SafeLogValue;
    public sealed record Decimal(decimal Value) : SafeLogValue;
    public sealed record Boolean(bool Value) : SafeLogValue;
    public sealed record Timestamp(DateTimeOffset Value) : SafeLogValue;
    public sealed record Identifier(string Type, string Value) : SafeLogValue;
}

public sealed record SafeLogField(string Name, SafeLogValue Value);
```

REJECTED CALL-SITE SHAPES 50-E

Shape	Why rejected	Replacement
object / dynamic	provider-specific expansion	typed scalar
Dictionary<string, object>	unreviewed schema growth	catalogue method
JSON element	payload laundering	selected field
exception as field	message/stack leak	ErrorDescriptor
raw headers/claims	credential and PII leak	bounded classification

PRIMARY CONTROL

Redaction is defense in depth. The primary control is that forbidden data has no parameter in the generated event method.

DATA CLASSIFICATION

Every field is classified before it is emitted

CLASSIFICATION MATRIX 50-F

Class	Examples	Log policy	Metric-label policy
bounded dimension	outcome, reason, event type	allowed	allow-listed
locator	MessageId, CaseId, RequestId	restricted access	never
security attribution	ActorRef, WorkloadRef	event-specific only	never
sensitive	token, raw subject, payload	forbidden	forbidden
unbounded diagnostic	stack, SQL, header map	quarantined/omitted	never

EXECUTABLE ASSET 50-E - CATALOGUE FIELD POLICY

```
public sealed record LogFieldDefinition(
    string Name,
    LogValueKind Kind,
    LogDataClass DataClass,
    int? MaxLength,
    FrozenSet<string>? AllowedValues,
    bool Required);

public enum LogDataClass
{
    BoundedDimension, Locator, SecurityAttribution, Forbidden
}
```

PRIVACY RULE

A hash is not automatically anonymous. A stable hash can remain linkable; it is emitted only under an approved classification, keying and retention policy.

OUTCOME VOCABULARY

Meaning lives in codes, not adjectives

OUTCOME CODE SET 50-G

Code	Meaning	Typical level
ACCEPTED	work admitted for execution	Information
COMMITTED	owner transaction accepted	Information
DUPLICATE_MATCH	same identity and fingerprint	Information
DEFERRED	bounded dependency delay	Warning
REJECTED	contract or policy refusal	Warning
FAILED	attempt ended unexpectedly	Error
ABSTAINED	insufficient evidence for verdict	Information

EXECUTABLE ASSET 50-F - BOUNDED CODES

```
public static class LogOutcome
{
    public const string Accepted = "ACCEPTED";
    public const string Committed = "COMMITTED";
    public const string DuplicateMatch = "DUPLICATE_MATCH";
    public const string Deferred = "DEFERRED";
    public const string Rejected = "REJECTED";
    public const string Failed = "FAILED";
    public const string Abstained = "ABSTAINED";
}
```

REASON DISCIPLINE

ReasonCode explains a bounded policy branch such as provider-timeout or fingerprint-mismatch. It never copies the provider's free-form error message.

SEVERITY SEMANTICS

LogLevel reports operational significance, not business preference

SEVERITY MATRIX 50-H

Level	Use	Misuse to reject
Trace	temporary fine-grained diagnostics	contractual evidence
Debug	bounded developer diagnostics	payload dumps
Information	expected state/outcome observation	every loop iteration
Warning	handled degradation or refusal	all client mistakes
Error	operation failed or evidence incomplete	retry merely scheduled
Critical	process/system integrity threatened	ordinary dependency outage

Severity is fixed by the catalogue for stable events. A dynamic level is allowed only for an explicitly reviewed diagnostic family; configuration cannot turn an ordinary success into an error or silence a mandatory security/audit obligation.

ERROR SEMANTICS

A handled retry can be Warning while its exhausted terminal attempt is Error. Severity follows the observed outcome, not the exception type alone.

HUMAN MESSAGE

Templates remain short, stable and non-authoritative

The formatted message supports local reading. It contains no sensitive or unbounded values and does not encode extra meaning absent from structured fields. Changing punctuation is not a schema change; changing a field's meaning, presence or unit is.

SOURCE-GENERATED EMITTER

The compiler sees the event contract

EXECUTABLE ASSET 50-G - CATALOGUE METHODS

```
public static partial class AlarmLog
{
    [LoggerMessage(
        EventId = LogEventIds.AlarmFloodCommitted,
        EventName = "alarm.flood.committed.v1",
        Level = LogLevel.Information,
        Message = "Alarm flood decision {OutcomeCode} ({ReasonCode})")]
    public static partial void FloodCommitted(
        this ILogger logger,
        string outcomeCode,
        string reasonCode,
        string alarmId,
        string correlationId,
        string causationId,
        string decisionId);
}
```

GENERATOR GATES 50-I

Gate	Failure caught
partial void method	runtime-format boilerplate drift
named placeholders	state lost in formatted prose
fixed EventId/EventName	unsearchable event family
typed parameters	object graph expansion
compiler diagnostics as errors	duplicate IDs and malformed templates

PERFORMANCE CONSEQUENCE

Source generation also avoids much of the boxing, allocation and parsing overhead of ad-hoc logging, but semantic safety remains the primary reason for the catalogue.

EXPLICIT CONTEXT

Chapter 49 identities enter through one immutable snapshot

EXECUTABLE ASSET 50-H - SAFE LOG CONTEXT

```
public sealed record SafeLogContext(  
    MessageId? MessageId,  
    CorrelationId? CorrelationId,  
    CausationId? CausationId,  
    RequestId? RequestId,  
    AttemptId AttemptId,  
    string? TraceId,  
    string? SpanId,  
    PairwiseActorRef? ActorRef,  
    string? WorkloadRef,  
    string PolicyRevision);  
  
public static SafeLogContext From(ExecutionContextSnapshot source) =>  
    SafeLogContextPolicy.Project(source);
```

PROJECTION RULES 50-J

Source	Log form	Rule
business lineage	typed string locators	preserve meaning/name
diagnostic context	nullable trace/span	observation only
security attribution	event-specific projection	no raw identity
ambient scope	never authoritative	may only duplicate safe snapshot

SCOPE INVARIANT

Background workers create one scope per item. Disposal and a post-item assertion prove that the next delivery cannot inherit the previous delivery's lineage.

HTTP ADMISSION EVENT

The edge records the admitted internal meaning

EXECUTABLE ASSET 50-I - HTTP COMMAND ADMITTED

```
[LoggerMessage(
    EventId = LogEventIds.HttpCommandAdmitted,
    EventName = "http.command.admitted.v1",
    Level = LogLevel.Information,
    Message = "Command {OperationName} admitted as {OutcomeCode}")]
public static partial void CommandAdmitted(
    this ILogger logger,
    string operationName,
    string outcomeCode,
    string requestId,
    string correlationId,
    string actorRef,
    string policyRevision,
    int httpStatusCode);
```

HTTP EXCLUSIONS 50-K

Never emit	Emit instead
request/response body	operation name + bounded outcome
Authorization/Cookie headers	authentication class when required
raw path/query	route template
raw subject/email	pairwise actor reference
arbitrary user agent	approved client class

ADMISSION TIMING

Emit after authentication, policy mapping and context creation. The event describes the internal command admitted by NEXUS-1, not the caller's untrusted headers.

ALARM EVENTS

One event names the business transition that was observed

ALARM CATALOGUE 50-L

EventName	Required fields	Outcome set
alarm.flood.committed.v1	alarmId, decisionId, lineage	COMMITTED, DUPLICATE_MATCH
alarm.state.changed.v1	alarmId, from, to, decisionId	COMMITTED
alarm.command.rejected.v1	operation, reason, requestId	REJECTED

EXECUTABLE ASSET 50-J - EMIT AFTER COMMIT OUTCOME

```
var outcome = await unitOfWork.CommitFloodAsync(decision, context, ct);

logger.FloodCommitted(
    outcome.Code,
    outcome.ReasonCode,
    decision.AlarmId.Value,
    context.Business.CorrelationId.Value,
    context.Business.CausationId.Value,
    decision.DecisionId.Value);

return outcome;
```

DURABILITY CAVEAT

The event reports the commit adapter's classified result. Incident reconstruction verifies that claim against AlarmManagementDb and the outbox; it never treats the log line as the commit oracle.

ROOT-CAUSE EVENTS

Abstention remains a first-class domain outcome

ROOTCAUSE CATALOGUE 50-M

EventName	Required fields	Meaning
rootcause.case.opened.v1	caseId, cause messageId	case durable
rootcause.verdict.issued.v1	caseId, verdictCode, evidenceCount	verdict produced
rootcause.verdict.abstained.v1	caseId, reasonCode	insufficient evidence

EXECUTABLE ASSET 50-K - VERDICT EVENT

```
[LoggerMessage(
    eventId = LogEventIds.RootCauseVerdictIssued,
    eventName = "rootcause.verdict.issued.v1",
    level = LogLevel.Information,
    message = "Case {CaseId} produced verdict {VerdictCode}")]]
public static partial void VerdictIssued(
    this ILogger logger,
    string caseId,
    string verdictCode,
    int evidenceCount,
    string messageId,
    string correlationId,
    string causationId);
```

MEANING PRESERVATION

A missing verdict is not logged as failure when the policy deliberately abstains. ABSTAINED and its bounded reason preserve the Domain decision.

OUTBOX DISPATCH EVENTS

Confirmed means broker confirmation was observed

DISPATCH CATALOGUE 50-N

EventName	Outcome	Does not claim
outbox.dispatch.started.v1	ATTEMPTED	delivery
outbox.dispatch.confirmed.v1	CONFIRMED	consumer commit
outbox.dispatch.deferred.v1	DEFERRED	terminal failure
outbox.dispatch.failed.v1	FAILED	business rollback

EXECUTABLE ASSET 50-L - CONFIRMED DISPATCH

```
[LoggerMessage(
    eventId = LogEventIds.OutboxDispatchConfirmed,
    eventName = "outbox.dispatch.confirmed.v1",
    level = LogLevel.Information,
    message = "Outbox message {MessageId} dispatch {OutcomeCode}")]]
public static partial void DispatchConfirmed(
    this ILogger logger,
    string messageId,
    string eventType,
    string outcomeCode,
    int attemptNumber,
    long durationMs,
    string correlationId,
    string causationId);
```

TRUTH BOUNDARY

The durable outbox row and broker confirm receipt remain the evidence. The log is an indexed observation of that adapter result.

CONSUMER EVENTS

Delivery outcome and business outcome are separate fields

CONSUME CATALOGUE 50-O

Field	Example	Why separate
handlerOutcome	DUPLICATE_MATCH	business/idempotency result
settlementOutcome	ACKED	broker action
redelivered	true	delivery observation
attemptNumber	3	bounded retry position
durationMs	47	one attempt duration

EXECUTABLE ASSET 50-M - CONSUME COMPLETION

```
[LoggerMessage(
    EventId = LogEventIds.MessageConsumeCompleted,
    EventName = "message.consume.completed.v1",
    Level = LogLevel.Information,
    Message = "Message {MessageId} consume {HandlerOutcome}")]
public static partial void ConsumeCompleted(
    this ILogger logger,
    string messageId,
    string eventType,
    string handlerOutcome,
    string settlementOutcome,
    bool redelivered,
    int attemptNumber,
    long durationMs,
    string correlationId);
```

EXACTLY-ONCE HONESTY

Duplicate delivery may produce multiple attempt events but one matching durable receipt. Logs therefore describe attempts, not exactly-once transport.

RETRY EVENTS

A retry schedule is a bounded policy decision

RETRY FIELDS 50-P

Field	Rule	Forbidden alternative
reasonCode	finite provider/policy class	raw exception message
attemptNumber	positive bounded integer	string counter
delayMs	actual scheduled delay	policy object dump
budgetRemaining	bounded count/time	entire configuration
nextAction	RETRY / DEADLETTER / STOP	free-form prose

EXECUTABLE ASSET 50-N - RETRY SCHEDULED

```
[LoggerMessage(
    EventId = LogEventIds.MessageRetryScheduled,
    EventName = "message.retry.scheduled.v1",
    Level = LogLevel.Warning,
    Message = "Message {MessageId} retry {AttemptNumber} in {DelayMs} ms")]
public static partial void RetryScheduled(
    this ILogger logger,
    string messageId,
    int attemptNumber,
    long delayMs,
    int budgetRemaining,
    string reasonCode,
    string nextAction,
    string correlationId);
```

NOISE RULE

Do not emit an Error for every handled transient attempt and another Error for exhaustion. The catalogue preserves Warning for bounded recovery and Error for the terminal failed operation.

DEAD-LETTER EVENTS

Dead-lettering records quarantine, not payload

EXECUTABLE ASSET 50-O - TERMINAL TRANSFER

```
[LoggerMessage(
    EventId = LogEventIds.MessageDeadLettered,
    EventName = "message.deadlettered.v1",
    Level = LogLevel.Error,
    Message = "Message {MessageId} dead-lettered as {ReasonCode}")]]
public static partial void MessageDeadLettered(
    this ILogger logger,
    string messageId,
    string eventType,
    string reasonCode,
    int finalAttemptNumber,
    string quarantineReference,
    string correlationId,
    string causationId);
```

QUARANTINE BOUNDARY 50-Q

Log contains	Quarantine contains	Access
locator + reason	retained original bytes if policy allows	operators
fingerprint	provider/broker metadata	restricted
attempt count	repair disposition	workflow-controlled
no payload	encrypted sensitive material	need-to-know

REPAIRABILITY

The quarantine reference is a locator under access control. The log never becomes a second, less protected payload store.

RECONCILIATION EVENTS

A mismatch names the compared invariants

EXECUTABLE ASSET 50-P - MISMATCH DETECTED

```
[LoggerMessage(
    EventId = LogEventIds.ReconciliationMismatch,
    EventName = "reconciliation.mismatch.detected.v1",
    Level = LogLevel.Warning,
    Message = "Reconciliation {RuleId} found {MismatchCode}")]]
public static partial void MismatchDetected(
    this ILogger logger,
    string ruleId,
    string mismatchCode,
    string ownerService,
    string sourceReference,
    string targetReference,
    string runId,
    long ageSeconds);
```

MISMATCH EXAMPLES 50-R

Code	Meaning	Repair precondition
OUTBOX_UNPUBLISHED	durable row exceeds age bound	row still pending
RECEIPT_WITHOUT_STATE	receipt/state oracle differs	fingerprint matches
PROJECTION_LAG_EXCEEDED	checkpoint behind bound	source position valid
ORPHAN_CASE	case lacks accepted cause	lineage record inspected

NO INFERENCE BY TIMESTAMP

Age is an observation used by a documented rule. It does not create a causal edge or prove which service failed first.

REPAIR EVENTS

Repair logs the decision and the verified postcondition

EXECUTABLE ASSET 50-Q - REPAIR COMPLETION

```
[LoggerMessage(
    EventId = LogEventIds.RepairActionCompleted,
    EventName = "repair.action.completed.v1",
    Level = LogLevel.Information,
    Message = "Repair {ActionCode} completed as {OutcomeCode}")]
public static partial void RepairCompleted(
    this ILogger logger,
    string actionCode,
    string outcomeCode,
    string reasonCode,
    string targetReference,
    string runId,
    string actorRef,
    string policyRevision,
    bool postconditionVerified);
```

REPAIR OUTCOMES 50-S

Outcome	Meaning	Required next evidence
COMMITTED	owner accepted repair	postcondition true
NO_CHANGE	target already converged	oracle comparison
REJECTED	precondition/policy failed	reason code
FAILED	repair attempt incomplete	bounded failure class

AUDIT SEPARATION

A business-required repair authorization remains durable audit evidence. This operational event may reference it; logging failure cannot erase or replace it.

SECURITY SEPARATION

Operational logs and durable audit have different failure contracts

SEPARATION MATRIX 50-T

Concern	Operational log	Durable audit intent
purpose	diagnosis and operations	attributable control evidence
commit	best effort after observation	same transaction when required
failure	must not mutate business result	may fail closed by policy
fields	safe locators and outcomes	approved attribution and policy
retention	operational policy	compliance/audit policy

Chapter 48's rule remains: logging failure cannot turn DENY into ALLOW. Conversely, a security/audit obligation that must survive is not delegated to ILogger; it is a durable owner-controlled record with its own authorization, retention and integrity proof.

NO DUAL USE BY ACCIDENT

A retained operational event can assist an investigation. It becomes neither a legally sufficient audit trail nor a business transaction merely because it contains an actor reference.

EXCEPTION POLICY

Classify the failure before it reaches the event

EXECUTABLE ASSET 50-R - SAFE ERROR DESCRIPTOR

```
public sealed record ErrorDescriptor(  
    string ErrorType,  
    string ErrorCode,  
    string FailureFingerprint,  
    bool Retryable);  
  
public ErrorDescriptor Describe(Exception exception) => exception switch  
{  
    TimeoutException => New("dependency-timeout", "DEP_TIMEOUT", true),  
    OperationCanceledException => New("operation-cancelled", "CANCELLED", false),  
    DbUpdateConcurrencyException => New(  
        "optimistic-concurrency", "DB_CONCURRENCY", true),  
    _ => New("unclassified", "UNCLASSIFIED", false)  
};
```

DEFAULT EXCEPTION EXCLUSIONS 50-U

Excluded	Reason
Exception.Message	provider/user data may be embedded
StackTrace	paths, code shape and unbounded size
Data dictionary	arbitrary objects and secrets
InnerException graph	unbounded recursive disclosure
SQL/provider detail	schema, values and credentials

DIAGNOSTIC ESCALATION

A time-bounded, access-controlled diagnostic channel may retain additional data under an incident policy. It is not the default application log contract.

SANITIZATION

Untrusted text cannot create fields or lines

EXECUTABLE ASSET 50-S - BOUNDED TOKEN NORMALIZER

```
public static String NormalizeBoundedToken(String? value, int maxLength)
{
    if (string.IsNullOrEmpty(value)) return "unknown";
    var result = new StringBuilder(Math.Min(value.Length, maxLength));
    foreach (var rune in value.EnumerateRunes())
    {
        if (result.Length >= maxLength) break;
        result.Append(Rune.IsLetterOrDigit(rune) || rune.Value is '-' or '_'
            ? rune.ToString().ToLowerInvariant()
            : "_");
    }
    return result.Length == 0 ? "unknown" : result.ToString();
}
```

INJECTION CASES 50-V

Input	Stored result	Invariant
CR/LF	underscore or rejection	one record remains one record
JSON delimiters	provider encoding	cannot escape field value
control characters	removed/rejected	no terminal control
overlong token	bounded/rejected	storage and cost bound
invalid Unicode	replacement/rejection	valid output encoding

ENCODING RULE

Normalization is for bounded classifications only. It is never used to make a bearer token, payload or personal value acceptable for logging.

CARDINALITY BUDGET

Search locators are useful, but their cost is explicit

FIELD BUDGET 50-W

Field	Class	Per-event limit	Index policy
EventName	bounded	1 of catalogue	indexed
OutcomeCode	bounded	<= 12 per event	indexed
ReasonCode	bounded	<= 64 per event	indexed
MessageId/Caseld	locator	one each	restricted locator index
TraceId/SpanId	diagnostic locator	one each	retention-limited
free-form text	forbidden	zero	none

EXECUTABLE ASSET 50-T - BUDGET VALIDATOR

```
public static void Validate(LogEventDefinition definition)
{
    Require(definition.Fields.Count <= 18, "too-many-fields");
    Require(definition.MaxEncodedBytes <= 8_192, "event-too-large");
    Require(definition.OutcomeCodes.Count <= 12, "outcome-unbounded");
    Require(definition.ReasonCodes.Count <= 64, "reason-unbounded");
    Require(definition.Fields.All(f => f.DataClass != LogDataClass.Forbidden),
        "forbidden-field");
}
```

METRICS BOUNDARY

High-cardinality locators may exist in protected logs. They are never metric labels. Chapter 52 will aggregate only bounded dimensions.

RESOURCE IDENTITY

Deployment metadata is injected from a trusted manifest

EXECUTABLE ASSET 50-U - BOUNDED SERVICE RESOURCE

```
public sealed record ServiceResource(  
    string ServiceName,  
    string ServiceNamespace,  
    string ServiceVersion,  
    string ServiceInstanceId,  
    string DeploymentEnvironment,  
    string BuildIdentity);  
  
public static ServiceResource Load(ValidatedRuntimeManifest manifest) => new(  
    manifest.ServiceName,  
    "nexus1",  
    manifest.ReleaseVersion,  
    manifest.InstanceReference,  
    manifest.EnvironmentName,  
    manifest.BuildDigest);
```

RESOURCE RULES 50-X

Rule	Reason
validated at startup	bad identity fails before emission
no hostname/IP by default	topology and cardinality control
instance reference bounded	locates runtime without secret
build identity immutable	joins evidence to deployment
same resource for batch	provider can encode efficiently

PROVENANCE

Resource metadata says which runtime emitted the observation. It does not prove that the runtime was authorized to make the underlying business decision.

JSON NORMALIZATION

Canonical evidence JSON is stricter than provider output

EXECUTABLE ASSET 50-V - DETERMINISTIC JSON PROFILE

```
public static readonly JsonSerializerOptions EvidenceJson = new()
{
    PropertyNamingPolicy = JsonNamingPolicy.CamelCase,
    DictionaryKeyPolicy = null,
    WriteIndented = false,
    DefaultIgnoreCondition = JsonIgnoreCondition.WhenWritingNull,
    Encoder = JavaScriptEncoder.Default,
    NumberHandling = JsonNumberHandling.Strict
};

public static byte[] Serialize(NexusLogRecordV1 record) =>
    JsonSerializer.SerializeToUtf8Bytes(record, EvidenceJson);
```

CANONICALIZATION RULES 50-Y

Concern	Rule
property names	fixed camelCase schema
dates	UTC RFC 3339 representation
numbers	JSON numbers; units in field names
null	omitted only when schema marks optional
order	tests compare parsed canonical model, not provider order
culture	invariant serialization

PROVIDER INDEPENDENCE

Console or exporter JSON may wrap State differently. The evidence adapter maps it back to the logical schema before validation; raw provider layout is not the business contract.

CONSOLE TRANSPORT

AddJsonConsole is a Phase-0 transport, not the schema owner

EXECUTABLE ASSET 50-W - BOUNDED CONSOLE CONFIGURATION

```
builder.Logging.ClearProviders();
builder.Logging.AddJsonConsole(options =>
{
    options.IncludeScopes = true;
    options.UseUtcTimestamp = true;
    options.TimestampFormat = "yyyy-MM-dd'T'HH:mm:ss.fff'Z'";
    options.JsonWriterOptions = new JsonWriterOptions
    {
        Indented = false,
        SkipValidation = false
    };
});
```

TRANSPORT CAVEATS 50-Z

Provider feature	NEXUS-1 treatment
formatted Message	human convenience only
State object	mapped to catalogue fields
Scopes	safe context duplication only
timestamp	provider observation; origin retained separately
exception output	disabled by safe emitter policy

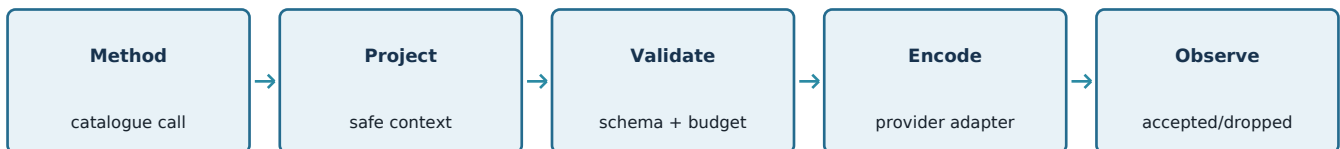
EVOLUTION RULE

Moving to OpenTelemetry or another provider changes an adapter and its verification corpus. It does not change event names, field meanings or outcome codes.

PROVIDER BOUNDARY

Normalize, validate, then hand off

PROOF PATH 50-B - EMISSION PIPELINE



EXECUTABLE ASSET 50-X - FAIL-SAFE EMITTER

```
public void Emit(LogEventDefinition definition, LogOccurrence occurrence)
{
    try
    {
        validator.Validate(definition, occurrence);
        provider.Write(mapper.Map(definition, occurrence, resource.Current));
        health.RecordAccepted(definition.EventId);
    }
    catch (Exception ex) when (!Fatal.IsProcessFatal(ex))
    {
        health.RecordDropped(definition.EventId, FailureClass.From(ex));
        emergency.TryWriteBounded("log-emission-failed", definition.EventId);
    }
}
```

BUSINESS ISOLATION

An operational logging failure does not roll back or alter a completed business result. Required durable audit intent is committed separately under its own policy.

BACKPRESSURE AND LOSS

The system makes dropped observations visible without blocking forever

FAILURE POLICY 50-AA

Condition	Application action	Evidence action
provider disabled	business path continues	health degraded + counter
bounded queue full	drop by documented priority	drop count/reason
sink timeout	do not await unboundedly	circuit state
disk full	preserve business policy	emergency channel if possible
shutdown	bounded flush	flushed/dropped manifest

EXECUTABLE ASSET 50-Y - BOUNDED SHUTDOWN

```
public async Task<LogFlushResult> FlushAsync(CancellationToken stopping)
{
    using var timeout = CancellationTokenSource.CreateLinkedTokenSource(stopping);
    timeout.CancelAfter(options.FlushTimeout);
    try
    {
        return await pipeline.FlushAsync(timeout.Token);
    }
    catch (OperationCanceledException)
    {
        return new(LogFlushOutcome.TimedOut, pipeline.PendingCount);
    }
}
```

AVAILABILITY RULE

Logging must not become an unbounded dependency. Loss is a degraded evidence condition that is measured and retained; it is never silently described as complete evidence.

EMISSION TIMING

One authoritative outcome observation per operation

TIMING MATRIX 50-AB

Moment	Event	Rule
before validation	none	untrusted bytes are not domain events
after admission	command admitted/rejected	edge meaning fixed
after owner commit	committed/duplicate/rejected	adapter outcome classified
after broker confirm	dispatch confirmed	confirm observed
after settlement	consume completed	handler and settlement both known

Started events are reserved for long-running operations where absence of completion is diagnostically valuable. They carry a new AttemptId and pair with exactly one terminal observation in the retained test corpus. Ordinary short operations emit only their terminal outcome.

DUPLICATE EVENTS

A retry can legitimately emit multiple attempt observations. The join key and attempt number make this explicit; consumers must not count log lines as unique business facts.

RETENTION AND ACCESS

Field classification drives storage policy

RETENTION PROFILE 50-AC

Class	Access	Illustrative retention	Deletion proof
bounded operational	on-call/service owner	short operational window	backend policy record
locator-rich	restricted investigation	minimum case window	expiry sample
security attribution	security/approved owner	separate policy	audited access
diagnostic escalation	incident-scoped	time-bounded	incident closure

Exact periods are deployment policy, not invented by this manuscript. The implementation must pin them in an approved manifest, enforce access at query and export surfaces, and retain evidence that expiry and deletion were exercised.

DATA MINIMIZATION

Collection, indexing, export, backup and support tools inherit the same classification. A safe application event is not safe if a downstream pipeline enriches it with raw headers or payloads.

CATALOGUE MODEL

The schema itself is versioned evidence

EXECUTABLE ASSET 50-Z - EVENT DEFINITION

```
public sealed record LogEventDefinition(  
    int EventId,  
    string EventName,  
    int SchemaVersion,  
    LogLevel Level,  
    string MessageTemplate,  
    ImmutableArray<LogFieldDefinition> Fields,  
    FrozenSet<string> OutcomeCodes,  
    FrozenSet<string> ReasonCodes,  
    int MaxEncodedBytes,  
    string Owner,  
    string DecisionReference);  
  
public sealed record LogCatalogue(  
    string CatalogueVersion,  
    ImmutableArray<LogEventDefinition> Events,  
    string CanonicalSha256);
```

CATALOGUE EVIDENCE 50-AD

Artifact	Proves
canonical JSON	exact reviewed schema
SHA-256 digest	artifact identity
owner + decision reference	change authority
compatibility report	versioning decision
generated-code inventory	methods match catalogue

CHANGE GATE

Adding a field, outcome or event is a reviewed catalogue change. Renaming or changing meaning requires a new event version and a compatibility decision.

CATALOGUE VALIDATOR

Fail closed before a malformed schema ships

VERIFICATION ASSET 50-A - SOLUTION-WIDE VALIDATION

```
public static IReadOnlyList<string> Validate(LogCatalogue catalogue)
{
    var errors = new List<string>();
    AddDuplicates(errors, catalogue.Events, x => x.EventId, "event-id");
    AddDuplicates(errors, catalogue.Events, x => x.EventName, "event-name");
    foreach (var e in catalogue.Events)
    {
        CheckName(e, errors);
        CheckTemplateMatchesFields(e, errors);
        CheckClassesAndLengths(e, errors);
        CheckBoundedCodes(e, errors);
        CheckVersionAndOwner(e, errors);
    }
    return errors;
}
```

RELEASE BLOCKERS 50-AE

Violation	Why blocked
duplicate EventId/Name	ambiguous query and schema
template-only parameter	hidden structured state
field without classification	unknown disclosure
unbounded string	cost/injection risk
locator marked metric-safe	cardinality explosion
breaking change without vNext	historic meaning corrupted

ARCHITECTURE TESTS

Application code can log only through approved methods

VERIFICATION ASSET 50-B - FORBIDDEN SURFACES

```
[Fact]
public void Production_code_uses_only_catalogue_emitters()
{
    var violations = solution.ProductionMethods()
        .Where(m => m.InvokeAny(
            "ILogger.Log", "LogInformation", "LogWarning", "LogError"))
        .Where(m => !m.DeclaringType.HasAttribute<GeneratedCodeAttribute>())
        .ToArray();

    violations.Should().BeEmpty();
}

[Fact]
public void Event_parameters_are_closed_scalars() =>
    catalogue.Events.SelectMany(x => x.Fields)
        .Should().OnlyContain(f => AllowedKinds.Contains(f.Kind));
```

STATIC GATES 50-AF

Gate	Rejects
no string interpolation	\$-strings at log call sites
no arbitrary ILogger calls	uncatalogued events
no Exception parameter	automatic message/stack output
no object/dictionary	unbounded expansion
no forbidden field names	token, body, password, connectionString

GENERATED-CODE CAVEAT

The rule recognizes compiler-generated implementations while keeping the authored partial methods under catalogue validation.

GOLDEN CORPUS

Every event has an exact safe example

VERIFICATION ASSET 50-C - CANONICAL RECORD TEST

```
[Fact]
public void Flood_commit_matches_golden_schema()
{
    var record = fixture.Capture(logger => logger.FloodCommitted(
        "COMMITTED", "policy-satisfied", "alarm-000042",
        "corr-000042", "decision-000041", "decision-000042"));

    logicalMapper.Map(record).Should().BeEquivalentTo(
        Golden.Load("alarm.flood.committed.v1.json"),
        o => o.WithStrictOrdering());
}
```

GOLDEN ASSERTIONS 50-AG

Assert	Reason
EventId + EventName	identity pair exact
field names/types	provider mapping stable
outcome/reason allow-list	domain meaning bounded
no extra fields	no enrichment leakage
encoded byte ceiling	storage/cost bound
UTC timestamps	portable observation

REVIEW PRACTICE

Golden changes require the catalogue diff and decision reference. A snapshot update cannot silently bless a leaked or renamed field.

LEAK AND INJECTION TESTS

Hostile input must not appear anywhere in encoded output

VERIFICATION ASSET 50-D - HOSTILE CORPUS

```
[Theory]
[MemberData(nameof(HostileValues))]
public void Untrusted_text_cannot_escape_or_leak(string hostile)
{
    var captured = fixture.EmitRejectedCommand(hostile);
    var json = fixture.Encode(captured);

    json.Should().NotContain(hostile);
    JsonDocument.Parse(json).RootElement.ValueKind
        .Should().Be(JsonValueKind.Object);
    fixture.ReadRecords(json).Should().ContainSingle();
    leakScanner.FindForbidden(json).Should().BeEmpty();
}
```

HOSTILE CORPUS 50-AH

Family	Examples
line injection	CR, LF, CRLF, fake timestamp
structured escape	quotes, braces, delimiters
Unicode	controls, bidi marks, invalid sequences
secret patterns	bearer, connection string, PEM header
size	oversized token and repeated nesting

TEST ORACLE

The test validates parsed record count and schema, not only string replacement. Encoding must preserve one valid record with no injected fields.

SCOPE ISOLATION TESTS

Concurrent work cannot exchange lineage

VERIFICATION ASSET 50-E - WORKER SCOPE ISOLATION

```
[Fact]
public async Task Concurrent_deliveries_keep_distinct_context()
{
    await worker.ProcessConcurrentlyAsync(
        Delivery("message-a", "corr-a"),
        Delivery("message-b", "corr-b"));

    captured.For("message-a").Should().OnlyContain(
        e => e.CorrelationId == "corr-a");
    captured.For("message-b").Should().OnlyContain(
        e => e.CorrelationId == "corr-b");
    diagnosticScope.Current.Should().BeNull();
}
```

ISOLATION CAMPAIGN 50-AI

Mutation	Expected
interleaved awaits	context stays item-local
handler exception	finally restores prior scope
cancellation	no stale context after disposal
batch channel reuse	lineage remains per delivery
fire-and-forget attempt	architecture gate rejects

NO AMBIENT RESCUE

The emitted fields originate from the explicit SafeLogContext. A missing ambient Activity may remove trace locators but cannot remove business lineage.

FAILURE TESTS

Logger disabled or broken never changes the business outcome

VERIFICATION ASSET 50-F - SINK FAILURE ISOLATION

```
[Theory]
[InlineData(LogFailure.Disabled)]
[InlineData(LogFailure.QueueFull)]
[InlineData(LogFailure.ThrowingProvider)]
[InlineData(LogFailure.FlushTimeout)]
public async Task Logging_failure_does_not_mutate_commit(LogFailure failure)
{
    fixture.Logging.Inject(failure);
    var result = await fixture.CommitAlarmAsync();

    result.Code.Should().Be("COMMITTED");
    await fixture.Database.ShouldContainCommittedAlarm();
    fixture.LogHealth.ShouldReport(failure);
    fixture.EvidenceCompleteness.Should().Be("DEGRADED");
}
```

FAILURE ORACLES 50-AJ

Claim	Oracle
business unaffected	provider-backed state
public fact unaffected	outbox bytes/fingerprint
loss visible	health/drop evidence
shutdown bounded	elapsed time + pending count
no recursive storm	emergency event count bounded

PROVIDER COMPONENT TESTS

The actual formatter must preserve the logical state

VERIFICATION ASSET 50-G - JSON CONSOLE ROUND TRIP

```
[Fact]
public async Task Production_logging_path_round_trips_catalogue_event()
{
    await using var host = await ProductionHost.StartAsync(
        output: capturedConsole,
        environment: "Evidence");

    await host.Services.GetRequiredService<Scenario>()
        .CommitFloodAsync();

    var records = jsonLines.Read(capturedConsole.Text);
    var logical = providerMapper.Map(records.Single(
        x => x.EventId == LogEventIds.AlarmFloodCommitted));
    catalogueValidator.Validate(logical).Should().BeEmpty();
}
```

COMPONENT OBLIGATIONS 50-AK

Surface	Required proof
JSON console	valid single-line JSON and scopes
provider filter	mandatory catalogue levels retained
culture/timezone	same logical field values
large locator	bounded rejection/normalization
process shutdown	bounded flush manifest

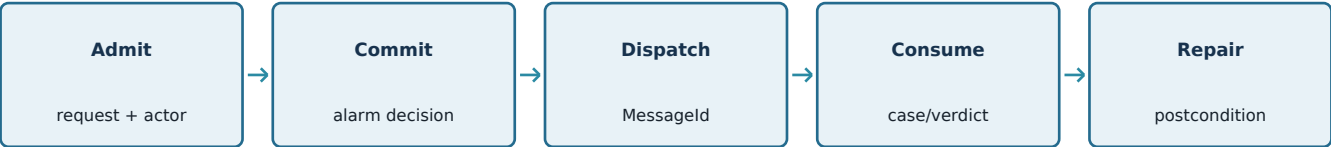
DEPLOYMENT HONESTY

This defines the test harness. A fresh executed result is required before the provider path can move from RUNTIME PENDING to VERIFIED.

ONE-CASE RECONSTRUCTION

Logs index the causal graph without replacing it

PROOF PATH 50-C - ONE DISTRIBUTED CASE



JOIN LEDGER 50-AL

Observation	Primary join	Authoritative cross-check
http.command.admitted	RequestId/CorrelationId	edge admission
alarm.flood.committed	DecisionId/CausationId	AlarmManagementDb
outbox.dispatch.confirmed	MessageId	outbox + confirm receipt
message.consume.completed	MessageId/AttemptId	inbox + owner state
rootcause.verdict.issued	CasId/new MessageId	RootCauseAnalysisDb
repair.action.completed	RunId/target reference	postcondition oracle

RECONSTRUCTION INVARIANT

Shared correlation finds the case. Causation and durable records establish the path. Timestamps and message prose help navigation but never create missing edges.

EVIDENCE MANIFEST

Completeness is an explicit verdict

EXECUTABLE ASSET 50-AA - LOGGING EVIDENCE RECORD

```
public sealed record LoggingEvidenceManifest(  
    string BuildIdentity,  
    string CatalogueVersion,  
    string CatalogueSha256,  
    string ProviderProfile,  
    int ExpectedTests,  
    int DiscoveredTests,  
    int PassedTests,  
    int DroppedRecords,  
    DateTimeOffset StartedAtUtc,  
    DateTimeOffset CompletedAtUtc,  
    string Verdict,  
    ImmutableArray<string> Residuals);
```

FAIL-CLOSED CHECKS 50-AM

Condition	Verdict
zero discovered tests	INVALID
catalogue digest mismatch	INVALID
required event absent	INCOMPLETE
leak/injection finding	FAILED
unexplained dropped records	DEGRADED
provider + corpus pass	VERIFIED for named environment

EVIDENCE STATUS

The current manuscript supplies the schema and verification design. Until repository tests and a retained provider run exist, the chapter remains SPECIFIED | RUNTIME PENDING.

EVIDENCE VALIDATOR

A green test command is not sufficient evidence

VERIFICATION ASSET 50-H - MANIFEST GATE

```
public static EvidenceVerdict Verify(LoggingEvidenceManifest m)  
{  
    if (m.DiscoveredTests == 0 || m.DiscoveredTests != m.ExpectedTests)  
        return EvidenceVerdict.Invalid("test-discovery-mismatch");  
    if (m.PassedTests != m.DiscoveredTests)  
        return EvidenceVerdict.Failed("logging-test-failure");  
    if (m.DroppedRecords > 0)  
        return EvidenceVerdict.Degraded("records-dropped");  
    if (m.CompletedAtUtc <= m.StartedAtUtc)  
        return EvidenceVerdict.Invalid("evidence-time-invalid");  
    return EvidenceVerdict.Verified(m.BuildIdentity, m.CatalogueSha256);  
}
```

RETAINED BUNDLE 50-AN

Artifact	Required coordinates
catalogue.json	version + digest
generated inventory	assembly + EventId/Name
golden corpus	event + schema version
test results	build + environment + discovery count
provider samples	sanitized bounded records
loss/flush result	accepted/dropped/pending

IMPLEMENTATION SEQUENCE

Build meaning before choosing a backend

SEQUENCE 50-AO

Step	Deliverable	Exit gate
1	ADR-050 + field taxonomy	owners approve purpose
2	logical record + catalogue	schema validator passes
3	generated emitters	compiler/static gates pass
4	safe context projection	scope isolation passes
5	domain/messaging/repair events	golden corpus passes
6	provider adapter	component round trip passes
7	failure/backpressure policy	fault campaign passes
8	retention/access/evidence	manifest verifies

ORDER INVARIANT

Do not start with dashboard queries or a vendor index template. First freeze domain meaning, safe fields and evidence obligations; then prove the chosen transport preserves them.

PROVIDER MIGRATION

Changing sinks must not rewrite history

A provider migration replays the golden corpus through both adapters and compares logical records. Differences in wrapper layout are acceptable; differences in event identity, units, optionality, outcome meaning or safe-field set require a compatibility decision.

STANDARDS ALIGNMENT

Use standards as translation surfaces, not design substitutes

SOURCE LEDGER 50-AP

Source	Applied rule	NEXUS-1 extension
.NET LoggerMessage	generated structured state	catalogue-only call sites
.NET JSON console	JSON formatter + UTC/scopes	transport not schema owner
OpenTelemetry Logs	timestamps, severity, resource, trace, EventName	typed domain operation
OWASP Logging	exclude secrets, sanitize, test failures	closed field catalogue
Chapter 49	typed lineage and attribution	safe explicit projection

VERIFICATION BOUNDARY

Standards alignment makes mapping possible. Only the retained NEXUS-1 catalogue, executed tests and environment evidence prove this implementation.

OPERATIONAL QUERIES

Queries use stable fields, never message text

QUERY EXAMPLES 50-AQ

Question	Filter/join
why was a verdict absent?	CorrelationId + outcome ABSTAINED + ReasonCode
which attempt exhausted retry?	MessageId + attemptNumber + nextAction
was dispatch observed?	MessageId + eventName + confirm cross-check
who approved repair?	RunId -> durable audit reference

COMPLETION LEDGER

What Chapter 50 now specifies

COMPLETION LEDGER 50-AR

Obligation	Asset	Status
stable event identities	name grammar + allocations	SPECIFIED
provider-neutral schema	NexusLogRecordV1	SPECIFIED
safe domain catalogue	alarm/case/verdict/messaging/repair	SPECIFIED
sensitive-data prevention	classification + closed emitters	SPECIFIED
cardinality and retention	budgets + policy boundaries	SPECIFIED
failure isolation	backpressure/flush contracts	SPECIFIED
runtime proof	provider/fault/evidence campaign	PENDING

OPEN RESIDUALS 50-AS

Residual	Consequence
no executed repository checkpoint	listings remain compile-oriented
no retained provider campaign	formatter mapping unproved
no collector/backend deployment	retention and loss behavior unproved
no approved environment retention values	periods remain policy inputs
no production incident corpus	query utility unproved at scale

EVIDENCE VERDICT

SPECIFIED | RUNTIME PENDING. The logical contract, event catalogue, safe-field rules and test obligations are complete; no fresh runtime, collector, backend or production evidence is claimed.

HANDOFF TO CHAPTER 51

Distributed Tracing with OpenTelemetry

Chapter 51 will create ActivitySource instrumentation for HTTP, messaging, database and background work; configure OpenTelemetry SDKs, sampling and exporters; and demonstrate both a complete and deliberately broken distributed trace. It will reuse the safe log identifiers without turning spans into business authority.

BOUNDARY CARRIED FORWARD 50-AT

Chapter 50 leaves	Chapter 51 must add
stable event and lineage fields	span names, kinds and attributes
trace/span IDs as optional locators	context propagation and links
provider-neutral log schema	OpenTelemetry log/trace correlation mapping
sampling-independent business meaning	sampling and tail/head decisions
runtime evidence pending	exporter and broken-trace campaign

NEXT INVARIANT

A trace explains an execution path that was observed. It never becomes the business cause, authorization proof or durability oracle for the transaction it surrounds.

FINAL CHAPTER RULE

If a field is not safe, bounded and meaningful, it is not a log field

NEXUS-1 structured logs are small versioned observations that can be joined to authoritative evidence. Their value comes from disciplined omission as much as from inclusion: no payloads, credentials, accidental identities, arbitrary baggage or prose-only outcomes.

PART IX CONTINUITY

Correlation gives the case a joinable vocabulary; structured logs preserve its observations; tracing will explain execution; metrics will bound aggregate health; resilience evidence will prove failure stays visible and recoverable.

PART IX - OBSERVABILITY, RESILIENCE, AND FAILURE EVIDENCE

51. Distributed Tracing with OpenTelemetry

A trace explains one observed execution path only when span identity, propagation, sampling and export are explicit - and when missing telemetry is never mistaken for missing business truth.

CHAPTER DECISION

Instrument meaningful HTTP, messaging, database and background-work boundaries with System.Diagnostics.ActivitySource and OpenTelemetry. Preserve Chapter 49 business lineage and Chapter 50 safe fields as bounded attributes, propagate only diagnostic context, and verify one complete and one deliberately broken trace with retained structural evidence.

PROOF PATH 51-A - FROM REQUEST TO VERDICT**GUARANTEE LEDGER 51-A**

Boundary	Trace mechanism	Allowed claim	Still forbidden
HTTP -> handler	remote parent + SERVER span	request path observed	caller is authorized
commit -> dispatch	link to stored producer context	attempt relation observed	log proves commit
message -> consumer	creation context + CONSUMER span	processing path joined	exactly-once delivery
span -> evidence	export + structural validator	named trace retained	business truth exists

CHAPTER EXIT CRITERIA 51-B

Question	Required answer
Can one case cross process boundaries?	yes - parent/link topology is explicit
Can sampling alter a business decision?	no - diagnostics are optional
Can a missing span be classified?	yes - propagation, registration, sampling and export checked
Can the trace be reproduced?	yes - build, packages, config and corpus retained

ORDINARY DEVELOPER PROBLEM

A waterfall can look complete while the case is wrong

Automatic HTTP and SQL instrumentation creates an attractive diagram, so the team assumes the distributed transaction is visible. The outbox dispatcher starts hours later with no request Activity, the broker carrier drops trace context, and the consumer opens a new root. The durable case may still be correct, but the visual story silently loses its causal bridge.

FAILURE LEDGER 51-1

Shortcut	Diagnostic failure	Required correction
auto-instrument everything	business seam remains invisible	custom spans at owned boundaries
Activity.Current is lineage	retry inherits wrong attempt	explicit Chapter 49 snapshot
trace ID proves one fact	sampling/export can erase it	durable MessageId evidence
raw object as tags	secrets and cardinality escape	bounded attribute catalogue
green waterfall means success	domain failure may be Unset	status + domain outcome

PLAIN-LANGUAGE PRINCIPLE

A trace is a map of telemetry that arrived. It is not a census of every operation that occurred.

CHAPTER BOUNDARY

Execution topology now; aggregate health next

Chapter 51 defines span sources, names, kinds, attributes, propagation, links, sampling, export and structural tests. Chapter 52 will aggregate requests, messages, queue age, errors and workflow duration into metrics and educational objectives. This chapter does not invent production SLOs or dashboards.

SCOPE INVARIANT

No collector, backend, trace UI or production retention policy is claimed until an executed environment manifest and retained evidence bundle prove it.

DECISION RECORD

ADR-051 - Diagnostic topology is a separate ledger

NEXUS-1 records each meaningful execution attempt as an Activity/Span. Parent-child relationships describe synchronous execution; links describe accepted asynchronous or fan-in relationships; safe attributes join the span to business lineage. Neither parentage nor TraceId creates business causation, authorization or durability.

DECISION LEDGER 51-2

Ledger	Owns	Stable join	Cannot decide
business lineage	fact and direct cause	MessageId/CausationId	span parent
trace topology	observed attempt path	TraceId/SpanId/links	business truth
structured logs	named observations	event + trace locators	span completeness
security evidence	bounded attribution	audit reference	baggage authority

DECISION IDENTITY

ADR-051 is hashed from retained canonical text by the evidence build. The manuscript does not invent a runtime digest.

SPAN MODEL

One span represents one operation with a bounded lifetime

SPAN ANATOMY 51-A

Element	NEXUS-1 meaning	Rule
name	low-cardinality operation	never an identifier
kind	remote/process relationship	SERVER/CLIENT/PRODUCER/CONSUMER/INTERNAL
parent	one execution predecessor	not global cause
links	additional async/fan-in relations	created at span start
attributes	bounded diagnostic facts	safe typed allow-list
status	operation error state	not domain outcome

INSTRUMENTATION TOPOLOGY

Trace the seams where latency or failure changes meaning

COVERAGE CATALOGUE 51-B

Surface	Span kind	Owner	Why retained
incoming HTTP	SERVER	ASP.NET Core	edge attempt
outgoing HTTP	CLIENT	HttpClient	remote dependency
owner orchestration	INTERNAL	application	decision/commit boundary
database operation	CLIENT	SqlClient	provider latency/error
message publish	PRODUCER	dispatcher	creation/send attempt
message process	CONSUMER	worker	delivery attempt
reconciliation/repair	INTERNAL	background owner	bounded run

NO SPAN PER METHOD

Pure Domain methods, value-object construction and trivial internal calls remain untraced. Trace volume follows diagnostic value, not call depth.

TRACE NARRATIVE

The complete case contains asynchronous bridges

PROOF PATH 51-B - EXPECTED TOPOLOGY



Database client spans hang beneath owner-local commit spans. The outbox dispatch attempt can occur in a new trace linked to the producer commit snapshot. The consumer process span uses the propagated message creation context. Business correlation remains joinable even when these diagnostic traces are separate.

ACTIVITYSOURCE CATALOGUE

Source name and version are part of the instrumentation contract

EXECUTABLE ASSET 51-A - SOURCE INVENTORY

```
public static class NexusActivitySources
{
    public const string Alarm = "Nexus1.AlarmManagement";
    public const string RootCause = "Nexus1.RootCauseAnalysis";
    public const string Messaging = "Nexus1.Messaging";
    public const string Repair = "Nexus1.Repair";

    public static readonly ActivitySource AlarmSource = new(Alarm, "1.0.0");
    public static readonly ActivitySource RootCauseSource = new(RootCause, "1.0.0");
    public static readonly ActivitySource MessagingSource = new(Messaging, "1.0.0");
    public static readonly ActivitySource RepairSource = new(Repair, "1.0.0");
}
```

SOURCE DECISIONS 51-C

Rule	Reason
one static source per component	listener registration and allocation discipline
hierarchical unique names	avoid library collisions
explicit version	instrumentation evolution is visible
source inventory in evidence	zero-registration failure is detectable
Dispose only at process shutdown	source lives with component

.NET BEHAVIOR

StartActivity may return null when no interested listener records the source. Business code must remain correct in that state.

SPAN NAMING

Names describe operations, never instances

NAME CATALOGUE 51-D

Operation	Span name	Rejected name
HTTP command	HTTP POST /alarms	POST /alarms/42
alarm commit	alarm flood commit	commit alarm-42
publish	publish alarm-flood-detected	publish msg-...
consume	process alarm-flood-detected	consume delivery-887
verdict	root-cause verdict commit	verdict CONFIRMED
repair	repair outbox gap	repair run-uuid

EXECUTABLE ASSET 51-B - REVIEWED SPAN NAMES

```
public static class SpanNames
{
    public const string AlarmFloodCommit = "alarm flood commit";
    public const string RootCauseOpen = "root-cause case open";
    public const string RootCauseVerdict = "root-cause verdict commit";
    public const string PublishAlarmFlood = "publish alarm-flood-detected";
    public const string ProcessAlarmFlood = "process alarm-flood-detected";
    public const string ReconcileOutbox = "reconcile outbox gap";
}
```

CARDINALITY INVARIANT

A query groups spans by name without parsing identifiers. Locators belong in classified attributes and never change the span name.

ATTRIBUTE CATALOGUE

Reuse safe meaning from Chapter 50, with trace-specific limits

ATTRIBUTE POLICY 51-E

Attribute	Class	Allowed on	Never used for
nexus1.message.id	locator	publish/process	metric label
nexus1.correlation.id	locator	case spans	parent selection
nexus1.causation.id	business edge	derived work	trace status
nexus1.outcome.code	bounded	terminal owner span	error inference
messaging.destination.name	bounded topology	broker spans	raw temporary queue
error.type	bounded class	failed operation	exception message

EXECUTABLE ASSET 51-C - SAFE TAG PROJECTION

```
public static ActivityTagsCollection SafeTags(
    MessageId messageId,
    CorrelationId correlationId,
    string outcomeCode) => new()
{
    ["nexus1.message.id"] = messageId.Value,
    ["nexus1.correlation.id"] = correlationId.Value,
    ["nexus1.outcome.code"] = outcomeCode
};
```

PRIMARY EXCLUSION

Payloads, headers, tokens, claims, SQL text with values, exception messages and arbitrary baggage have no tag helper and no approved attribute definition.

RESOURCE IDENTITY

Every exported span names the runtime that observed it

EXECUTABLE ASSET 51-D - VALIDATED RESOURCE

```
var resource = ResourceBuilder.CreateDefault()
    .AddService(
        serviceName: manifest.ServiceName,
        serviceNamespace: "nexus1",
        serviceVersion: manifest.ReleaseVersion,
        serviceInstanceId: manifest.InstanceReference)
    .AddAttributes(new Dictionary<string, object>
    {
        ["deployment.environment.name"] = manifest.EnvironmentName,
        ["nexus1.build.identity"] = manifest.BuildDigest
    });
```

RESOURCE PROOF 51-F

Field	Source	Gate
service.name	service manifest	bounded allow-list
service.version	release artifact	immutable
service.instance.id	runtime reference	not host/IP by default
deployment.environment.name	validated config	startup validation
build identity	CI artifact digest	evidence join

PROVENANCE RULE

Resource identifies the emitter. It does not authorize the service or authenticate the message producer; Chapters 46-48 own those proofs.

OPENTELEMETRY COMPOSITION

The production host registers every expected source explicitly

EXECUTABLE ASSET 51-E - TRACING REGISTRATION

```
services.AddOpenTelemetry()
    .ConfigureResource(builder => builder.AddService(manifest.ServiceName))
    .WithTracing(builder => builder
        .SetSampler(new AlwaysOnSampler()) // Phase-0 evidence profile
        .AddSource(
            NexusActivitySources.Alarm,
            NexusActivitySources.RootCause,
            NexusActivitySources.Messaging,
            NexusActivitySources.Repair)
        .AddAspNetCoreInstrumentation()
        .AddHttpClientInstrumentation()
        .AddSqlClientInstrumentation()
        .AddOtlpExporter(options => options.Endpoint = manifest.OtlpEndpoint));
```

REGISTRATION LEDGER 51-G

Registration	Owns	Evidence check
AddSource	custom spans	source inventory matches
ASP.NET Core	incoming SERVER spans	route/status corpus
HttpClient	outgoing CLIENT spans	dependency corpus
SqlClient	database CLIENT spans	export corpus is redaction-gated
OTLP exporter	transport only	endpoint + export result

PACKAGE HONESTY

Exact OpenTelemetry package versions and supported redaction hooks come from retained lock/restore artifacts. The evidence profile rejects query text or parameters unless a reviewed package-supported filter has sanitized them.

HTTP SERVER SPANS

Automatic instrumentation owns transport; application spans own meaning

HTTP SEPARATION 51-H

Span	Kind	Fields	Owner
HTTP POST /alarms	SERVER	method, route, status	ASP.NET Core
alarm flood commit	INTERNAL	outcome, safe lineage	Alarm application
SQL operation	CLIENT	db.system, safe operation	SqlClient

EXECUTABLE ASSET 51-F - OWNER-LOCAL OPERATION

```
using var activity = NexusActivitySources.AlarmSource.StartActivity(
    SpanNames.AlarmFloodCommit,
    ActivityKind.Internal,
    tags: SafeTags(messageId, context.Business.CorrelationId, "ATTEMPTED"));

var outcome = await handler.CommitAsync(command, context, cancellationTokens);
activity?.SetTag("nexus1.outcome.code", outcome.Code);
if (outcome.IsFailure)
    activity?.SetStatus(ActivityStatusCode.Error);
return outcome;
```

STATUS DISCIPLINE

Expected REJECTED or ABSTAINED business outcomes can remain Status=Unset with an explicit outcome attribute. Error status means the traced operation failed unexpectedly or could not complete.

OUTGOING HTTP

Client spans describe remote attempts, not remote truth

EXECUTABLE ASSET 51-G - BOUNDED HTTPCLIENT ENRICHMENT

```
services.AddOpenTelemetry().WithTracing(builder => builder
    .AddHttpClientInstrumentation(options =>
    {
        options.FilterHttpRequestMessage = request =>
            allowedDependencies.Contains(request.RequestUri!.Host);
        options.EnrichWithHttpRequestMessage = (activity, request) =>
            activity.SetTag("nexus1.dependency.class", Classify(request));
        options.RecordException = false;
    }));
```

HTTP CLIENT EXCLUSIONS 51-I

Excluded	Retained alternative
query string	route/template and dependency class
Authorization/Cookie	no credential attributes
request/response body	status and bounded outcome
raw exception	error.type classification
unbounded peer address	approved service identity

REMOTE-CALL RULE

A CLIENT span ending successfully proves only that this client observed a completed protocol attempt. The remote service's business commit requires its own evidence.

DATABASE SPANS

Trace provider latency without turning SQL into data export

DATABASE POLICY 51-J

Attribute	Disposition	Reason
db.system.name	allow	bounded provider class
db.operation.name	allow	SELECT/INSERT/COMMIT class
server.address	environment policy	topology disclosure
db.query.summary	allow-listed only	low-cardinality intent
db.query.text	reject unless verified sanitized	values/schema may leak
connection string	never	credential disclosure

EXECUTABLE ASSET 51-H - COMMIT WRAPPER

```
using var activity = source.StartActivity(
    "alarm transaction commit",
    ActivityKind.Internal,
    tags: new ActivityTagsCollection
    {
        ["nexus1.transaction.owner"] = "alarm-management",
        ["nexus1.transaction.effects"] = 3
    });

await db.SaveChangesAsync(cancellationToken);
activity?.SetTag("nexus1.outcome.code", "COMMITTED");
```

NO DOUBLE TRUTH

The custom commit span groups the owner operation; SqlClient spans diagnose provider calls. Neither replaces the committed rows, outbox fingerprint or transaction receipt.

OUTBOX DIAGNOSTIC SNAPSHOT

Capture optional trace coordinates at the business commit

EXECUTABLE ASSET 51-I - PRODUCER CONTEXT SNAPSHOT

```
public sealed record ProducerTraceSnapshot(  
    string TraceId,  
    string SpanId,  
    byte TraceFlags,  
    string? TraceState);  
  
public static ProducerTraceSnapshot? Capture(Activity? activity) =>  
    activity is null ? null : new(  
        activity.TraceId.ToHexString(),  
        activity.SpanId.ToHexString(),  
        (byte)activity.ActivityTraceFlags,  
        activity.TraceStateString);
```

SNAPSHOT RULES 51-K

Rule	Effect
nullable	sampling/listener state cannot block commit
stored beside outbox	dispatcher does not read stale ambient context
not in envelope fingerprint	diagnostic changes do not mutate fact
used as link, not authority	business cause remains CausationId
bounded fields only	no baggage or actor claims

CHAPTER 49 COMPATIBILITY

MessageId, CorrelationId and CausationId remain stable public lineage. Trace context is optional attempt-local diagnostics stored and propagated under a separate contract.

DISPATCHER SPAN

A delayed publish is a new attempt linked to its origin

EXECUTABLE ASSET 51-J - PUBLISH ATTEMPT WITH LINK

```

var links = row.ProducerTrace is { } snapshot
? new[] { new ActivityLink(Parse(snapshot)) }
: Array.Empty<ActivityLink>();

using var activity = NexusActivitySources.MessagingSource.StartActivity(
    SpanNames.PublishAlarmFlood,
    ActivityKind.Producer,
    parentContext: default,
    tags: MessagingTags.ForPublish(row),
    links: links);

propagator.Inject(
    new PropagationContext(activity?.Context ?? default, Baggage.Current),
    properties.Headers,
    AmqpCarrier.Set);

```

PUBLISH SEMANTICS 51-L

Value	First attempt	Retry
MessageId/business lineage	stable	same
publish span	new	new
AttemptId	new	new
link to producer commit	optional	same retained snapshot
injected traceparent	current publish context	new attempt context

NO AMBIENT REQUEST

The dispatcher never assumes `Activity.Current` belongs to the original HTTP request. It starts an owned attempt and uses the persisted diagnostic snapshot only as an optional link.

AMQP PROPAGATION

Trace context travels in allow-listed headers

EXECUTABLE ASSET 51-K - BOUNDED CARRIER

```

public static void Set(
    IDictionary<string, object?> headers,
    string key,
    string value)
{
    if (key is not ("traceparent" or "tracestate")) return;
    if (value.Length > (key == "traceparent" ? 55 : 512))
        throw new TraceCarrierRejected("trace-header-oversized");
    headers[key] = Encoding.ASCII.GetBytes(value);
}

public static IEnumerable<string> Get(
    IReadOnlyDictionary<string, object?> headers,
    string key) => AmqpCarrier.ReadSingleAscii(headers, key);

```

CARRIER CONTRACT 51-M

Header	Count	Trust	Never contains
traceparent	0..1	validate W3C grammar	business identity
tracestate	0..1	only with valid parent	owner policy
baggage	absent by default	deny	token/actor/role

SEMANTIC-CONVENTION STATUS

The current OpenTelemetry messaging conventions are still marked Development. NEXUS-1 pins the convention version and migration mode in its evidence manifest rather than calling the shape permanently stable.

CONSUMER EXTRACTION

Extract diagnostics only after carrier bounds are enforced

EXECUTABLE ASSET 51-L - PROCESS SPAN

```
var extracted = propagator.Extract(
    default,
    delivery.Headers,
    AmqpCarrier.Get);

var parent = extracted.ActivityContext.IsRemote
    ? extracted.ActivityContext
    : default;

using var activity = NexusActivitySources.MessagingSource.StartActivity(
    SpanNames.ProcessAlarmFlood,
    ActivityKind.Consumer,
    parent,
    tags: MessagingTags.ForProcess(admittedEnvelope, delivery));

await handler.HandleAsync(admittedEnvelope, explicitContext, cancellationTokens);
```

ADMISSION ORDER 51-N

Step	Failure meaning
bound header count/bytes	carrier abuse rejected
validate envelope/fingerprint	business fact rejected
authenticate producer path	origin rejected
extract trace context	diagnostic parent accepted or absent
create explicit context + span	business and diagnostic ledgers joined

INDEPENDENCE RULE

Malformed or absent trace context can start a new diagnostic root under route policy. It cannot rewrite the accepted envelope's CorrelationId or CausationId.

MESSAGING SPAN KINDS

Creation, sending, processing and settlement are different operations

MESSAGING MODEL 51-O

Operation	Kind	Parent/link rule	NEXUS-1 use
create/send	PRODUCER	creation context	outbox publish
send without creation parent	CLIENT	client request	adapter-specific
receive	CLIENT	broker request	pull clients only
process	CONSUMER	message creation context	worker handler
settle	CLIENT	process child	ack/nack when separately useful

The Phase-0 RabbitMQ path focuses on one PRODUCER publish attempt and one CONSUMER process span. Settlement is an event/attribute unless it is a separately timed remote operation. Waiting on a subscription is not traced per idle loop.

CONVENTION PINNING

The manifest records OpenTelemetry semantic-convention version 1.44.0 as the reviewed source for this manuscript date, but runtime evidence must record the version actually restored and emitted.

BATCH AND FAN-IN

One parent cannot represent many accepted inputs

EXECUTABLE ASSET 51-M - BOUNDED LINKS

```

var accepted = batch.Take(options.MaxLinkedInputs).ToArray();
var links = accepted
    .Select(x => new ActivityLink(x.CreationContext, new ActivityTagsCollection
    {
        ["nexus1.message.id"] = x.MessageId.Value
    }))
    .ToArray();

using var activity = source.StartActivity(
    "root-cause evidence batch",
    ActivityKind.Consumer,
    parentContext: default,
    links: links);

```

FAN-IN RULES 51-P

Concern	Rule
primary business cause	Chapter 49 policy chooses and records it
additional diagnostic relations	ActivityLink per accepted input
link count	bounded by explicit configuration
overflow	count + classification, not silent truncation
late input	new attempt/span under owner policy

IMMUTABLE LINKS

Activity links are supplied when the span starts. Batch admission therefore freezes the accepted diagnostic relation set before processing.

BACKGROUND WORK

Each work item owns its scope, attempt and span

EXECUTABLE ASSET 51-N - SCOPED WORKER

```

public async Task ProcessAsync(WorkItem item, CancellationToken stop)
{
    await using var scope = scopeFactory.CreateAsyncScope();
    var context = contextFactory.Create(item);
    using var activity = repairSource.StartActivity(
        "repair outbox gap",
        ActivityKind.Internal,
        parentContext: default,
        links: context.DiagnosticLinks,
        tags: RepairTags.For(item));

    try { await repair.RunAsync(item, context, stop); }
    catch (Exception ex) { SafeError.Record(activity, ex); throw; }
}

```

WORKER GATES 51-Q

Hazard	Gate
singleton mutable Activity	one using scope per item
fire-and-forget child task	structured await/owned queue
AsyncLocal leak	post-item Activity.Current assertion
batch channel reuse	item-local span/context
cancellation skips status	finally and bounded stop evidence

SPAN STATUS

Diagnostic error and domain outcome remain orthogonal

STATUS MATRIX 51-R

Domain outcome	Span status	Reason
COMMITTED	Unset	normal completion
DUPLICATE_MATCH	Unset	idempotent normal path
ABSTAINED	Unset	valid domain decision
REJECTED policy	Unset or Error by operation contract	expected refusal
provider timeout	Error	operation failed
unclassified exception	Error	unexpected failure

EXECUTABLE ASSET 51-O - SAFE ERROR RECORDING

```
public static void Record(Activity? activity, Exception exception)
{
    if (activity is null) return;
    var error = SafeErrorClassifier.Describe(exception);
    activity.SetTag("error.type", error.ErrorType);
    activity.SetTag("nexus1.error.code", error.ErrorCode);
    activity.SetStatus(ActivityStatusCode.Error);
}
```

NO RAW EXCEPTION

Exception.Message, stack traces, SQL details and Data dictionaries are omitted from the default trace profile. Chapter 50's classified ErrorDescriptor remains the safe source.

SPAN EVENTS

Events mark a few meaningful instants, not every log line

EVENT CATALOGUE 51-S

Event	When	Attributes
outbox.confirmed	broker confirm observed	attempt, bounded outcome
message.redelivered	delivery flag true	attempt number
retry.scheduled	bounded recovery decision	delay, reason
repair.postcondition	oracle evaluated	verified boolean

EXECUTABLE ASSET 51-P - BOUNDED EVENT

```
activity?.AddEvent(new ActivityEvent(
    "retry.scheduled",
    clock.UtcNow,
    new ActivityTagsCollection
    {
        ["nexus1.retry.attempt"] = attempt,
        ["nexus1.retry.delay_ms"] = delay.TotalMilliseconds,
        ["nexus1.reason.code"] = reasonCode
    }
));
```

VOLUME RULE

Activity events are retained in-memory until export and therefore remain few and bounded. Repeated operational observations belong in the structured-log catalogue.

SAMPLING CONTRACT

Sampling controls telemetry cost, never business execution

SAMPLING PROFILES 51-T

Profile	Decision	Use	Trade-off
evidence	AlwaysOn	deterministic Phase-0 campaign	higher volume
developer	AlwaysOn/console	local diagnosis	not production proof
bounded head	ParentBased ratio	known high volume	error traces may be missed
tail	collector after trace	error/latency selection	stateful operational cost

EXECUTABLE ASSET 51-Q - PROFILE-OWNED SAMPLER

```
Sampler sampler = profile.Name switch
{
    "evidence" => new AlwaysOnSampler(),
    "bounded-head" => new ParentBasedSampler(
        new TraceIdRatioBasedSampler(profile.Ratio)),
    _ => throw new InvalidOperationException("trace-profile-unknown")
};
```

SAMPLING INDEPENDENCE

A non-recording Activity or null StartActivity result must not change handler calls, transaction effects, outbox bytes, settlement or authorization.

HEAD AND TAIL DECISIONS

Early efficiency and late knowledge have different costs

DECISION LEDGER 51-U

Question	Head sampling	Tail sampling
when decided?	at/near trace start	after most/all spans arrive
sees full error/latency?	no	yes, by configured criteria
runtime state?	small	stateful buffering
Phase-0 required?	not for evidence profile	not claimed
failure mode	interesting trace dropped	collector overload/incomplete trace

The book does not prescribe a production sampling rate. That value depends on traffic, diagnostic need, regulation, storage and budget. Any chosen rate belongs in an environment manifest and must be evaluated against known error and latency scenarios.

COMPLETENESS CAVEAT

A sampled trace can still be incomplete because instrumentation, propagation, export or backend ingestion failed. Sampling policy alone cannot certify trace completeness.

EXPORTER BOUNDARY

OTLP transport is replaceable; the span contract is not

EXECUTABLE ASSET 51-R - VALIDATED EXPORTER OPTIONS

```
public sealed record OtlpTraceOptions(
    Uri Endpoint,
    TimeSpan ExportTimeout,
    int MaxQueueSize,
    int MaxExportBatchSize);

public sealed class OtlpTraceOptionsValidator
    : IValidateOptions<OtlpTraceOptions>
{
    public ValidateOptionsResult Validate(string? name, OtlpTraceOptions value) =>
        TraceOptionRules.Validate(value);
}
```

EXPORTER INVARIANTS 51-V

Rule	Failure behavior
HTTPS outside local evidence	startup fails closed
bounded queue/batch/timeout	no unbounded memory/wait
credential from secret provider	never source/image/log/tag
exporter unavailable	business continues; evidence degrades
drop/error counters retained	incomplete dossier cannot say VERIFIED

NO BACKEND CLAIM

An exporter configuration shows intended transport. Only a retained receiver acknowledgement and queried trace corpus prove delivery to a named environment.

COLLECTOR EVIDENCE PROFILE

A small deterministic pipeline is enough for Phase-0 proof

CONFIGURATION ASSET 51-A - TRACE PIPELINE

```
receivers:
  otlp:
    protocols:
      grpc: {}
processors:
  memory_limiter:
    limit_mib: 256
  batch:
    send_batch_size: 256
exporters:
  file/traces:
    path: /evidence/traces.json
service:
  pipelines:
    traces:
      receivers: [otlp]
      processors: [memory_limiter, batch]
      exporters: [file/traces]
```

PROFILE BOUNDARIES 51-W

Provides	Does not provide
reproducible OTLP receiver	production availability
bounded batch/memory config	capacity sizing
retained machine-readable traces	dashboard usability
collector self-evidence	long-term retention policy

CONFIGURATION STATUS

This is a reference asset. Exact Collector image digest, supported exporter syntax and executed startup evidence remain runtime obligations.

COMPLETE TRACE SPECIMEN

One successful case has an explicit structural oracle

EXPECTED TRACE 51-X

#	Span	Kind	Relation	Required join
1	HTTP POST /alarms	SERVER	root	RequestId
2	alarm flood commit	INTERNAL	child of 1	DecisionId
3	SQL commit	CLIENT	child of 2	owner db
4	publish alarm-flood-detected	PRODUCER	linked to 2	MessageId
5	process alarm-flood-detected	CONSUMER	child of 4 context	MessageId
6	root-cause case open	INTERNAL	child of 5	CasId
7	root-cause verdict commit	INTERNAL	child of 6	VerdictId

MULTIPLE TRACE IDS ALLOWED

If delayed dispatch starts a linked trace, the structural oracle becomes a trace set joined by ActivityLink and MessageId. Completeness means the documented graph exists, not that every operation shares one TraceId.

SUCCESSFUL TRACE INTERPRETATION

The waterfall answers execution questions

QUESTIONS ANSWERED 51-Y

Question	Evidence
where did latency accumulate?	span durations by boundary
which dependency failed?	kind, status and error.type
did consumer context join?	parent/link and MessageId
which build emitted work?	resource build identity
did business commit?	not by trace - cross-check owner state

BROKEN TRACE EXPERIMENT

Remove propagation and preserve every business input

FAULT ASSET 51-A - DELIBERATE CARRIER OMISSION

```
public sealed class DropTraceContextFault : IMessageTraceInjector
{
    public void Inject(
        ActivityContext context,
        IDictionary<string, object?> headers)
    {
        // Deliberate experiment: business envelope and MessageId are untouched.
        headers.Remove("traceparent");
        headers.Remove("tracestate");
    }
}
```

CONTROLLED VARIABLES 51-Z

Held constant	Changed
envelope bytes/fingerprint	trace headers omitted
MessageId/CorrelationId/CausationId	consumer gets no remote parent
database and broker versions	fault profile identity
sampler AlwaysOn	one propagation seam
test input and expected state	trace topology only

SAFETY OF EXPERIMENT

The fault adapter operates only in an isolated evidence environment. It cannot be enabled by an untrusted request and is absent from production composition.

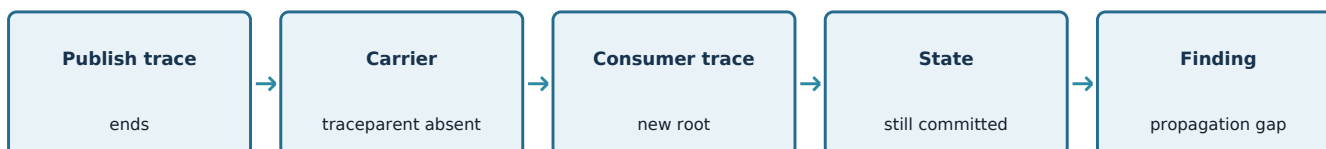
BROKEN TRACE RESULT

The missing bridge reveals a propagation defect

EXPECTED BROKEN TOPOLOGY 51-AA

Observation	Interpretation	Not justified
publish trace ends at broker	producer path exported	message lost
consumer starts a new root	no accepted creation context	different business case
same MessageId in both traces	diagnostic join still possible	parentage valid
owner state and receipt exist	business processing committed	trace was complete
fault manifest says headers dropped	controlled cause identified	real network fault

PROOF PATH 51-C - BROKEN DIAGNOSTIC GRAPH

**MISSING-SPAN RULE**

Absence narrows hypotheses only after listener registration, sampling, exporter and ingestion are checked. A missing span alone never proves code did not execute.

MISSING SPAN TAXONOMY

Four telemetry failures can look identical in a UI

DIAGNOSTIC MATRIX 51-AB

Cause	Local symptom	Discriminator	Business effect
source not registered	StartActivity null/not recorded	source inventory test	none
context not propagated	new consumer root	carrier capture	none by itself
sampler drops	non-recording context	sampling manifest	none
exporter drops	span created locally	export counters	none
backend ingestion gap	export accepted, query absent	receiver evidence	none
operation skipped	no durable/log evidence	business oracle	scenario-specific

INVESTIGATION ORDER

Check authoritative state and MessageId lineage first, then local recording, propagation, sampling, export and backend ingestion. Do not begin with a waterfall screenshot.

BROKEN BUSINESS PATH

A real failure needs owner evidence too

If consumer state and inbox receipt are absent, the same trace gap may coexist with a processing failure. The investigation distinguishes them by broker delivery evidence, structured logs, database state, retry/dead-letter records and the controlled fault manifest.

IN-MEMORY TRACE TESTS

Verify topology without depending on a UI

VERIFICATION ASSET 51-A - ACTIVITYLISTENER CAPTURE

```
using var listener = new ActivityListener
{
    ShouldListenTo = source => expectedSources.Contains(source.Name),
    Sample = (ref ActivityCreationOptions<ActivityContext> _) =>
        ActivitySamplingResult.AllDataAndRecorded,
    ActivityStopped = activity => captured.Add(ActivitySnapshot.From(activity))
};
ActivitySource.AddActivityListener(listener);

await scenario.RunAsync();
traceValidator.Validate(captured).Should().BeEmpty();
```

LISTENER ASSERTIONS 51-AC

Assert	Failure caught
source name/version	unregistered/renamed instrumentation
kind and name	wrong boundary semantics
parent/link graph	propagation or fan-in loss
safe attributes only	leak/cardinality regression
status/outcome pair	domain/error conflation
stop time/duration	undisposed Activity

REGISTRATION TESTS

Zero custom spans is a failed test, not a green run

VERIFICATION ASSET 51-B - EXPECTED SOURCE INVENTORY

```
[Fact]
public async Task Every_expected_source_emits_a_recorded_span()
{
    await productionPathScenario.RunAsync();
    var emitted = captured.Select(x => x.SourceName).ToHashSet();

    emitted.Should().BeSupersetOf(new[]
    {
        NexusActivitySources.Alarm,
        NexusActivitySources.RootCause,
        NexusActivitySources.Messaging
    });
    captured.Should().NotBeEmpty();
}
```

ZERO-DISCOVERY GATES 51-AD

Count	Required
expected sources	manifest value > 0
discovered tests	equals expected test catalogue
captured spans	meets scenario minimum
exported spans	equals accepted minus explained drops
complete traces	exact named scenario count

FALSE-GREEN PREVENTION

A missing package, source-name typo or disabled listener cannot produce a successful evidence verdict merely because the application returned 202.

PROPAGATION TESTS

The exact message creation context must survive the carrier

VERIFICATION ASSET 51-C - PRODUCER TO CONSUMER

```
[Fact]
public async Task Consumer_uses_propagated_creation_context()
{
    var delivery = await harness.PublishAndCaptureAsync(envelope);
    var carrier = harness.ReadTraceParent(delivery.Headers);
    var result = await harness.ConsumeAndCaptureAsync(delivery);

    carrier.Should().NotBeNull();
    result.Consumer.ParentSpanId.Should().Be(result.Producer.SpanId);
    result.Consumer.TraceId.Should().Be(result.Producer.TraceId);
    result.Consumer.MessageId.Should().Be(envelope.MessageId);
}
```

CARRIER MUTATIONS 51-AE

Mutation	Expected topology	Business rule
traceparent absent	new consumer root	lineage unchanged
traceparent invalid	reject/ignore by route policy	no inferred cause
tracestate alone	discard	handler policy unchanged
duplicate header	bounded rejection	no concatenation
oversized header	bounded rejection	no allocation escape

SAMPLING TESTS

Recorded and unrecorded executions produce identical truth

VERIFICATION ASSET 51-D - SAMPLER INDEPENDENCE

```
[Theory]
[InlineData(true)]
[InlineData(false)]
public async Task Sampling_never_changes_business_effect(bool record)
{
    harness.UseSampler(record ? new AlwaysOnSampler() : new AlwaysOffSampler());
    var result = await harness.ExecuteAlarmCaseAsync();

    result.Outcome.Should().Be("COMMITTED");
    await ownerState.ShouldContainExpectedCase();
    await outbox.ShouldContainExactEnvelope();
    settlement.Should().Be("ACKED");
}
```

INDEPENDENCE ORACLES 51-AF

Surface	Must match across samplers
Domain decision	outcome and decision ID
database	state, receipt and outbox
broker	published bytes and settlement
security	allow/deny result
logs	business event fields; trace locators optional

LEAK AND CARDINALITY TESTS

No hostile value becomes a span name or unsafe attribute

VERIFICATION ASSET 51-E - SENSITIVE CORPUS

```
[Theory]
[MemberData(nameof(HostileAndSensitiveValues))]
public async Task Trace_corpus_contains_no_forbidden_value(string value)
{
    await harness.SendRejectedRequestAsync(value);
    var serialized = exporter.SerializeCapturedSpans();

    serialized.Should().NotContain(value);
    leakScanner.FindForbidden(serialized).Should().BeEmpty();
    captured.Select(x => x.DisplayName)
        .Should().OnlyContain(SpanNamePolicy.IsApproved);
}
```

BUDGET ASSERTIONS 51-AG

Budget	Gate
attributes per span	<= reviewed maximum
string length	per-attribute bound
events per span	finite event catalogue
links per span	configured maximum
span names	closed operation catalogue
payload/header/claim fields	zero

EXPORTER SCAN

Tests inspect the serialized exporter representation, not only in-memory Activity tags, so adapter enrichment cannot reintroduce forbidden material.

CONCURRENCY AND SCOPE TESTS

Parallel deliveries cannot exchange parentage

VERIFICATION ASSET 51-F - ITEM-LOCAL TOPOLOGY

```
[Fact]
public async Task Parallel_deliveries_keep_distinct_trace_context()
{
    await worker.ProcessConcurrentlyAsync(deliveryA, deliveryB);

    trace.ForMessage("message-a").Should().OnlyContain(
        s => s.TraceId == producerA.TraceId);
    trace.ForMessage("message-b").Should().OnlyContain(
        s => s.TraceId == producerB.TraceId);
    Activity.Current.Should().BeNull();
}
```

SCOPE MUTATIONS 51-AH

Mutation	Expected
interleaved awaits	parent remains item-local
handler exception	Activity disposed and status Error
cancellation	span stops; next item clean
batch reuse	links reflect accepted inputs only
fire-and-forget	architecture gate rejects

EXPORTER FAILURE TESTS

Telemetry loss degrades evidence, not business state

VERIFICATION ASSET 51-G - UNAVAILABLE EXPORTER

```
[Theory]
[InlineData(ExportFault.EndpointUnavailable)]
[InlineData(ExportFault.QueueFull)]
[InlineData(ExportFault.Timeout)]
public async Task Export_failure_preserves_commit(ExportFault fault)
{
    exporter.Inject(fault);
    var result = await scenario.ExecuteAsync();

    result.Outcome.Should().Be("COMMITTED");
    await database.ShouldContainCommittedState();
    telemetryHealth.ShouldReport(fault);
    evidence.Verdict.Should().Be("DEGRADED");
}
```

FAILURE EVIDENCE 51-AI

Claim	Oracle
business unaffected	provider-backed state and outbox
span created locally	ActivityListener capture
export failed	exporter callback/counter
loss bounded	queue/drop count and timeout
verdict honest	manifest DEGRADED, never VERIFIED

COMPLETE-TRACE VALIDATOR

Topology is checked as a graph, not a screenshot

VERIFICATION ASSET 51-H - STRUCTURAL ORACLE

```
public IReadOnlyList<string> ValidateComplete(TraceCorpus corpus)
{
    var errors = new List<string>();
    RequireSpan(corpus, "HTTP POST /alarms", ActivityKind.Server, errors);
    RequireChild(corpus, "alarm flood commit", "HTTP POST /alarms", errors);
    RequireLinked(corpus, "publish alarm-flood-detected",
        "alarm flood commit", errors);
    RequireMessageParent(corpus, "process alarm-flood-detected",
        "publish alarm-flood-detected", errors);
    RequireBusinessJoin(corpus, "nexus1.message.id", errors);
    return errors;
}
```

GRAPH INVARIANTS 51-AJ

Invariant	Why
unique SpanId within trace	identity integrity
parent exists or declared remote	no orphan child
link target retained or external	async relation explicit
time interval non-negative	valid lifecycle
expected source/version	instrumentation provenance
business joins agree	no identifier mutation

BROKEN-TRACE VALIDATOR

The fault campaign must fail the complete oracle for one reason

VERIFICATION ASSET 51-I - EXPECTED DEFECT

```
[Fact]
public async Task Dropped_carrier_produces_declared_trace_gap()
{
    var result = await faultLab.RunAsync(DropTraceContextFault.Id);

    result.BusinessOutcome.Should().Be("COMMITTED");
    result.CompleteTraceErrors.Should().ContainSingle(
        "consumer-creation-context-missing");
    result.Consumer.ParentSpanId.Should().BeNull();
    result.Consumer.MessageId.Should().Be(result.Producer.MessageId);
    result.UnexpectedTraceErrors.Should().BeEmpty();
}
```

EXPERIMENT EXIT GATES 51-AK

Gate	Required result
fault identity	exact adapter/config digest
only changed variable	trace carrier omission
business state	expected commit
trace defect	one classified missing bridge
recovery	normal profile restores complete graph

EVIDENCE MANIFEST

A trace claim names its build, configuration and losses

EXECUTABLE ASSET 51-S - TRACE EVIDENCE RECORD

```
public sealed record TraceEvidenceManifest(
    string BuildIdentity,
    string RuntimeIdentity,
    string InstrumentationCatalogueSha256,
    string PackageLockSha256,
    string SemanticConventionVersion,
    string SamplerProfile,
    string ExporterProfileSha256,
    int ExpectedSpans,
    int CapturedSpans,
    int ExportedSpans,
    int DroppedSpans,
    string CompleteTraceSha256,
    string BrokenTraceSha256,
    string Verdict,
    ImmutableArray<string> Residuals);
```

MANIFEST CLAIMS 51-AL

Field	Prevents
build/package digests	stale binary/package evidence
convention version	silent semantic drift
sampler/exporter profile	unknown omission policy
captured/exported/dropped	loss hidden as completeness
complete/broken corpus digest	screenshot-only proof

CURRENT VERDICT

SPECIFIED | RUNTIME PENDING. The manuscript defines assets and gates; no fresh repository build, collector run, exporter receipt or trace corpus is claimed.

EVIDENCE VALIDATOR

Fail closed on zero discovery, loss or stale identity

VERIFICATION ASSET 51-J - MANIFEST GATE

```

public static EvidenceVerdict Verify(TraceEvidenceManifest m)
{
    if (m.ExpectedSpans <= 0 || m.CapturedSpans <= 0)
        return EvidenceVerdict.Invalid("zero-span-evidence");
    if (m.CapturedSpans != m.ExportedSpans + m.DroppedSpans)
        return EvidenceVerdict.Invalid("span-accounting-mismatch");
    if (m.DroppedSpans > 0)
        return EvidenceVerdict.Degraded("trace-export-loss");
    if (string.IsNullOrEmpty(m.CompleteTraceSha256) ||
        string.IsNullOrEmpty(m.BrokenTraceSha256))
        return EvidenceVerdict.Incomplete("trace-corpus-missing");
    return EvidenceVerdict.Verified(m.BuildIdentity);
}

```

RETAINED BUNDLE 51-AM

Artifact	Required identity
instrumentation catalogue	source/name/kind/attributes digest
package lock	restored package versions
collector/exporter config	exact profile digest
complete trace corpus	scenario + graph verdict
broken trace corpus	fault + expected defect
test results	discovery count + environment

OPERATIONAL INVESTIGATION

Start with the business case, then open the diagnostic graph

INVESTIGATION SEQUENCE 51-AN

Step	Question	Evidence
1	what business fact/case?	MessageId/CasId and owner state
2	which attempts observed it?	structured logs + AttemptId
3	which traces join?	TraceId/SpanId/ActivityLink
4	where is the gap/error?	span graph/status/duration
5	is telemetry complete?	sampler/exporter/backend manifest
6	what may be concluded?	bounded evidence verdict

QUERY INVARIANT

Use low-cardinality span names and safe business locators. Never search by raw payload, credential, exception message or timestamp coincidence.

LOG CORRELATION

Chapter 50 events carry optional trace locators

When an Activity is present, the logging provider records TraceId and SpanId beside the stable EventName, MessageId and outcome fields. When it is absent or unsampled, the structured business event remains meaningful and joinable through Chapter 49 lineage.

IMPLEMENTATION SEQUENCE

Freeze topology before adding an exporter

SEQUENCE 51-AO

Step	Deliverable	Exit gate
1	ADR-051 + span catalogue	owners approve boundaries
2	ActivitySource inventory	registration test passes
3	safe attribute catalogue	leak/cardinality test passes
4	HTTP/database instrumentation	component graph passes
5	outbox/message propagation	carrier test passes
6	background/link policy	scope/fan-in tests pass
7	sampler/exporter profile	failure isolation passes
8	complete/broken campaigns	manifest verifies

ORDER INVARIANT

A backend cannot repair missing instrumentation or incorrect meaning. First prove the local graph and carrier; then prove export and ingestion.

CHANGE CONTROL

Instrumentation evolution is a compatibility decision

Renaming a span, changing kind, removing a required attribute, changing parent/link topology or adopting a new semantic-convention version requires a catalogue diff and updated golden graph. Provider/UI changes alone must preserve the logical corpus.

STANDARDS ALIGNMENT

Standards guide mapping; NEXUS-1 still owns meaning

SOURCE LEDGER 51-AP

Source	Applied rule	NEXUS-1 extension
.NET ActivitySource	static unique source, nullable Activity	source inventory gate
OpenTelemetry traces	span, context, links, status, kind	business-ledger separation
OTel messaging	creation context and operation kinds	pinned development convention
OTel sampling	head/tail trade-offs	AlwaysOn evidence profile
Chapters 49-50	lineage and safe log fields	typed trace projection

VERIFICATION BOUNDARY

Standards alignment makes traces interoperable. Only retained NEXUS-1 graphs, faults, package/config identities and executed tests prove this runtime.

ANTI-PATTERN REVIEW

Five attractive trace claims remain invalid

REJECTED CLAIMS 51-AQ

Claim	Correction
same Traceld means same business transaction	use business lineage
no span means no execution	classify telemetry gap
Status=Ok proves commit	cross-check owner state
parent span is direct business cause	use CausationId/edge
sampled trace is complete	validate instrumentation/export corpus

COMPLETION LEDGER

What Chapter 51 now specifies

COMPLETION LEDGER 51-AR

Obligation	Asset	Status
HTTP/database coverage	auto + owner spans	SPECIFIED
messaging propagation	bounded carrier + process span	SPECIFIED
background/fan-in	item scope + ActivityLink policy	SPECIFIED
safe names/attributes	catalogues and static gates	SPECIFIED
sampling/export failure	profiles and independence tests	SPECIFIED
complete trace	structural graph oracle	SPECIFIED
broken trace	controlled carrier fault	SPECIFIED
runtime proof	executed exporter/collector corpus	PENDING

OPEN RESIDUALS 51-AS

Residual	Consequence
no executed repository checkpoint	listings remain compile-oriented
no retained package restore	runtime APIs/versions unproved
no collector/exporter run	delivery and loss behavior unproved
no production sampling profile	cost/completeness trade-off open
messaging conventions still evolving	version/migration must remain pinned
no production backend	query and retention behavior unproved

EVIDENCE VERDICT

SPECIFIED | RUNTIME PENDING. Instrumentation topology, propagation, safe attributes, sampling and two trace campaigns are fully designed; no fresh runtime or production trace evidence is claimed.

HANDOFF TO CHAPTER 52

Metrics, Service Objectives, and Dashboards

Chapter 52 will aggregate requests, messages, outbox/inbox state, queue age, errors, retries, projection lag and workflow duration into bounded metrics. It will define explicit educational objectives and a small dashboard without inventing production SLOs or turning high-cardinality trace locators into metric labels.

BOUNDARY CARRIED FORWARD 51-AT

Chapter 51 leaves	Chapter 52 must add
low-cardinality operation names	metric instruments and bounded labels
span status/duration	error and latency aggregations
message/queue seam catalogue	outbox, inbox and queue-age measures
sampling-aware trace evidence	metric collection independence
runtime evidence pending	dashboard and objective campaign

NEXT INVARIANT

A metric summarizes many observations under a bounded label set. It never carries a MessageId, CaseId, TraceId, payload or other unbounded locator.

FINAL CHAPTER RULE

A missing span is a finding, not a verdict

NEXUS-1 traces explain observed execution by joining carefully named spans, safe attributes, explicit context propagation and asynchronous links. When the graph breaks, the evidence process classifies whether instrumentation, carrier, sampling, export, ingestion or the operation itself is missing.

PART IX CONTINUITY

Business lineage identifies the case; structured logs preserve domain observations; traces explain attempts and gaps; metrics will bound aggregate health; resilience experiments will prove recovery remains visible.

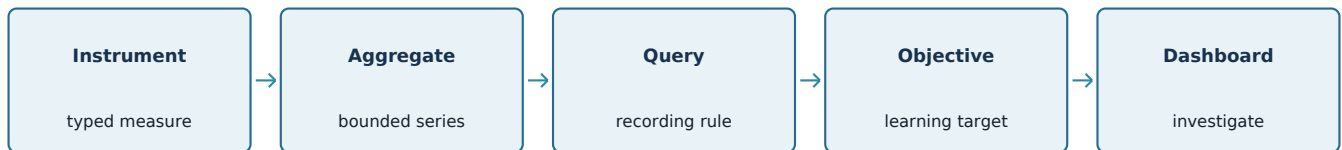
PART IX - OBSERVABILITY, RESILIENCE, AND FAILURE EVIDENCE

52. Metrics, Service Objectives, and Dashboards

A useful metric compresses many observations into a bounded question. It reveals a trend or breach without pretending to identify the case, prove the cause, or define a production promise.

CHAPTER DECISION

Define request, message, outbox, inbox, queue-age, error, retry, projection-lag and workflow-duration metrics with explicit units, temporality and bounded labels. Turn them into educational objectives and a small dashboard, then verify one complete and one deliberately broken metric campaign.

PROOF PATH 52-A - FROM MEASUREMENT TO ACTION**GUARANTEE LEDGER 52-A**

Boundary	Allowed claim	Required proof	Still forbidden
measurement	one value recorded	instrument + unit	business fact exists
series	bounded aggregate changed	labels + temporality	specific case failed
objective	campaign met a declared target	window + query + corpus	production SLO
dashboard	view matches query results	provisioned artifact	root cause

CHAPTER EXIT CRITERIA 52-B

Question	Required answer
Are all labels bounded?	yes - catalogue and runtime distinct-set gate
Can traces be sampled independently?	yes - metrics do not depend on Activity recording
Can no-data be distinguished?	yes - staleness and scrape/export health are explicit
Are objectives honest?	yes - educational, campaign-scoped and non-production

ORDINARY DEVELOPER PROBLEM

A green average can hide a growing distributed backlog

The request dashboard stays green because HTTP latency is low. Meanwhile outbox rows age, retries amplify broker calls, the projection falls behind and one workflow takes minutes. A team that watches only averages and process health sees no failure until users report stale results.

FAILURE LEDGER 52-1

Shortcut	Failure	Correction
one request counter	async seams disappear	owner seam catalogue
average duration	tail and multimodal latency hidden	histogram + thresholds
queue depth only	oldest item can starve	age and depth together
TraceId as label	series explosion	exemplar or investigation link
dashboard number as truth	stale/no-data looks healthy	freshness panel + query tests
99.9% copied from elsewhere	invented promise	campaign-derived learning objective

PLAIN-LANGUAGE PRINCIPLE

Metrics answer how often, how many and how long across a population. Logs and traces answer which observation or path deserves investigation.

CHAPTER BOUNDARY

Aggregate health now; resilience policy next

This chapter owns instruments, views, queries, learning objectives, dashboard panels and their evidence. Chapter 53 will decide where timeout, retry, circuit breaker and bulkhead policies are justified. A red panel does not authorize an automatic retry policy.

DECISION RECORD

ADR-052 - Metrics are a bounded aggregate ledger

NEXUS-1 records measurements only through reviewed instruments. Each instrument declares its semantic owner, type, unit, allowed attributes, collection semantics and evidence query. Aggregation is diagnostic compression; it never becomes business authority.

DECISION LEDGER 52-2

Ledger	Identity	Owns	Cannot decide
domain state	aggregate key	durable truth	metric sample
structured log	event + locators	named observation	population rate
trace	trace/span graph	attempt path	unsampled population
metric	name + bounded attributes	aggregate behavior	case cause
objective	spec + implementation + window	campaign result	SLA

DECISION IDENTITY

ADR-052 is retained as canonical text and hashed by the evidence build. No runtime digest is claimed until the repository checkpoint executes.

INVARIANT CARRIED FROM CHAPTER 51

No unbounded locator enters a measurement label

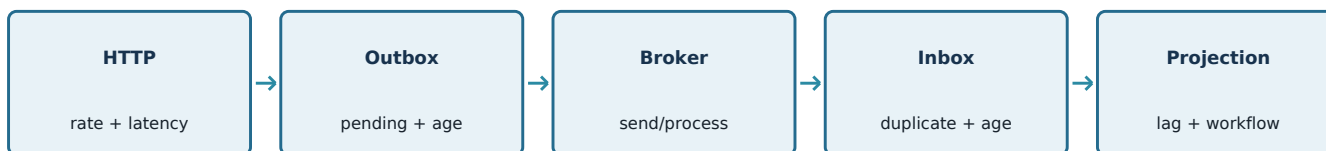
FORBIDDEN LABEL CATALOGUE 52-A

Forbidden	Why	Use instead
Messageld / Caseld	one series per case	log or trace lookup
Traceld / Spanld	one series per attempt	exemplar when supported
raw URL / queue	attacker/runtime controlled	route or destination class
exception message	secret and cardinality leak	bounded error.type
payload / claim	privacy and unbounded content	reviewed outcome class

MEASUREMENT TOPOLOGY

Every seam needs a question before it needs a metric

PROOF PATH 52-B - DISTRIBUTED HEALTH SURFACE



QUESTION CATALOGUE 52-B

Surface	Question	Metric form
request	are edge attempts completing?	count + duration
message	are send/process attempts healthy?	count + duration
outbox	is committed work waiting?	pending + oldest age
inbox	are duplicates or failures accumulating?	outcome count + oldest age
queue	is the oldest accepted item progressing?	depth + age
projection	how stale is the read model?	time + position lag
workflow	how long from accepted start to terminal fact?	duration histogram

MEASUREMENT RULE

A metric exists because a named engineering question needs an aggregate answer. Instrumenting everything first produces cost, ambiguity and dashboards without ownership.

INSTRUMENT CATALOGUE

Name, type and unit form one reviewed contract

CORE INSTRUMENTS 52-C

Instrument	Type	Unit	Owner
nexus1.edge.requests	Counter<long>	{request}	Edge
nexus1.message.attempts	Counter<long>	{attempt}	Messaging
nexus1.message.duration	Histogram<double>	s	Messaging
nexus1.outbox.pending	ObservableGauge<long>	{item}	Outbox
nexus1.outbox.oldest_age	ObservableGauge<double>	s	Outbox
nexus1.inbox.outcomes	Counter<long>	{message}	Inbox
nexus1.projection.lag	ObservableGauge<double>	s	Projection
nexus1.workflow.duration	Histogram<double>	s	Workflow owner

INSTRUMENT CHOICE 52-D

Intent	Instrument	Constraint
monotonic occurrences	Counter	record positive increments
current non-additive state	ObservableGauge	fresh bounded callback
distribution	Histogram	unit and useful buckets
in-flight value	UpDownCounter	balanced add/remove

NAMING RULE

Names are lowercase dotted identifiers, globally meaningful without relying on a backend-specific prefix. Units use UCUM-compatible values such as s, B and {item}.

.NET METRIC OWNER

One DI-managed instrument set serves the process lifetime

EXECUTABLE ASSET 52-A - INSTRUMENT OWNER

```
public sealed class NexusRuntimeMetrics
{
    public const string MeterName = "Nexus1.Runtime";
    public Counter<long> Requests { get; }
    public Counter<long> MessageAttempts { get; }
    public Histogram<double> MessageDuration { get; }
    public Histogram<double> WorkflowDuration { get; }

    public NexusRuntimeMetrics(IMeterFactory factory)
    {
        var meter = factory.Create(MeterName);
        Requests = meter.CreateCounter<long>("nexus1.edge.requests", "{request}");
        MessageAttempts = meter.CreateCounter<long>("nexus1.message.attempts", "{attempt}");
        MessageDuration = meter.CreateHistogram<double>("nexus1.message.duration", "s");
        WorkflowDuration = meter.CreateHistogram<double>("nexus1.workflow.duration", "s");
    }
}
```

LIFETIME PROOF 52-E

Object	Lifetime	Gate
NexusRuntimeMetrics	singleton	one registration
Meter	factory/container owned	not created per request
Instrument	process lifetime	catalogue identity unique
measurement tags	per observation	stable key order

NO-LISTENER BEHAVIOR

Recording is safe when no reader subscribes. Metric availability can never be a precondition for business execution.

BOUNDED ATTRIBUTE VOCABULARY

Callers choose typed classes, not arbitrary strings

EXECUTABLE ASSET 52-B - BOUNDED TAGS

```
public readonly record struct MetricLabels(
    string Operation,
    string Outcome,
    string Component)
{
    public TagList ToTagList() => new()
    {
        { "nexus1.operation", Operation },
        { "nexus1.outcome", Outcome },
        { "nexus1.component", Component }
    };
}

public static class MetricVocabulary
{
    public static readonly FrozenSet<string> Operations =
        new[] { "accept-alarm", "dispatch-outbox", "process-alarm",
            "commit-verdict", "repair-gap" }.ToFrozenSet();
    public static readonly FrozenSet<string> Outcomes =
        new[] { "succeeded", "rejected", "abstained", "failed" }
            .ToFrozenSet();
}
```

LABEL CATALOGUE 52-F

Key	Maximum domain	Owner
nexus1.operation	5 reviewed values	architecture catalogue
nexus1.outcome	4 reviewed values	application outcome map
nexus1.component	7 service classes	runtime manifest
error.type	12 classified values	failure taxonomy
messaging.destination.name	6 logical names	contract registry

ADMISSION GATE

A measurement with an unknown label value is rejected by the instrumentation adapter and counted once under `telemetry_rejected`, never admitted as a new series.

CARDINALITY BUDGET

The series ceiling is calculated before traffic arrives

BUDGET WORKSHEET 52-G

Metric	Label domains	Upper bound
edge.requests	5 operations x 4 outcomes	20
message.attempts	5 operations x 4 outcomes x 2 directions	40
message.duration	5 operations x 4 outcomes	20 x buckets
outbox gauges	7 components x 3 states	21 each
projection.lag	4 projection classes	4
workflow.duration	3 workflow classes x 4 outcomes	12 x buckets

EXECUTABLE ASSET 52-C - FAIL-CLOSED BUDGET

```
public static int Product(params IReadOnlyCollection<string>[] domains)
{
    checked
    {
        var total = 1;
        foreach (var domain in domains) total *= domain.Count;
        return total;
    }
}

Assert.True(Product(Operations, Outcomes) <= 64);
Assert.DoesNotContain("message.id", ApprovedMetricTagKeys);
Assert.DoesNotContain("trace.id", ApprovedMetricTagKeys);
```

SDK GUARD IS SECONDARY

A configured SDK cardinality limit protects memory after a mistake. It does not make an unbounded label design acceptable; overflow is itself a telemetry defect.

OPENTELEMETRY COMPOSITION

The host subscribes only to approved meters and views

EXECUTABLE ASSET 52-D - METRIC REGISTRATION

```
services.AddSingleton<NexusRuntimeMetrics>();

services.AddOpenTelemetry()
    .ConfigureResource(r => r.AddService(manifest.ServiceName))
    .WithMetrics(metrics => metrics
        .AddMeter(NexusRuntimeMetrics.MeterName)
        .AddAspNetCoreInstrumentation()
        .AddHttpClientInstrumentation()
        .AddRuntimeInstrumentation()
        .AddView("nexus1.message.duration",
            new ExplicitBucketHistogramConfiguration
            {
                Boundaries = new[] { .005, .01, .025, .05, .1,
                                    .25, .5, 1, 2.5, 5, 10 },
                CardinalityLimit = 64
            })
        .AddOtlpExporter());
```

REGISTRATION LEDGER 52-H

Registration	Purpose	Evidence
AddMeter	custom contract	inventory match
ASP.NET Core	server request metrics	route corpus
HttpClient	remote attempt metrics	dependency corpus
Runtime	process diagnosis	source/version manifest
View	buckets + point limit	exported descriptor
OTLP	transport	collector receipt

PACKAGE HONESTY

Exact packages, semantic-convention mode and supported View properties are resolved from retained restore artifacts. This listing remains compile-oriented until that checkpoint executes.

HTTP MEASUREMENTS

Use framework semantics for transport and owner metrics for meaning

HTTP METRIC CONTRACT 52-I

Metric	Meaning	Labels retained
http.server.request.duration	server request duration	method, route, status/error
http.client.request.duration	client attempt duration	method, dependency, error
nexus1.edge.requests	owner outcome count	operation, outcome, component

EXECUTABLE ASSET 52-E - OWNER OUTCOME

```
var labels = MetricLabelPolicy.For(  
    operation: "accept-alarm",  
    outcome: outcome.Code,  
    component: "alarm-management");  
  
metrics.Requests.Add(1, labels.ToTagList());  
return outcome.ToHttpResult();
```

HTTP EXCLUSIONS 52-J

Excluded	Retained alternative
raw path or query	matched route template
user/tenant identity	authorized service class if needed
request body size by case	aggregate body-size histogram only
exception message	bounded error.type

OUTCOME SEPARATION

HTTP 202, domain REJECTED and unexpected failure are different dimensions. The metric adapter maps each explicitly; it does not infer domain meaning from status alone.

MESSAGING MEASUREMENTS

Creation, send, receive and process are different operations

MESSAGING CATALOGUE 52-K

Operation	Counter	Duration	Outcome
create	message.attempts	owner-local optional	created/failed
send	message.attempts	client operation	sent/failed
receive	message.attempts	client operation	received/failed
process	message.attempts	process duration	succeeded/rejected/failed
settle	message.attempts	client operation	acked/nacked/failed

EXECUTABLE ASSET 52-F - PROCESS ATTEMPT

```
var started = time.GetTimestamp();  
var outcome = await consumer.ProcessAsync(envelope, cancellationTokens);  
var seconds = time.GetElapsedTime(started).TotalSeconds;  
  
var tags = labels.ForMessage("process-alarm", outcome.Code);  
metrics.MessageAttempts.Add(1, tags);  
metrics.MessageDuration.Record(seconds, tags);
```

CONVENTION STATUS

Messaging semantic conventions remain version-sensitive. NEXUS-1 pins the emitted convention profile and compares descriptor snapshots before migration; it does not silently mix old and new names.

OUTBOX STATE

Pending count and oldest age answer different failure questions

OUTBOX MEASURES 52-L

Measure	Answers	Cannot prove
pending rows	how much committed work waits?	oldest row progresses
oldest pending age	how long has any row waited?	all rows are blocked
dispatch attempts	how often is delivery tried?	broker accepted fact
dispatch outcomes	which bounded result classes occur?	business effect consumed

EXECUTABLE ASSET 52-G - OWNER SNAPSHOT

```
public sealed record OutboxMetricSnapshot(
    long Pending,
    DateTimeOffset? OldestOccurredUtc,
    DateTimeOffset ObservedUtc)
{
    public double OldestAgeSeconds => OldestOccurredUtc is null
        ? 0
        : Math.Max(0, (ObservedUtc - OldestOccurredUtc.Value).TotalSeconds);
}

public interface IOutboxMetricSnapshotReader
{
    ValueTask<OutboxMetricSnapshot> ReadAsync(Cancellation token);
}
```

SNAPSHOT CONSISTENCY

Count, oldest timestamp and observed time come from one owner read. A dashboard must not join unrelated queries taken at materially different instants and call the result one state.

OBSERVABLE CALLBACK BOUNDARY

Collection callbacks return cached snapshots, not database work

EXECUTABLE ASSET 52-H - CACHED OBSERVABLE STATE

```
public sealed class OutboxMetricState
{
    private OutboxMetricSnapshot snapshot =
        new(0, null, DateTimeOffset.UnixEpoch);

    public OutboxMetricState(IMeterFactory factory)
    {
        var meter = factory.Create(NexusRuntimeMetrics.MeterName);
        meter.CreateObservableGauge("nexus1.outbox.pending",
            () => Volatile.Read(ref snapshot).Pending, "{item}");
        meter.CreateObservableGauge("nexus1.outbox.oldest_age",
            () => Volatile.Read(ref snapshot).OldestAgeSeconds, "s");
    }

    public void Publish(OutboxMetricSnapshot next) =>
        Volatile.Write(ref snapshot, next);
}
```

CALLBACK RULES 52-M

Rule	Reason
no async I/O	collection callback is synchronous
no lock convoy	reader/exporter must remain bounded
one immutable snapshot	values share observation time
staleness exported separately	cached zero is not freshness
refresh worker has timeout	database pause cannot block collection

ZERO IS DATA

Zero pending rows is healthy only when the snapshot is fresh. An unrefreshed or failed read is no-data/fault, never silently converted to zero.

BOUNDED SNAPSHOT REFRESH

The worker owns I/O, timeout and last-good semantics

EXECUTABLE ASSET 52-I - REFRESH CYCLE

```
public async Task RefreshAsync(CancellationToken stoppingToken)
{
    using var budget = CancellationTokenSource.CreateLinkedTokenSource(
        stoppingToken);
    budget.CancelAfter(options.SnapshotTimeout);

    try
    {
        var next = await reader.ReadAsync(budget.Token);
        state.Publish(next);
        health.MarkSuccess(next.ObservedUtc);
    }
    catch (Exception ex) when (!stoppingToken.IsCancellationRequested)
    {
        health.MarkFailure(ErrorClassifier.Classify(ex), clock.GetUtcNow());
        metrics.SnapshotFailures.Add(1, health.SafeTags());
    }
}
```

FAILURE SEMANTICS 52-N

Condition	Gauge behavior	Health metric
fresh success	publish new snapshot	age since success = 0
read timeout	retain last good	failure + staleness grows
database unavailable	retain last good	bounded error.type
first read failed	no business value	initialized=0
shutdown	no new refresh	scrape may cease

LAST-GOOD RULE

A retained value is shown only beside its observation age. Without age, a perfectly rendered historical value becomes a lie about the present.

INBOX OUTCOMES

Duplicate, replay and failure are separate observations

INBOX CATALOGUE 52-O

Outcome	Counter value	Interpretation
first-seen	+1	new message admitted
duplicate-completed	+1	idempotency prevented repeat effect
duplicate-inflight	+1	concurrent delivery observed
replay-authorized	+1	operator-approved reprocessing
rejected-origin	+1	security policy rejected source
processing-failed	+1	attempt ended unexpectedly

EXECUTABLE ASSET 52-J - ONE TERMINAL OBSERVATION

```
var disposition = await inbox.AdmitAsync(envelope, cancellationToken);
var outcome = disposition switch
{
    InboxDisposition.FirstSeen => "first-seen",
    InboxDisposition.DuplicateCompleted => "duplicate-completed",
    InboxDisposition.DuplicateInflight => "duplicate-inflight",
    _ => "rejected"
};

metrics.InboxOutcomes.Add(1,
    new("nexus1.outcome", outcome),
    new("nexus1.message.type", catalogue.ClassOf(envelope.Type)));
```

ACCOUNTING RULE

The counter records the admission result once. Redelivery loops must not emit both duplicate and failed for the same admission branch unless two explicitly named operations occurred.

QUEUE AGE AND DEPTH

Backlog size without age can miss starvation

QUEUE SIGNALS 52-P

Signal	Best for	Failure mode
ready depth	amount waiting now	young burst looks severe
unacked depth	work currently leased	stuck lease needs age
oldest ready age	starvation and stalled progress	clock/source quality
enqueue rate	incoming pressure	no completion information
completion rate	drain capacity	retries can inflate work

EXECUTABLE ASSET 52-K - AGE PROJECTION

```
public static double QueueAgeSeconds(
    DateTimeOffset enqueuedUtc,
    DateTimeOffset observedUtc)
{
    var age = observedUtc - enqueuedUtc;
    return Math.Max(0, age.TotalSeconds);
}

metrics.QueueAge.Record(
    QueueAgeSeconds(envelope.EnqueuedUtc, clock.GetUtcNow()),
    new("nexus1.queue.class", "alarm-events"));
```

TIMESTAMP PROVENANCE

Queue age is valid only when the producer timestamp is authenticated/bounded and clock quality is known. Otherwise use broker-observed age or expose age_quality=unknown in a separate bounded signal.

ERROR MEASUREMENT

Count operational failures without converting domain outcomes

ERROR MAP 52-Q

Event	error.type	Domain outcome	Error counter
policy abstains	none	abstained	no
validation rejects	none	rejected	no
SQL timeout	timeout	not completed	yes
broker unavailable	dependency_unavailable	not completed	yes
contract invalid	contract_invalid	rejected	yes - boundary defect
cancellation by shutdown	shutdown_cancelled	not completed	classified

EXECUTABLE ASSET 52-L - CLASSIFIED FAILURE

```
catch (Exception ex) when (!cancellationToken.IsCancellationRequested)
{
    var errorType = errors.Classify(ex); // finite reviewed vocabulary
    metrics.OperationErrors.Add(1,
        new("nexus1.operation", "dispatch-outbox"),
        new("error.type", errorType));
    throw;
}
```

NO RAW EXCEPTION TAG

Type names are admitted only through a bounded classifier. Exception messages, stack traces and provider text remain in protected logs/traces, not metric attributes.

RETRY MEASUREMENT

Attempts and completed work must never share one counter

RETRY LEDGER 52-R

Metric	Increment	Question
operation.starts	every logical operation	how much work entered?
dependency.attempts	every remote attempt	how much downstream load?
retry.scheduled	each planned retry	how often policy repeats?
operation.completed	one terminal result	how much work finished?
retry.exhausted	terminal budget exhaustion	where recovery stopped?

EXECUTABLE ASSET 52-M - AMPLIFICATION RATIO

```
// Recording rule intent, not an in-process calculation:
// dependency_attempt_rate / logical_operation_rate

metrics.LogicalStarts.Add(1, operationTags);
await retryPolicy.ExecuteAsync(async token =>
{
    metrics.DependencyAttempts.Add(1, dependencyTags);
    await dependency.CallAsync(token);
}, cancellationToken);
```

BOUNDARY TO CHAPTER 53

Chapter 52 exposes retry amplification. Chapter 53 must justify the retry policy, timeout budget and circuit behavior that produced it.

PROJECTION LAG

Time lag and position lag reveal different kinds of staleness

PROJECTION MEASURES 52-S

Measure	Formula	Best interpretation
time lag	observedUtc - lastAppliedOccurredUtc	age of represented fact
position lag	sourceHighWater - lastAppliedPosition	number of unapplied facts
apply duration	completion - apply start	processing cost
failure count	classified failed applies	defect pressure

EXECUTABLE ASSET 52-N - PROJECTION SNAPSHOT

```
public sealed record ProjectionMetricSnapshot(
    string ProjectionClass,
    long SourceHighWater,
    long LastAppliedPosition,
    DateTimeOffset LastAppliedOccurredUtc,
    DateTimeOffset ObservedUtc)
{
    public long PositionLag => Math.Max(0,
        SourceHighWater - LastAppliedPosition);
    public double TimeLagSeconds => Math.Max(0,
        (ObservedUtc - LastAppliedOccurredUtc).TotalSeconds);
}
```

TWO CLOCKS, TWO CLAIMS

Position lag needs a comparable ordered source. Time lag needs trustworthy event time. When either prerequisite is absent, export its quality/freshness state instead of manufacturing zero.

WORKFLOW DURATION

End-to-end duration comes from durable milestones

WORKFLOW MILESTONES 52-T

Milestone	Timestamp owner	Retained evidence
alarm accepted	AlarmManagementDb	accepted fact
message committed	outbox owner	pending public fact
case opened	RootCauseDb	case event
verdict committed	RootCauseDb	terminal fact
projection visible	projection owner	checkpoint

EXECUTABLE ASSET 52-O - TERMINAL DURATION

```
public static void RecordTerminalDuration(
    WorkflowFacts facts,
    NexusRuntimeMetrics metrics)
{
    var seconds = Math.Max(0,
        (facts.TerminalOccurredUtc - facts.AcceptedOccurredUtc).TotalSeconds);
    metrics.WorkflowDuration.Record(seconds,
        new("nexus1.workflow", "alarm-to-verdict"),
        new("nexus1.outcome", facts.TerminalOutcomeClass));
}
```

NO AMBIENT STOPWATCH

A process stopwatch cannot cross restarts or queues. End-to-end workflow duration is reconstructed from durable, semantically owned timestamps after the terminal fact commits.

TIME DISCIPLINE

Use monotonic elapsed time locally and UTC facts across boundaries

CLOCK MATRIX 52-U

Measure	Clock	Why
handler duration	TimeProvider timestamp	monotonic elapsed time
dependency duration	instrumentation clock	owned attempt lifetime
queue age	trusted UTC timestamps	cross-process comparison
workflow duration	durable UTC facts	survives restart
snapshot staleness	observer UTC	collection freshness

EXECUTABLE ASSET 52-P - ELAPSED HELPER

```
public static async Task<(T Result, double Seconds)> Timed<T>(
    TimeProvider time,
    Func<Task<T>> operation)
{
    var start = time.GetTimestamp();
    var result = await operation();
    return (result, time.GetElapsedTime(start).TotalSeconds);
}
```

CLOCK FAULT

Negative cross-boundary age is clamped only for export safety and increments clock_anomaly. Silent clamping without an anomaly signal would hide a broken evidence prerequisite.

MEASUREMENT PLACEMENT

Record at the semantic boundary, not at every method

PLACEMENT CATALOGUE 52-V

Boundary	Record when	Why
HTTP owner outcome	result mapping is final	one logical request result
outbox commit	transaction succeeds	pending fact became durable
dispatch attempt	remote call begins/ends	dependency load and latency
inbox admission	disposition chosen	duplicate semantics known
projection apply	checkpoint commits	progress became durable
workflow duration	terminal fact commits	end state exists

CRASH WINDOW

A process can commit state and crash before incrementing a counter. Therefore counters describe observed instrumentation, while reconciliation and durable tables remain the authority for exact accounting.

NO DUAL TRUTH

Metrics never replace service-owned state

The dashboard may show a pending outbox count from a database snapshot, but only the owning database decides which rows are pending. A metric reset, scrape gap or exporter failure cannot change that state.

METRIC AND TRACE INDEPENDENCE

Trace sampling must not change population metrics

INDEPENDENCE LEDGER 52-W

Condition	Trace result	Metric result
AlwaysOn	all eligible spans	all measurements
ParentBased sampled out	no recorded child span	measurement retained
carrier missing	new/broken trace	message outcome retained
trace exporter down	trace loss	metric pipeline independent
metric exporter down	metric loss	business and trace continue

EXECUTABLE ASSET 52-Q - NO ACTIVITY GUARD

```
// Correct: measurement follows the operation outcome.
metrics.MessageAttempts.Add(1, tags);

// Rejected: sampling changes the population count.
if (Activity.Current?.IsAllDataRequested == true)
    metrics.MessageAttempts.Add(1, tags);
```

CORRELATION RULE

Do not add `TraceId` or `SpanId` as metric attributes. When supported and reviewed, exemplars can attach sampled trace context without multiplying metric series.

EXEMPLARS

A sampled trace may illustrate a bucket without becoming a label

EXEMPLAR CONTRACT 52-X

Element	Allowed	Guard
trace context	sampled exemplar reference	backend/SDK support verified
measurement labels	unchanged bounded set	no trace locator
payload fields	never	redaction independent of View
investigation	jump from aggregate to example	not representative proof

REDACTION WARNING

Dropping an attribute with a View is not sufficient redaction when exemplars are enabled. The evidence profile keeps exemplars off until a reviewed filter/reservoir and exported-corpus scan prove the policy.

RESOURCE IDENTITY

Service identity is not measurement cardinality

IDENTITY PLACEMENT 52-Y

Data	Placement	Reason
service.name/version	Resource	emitter identity
environment	Resource	deployment context
meter name/version	scope	instrumentation identity
operation/outcome	measurement attribute	observation dimension

TEMPORALITY AND RESET

Counter values need a window, not visual subtraction

AGGREGATION SEMANTICS 52-Z

Signal	Stored/exported form	Query rule
Counter	monotonic sum per process	rate/increase over window
ObservableGauge	last observed value	show freshness/no-data
Histogram	count, sum and buckets	rate buckets before quantile
process restart	new resource/series lifetime	never treat reset as negative work

EXECUTABLE ASSET 52-R - RESTART-SAFE QUERY INTENT

```
sum by (nexus1_operation, nexus1_outcome) (  
  rate(nexus1_edge_requests_total[5m])  
)  
  
sum by (le, nexus1_operation) (  
  rate(nexus1_workflow_duration_seconds_bucket[15m])  
)
```

EXPORTER MAPPING

Backend names and suffixes are verified from the exported descriptor corpus. The canonical contract remains the .NET/OpenTelemetry instrument name, type and unit; queries are pinned to the chosen exporter mapping.

HISTOGRAM DESIGN

Buckets encode the questions the team intends to ask

BUCKET CATALOGUE 52-AA

Histogram	Illustrative boundaries (s)	Reason
HTTP/message attempt	.005 .01 .025 .05 .1 .25 .5 1 2.5 5 10	transport tails
workflow	.1 .25 .5 1 2.5 5 10 30 60 120 300	async completion
queue age	1 5 10 30 60 300 900	stagnation classes
projection apply	.005 .01 .025 .05 .1 .25 .5 1 2.5	handler cost

HISTOGRAM CAVEATS 52-AB

Claim	Qualification
p95 = 0.42 s	estimate from bucket distribution
average = sum/count	tail may still be poor
bucket threshold ratio	direct good/total population
cross-instance aggregation	compatible buckets/temporality required

NO MAGICAL PERCENTILE

A percentile is an aggregate estimate over a selected window and label set. It does not describe every request, and it must not be compared across incompatible bucket schemas.

COLLECTION PIPELINE

Scrape and OTLP are transport choices, not metric semantics

PROOF PATH 52-C - RETAINED PIPELINE



PIPELINE PROOF 52-AC

Stage	Failure signal	Retained evidence
instrument	zero discovery	descriptor inventory
SDK	overflow/internal error	diagnostic logs + point census
exporter	failed export	self-telemetry
collector	refused/dropped points	collector metrics/config
backend	ingestion/query gap	query result + timestamps
dashboard	stale/no-data	panel JSON + screenshot/hash

METRIC ABOUT METRICS

Telemetry pipeline health is visible through separate self-telemetry. Application metrics must not recursively claim their own completeness.

COLLECTOR REFERENCE PROFILE

Configuration is versioned and value-free

EXECUTABLE ASSET 52-S - REFERENCE COLLECTOR

```
receivers:
  otlp:
    protocols:
      grpc: {}
      http: {}

processors:
  memory_limiter:
    check_interval: 1s
    limit_mib: 256
    spike_limit_mib: 64
  batch:
    send_batch_size: 1024
    timeout: 5s

exporters:
  prometheus:
    endpoint: "0.0.0.0:9464"

service:
  pipelines:
    metrics:
      receivers: [otlp]
      processors: [memory_limiter, batch]
      exporters: [prometheus]
```

CONFIGURATION GATES 52-AD

Gate	Required proof
component availability	collector build exposes named components
config validation	startup command succeeds
endpoint isolation	network policy and no public exposure
label scan	forbidden keys absent after processing
loss accounting	receiver/exporter accepted/refused counters retained

VERSION HONESTY

Collector component names and configuration schema are verified against the pinned image. The manuscript does not imply that an arbitrary latest image accepts this reference profile.

PROMETHEUS RECORDING RULES

Canonical queries are reviewed artifacts, not dashboard text

EXECUTABLE ASSET 52-T - RECORDING RULES

```
groups:
- name: nexus1-learning
  rules:
  - record: nexus1:edge_success:ratio_5m
    expr: |
      sum(rate(nexus1_edge_requests_total{nexus1_outcome="succeeded"}[5m]))
      /
      clamp_min(sum(rate(nexus1_edge_requests_total[5m])), 1e-12)
  - record: nexus1:workflow_duration:p95_15m
    expr: |
      histogram_quantile(0.95,
        sum by (le) (rate(nexus1_workflow_duration_seconds_bucket[15m])))
  - record: nexus1:retry_amplification:ratio_5m
    expr: |
      sum(rate(nexus1_dependency_attempts_total[5m]))
      / clamp_min(sum(rate(nexus1_logical_starts_total[5m])), 1e-12)
```

RULE OWNERSHIP 52-AE

Artifact	Owner	Gate
metric descriptors	instrumentation	schema snapshot
recording rules	observability	promtool/backend validation
objective definition	engineering review	scope + rationale
dashboard panel	observability	query reference only

INDICATOR BEFORE OBJECTIVE

Specify the user-relevant ratio before choosing a target

SLI ANATOMY 52-AF

Element	Required content	NEXUS-1 example
population	which eligible observations	synthetic alarm workflows
good event	explicit success condition	terminal within threshold
total event	explicit denominator	all accepted campaign workflows
window	fixed or rolling interval	one retained test campaign
implementation	exact metrics and query	recording rule digest
limitations	coverage/precision gaps	no external user traffic

SPECIFICATION VERSUS IMPLEMENTATION

The indicator specification states the outcome that matters independently of measurement. The implementation names the available data and its known gaps. NEXUS-1 retains both.

EDUCATIONAL OBJECTIVE

A campaign target is not a production promise

Phase-0 has no validated workload, users, traffic seasonality, support model or operational consequence analysis. Therefore this chapter defines learning objectives that test whether the demonstrator exposes the intended behavior under a retained campaign.

OBJECTIVE CATALOGUE

Each target is scoped to a named reproducible campaign

EDUCATIONAL OBJECTIVES 52-AG

ID	Indicator	Campaign target	Not claimed
EO-52-1	edge successful / eligible	≥ 0.99 over baseline run	production availability
EO-52-2	workflow ≤ 5 s / terminal	≥ 0.95 over baseline run	user latency SLO
EO-52-3	outbox oldest age	≤ 10 s after drain point	broker guarantee
EO-52-4	projection position lag	returns to 0 after pause	real deployment recovery
EO-52-5	retry amplification	≤ 1.20 without injected fault	retry policy approved
EO-52-6	metric freshness	all required series observed	telemetry loss impossible

OBJECTIVE METADATA 52-AH

Field	Required value
status	PROPOSED / EXECUTED / REJECTED / RETIRED
owner/reviewer	named roles from evidence manifest
query digest	canonical recording-rule artifact
campaign identity	workload, build, config and time interval
rationale	why target teaches the intended behavior
review date	prevents accidental permanent promise

TARGET HONESTY

The numerical targets above are proposed test thresholds. They become evidence only after the campaign executes and never become an SLA or production SLO by appearing on a dashboard.

OBJECTIVE DOCUMENT

The artifact carries scope, math, provenance and limitations

EXECUTABLE ASSET 52-U - OBJECTIVE SPECIFICATION

```
public sealed record EducationalObjective(
    string Id,
    string IndicatorSpecification,
    string RecordingRule,
    string Comparator,
    double Target,
    TimeSpan Window,
    string CampaignId,
    string Rationale,
    string[] Limitations,
    EvidenceStatus Status);

var workflowObjective = new EducationalObjective(
    "EO-52-2",
    "terminal workflows completed within five seconds / all terminal workflows",
    "nexus1:workflow_within_5s:ratio_campaign",
    ">=", 0.95, TimeSpan.FromMinutes(15),
    "baseline-v1", "exercise end-to-end visibility",
    ["synthetic load", "single environment", "not a production SLO"],
    EvidenceStatus.Proposed);
```

REVIEW GATE 52-AI

Question	Reject when
Does population match the intended user outcome?	proxy omits a critical seam
Can numerator exceed denominator?	event accounting is inconsistent
Is no-data explicit?	absence can pass as zero
Is target provenance stated?	number is copied/invented
Can one run reproduce the query?	artifact/digest missing

NO TRAFFIC AND NO DATA

An empty denominator is unknown, not one hundred percent

ZERO-DATA TRUTH TABLE 52-AJ

Good	Total	Pipeline fresh	Verdict
0	0	yes	NO_ELIGIBLE_TRAFFIC
0	0	no	TELEMETRY_UNKNOWN
0	>0	yes	OBJECTIVE_MISSED
>0	>0	yes	evaluate ratio
any	any	no	TELEMETRY_UNKNOWN

EXECUTABLE ASSET 52-V - OBJECTIVE VERDICT

```
public static ObjectiveVerdict Evaluate(
    long good, long total, bool telemetryFresh, double target)
{
    if (!telemetryFresh) return ObjectiveVerdict.TelemetryUnknown;
    if (total == 0) return ObjectiveVerdict.NoEligibleTraffic;
    if (good < 0 || good > total) return ObjectiveVerdict.InvalidEvidence;
    return (double)good / total >= target
        ? ObjectiveVerdict.Met
        : ObjectiveVerdict.Missed;
}
```

NO CLAMP-TO-SUCCESS

PromQL denominator guards can prevent division errors, but the evidence evaluator must still inspect the raw denominator and freshness. A numerical zero or NaN is not an objective verdict.

DASHBOARD INFORMATION ARCHITECTURE

Six panels tell one operational learning story

DASHBOARD MANIFEST 52-AK

Panel	Question	Visual	Drill path
Edge outcomes	are attempts completing?	rate/ratio	logs by event
Workflow latency	is end-to-end work timely?	p50/p95 + threshold	trace exemplar
Outbox pressure	is durable work waiting?	pending + oldest age	owner query
Inbox outcomes	are duplicates/failures rising?	stacked rate	security/log events
Projection lag	are reads stale?	time + position	checkpoint evidence
Retry amplification	is remote load multiplying?	ratio	Chapter 53 policy

GLOBAL CONTEXT STRIP 52-AL

Field	Source	Reason
environment/build	Resource manifest	provenance
selected window	dashboard control	query meaning
last successful collection	pipeline health	freshness
objective status	evidence evaluator	not inferred by color

SMALL BY DESIGN

The dashboard contains the minimum panels needed to follow the distributed case. Runtime/CPU panels may support diagnosis, but they do not displace workflow health.

DASHBOARD AS CODE

Provisioned artifacts preserve review and rollback

EXECUTABLE ASSET 52-W - CLASSIC PROVISIONING

```
apiVersion: 1
providers:
- name: nexus1-runtime
  orgId: 1
  folder: NEXUS-1 Phase-0
  type: file
  disableDeletion: true
  updateIntervalSeconds: 30
  allowUiUpdates: false
  options:
    path: /var/lib/grafana/dashboards/nexus1
```

EXECUTABLE ASSET 52-X - REVIEWED PANEL MANIFEST

```
public sealed record DashboardPanel(
  string Id, string Title, string RecordingRule,
  string Unit, string EvidenceQuestion);

DashboardPanel[] panels =
[
  new("edge", "Edge outcomes", "nexus1:edge_success:ratio_5m",
    "percentunit", "Are eligible edge attempts completing?"),
  new("workflow", "Workflow p95", "nexus1:workflow_duration:p95_15m",
    "s", "Is terminal workflow duration within the campaign target?"),
  new("retry", "Retry amplification", "nexus1:retry_amplification:ratio_5m",
    "none", "Are dependency attempts multiplying logical work?")
];
```

SCHEMA PIN

A generator renders this reviewed manifest into the exact dashboard schema supported by the pinned Grafana image. The retained JSON, image digest and screenshot are the executable evidence, not a hand-edited UI.

FROM SERVICES TO RUNTIME

PANEL QUERY CONTRACTS

Every panel names its source rule, unit and no-data behavior

PANEL CONTRACT 52-AM

Panel	Source	Unit	No-data text
Edge outcomes	edge_success:ratio_5m	%	no eligible traffic / unknown
Workflow latency	workflow_duration:p95_15m	s	no terminal workflows
Outbox pressure	pending + oldest_age	items / s	snapshot stale
Inbox outcomes	rate(inbox_outcomes)	msg/s	no admissions / unknown
Projection lag	time + position gauges	s / facts	snapshot stale
Retry amplification	attempts / starts	ratio	no logical starts

EXECUTABLE ASSET 52-Y - QUERY REFERENCE

```
public static void ValidatePanel(DashboardPanel panel,
    IReadOnlyDictionary<string, RecordingRule> rules)
{
    if (!rules.TryGetValue(panel.RecordingRule, out var rule))
        throw new EvidenceException($"Unknown rule for panel {panel.Id}");
    if (rule.Unit != panel.Unit)
        throw new EvidenceException($"Unit mismatch for panel {panel.Id}");
    if (string.IsNullOrEmpty(rule.NoDataSemantics))
        throw new EvidenceException($"No-data policy missing for {panel.Id}");
}
```

NO DUPLICATED PROMQL

Panels reference canonical recording rules. Copying long expressions into each panel creates silent divergence between dashboard, objective evaluator and tests.

FRESHNESS AND STALENESS

A dashboard must reveal when it stopped knowing

FRESHNESS SIGNALS 52-AN

Layer	Signal	Stale consequence
snapshot worker	seconds since last success	gauges are last-good
application SDK	export failures / overflow	points incomplete
collector	accepted/refused/exported	pipeline loss
backend	latest sample timestamp	query lag
dashboard	data-source/query error	panel unknown

EXECUTABLE ASSET 52-Z - FRESHNESS VERDICT

```
public static FreshnessVerdict CheckFreshness(
    DateTimeOffset observedUtc,
    DateTimeOffset nowUtc,
    TimeSpan maximumAge,
    bool pipelineHealthy)
{
    if (!pipelineHealthy) return FreshnessVerdict.Unknown;
    var age = nowUtc - observedUtc;
    if (age < TimeSpan.Zero) return FreshnessVerdict.ClockInvalid;
    return age <= maximumAge
        ? FreshnessVerdict.Fresh
        : FreshnessVerdict.Stale;
}
```

COLOR RULE

Green is permitted only for a fresh evaluated result. Stale, unknown, query-error and no-traffic states use distinct text and color; they never inherit the last green state.

COMPLETE METRIC CAMPAIGN

The baseline run must exercise every declared seam

CAMPAIGN INPUT 52-AO

Phase	Stimulus	Expected observation
baseline	200 accepted alarm workflows	eligible totals known
duplicates	20 deterministic redeliveries	duplicate-completed +20
outbox pause	dispatcher held for 12 s	pending/oldest age rises
resume	dispatcher released	age and position lag drain
dependency fault	10 classified transient failures	attempts exceed starts
terminal	all cases reach bounded outcomes	workflow corpus closes

COMPLETE ORACLE 52-AP

Proof	Required result
descriptor discovery	all required instruments found once
population conservation	terminal <= accepted; reasons explain difference
series census	within calculated cardinality budget
freshness	all required snapshots and pipeline stages fresh
objective evaluation	verdict includes raw good/total and target
dashboard	six panels match canonical query snapshot

CAMPAIGN IDENTITY

The run retains build digest, config digest, package lock, seed, workload manifest, UTC interval, query digests and raw backend result bundle.

DELIBERATELY BROKEN METRIC CAMPAIGN

An illegal locator exposes the cardinality defense

EXECUTABLE ASSET 52-AA - CONTROLLED FAULT

```
public sealed class UnsafeMetricFault(Meter meter)
{
    private readonly Counter<long> counter =
        meter.CreateCounter<long>("nexus1.test.unsafe_measurements");

    public void Emit(int count)
    {
        for (var i = 0; i < count; i++)
            counter.Add(1, new("nexus1.message.id", $"fault-{i:D6}"));
    }
}
```

BROKEN ORACLE 52-AQ

Layer	Expected finding
static catalogue	forbidden nexus1.message.id detected
runtime series census	distinct series exceeds declared budget
SDK limit	overflow point/internal diagnostic may appear
objective evaluator	TELEMETRY_INVALID, never MET
dashboard	cardinality fault banner, panels not trusted
business state	unchanged; fault is telemetry-only

FAULT ISOLATION

UnsafeMetricFault exists only in the evidence fixture, is absent from production composition and requires an explicit test profile. The campaign proves the gate, not a recommended API.

MISSING METRIC TAXONOMY

No sample has several causes, and each demands a different action

DIAGNOSTIC MATRIX 52-AR

Finding	Likely cause	Next proof
instrument undiscovered	Meter name not subscribed	registration inventory
descriptor present, zero points	code path/no traffic	stimulus receipt
points local, absent backend	export/collector loss	pipeline self-metrics
series overflow	label cardinality defect	distinct-set and forbidden-key scan
gauge frozen	snapshot refresh failed	last-success age
panel no-data only	query/name mapping drift	rule and descriptor snapshot
objective unknown	freshness or denominator absent	raw evaluator inputs

DIAGNOSIS ORDER

Start with the business stimulus and durable state, then instrument discovery, local points, export, collection, backend query and dashboard. Do not edit the panel until the failing layer is identified.

INSTRUMENT UNIT TESTS

MeterListener verifies names, types, values and tags in process

EXECUTABLE ASSET 52-AB - LISTENER HARNESS

```
using var listener = new MeterListener();
var seen = new List<(string Name, long Value, TagList Tags)>();

listener.InstrumentPublished = (instrument, owner) =>
{
    if (instrument.Meter.Name == NexusRuntimeMetrics.MeterName)
        owner.EnableMeasurementEvents(instrument);
};
listener.SetMeasurementEventCallback<long>((instrument, value, tags, _) =>
{
    var copy = new TagList();
    foreach (var tag in tags) copy.Add(tag);
    seen.Add((instrument.Name, value, copy));
});
listener.Start();

metrics.Requests.Add(1, MetricLabelPolicy.Success("accept-alarm"));
Assert.Single(seen, x => x.Name == "nexus1.edge.requests");
```

UNIT ASSERTIONS 52-AS

Assertion	Failure caught
instrument name/type/unit	contract drift
exact measurement count	double recording
bounded tag keys/values	cardinality/privacy leak
no Activity dependency	sampling-coupled metrics
observable callback fresh	blocking/stale gauge

COLLECTION AND QUERY TESTS

An in-memory reader proves aggregation before a backend

EXECUTABLE ASSET 52-AC - COLLECTION ORACLE

```
var exported = new List<Metric>();
using var provider = Sdk.CreateMeterProviderBuilder()
    .AddMeter(NexusRuntimeMetrics.MeterName)
    .AddInMemoryExporter(exported, exportIntervalMilliseconds: 10)
    .Build();

fixture.EmitKnownMeasurements();
Assert.True(provider.ForceFlush(5_000));

var census = MetricCorpus.Read(exported);
Assert.Equal(200, census.Sum("nexus1.edge.requests"));
Assert.True(census.SeriesCount("nexus1.edge.requests") <= 20);
Assert.DoesNotContain(census.TagKeys, ForbiddenMetricTags.Contains);
```

COMPONENT GATES 52-AT

Gate	Environment
aggregation and views	in-memory exporter
export mapping	real OTLP collector
recording rules	pinned Prometheus-compatible backend
dashboard provisioning	pinned Grafana image
objective campaign	full reproducible Compose runtime

API PINNING

Exporter and test-helper signatures are verified against restored packages. If the retained SDK exposes a different supported overload, the executable artifact wins and the manuscript is updated.

STATIC AND STRUCTURAL GATES

Architecture tests fail before cardinality reaches a backend

EXECUTABLE ASSET 52-AD - CONTRACT VALIDATOR

```
public static void Validate(MetricDefinition metric)
{
    Require(MetricNames.IsCanonical(metric.Name), "invalid name");
    Require(metric.Unit is "s" or "By" or "{item}" or
        "{request}" or "{attempt}" or "{message}", "invalid unit");
    Require(metric.LabelKeys.All(ApprovedMetricTagKeys.Contains),
        "unapproved label key");
    Require(!metric.LabelKeys.Any(ForbiddenMetricTags.Contains),
        "locator label forbidden");
    Require(metric.MaximumSeries > 0 && metric.MaximumSeries <= 128,
        "series budget invalid");
}
```

REPOSITORY GATES 52-AU

Scan	Rejects
instrument inventory	undeclared or duplicate names
literal tag keys	calls outside typed adapter
forbidden locator terms	MessageId/TraceId/user/payload labels
dashboard query refs	unknown recording rules
objective docs	target without rationale/status/window
evidence manifest	zero discovery or stale artifact

DASHBOARD VALIDATION

A rendered panel must match its canonical query snapshot

DASHBOARD TEST MATRIX 52-AV

Test	Pass condition
provisioning load	dashboard UID/title/folder present once
panel census	six required panels and freshness strip
query identity	recording-rule references match manifest
unit mapping	seconds, ratio, items and facts correct
no-data fixture	unknown/no-traffic are not green
stale fixture	last-good value marked stale
golden screenshot	labels/thresholds legible and current

EXECUTABLE ASSET 52-AE - MANIFEST CHECK

```
var dashboard = await grafana.ReadByIdAsync("nexus1-runtime-learning");
Assert.Equal(expected.Digest, DashboardCanonicalizer.Hash(dashboard));
Assert.Equal(expected.PanelIds.Order(), dashboard.PanelIds.Order());
Assert.All(dashboard.Panels, panel =>
{
    Assert.Contains(panel.RuleName, recordingRules.Keys);
    Assert.False(string.IsNullOrEmpty(panel.NoDataText));
});
```

UI IS A PROJECTION

The screenshot proves rendering, not metric correctness. Query results, descriptors, evaluator inputs and pipeline evidence remain separately retained.

EVIDENCE MANIFEST

A metric claim names build, corpus, query and freshness

EXECUTABLE ASSET 52-AF - EVIDENCE RECORD

```
public sealed record MetricEvidenceManifest(
    string BuildDigest,
    string PackageLockDigest,
    string ConfigurationDigest,
    string InstrumentCatalogueDigest,
    string RecordingRulesDigest,
    string DashboardDigest,
    string CampaignId,
    DateTimeOffset StartedUtc,
    DateTimeOffset EndedUtc,
    int InstrumentsDiscovered,
    int SeriesObserved,
    bool PipelineFresh,
    ObjectiveResult[] Objectives,
    ArtifactRef[] Artifacts,
    EvidenceVerdict Verdict);
```

REQUIRED ARTIFACTS 52-AW

Artifact	Purpose
descriptor corpus	names, types, units, resources
series census	cardinality and forbidden-label scan
raw query results	objective numerator/denominator
pipeline self-metrics	loss/freshness qualification
dashboard JSON + screenshot	provisioning/render proof
campaign manifest	reproduction and scope

FAIL-CLOSED EVIDENCE VERDICT

Zero discovery, overflow, staleness or missing artifacts invalidate the claim

EXECUTABLE ASSET 52-AG - VERIFIER

```
public static EvidenceVerdict Verify(MetricEvidenceManifest m)
{
    if (m.InstrumentsDiscovered == 0)
        return EvidenceVerdict.Invalid("zero instrument discovery");
    if (!m.PipelineFresh)
        return EvidenceVerdict.Invalid("telemetry pipeline not fresh");
    if (m.SeriesObserved <= 0)
        return EvidenceVerdict.Invalid("zero series observed");
    if (m.Artifacts.Any(a => !a.Exists || !a.DigestVerified))
        return EvidenceVerdict.Invalid("missing or stale artifact");
    if (m.Objectives.Any(o => o.Verdict is ObjectiveVerdict.InvalidEvidence
        or ObjectiveVerdict.TelemetryUnknown))
        return EvidenceVerdict.Invalid("objective evidence incomplete");
    return EvidenceVerdict.Valid;
}
```

VERDICT VOCABULARY 52-AX

Status	Meaning
SPECIFIED	contract and campaigns fully designed
EXECUTED	retained runtime artifacts pass all gates
PARTIAL	named subset executed; residuals explicit
INVALID	evidence cannot support claim
PENDING	no fresh runtime campaign retained

CURRENT VERDICT

SPECIFIED | RUNTIME PENDING. This manuscript defines the metric system and evidence campaigns; it does not claim a fresh build, collector, backend or dashboard execution.

IMPLEMENTATION SEQUENCE

Build the contract before the dashboard

SEQUENCE 52-AY

Step	Deliverable	Gate
1	question and instrument catalogue	owner/unit/type reviewed
2	typed adapters and bounded vocabulary	unit tests pass
3	views, buckets and cardinality limits	descriptor snapshot
4	outbox/inbox/projection snapshots	freshness tests
5	collector and backend profile	export/query component tests
6	recording rules and objectives	math/no-data review
7	provisioned dashboard	panel/golden tests
8	complete and broken campaigns	evidence verifier

PULL-REQUEST EVIDENCE 52-AZ

Change	Required diff
new label	domain size + series budget
new histogram bucket	storage/query/target impact
semantic convention	old/new descriptor migration
objective target	rationale, owner and review
dashboard panel	question, rule, unit, no-data

REVIEW ORDER

Meaning and boundedness are reviewed before visualization. A beautiful panel cannot rescue an invalid metric contract.

STANDARDS AND SOURCE LEDGER

Specifications guide transport; NEXUS-1 owns local meaning

SOURCE LEDGER 52-BA

Source	Used for	Local qualification
Microsoft .NET metrics	Meter/IMeterFactory/instruments	DI lifetime + naming
OpenTelemetry Metrics API/SDK	provider, views, limits	pinned packages
OTel HTTP conventions	request duration/route	stable HTTP profile
OTel messaging conventions	operation/process metrics	development profile pinned
Prometheus	rate, histogram and rules	export-name snapshot
Google SRE Workbook	SLI spec/implementation	educational objectives only
Grafana provisioning	dashboard as code	pinned schema/image

VERSION POLICY

Every external convention, package and dashboard schema is recorded with version/digest. Experimental or development semantics remain explicitly labeled and migration-tested.

ANTI-PATTERN REVIEW

Five attractive metric claims remain invalid

REJECTED CLAIMS 52-BB

Claim	Why invalid
green dashboard means system correct	metrics are aggregate diagnostics
zero errors means no failures	no-data/export loss may exist
p95 describes every workflow	percentile is aggregate estimate
counter equals durable facts	commit/crash instrumentation gap
target is an SLO	no production evidence or approval

COMPLETION LEDGER

What Chapter 52 now specifies

COMPLETION LEDGER 52-BC

Obligation	Asset	Status
request/message coverage	typed counters + histograms	SPECIFIED
outbox/inbox state	owner snapshots + freshness	SPECIFIED
queue/projection/workflow	age, lag and duration contracts	SPECIFIED
cardinality/privacy	vocabulary, budget and fail-closed gates	SPECIFIED
objective math	indicator documents + evaluator	SPECIFIED
dashboard	six-panel provisioned manifest	SPECIFIED
complete/broken campaigns	structural oracles	SPECIFIED
runtime proof	executed metric evidence manifest	PENDING

OPEN RESIDUALS 52-BD

Residual	Consequence
no executed repository checkpoint	listings remain compile-oriented
no retained package restore	exact API/version unproved
no collector/backend run	transport/query behavior unproved
no dashboard provisioning run	render and no-data behavior unproved
no representative production workload	no production SLO can be set
messaging conventions evolving	profile migration remains pending

EVIDENCE VERDICT

SPECIFIED | RUNTIME PENDING. Instrumentation, aggregation, objective, dashboard and fault campaigns are fully designed; no fresh runtime evidence is claimed.

HANDOFF TO CHAPTER 53

Timeout, Retry, Circuit Breaker, and Bulkhead

Chapter 53 will use the bounded rates, latency distributions, queue age, retry amplification and dependency failure classes defined here to justify resilience at specific remote boundaries. It must demonstrate timeout budgets, retry multiplication, circuit state and resource isolation without changing domain truth.

BOUNDARY CARRIED FORWARD 52-BE

Chapter 52 leaves	Chapter 53 must add
dependency attempt rate/duration	remote timeout budget
retry amplification ratio	bounded retry policy and proof
classified dependency errors	circuit-breaker predicates/state
queue age and workflow duration	backpressure and bulkhead evidence
fresh metric pipeline	resilience campaign observations

NEXT INVARIANT

A resilience policy is admitted only at a named remote or resource boundary, with a bounded budget and evidence that it reduces failure propagation without multiplying load or hiding an incomplete business operation.

FINAL CHAPTER RULE

A dashboard is an investigation surface, not an oracle

NEXUS-1 metrics compress bounded populations into rates, distributions and current-state observations. Objectives evaluate a declared campaign. Dashboards expose those results and their freshness. Durable state, security evidence, logs and traces remain necessary to decide what actually happened.

PART IX CONTINUITY

Business lineage identifies the case; logs preserve domain observations; traces explain attempts; metrics bound aggregate behavior; Chapter 53 will constrain resilience; Chapter 54 will inject failure and prove recovery.

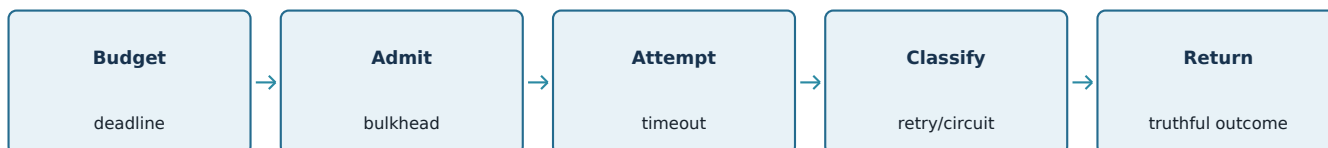
PART IX - OBSERVABILITY, RESILIENCE, AND FAILURE EVIDENCE

53. Timeout, Retry, Circuit Breaker, and Bulkhead

Resilience is a bounded admission policy at a named remote or resource boundary. It buys time or isolation; it never creates business truth, erases uncertainty, or grants unlimited extra attempts.

CHAPTER DECISION

Admit a policy only after classifying the boundary, operation semantics, remaining deadline and load budget. Compose one explicit pipeline, prove its order, and retain campaigns for slow calls, transient failures, open circuits and saturation.

PROOF PATH 53-A - FROM REMOTE FAULT TO BOUNDED RESULT**GUARANTEE LEDGER 53-A**

Mechanism	Allowed claim	Required proof	Cannot claim
timeout	caller stops waiting within budget	elapsed-time campaign	remote work rolled back
retry	eligible call was re-attempted	attempt ledger + idempotency	effect happened once
circuit	known-failing calls were short-circuited	state-transition corpus	dependency recovered
bulkhead	concurrency was bounded	permit/queue census	fairness or success

CHAPTER EXIT CRITERIA 53-B

Question	Required answer
Can policies multiply invisibly?	no - one owner and amplification gate
Can unsafe work be retried?	only with explicit idempotency proof
Can timeout mean rollback?	no - completion remains success/failure/unknown
Can saturation spread?	no - per-dependency concurrency and queue bounds

ORDINARY DEVELOPER PROBLEM

Three helpful retries can become twenty-seven calls

An edge client retries twice, the service retries twice and the SDK retries twice. One logical operation can now produce $3 \times 3 \times 3 = 27$ dependency attempts. During an outage, latency rises, queues grow and every layer adds more work precisely when the dependency has the least capacity.

FAILURE LEDGER 53-1

Shortcut	Failure	Correction
retry every exception	permanent faults amplified	finite classifier
timeout means cancelled	remote effect may continue	unknown outcome branch
circuit per request	no shared failure memory	stable dependency partition
unbounded wait queue	latency and memory move upstream	zero or bounded queue
nested handlers	unseen multiplication	single composition owner
fallback success	business truth fabricated	truthful degraded result

PLAIN-LANGUAGE PRINCIPLE

The safest retry is the one the system can prove it does not need. The next safest is bounded, delayed, idempotent and owned by exactly one layer.

CHAPTER BOUNDARY

Policy is specified here; destructive recovery is proved next

This chapter defines policy placement, budgets, ordering, telemetry and controlled campaigns. Chapter 54 will stop the broker and database, kill consumers, duplicate messages, delay projections and restart services to produce retained recovery evidence.

DECISION RECORD

ADR-053 - Resilience belongs to named boundaries

NEXUS-1 admits timeout, retry, circuit breaker or concurrency limiting only at a catalogue entry that names the caller, dependency, operation class, idempotency contract, total budget, maximum attempts, concurrency capacity and evidence campaign. Domain and application decisions remain free of transport policies.

DECISION LEDGER 53-2

Layer	May own	Must not own
Domain	deterministic decision	retry, timeout, breaker
Application	outcome and port semantics	provider exception loops
Infrastructure	remote policy adapter	business fallback
Edge/Worker	caller deadline and backpressure	domain truth rewrite
Platform	shared defaults and telemetry	silent per-service override

DECISION IDENTITY

ADR-053 and the boundary catalogue are hashed by the evidence build. A package default is not an architectural decision until its effective options are inventoried.

INVARIANT CARRIED FROM CHAPTER 52

Aggregate evidence justifies policy; it does not authorize it

Latency distributions, retry amplification, error classes and queue age identify a candidate boundary. Admission still requires operation semantics, a bounded deadline and proof that the policy reduces propagation without hiding incomplete work.

BOUNDARY CATALOGUE

Every policy row names one caller-dependency relationship

NEXUS-1 BOUNDARY INVENTORY 53-C

Key	Operation	Semantics	Candidate policy
alarm.rootcause.read	GET case projection	safe read	deadline/retry/circuit/limit
alarm.rootcause.open	POST open case	idempotency key	deadline/retry/circuit/limit
outbox.broker.publish	publish public fact	confirm + Messageld	deadline/circuit/limit
projection.db.apply	apply fact	Inbox + checkpoint	command timeout/concurrency
repair.source.scan	read owner range	restartable cursor	deadline/retry/limit

EXPLICIT NON-BOUNDARIES 53-D

Surface	Why no resilience policy
aggregate method	deterministic local computation
application validation	permanent caller defect
local mapping	a bug must fail visibly
transaction commit wrapper	unknown commit cannot be blindly replayed
health endpoint body	diagnosis is not remote work recovery

CATALOGUE RULE

The key is stable and bounded. It identifies policy configuration and telemetry; Messageld, Caseld, URL and tenant never become pipeline partitions or metric labels.

FAILURE CLASSIFICATION

Transient, permanent, overload and unknown are different outcomes

CLASSIFIER 53-E

Class	Examples	Retry?	Circuit sample?
transient	connect reset, selected 5xx	bounded	yes
throttled	429 with Retry-After	only within budget	policy-specific
permanent	400, 401, 403, contract invalid	no	no
caller-cancelled	request/shutdown token	no	no
attempt-timeout	policy cancelled attempt	if idempotent	yes
unknown-effect	connection lost after send/commit	reconcile first	yes
local-overload	bulkhead rejected	no local retry	no dependency blame

EXECUTABLE ASSET 53-A - FINITE FAILURE VOCABULARY

```
public enum RemoteFailureClass
{
    Transient, Throttled, Permanent, CallerCancelled,
    AttemptTimedOut, UnknownEffect, LocalOverload
}

public sealed record RemoteFailure(
    RemoteFailureClass Class,
    string Code,
    TimeSpan? RetryAfter = null);
```

CLASSIFIER SCOPE

Provider text and exception messages are retained in protected diagnostics. Policy predicates use reviewed types, status codes and bounded failure codes only.

DEADLINE MODEL

One monotonic deadline bounds queueing, attempts and delays

EXECUTABLE ASSET 53-B - MONOTONIC BUDGET

```
public sealed class DeadlineBudget
{
    private readonly TimeProvider time;
    private readonly long started;
    private readonly TimeSpan total;

    public DeadlineBudget(TimeProvider time, TimeSpan total)
    {
        if (total <= TimeSpan.Zero) throw new ArgumentOutOfRangeException(nameof(total));
        this.time = time;
        this.total = total;
        started = time.GetTimestamp();
    }

    public TimeSpan Remaining()
    {
        var remaining = total - time.GetElapsedTime(started);
        return remaining > TimeSpan.Zero ? remaining : TimeSpan.Zero;
    }
}
```

CLOCK CONTRACT 53-F

Use	Clock	Reason
local elapsed budget	TimeProvider timestamp	monotonic
cross-service expiry metadata	authenticated UTC deadline	portable
Retry-After	bounded duration/date	dependency guidance
evidence timestamps	UTC + monotonic elapsed	reproduction

DEADLINE AUTHORITY

A caller deadline can shorten the local maximum but cannot extend it. Untrusted arbitrary headers never grant a larger budget.

BUDGET ALGEBRA

The arithmetic is reviewed before the policy is enabled

BUDGET WORKSHEET 53-G

Term	Read profile	Command profile
caller maximum	1,500 ms	2,500 ms
queue allowance	50 ms	0 ms
maximum attempts	3	2 with idempotency
attempt timeout	350 ms	800 ms
maximum delays	100 + 250 ms	250 ms
serialization/overhead	100 ms	150 ms
worst configured sum	1,500 ms	2,000 ms

EXECUTABLE ASSET 53-C - BUDGET GATE

```
public static void Validate(ResilienceProfile p)
{
    var attempts = p.MaxRetries + 1;
    var worst = TimeSpan.FromTicks(
        checked(p.AttemptTimeout.Ticks * attempts
            + p.MaximumDelay.Ticks
            + p.OverheadAllowance.Ticks));

    Require(worst <= p.TotalTimeout, "attempts exceed total timeout");
    Require(p.TotalTimeout <= p.CallerMaximum, "policy exceeds caller maximum");
}
```

UPPER-BOUND NOTE

A total timeout remains the enforcement authority even when the worksheet is conservative. Scheduling, DNS, connection pooling and cancellation cooperation can change observed timing.

TIMEOUT SEMANTICS

Total timeout, attempt timeout and caller cancellation are not synonyms

TIMEOUT MAP 53-H

Signal	Owner	Meaning	Result
caller token	upstream/shutdown	caller no longer waits	cancelled
total timeout	boundary profile	policy budget exhausted	incomplete/unknown
attempt timeout	inner attempt	one callback exceeded limit	retry candidate
HttpClient.Timeout	client instance	competing global clock	disabled when policy owns
DB command timeout	provider command	server call exceeded bound	classified failure

CRITICAL DISTINCTION

A timeout proves only that this caller stopped waiting. It does not prove that a remote server, broker or database rolled back the effect.

UNKNOWN EFFECT

After transmission, uncertainty is a first-class outcome

If the connection fails after bytes were sent or during commit acknowledgement, NEXUS-1 returns UNKNOWN_EFFECT. The next action is deduplication lookup or reconciliation by operation identity, not an unconditional replay.

CANCELLATION COOPERATION

A timeout works only when the callback honors its token

EXECUTABLE ASSET 53-D - COOPERATIVE REMOTE CALL

```
public async Task<RootCauseProjection> ReadAsync(
    CaseId caseId,
    CancellationToken cancellationToken)
{
    using var request = new HttpRequestMessage(
        HttpMethod.Get, $"v1/root-cause/{caseId.Value:D}");

    using var response = await client.SendAsync(
        request,
        HttpCompletionOption.ResponseHeadersRead,
        cancellationToken);
    response.EnsureSuccessStatusCode();

    await using var body = await response.Content.ReadAsStreamAsync(cancellationToken);
    return await JsonSerializer.DeserializeAsync<RootCauseProjection>(
        body, jsonOptions, cancellationToken)
        ?? throw new InvalidDataException("empty response");
}
```

CANCELLATION AUDIT 53-I

Operation	Token reaches
HTTP	SendAsync and body read
EF Core	query, SaveChanges and transaction APIs
broker	publish confirm / receive / settle
delay	TimeProvider.Delay or Task.Delay
channel	ReadAsync / WriteAsync

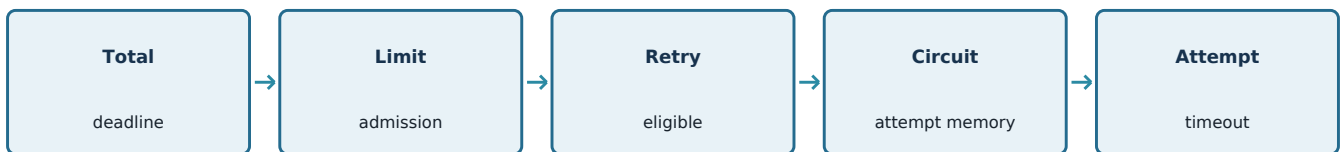
REJECTED IMPLEMENTATION

Starting unbounded work and racing it against Task.Delay only stops the wait; it leaves the work consuming sockets, connections or threads.

PIPELINE TOPOLOGY

Order determines which time and failures each strategy observes

NEXUS-1 HTTP PROFILE 53-B - OUTERMOST TO INNERMOST



ORDER PROOF 53-J

Strategy	Why here	Observed surface
total timeout	includes queue, delays and attempts	logical call
concurrency limiter	protects dependency capacity	admitted call
retry	owns repeat count and delay	logical operation
circuit breaker	samples each direct attempt	dependency partition
attempt timeout	bounds one callback	single attempt

ORDER QUALIFICATION

Microsoft's standard handler places its rate limiter outside the total timeout. NEXUS-1 uses a custom profile when queue wait must count against the caller budget; the descriptor test makes the difference explicit.

FROM SERVICES TO RUNTIME

ONE HANDLER OWNER

Do not stack a default handler and a custom handler

EXECUTABLE ASSET 53-E - ONE TYPED HTTP PIPELINE

```

services.AddHttpClient<RootCauseReadClient>((sp, client) =>
{
    var endpoint = sp.GetRequiredService<IOptions<RootCauseEndpoint>>().Value;
    client.BaseAddress = endpoint.BaseAddress;
    client.Timeout = Timeout.InfiniteTimeSpan;
})
.AddResilienceHandler("root-cause-read", static builder =>
{
    builder.AddTimeout(TimeSpan.FromMilliseconds(1500));
    builder.AddConcurrencyLimiter(permitLimit: 24, queueLimit: 0);
    builder.AddRetry(HttpProfiles.ReadRetry());
    builder.AddCircuitBreaker(HttpProfiles.ReadCircuit());
    builder.AddTimeout(TimeSpan.FromMilliseconds(350));
});

```

COMPOSITION GATES 53-K

Gate	Required finding
handler census	one resilience handler on client
descriptor order	timeout/limit/retry/circuit/timeout
client timeout	infinite; pipeline owns budget
profile key	catalogued and bounded
options	validated at startup

PACKAGE STATUS

The listing targets the current Microsoft.Extensions.Http.Resilience and Polly v8 model. Exact restored versions and overloads remain part of the repository evidence checkpoint.

STANDARD HANDLER BASELINE

Defaults are a starting inventory, not universal requirements

CURRENT DOCUMENTED STANDARD PROFILE 53-L

Order	Strategy	Documented default
1	rate limiter	queue 0; permits 1,000
2	total timeout	30 seconds
3	retry	3 retries; exponential; jitter; 2 s base
4	circuit breaker	10%; 100/30 s; break 5 s
5	attempt timeout	10 seconds

EXECUTABLE ASSET 53-F - SAFE-METHOD BASELINE

```

services.AddHttpClient<CatalogueReadClient>()
.AddStandardResilienceHandler(options =>
{
    options.Retry.DisableForUnsafeHttpMethods();
    options.TotalRequestTimeout.Timeout = TimeSpan.FromSeconds(2);
    options.AttemptTimeout.Timeout = TimeSpan.FromMilliseconds(450);
    options.Retry.MaxRetryAttempts = 2;
});

```

REVIEW RULE

The effective profile is emitted from options and compared with the architecture catalogue. Copying a default without latency, load and idempotency evidence is rejected.

RETRY ELIGIBILITY

Method safety is useful, but operation semantics decide

ELIGIBILITY MATRIX 53-M

Operation	Precondition	Retry decision
GET projection	no server mutation	yes for classified transient fault
PUT replacement	stable resource + conditional token	profile-specific
POST command	operation key + durable dedup receipt	bounded
DELETE	explicit idempotent contract	otherwise no
broker publish	stable MessageId + confirm semantics	only known pre-accept failure
DB commit	unknown commit state	reconcile; no blind retry

UNSAFE METHODS

The standard HTTP handler retries all methods unless configured otherwise. NEXUS-1 disables unsafe-method retries by default and enables a command profile only after deduplication evidence.

SAME CALLBACK

Retry re-executes the operation body

Therefore the callback contains one remote attempt, not domain mutation, SaveChanges, outbox insertion or a multi-step workflow. Retrying a broad handler repeats every side effect inside it.

IDEMPOTENT COMMAND ENVELOPE

A stable operation identity makes replay observable

EXECUTABLE ASSET 53-G - COMMAND REQUEST

```
public async Task<OpenCaseReply> OpenAsync(
    OpenCaseCommand command,
    OperationId operationId,
    CancellationToken cancellationToken)
{
    using var request = new HttpRequestMessage(HttpMethod.Post, "v1/root-cause-cases");
    request.Headers.TryAddWithoutValidation(
        "Idempotency-Key", operationId.Value.ToString("D"));
    request.Content = JsonContent.Create(command, options: jsonOptions);

    using var response = await commandClient.SendAsync(request, cancellationToken);
    return await replies.ReadAsync(response, operationId, cancellationToken);
}
```

RECEIVER PROOF 53-N

Condition	Receiver result
first operation key	commit effect + receipt atomically
same key, same digest	return retained result
same key, different digest	409 idempotency conflict
in-flight duplicate	bounded pending/accepted response
receipt expired	policy-defined non-retryable state

PROOF BURDEN

An Idempotency-Key header alone proves nothing. The receiver needs durable uniqueness, request-digest conflict detection, an atomic receipt and a retained duplicate campaign.

RETRY PREDICATE

Handle a finite set of transport outcomes

EXECUTABLE ASSET 53-H - READ RETRY OPTIONS

```
public static HttpRetryStrategyOptions ReadRetry() => new()
{
    MaxRetryAttempts = 2,
    Delay = TimeSpan.FromMilliseconds(100),
    BackoffType = DelayBackoffType.Exponential,
    UseJitter = true,
    ShouldHandle = new PredicateBuilder<HttpResponseMessage>()
        .Handle<HttpRequestException>()
        .Handle<TimeoutRejectedException>()
        .HandleResult(static response => response.StatusCode is
            HttpStatusCode.RequestTimeout or
            HttpStatusCode.TooManyRequests or
            HttpStatusCode.BadGateway or
            HttpStatusCode.ServiceUnavailable or
            HttpStatusCode.GatewayTimeout)
};
```

PREDICATE EXCLUSIONS 53-O

Excluded	Reason
OperationCanceledException	caller/shutdown authority
400/401/403/404	caller, security or durable absence
contract parse failure	version/implementation defect
RateLimiterRejectedException	local saturation must escape
BrokenCircuitException	breaker is already refusing load

RESPONSE LIFECYCLE

Every response consumed or abandoned by application code is disposed. Component tests track response and stream disposal across success and terminal failure.

BACKOFF AND SERVER GUIDANCE

Delay is bounded by both Retry-After and remaining budget

EXECUTABLE ASSET 53-I - CAPPED DELAY GENERATOR

```
static ValueTask<TimeSpan?> NextDelay(
    RetryDelayGeneratorArguments<HttpResponseMessage> args)
{
    var requested = args.Outcome.Result?.Headers.RetryAfter?.Delta;
    if (requested is { } retryAfter)
        return ValueTask.FromResult<TimeSpan?>(
            TimeSpan.FromMilliseconds(Math.Min(retryAfter.TotalMilliseconds, 500)));

    var milliseconds = 100 * Math.Pow(2, args.AttemptNumber);
    return ValueTask.FromResult<TimeSpan?>(
        TimeSpan.FromMilliseconds(Math.Min(milliseconds, 500)));
}
```

DELAY GATES 53-P

Gate	Rule
Retry-After date	convert with trusted clock; clamp negative
maximum delay	never exceeds profile cap
remaining deadline	no retry when next attempt cannot fit
jitter	enabled in runtime; deterministic source in tests
shutdown	delay observes stopping token

NO LOCAL RETRY STORM

A bulkhead rejection is not fed back into the same local retry loop. That would turn intentional load shedding into delayed self-amplification.

RETRY AMPLIFICATION

Count logical operations and dependency attempts separately

AMPLIFICATION MODEL 53-Q

Layers	Max attempts per layer	Worst dependency calls
one owner	3	3
edge + service	3 x 3	9
edge + service + SDK	3 x 3 x 3	27
two hedges + two retries	2 x 3	6 concurrent/sequential

EXECUTABLE ASSET 53-J - AMPLIFICATION VERDICT

```
public static decimal Amplification(long attempts, long logicalStarts) =>
    logicalStarts == 0 ? 0 : (decimal)attempts / logicalStarts;

Require(corpus.LogicalStarts == 200, "workload drift");
Require(corpus.DependencyAttempts <= 600, "attempt budget exceeded");
Require(Amplification(corpus.DependencyAttempts, corpus.LogicalStarts) <= 3m,
    "retry amplification exceeded profile");
```

METRIC INTERPRETATION

High amplification can be policy behavior, redelivery, hedging or caller duplication. The campaign joins the bounded metric with traces and operation receipts before assigning cause.

CIRCUIT BREAKER MODEL

A circuit remembers recent dependency outcomes

STATE MACHINE 53-R

State	Admission	Transition evidence
Closed	normal	sampled successes/failures
Open	short-circuit	threshold and break deadline
HalfOpen	bounded probe	probe result
Isolated	manual refusal	operator identity/reason

EXECUTABLE ASSET 53-K - CIRCUIT OPTIONS

```
public static HttpCircuitBreakerStrategyOptions ReadCircuit(
    CircuitBreakerStateProvider state) => new()
{
    FailureRatio = 0.25,
    MinimumThroughput = 20,
    SamplingDuration = TimeSpan.FromSeconds(20),
    BreakDuration = TimeSpan.FromSeconds(10),
    StateProvider = state,
    ShouldHandle = static args => ValueTask.FromResult(
        HttpClientResiliencePredicates.IsTransient(args.Outcome))
};
```

CIRCUIT CLAIM

The breaker does not retry or repair a dependency. During sampling it returns the original result/exception; while open it rejects calls with a broken-circuit signal.

THRESHOLD SELECTION

Failure ratio is meaningless without minimum throughput and time

THRESHOLD WORKSHEET 53-S

Input	Evidence	Chosen value
normal call rate	Chapter 52 campaign	60-90 / 20 s
normal transient ratio	baseline corpus	< 2%
incident failure ratio	fault fixture	> 60%
minimum throughput	avoid one-call opening	20 / window
failure ratio	separate baseline/incident	25%
break duration	recovery vs refusal cost	10 s

TRAFFIC QUALIFICATION 53-T

Traffic	Consequence
below minimum throughput	circuit remains closed; other defenses still apply
burst above threshold	open after sampling criteria are satisfied
no traffic while open	time can permit half-open probe
partitioned clients	each partition has independent sample

NO MAGIC PERCENTAGES

The values are an educational NEXUS-1 profile derived from a declared campaign. They are not production recommendations until representative traffic evidence and ownership approval exist.

CIRCUIT PARTITIONING

Share memory by dependency capacity, not by business identity

PARTITION CHOICE 53-U

Candidate	Decision	Reason
scheme + authority	baseline	shared remote capacity
endpoint class	when capacity differs	read/write pools may differ
region	when routed independently	fault domain separation
tenant	usually reject	unbounded state/cardinality
Messaged / Caseld	forbidden	one circuit per operation
random instance	reject	no stable memory

EXECUTABLE ASSET 53-L - BOUNDED KEY

```
public readonly record struct DependencyPartition(string Value)
{
    public static DependencyPartition RootCauseRead() => new("root-cause/read");
    public static DependencyPartition RootCauseCommand() => new("root-cause/command");
    public static DependencyPartition BrokerPublish() => new("broker/publish");
}
```

STATE LIFETIME

The pipeline and its breaker state are container-managed and long-lived. Constructing a pipeline per call discards the failure history and defeats the pattern.

STATE OBSERVATION

Expose transitions and current state without polling per request

EXECUTABLE ASSET 53-M - TRANSITION CALLBACKS

```
var state = new CircuitBreakerStateProvider();
var options = HttpProfiles.ReadCircuit(state);
options.OnOpened = args =>
{
    observer.Record("opened", args.BreakDuration, args.Outcome.Exception);
    return ValueTask.CompletedTask;
};
options.OnHalfOpened = _ =>
{
    observer.Record("half-opened");
    return ValueTask.CompletedTask;
};
options.OnClosed = _ =>
{
    observer.Record("closed");
    return ValueTask.CompletedTask;
};
```

TRANSITION EVIDENCE 53-V

Field	Constraint
pipeline/partition	bounded catalogue value
from/to state	finite vocabulary
occurred UTC	observer clock
break duration	configured/effective
failure class	bounded; no raw exception label
trace link	log/trace locator, never metric label

OBSERVATION RULE

A gauge of current circuit state is diagnostic. The breaker implementation remains the admission authority; a dashboard cannot force a transition.

MANUAL ISOLATION

Isolated is an operator action, not an automatic fallback

CONTROL CONTRACT 53-W

Action	Required evidence
isolate	authorized actor, dependency, reason, expiry
extend	new approval and reason
close	recovery evidence or emergency decision
startup	isolation state source and freshness
audit	before/after state and configuration digest

EXECUTABLE ASSET 53-N - BOUNDED CONTROL

```
public sealed record IsolationCommand(
    DependencyPartition Partition,
    string ReasonCode,
    DateTimeOffset ExpiresUtc,
    ActorId Actor);

public interface ICircuitIsolationControl
{
    Task IsolateAsync(IsolationCommand command, CancellationToken token);
    Task CloseAsync(DependencyPartition partition, ActorId actor,
        CancellationToken token);
}
```

FAIL-SAFE EXPIRY

The control plane never silently converts an expired isolation into healthy. It records expiry, evaluates the configured release rule and retains the operator event.

BULKHEAD SEMANTICS

In the current Polly model, outbound bulkhead is a concurrency limiter

RESOURCE ISOLATION 53-X

Control	Bounds	Does not bound
concurrency permits	active callbacks	work outside pipeline
queue limit	waiting callbacks	upstream queues
queue wait	admission latency	remote attempt duration
separate partition	cross-dependency interference	shared OS exhaustion
connection pool	open/leased connections	application tasks

EXECUTABLE ASSET 53-O - ZERO-QUEUE PROFILE

```
var pipeline = new ResiliencePipelineBuilder<HttpResponseBody>()
    .AddTimeout(TimeSpan.FromMilliseconds(1500))
    .AddConcurrencyLimiter(permitLimit: 24, queueLimit: 0)
    .AddRetry(HttpProfiles.ReadRetry())
    .AddCircuitBreaker(HttpProfiles.ReadCircuit(new()))
    .AddTimeout(TimeSpan.FromMilliseconds(350))
    .Build();
```

TERMINOLOGY

This chapter retains 'bulkhead' because it appears in the Contents, but implements it with modern concurrency/rate-limiting primitives rather than the legacy Polly BulkheadPolicy API.

QUEUE POLICY

Waiting is work, so its budget and order must be explicit

QUEUE DECISION 53-Y

Profile	Queue	Use
interactive read	0	shed immediately; upstream can respond
interactive command	0	avoid hidden command latency
bounded repair	small FIFO	operator-owned throughput smoothing
broker consumer	bounded channel	pause intake / prefetch
unbounded	forbidden	memory and latency become outage

EXECUTABLE ASSET 53-P - OVERLOAD OUTCOME

```
catch (RateLimiterRejectedException ex)
{
    var retryAfter = ex.RetryAfter is { } value
        ? TimeSpan.FromMilliseconds(Math.Min(value.TotalMilliseconds, 500))
        : (TimeSpan?)null;
    return RemoteCallResult<Reply>.Overloaded("dependency_bulkhead", retryAfter);
}
```

NO RETRY LOOP

The caller may return 429/503 with a bounded Retry-After when its contract permits. The same process does not immediately retry its own admission rejection.

WORKER BACKPRESSURE

Prefetch, channel capacity and processing concurrency form one budget

EXECUTABLE ASSET 53-Q - BOUNDED WORKER CHANNEL

```
var channel = Channel.CreateBounded<InboundEnvelope>(
    new BoundedChannelOptions(capacity: 64)
    {
        FullMode = BoundedChannelFullMode.Wait,
        SingleWriter = true,
        SingleReader = false
    });

await broker.ConfigurePrefetchAsync(64, stoppingToken);
await Parallel.ForEachAsync(
    channel.Reader.ReadAllAsync(stoppingToken),
    new ParallelOptions { MaxDegreeOfParallelism = 16,
        CancellationToken = stoppingToken },
    (envelope, token) => processor.ProcessAsync(envelope, token));
```

CONSERVATION GATES 53-Z

Quantity	Gate
broker prefetch	<= channel capacity
processor concurrency	<= DB/dependency safe capacity
lease/lock duration	exceeds bounded processing budget
shutdown drain	bounded and observable
unsettled messages	count reconciles with broker state

BACKPRESSURE PATH

When the local channel fills, intake pauses or broker credit is withheld. The system does not keep accepting work into a hidden unbounded list.

CAPACITY DERIVATION

Permit count follows dependency capacity and latency evidence

CAPACITY WORKSHEET 53-AA

Input	Observed/declared	Use
dependency safe concurrency	32 calls	hard upper bound
connection pool allocation	30 connections	cross-check
p95 attempt duration	180 ms	throughput estimate
reserved diagnostic capacity	4 calls	incident access
client permit limit	24	headroom retained
queue limit	0	no hidden interactive wait

ISOLATION LEDGER 53-AB

Dependency	Permits	Queue	Separate pool?
RootCause read	24	0	HTTP authority
RootCause command	8	0	logical partition
broker publish	16	0	broker channel pool
projection DB	12	0	DbContext/connection budget
repair scan	2	8	low-priority worker

EVIDENCE STATUS

These are campaign values, not production capacity claims. A representative load test, provider limits and operational approval must replace them before deployment.

DATABASE BOUNDARY

Provider retries must not hide an unknown commit

DATABASE OPERATION MATRIX 53-AC

Operation	Policy
read-only query	command timeout; provider retry if explicitly verified
single idempotent statement	provider-specific bounded strategy
transaction before commit sent	new scope may replay full unit
commit acknowledgement lost	UNKNOWN_COMMIT; reconcile key
concurrency conflict	application reload/redecide, not transport retry
constraint violation	permanent mapped outcome

EXECUTABLE ASSET 53-R - FRESH-SCOPE REPLAY BOUNDARY

```
public async Task<T> ExecuteUnitAsync<T>(  
    Func<IServiceProvider, CancellationToken, Task<T>> unit,  
    CancellationToken token)  
{  
    return await transactionReplay.ExecuteAsync(async attemptToken =>  
    {  
        await using var scope = scopes.CreateAsyncScope();  
        return await unit(scope.ServiceProvider, attemptToken);  
    }, token);  
}
```

SEMANTIC RETRY

A fresh DbContext prevents tracked-state reuse, but replay is still legal only when the whole unit is idempotent and the provider can classify whether commit began. Otherwise reconciliation wins.

BROKER PUBLISH BOUNDARY

Publish confirmation produces accepted, rejected or unknown

EXECUTABLE ASSET 53-S - TRUTHFUL PUBLISH OUTCOME

```
public enum PublishDisposition { Accepted, Rejected, Unknown }
```

```
public sealed record PublishResult(  
    PublishDisposition Disposition,  
    MessageId MessageId,  
    string? FailureCode);
```

```
public interface IConfirmingPublisher  
{  
    Task<PublishResult> PublishAsync(  
        PublicEnvelope envelope,  
        CancellationToken cancellationToken);  
}
```

PUBLISH POLICY 53-AD

Finding	Action
connection unavailable before send	bounded reconnect/retry
broker negative acknowledgement	classified rejected
positive confirmation	mark outbox dispatched
timeout after send	retain pending/unknown; reconcile or safe redeliver
duplicate accepted	consumer Inbox prevents repeat effect

OUTBOX AUTHORITY

The publisher never invents delivery from a timeout. The outbox row, stable MessageId, confirm corpus and downstream Inbox together bound recovery.

FALLBACK POLICY

Degradation may change availability, never business truth

FALLBACK MATRIX 53-AE

Fallback	Decision	Required contract
fabricated default verdict	forbidden	none can make it true
treat timeout as rejected	forbidden	unknown is not rejection
stale read projection	conditional	age/source visible; read-only
cached capability catalogue	conditional	signed version + expiry
enqueue for later	conditional	durable acceptance and receipt
fail fast	allowed	truthful unavailable/overloaded result

EXECUTABLE ASSET 53-T - EXPLICIT STALE READ

```
public sealed record DegradedProjection<T>(  
    T Value,  
    DateTimeOffset ObservedUtc,  
    TimeSpan Age,  
    string Source,  
    bool IsStale);
```

NO SEMANTIC CAMOUFLAGE

A fallback result has a distinct contract and telemetry outcome. It cannot reuse the normal success code while hiding staleness or incomplete work.

POLICY REGISTRY

Typed catalogue entries become named DI pipelines

EXECUTABLE ASSET 53-U - KEYED PIPELINE REGISTRATION

```

services.AddResiliencePipeline<string, PublishResult>(
    "broker-publish", static builder =>
    {
        builder.AddTimeout(TimeSpan.FromSeconds(2));
        builder.AddConcurrencyLimiter(16, 0);
        builder.AddCircuitBreaker(new CircuitBreakerStrategyOptions<PublishResult>
        {
            ShouldHandle = new PredicateBuilder<PublishResult>()
                .HandleResult(r => r.Disposition == PublishDisposition.Rejected),
            FailureRatio = 0.25,
            MinimumThroughput = 20,
            SamplingDuration = TimeSpan.FromSeconds(20),
            BreakDuration = TimeSpan.FromSeconds(10)
        });
    });

```

REGISTRY GATES 53-AF

Gate	Rejects
key inventory	unknown or duplicate pipeline
typed result	exception-only business mapping
owner	two retry layers for same call
lifetime	pipeline constructed per operation
options digest	runtime drift from approved profile

DI USE

Callers resolve a long-lived `ResiliencePipelineProvider` key or typed client. Domain assemblies do not reference `Polly` or `Microsoft.Extensions.Http.Resilience`.

OPTIONS AND STARTUP VALIDATION

Invalid budgets fail before traffic is accepted

EXECUTABLE ASSET 53-V - PROFILE CONTRACT

```

public sealed class ResilienceProfile
{
    public required string BoundaryKey { get; init; }
    public required TimeSpan CallerMaximum { get; init; }
    public required TimeSpan TotalTimeout { get; init; }
    public required TimeSpan AttemptTimeout { get; init; }
    public required TimeSpan MaximumDelay { get; init; }
    public required TimeSpan OverheadAllowance { get; init; }
    public required int MaxRetries { get; init; }
    public required int PermitLimit { get; init; }
    public required int QueueLimit { get; init; }
    public required bool IdempotencyRequired { get; init; }
}

```

VALIDATION RULES 53-AG

Rule	Failure
known boundary key	orphan policy
positive ordered timeouts	ineffective/contradictory budget
attempts and delays fit total	unreachable configuration
unsafe retry requires idempotency	duplicate effect risk
positive permit; bounded queue	resource exhaustion
circuit sampling coherent	never-open or flapping circuit

RELOAD RULE

A reloaded profile is bound, fully validated and swapped atomically. Existing executions retain their pipeline snapshot; partial mutation is forbidden.

REMOTE RESULT VOCABULARY

Application code receives outcomes, not provider folklore

EXECUTABLE ASSET 53-W - BOUNDARY RESULT

```
public abstract record RemoteCallResult<T>
{
    public sealed record Completed(T Value) : RemoteCallResult<T>;
    public sealed record Rejected(string Code) : RemoteCallResult<T>;
    public sealed record Unavailable(string Code, TimeSpan? RetryAfter)
        : RemoteCallResult<T>;
    public sealed record UnknownEffect(OperationId OperationId, string Code)
        : RemoteCallResult<T>;
    public sealed record Cancelled : RemoteCallResult<T>;
}
```

MAPPING RULES 53-AH

Infrastructure finding	Application outcome
2xx + valid contract	Completed
domain-declared rejection	Rejected
open circuit / overload	Unavailable
timeout after possible effect	UnknownEffect
caller cancellation	Cancelled
contract defect	throw/defect evidence; not retry

TRUTH PRESERVATION

The application can defer, reconcile or return a problem response. It cannot reinterpret Unavailable or UnknownEffect as a domain rejection or success.

RESILIENCE TELEMETRY

Observe policies with bounded pipeline and strategy names

TELEMETRY CATALOGUE 53-AI

Event/metric	Question	Bounded attributes
logical starts	how much work entered?	boundary, operation
dependency attempts	how much remote load?	boundary, outcome
OnRetry	why was repeat scheduled?	failure class, attempt
OnTimeout	which budget expired?	total/attempt
circuit transition	when did admission change?	partition, state
limiter rejection	where was load shed?	boundary

EXECUTABLE ASSET 53-X - BOUNDED OBSERVER

```
public void OnRetry(string boundary, int attempt, RemoteFailure failure)
{
    metrics.RetryScheduled.Add(1,
        new("nexus1.boundary", boundary),
        new("error.type", failure.Code));
    logger.LogWarning("Retry scheduled {Boundary} attempt {Attempt} class {Class}",
        boundary, attempt, failure.Class);
}
```

CARDINALITY RULE

Polly operation keys and custom enrichers use reviewed vocabularies. URLs, operation IDs, trace IDs and exception messages remain out of metric tags.

TRACE SEMANTICS

One logical span may contain several dependency attempt spans

TRACE MODEL 53-AJ

Span	Represents	Key attributes/events
application operation	logical owner call	operation, final outcome
dependency attempt	one transmitted attempt	attempt number, dependency
retry event	repeat decision	delay, failure class
circuit event	admission decision	state, retry-after
bulkhead event	local refusal/wait	permit/queue class

SAMPLING QUALIFICATION

Chapter 52 metrics retain the full population even when attempt spans are sampled out. A trace illustrates one execution; amplification metrics quantify the campaign.

LOG SEMANTICS

A policy event logs the decision, not the full exception payload

Logs carry stable boundary, strategy, attempt, state and error class plus protected trace linkage. Raw response bodies, credentials and payloads remain excluded.

BASELINE CAMPAIGN

First prove that healthy traffic is not damaged

CAMPAIGN 53-AK

Input	Expected
200 safe reads	200 completions; 200 attempts
50 idempotent commands	50 durable effects and receipts
no injected faults	no retry, timeout or circuit event
concurrency below permits	no limiter rejection
fresh metric pipeline	attempt/logical ratio = 1

BASELINE ORACLE 53-AL

Proof	Gate
latency	within declared campaign distribution
conservation	logical starts = terminal results
attempts	exactly one per logical call
circuit	Closed throughout
business state	expected durable facts exactly once

CAMPAIGN IDENTITY

The run retains build, config, profile, dependency fixture, workload seed, monotonic timings, telemetry corpus and durable-state snapshot digests.

RETRY CAMPAIGN

Transient failures recover without exceeding amplification

CAMPAIGN 53-AM

Stimulus	Expected
first attempt returns 503 for 40 reads	one bounded retry each
next attempt succeeds	40 logical completions
20 permanent 400 responses	zero retry
10 caller cancellations	zero retry and circuit sample
no layered retry fixture	attempt ceiling remains profile-bound

EXECUTABLE ASSET 53-Y - CAMPAIGN ASSERTIONS

```
Assert.Equal(70, corpus.LogicalStarts);
Assert.Equal(110, corpus.DependencyAttempts);
Assert.Equal(40, corpus.RetryScheduled);
Assert.Equal(0, corpus.UnexpectedDuplicateEffects);
Assert.True(corpus.Amplification <= 2m);
Assert.DoesNotContain(corpus.RetryCodes, code => code is "bad_request");
```

EVIDENCE MEANING

Recovery is valid only if remote attempts, operation receipts and durable effects reconcile. A final 200 alone cannot prove the first attempt produced no duplicate effect.

TIMEOUT CAMPAIGN

A slow dependency consumes no more than the declared budget

CAMPAIGN 53-AN

Stimulus	Expected evidence
attempt sleeps 600 ms cooperatively	350 ms attempt timeout
two eligible retries	total remains <= 1,500 ms + scheduler tolerance
callback observes token	active work returns to zero
command loses acknowledgement	UNKNOWN_EFFECT, not rejected
reconciliation finds receipt	terminal completed exactly once

TIMING ORACLE 53-AO

Measure	Gate
attempt elapsed	timeout event near configured bound
logical elapsed	within total bound and tolerance
post-return active calls	zero after grace window
retry delay	within capped generated sequence
business state	unknown until reconciled

SCHEDULER TOLERANCE

The campaign records a platform-specific tolerance separately from the configured budget. A wide unexplained tolerance invalidates the timing claim.

CIRCUIT CAMPAIGN

Closed, Open, HalfOpen and Closed are retained as a sequence

CAMPAIGN 53-AP

Phase	Stimulus	Expected state/action
baseline	20 successes	Closed
fault	20 attempts; >25% transient failures	Open
open interval	10 logical calls	short-circuited; zero remote
probe	break duration expires	HalfOpen; bounded remote probe
recovery	probe succeeds	Closed
post	10 successes	normal admission

EXECUTABLE ASSET 53-Z - STATE ORACLE

```
Assert.Equal(new[] { "closed", "opened", "half-opened", "closed" },  
    evidence.StateSequence);  
Assert.Equal(0, evidence.RemoteAttemptsDuringOpen);  
Assert.InRange(evidence.OpenDuration, TimeSpan.FromSeconds(10), tolerance.Maximum);  
Assert.Equal(CircuitState.Closed, evidence.FinalState);
```

SHORT-CIRCUIT RESULT

An open circuit produces an unavailable result with bounded `RetryAfter` when present. It never produces a cached success unless a separate explicit degraded-read contract applies.

BULKHEAD CAMPAIGN

Saturation is contained at the named dependency

CAMPAIGN 53-AQ

Stimulus	Expected
hold 24 admitted reads	active = 24
submit 16 more	16 immediate local rejections
queue limit zero	waiting = 0
release dependency	all 24 admitted calls complete
send broker publish concurrently	separate partition remains available
inspect memory/tasks	no unbounded growth

ISOLATION ORACLE 53-AR

Proof	Gate
maximum active	never above permit limit
rejection count	matches excess submissions
queue wait	zero for interactive profile
other dependency	latency/error remain baseline
business state	rejected admissions create no effects

LOAD-SHEDDING TRUTH

A rejected admission is an availability outcome. It is not a dependency failure sample and must not open the dependency circuit.

DELIBERATELY BROKEN CAMPAIGN

Nested retry proves the amplification gate

EXECUTABLE ASSET 53-AA - CONTROLLED ANTI-PATTERN

```
var inner = new ResiliencePipelineBuilder()
    .AddRetry(new RetryStrategyOptions { MaxRetryAttempts = 2 })
    .Build();
var outer = new ResiliencePipelineBuilder()
    .AddRetry(new RetryStrategyOptions { MaxRetryAttempts = 2 })
    .Build();

await outer.ExecuteAsync(token =>
    inner.ExecuteAsync(_ => dependency.AlwaysFailsAsync(), token));
```

BROKEN ORACLE 53-AS

Layer	Expected finding
descriptor/registration	two retry owners detected
remote fixture	9 attempts for one logical start
amplification gate	ratio 9 > approved maximum
metric campaign	REJECTED_CONFIGURATION
business state	no fabricated completion
production composition	fault fixture absent

FAULT ISOLATION

The nested pipeline exists only in the evidence project under an explicit fault profile. It demonstrates detection; it is not copied into production code.

DETERMINISTIC DEPENDENCY FIXTURE

Scripts drive attempts by number and virtual time

EXECUTABLE ASSET 53-AB - SCRIPTED DEPENDENCY

```
public sealed class ScriptedDependency(Queue<AttemptScript> scripts)
{
    public int Attempts { get; private set; }

    public async Task<HttpResponseMessage> SendAsync(CancellationToken token)
    {
        Attempts++;
        var script = scripts.Dequeue();
        await Task.Delay(script.Delay, token);
        if (script.Exception is not null) throw script.Exception;
        return new HttpResponseMessage(script.StatusCode);
    }
}

public sealed record AttemptScript(
    TimeSpan Delay, HttpStatusCode StatusCode, Exception? Exception = null);
```

FIXTURE CONTROLS 53-AT

Control	Purpose
attempt script	exact fault sequence
TimeProvider	bounded deterministic timing
concurrency latch	hold/release saturation
operation receipt store	duplicate-effect oracle
attempt ledger	independent call count

TEST BOUNDARY

Tests assert NEXUS-1 options, predicates, ordering and outcomes. They do not re-test Polly's internal algorithms.

DESCRIPTOR TESTS

Pipeline order and settings are executable architecture

EXECUTABLE ASSET 53-AC - POLLY.TESTING DESCRIPTOR

```
var descriptor = pipeline.GetPipelineDescriptor();
Assert.Collection(descriptor.Strategies,
    s => Assert.IsType<TimeoutStrategyOptions>(s.Options),
    s => Assert.IsType<RateLimiterStrategyOptions>(s.Options),
    s => Assert.IsType<RetryStrategyOptions<HttpResponseMessage>>(s.Options),
    s => Assert.IsType<CircuitBreakerStrategyOptions<HttpResponseMessage>>(s.Options),
    s => Assert.IsType<TimeoutStrategyOptions>(s.Options));

var retry = (RetryStrategyOptions<HttpResponseMessage>)
    descriptor.Strategies[2].Options;
Assert.Equal(2, retry.MaxRetryAttempts);
```

DESCRIPTOR GATES 53-AU

Assertion	Failure caught
strategy count/order	stacking or reorder
timeouts	budget drift
retry count/delay	amplification drift
circuit thresholds	never-open/flapping profile
permit/queue	resource-isolation drift

TESTING PACKAGE

Polly.Testing exposes pipeline descriptors specifically so application tests can verify composition and options without mocking implementation internals.

HTTP COMPONENT TESTS

A real handler chain proves request reuse and terminal mapping

EXECUTABLE ASSET 53-AD - COMPONENT ORACLE

```
var fixture = new DependencyServer()
    .Then(HttpStatusCode.ServiceUnavailable)
    .Then(HttpStatusCode.OK, json: "{ \"status\": \"ready\" }");

await using var host = await TestHost.StartAsync(fixture);
var result = await host.RootCauseRead.ReadAsync(caseId, CancellationToken.None);

Assert.Equal("ready", result.Status);
Assert.Equal(2, fixture.Requests.Count);
Assert.Equal(1, host.ResilienceEvents.Count(e => e.Name == "OnRetry"));
Assert.All(fixture.Requests, r => Assert.Equal("traceparent", r.TraceHeaderName));
```

HTTP CASES 53-AV

Case	Required assertion
503 then 200	one retry, one logical success
400	no retry
slow body	body read honors token
caller cancellation	no retry/circuit count
unsafe POST without contract	one attempt only
idempotent POST lost reply	receipt reconciliation

STRUCTURAL ARCHITECTURE TESTS

Resilience dependencies stay outside Domain and Application

EXECUTABLE ASSET 53-AE - ASSEMBLY GATE

```
var result = Types.InAssemblies(DomainAssembly, ApplicationAssembly)
    .ShouldNot()
    .HaveDependencyOnAny(
        "Polly",
        "Microsoft.Extensions.Http.Resilience",
        "System.Net.Http",
        "System.Threading.RateLimiting")
    .GetResult();

Assert.True(result.IsSuccessful, result.FormatFailures());
```

REPOSITORY SCANS 53-AW

Scan	Rejects
AddResilienceHandler census	more than one per client
retry literals	policy outside registry
catch-all predicates	permanent/caller cancellation retry
Task.WhenAny timeout	abandoned unbounded work
unbounded Channel	hidden queue
fallback result	normal-success camouflage

GENERATED-CODE CAVEAT

Static scans produce candidates; descriptor, component and campaign evidence decide. Suppressions name owner, reason, expiry and reviewed replacement proof.

EVIDENCE MANIFEST

A resilience claim binds policy, workload, attempts and durable effects

EXECUTABLE ASSET 53-AF - EVIDENCE RECORD

```
public sealed record ResilienceEvidenceManifest(
    string BuildDigest,
    string PackageLockDigest,
    string ConfigurationDigest,
    string BoundaryCatalogueDigest,
    string PipelineDescriptorDigest,
    string CampaignId,
    DateTimeOffset StartedUtc,
    DateTimeOffset EndedUtc,
    long LogicalOperations,
    long DependencyAttempts,
    int MaximumObservedConcurrency,
    CircuitTransition[] CircuitTransitions,
    ArtifactRef[] Artifacts,
    EvidenceVerdict Verdict);
```

REQUIRED ARTIFACTS 53-AX

Artifact	Purpose
effective option snapshot	policy actually executed
pipeline descriptor	strategy order and settings
attempt ledger	amplification and timing
transition corpus	circuit chronology
concurrency census	bulkhead bound
operation receipts/state	effect conservation
telemetry bundle	logs, traces and metrics

FAIL-CLOSED EVIDENCE VERDICT

Zero traffic, missing receipts or excess attempts invalidate the claim

EXECUTABLE ASSET 53-AG - VERIFIER

```
public static EvidenceVerdict Verify(ResilienceEvidenceManifest m)
{
    if (m.LogicalOperations <= 0)
        return EvidenceVerdict.Invalid("zero logical operations");
    if (m.DependencyAttempts < m.LogicalOperations)
        return EvidenceVerdict.Invalid("attempt ledger incomplete");
    if (m.MaximumObservedConcurrency <= 0)
        return EvidenceVerdict.Invalid("concurrency not observed");
    if (m.Artifacts.Any(a => !a.Exists || !a.DigestVerified))
        return EvidenceVerdict.Invalid("missing or stale artifact");
    if (!EffectConservation.Verified(m.CampaignId))
        return EvidenceVerdict.Invalid("durable effects not reconciled");
    return EvidenceVerdict.Valid;
}
```

VERDICT VOCABULARY 53-AY

Status	Meaning
SPECIFIED	boundary, policy and campaigns designed
EXECUTED	fresh retained campaigns pass
PARTIAL	named subset with residuals
INVALID	evidence cannot support claim
PENDING	no fresh runtime campaign retained

CURRENT VERDICT

SPECIFIED | RUNTIME PENDING. The manuscript defines executable policy composition and evidence campaigns; it does not claim that a fresh NEXUS-1 runtime executed them.

IMPLEMENTATION SEQUENCE

Start with semantics and a deadline, not a retry count

SEQUENCE 53-AZ

Step	Deliverable	Gate
1	remote/resource boundary catalogue	owner and semantics reviewed
2	failure and result vocabulary	unknown remains explicit
3	deadline and budget worksheet	worst sum bounded
4	idempotency/receipt contract	duplicate campaign
5	single ordered pipeline	descriptor test
6	bounded telemetry	cardinality scan
7	baseline and four fault campaigns	all oracles pass
8	evidence manifest	fail-closed verifier

PULL-REQUEST EVIDENCE 53-BA

Change	Required proof
new retry	idempotency + amplification budget
timeout increase	caller budget + load impact
circuit threshold	traffic and failure campaign
permit/queue	dependency capacity and saturation run
fallback	distinct truthful contract

STANDARDS AND SOURCE LEDGER

Current APIs guide mechanics; NEXUS-1 owns semantic admission

SOURCE LEDGER 53-BB

Source	Used for	Local qualification
Microsoft .NET HTTP resilience	standard/custom handler and defaults	one handler; profile reviewed
Polly retry	predicate, delay, callback replay	idempotency and owner gate
Polly timeout	cooperative token and timeout signal	unknown effect retained
Polly circuit breaker	states, thresholds and providers	bounded partition
Polly rate limiter	outbound concurrency limiting	bulkhead terminology
Polly telemetry/testing	events and descriptors	bounded names; app settings only

VERSION POLICY

Microsoft.Extensions.Http.Resilience, Polly, Polly.RateLimiting and Polly.Testing are pinned by package lock and descriptor snapshots. The retained executable artifact overrides a stale manuscript signature.

ANTI-PATTERN REVIEW

Six attractive resilience claims remain invalid

REJECTED CLAIMS 53-BC

Claim	Why invalid
retry makes POST safe	receiver idempotency is separate
timeout cancels server work	only caller wait is bounded
open circuit means dependency down	it reflects local sample
bulkhead guarantees fairness	it bounds concurrency/queue
fallback preserves success	may fabricate/stale truth
default profile is best practice	workload and semantics differ

COMPLETION LEDGER

What Chapter 53 now specifies

COMPLETION LEDGER 53-BD

Obligation	Asset	Status
boundary admission	ADR + typed catalogue	SPECIFIED
timeout budget	monotonic deadline + algebra	SPECIFIED
retry safety	predicate, idempotency and amplification	SPECIFIED
circuit behavior	thresholds, partitions and transitions	SPECIFIED
resource isolation	concurrency/queue/worker bounds	SPECIFIED
truthful outcomes	unknown, unavailable and degraded contracts	SPECIFIED
complete/broken campaigns	structural oracles	SPECIFIED
runtime proof	executed evidence manifest	PENDING

OPEN RESIDUALS 53-BE

Residual	Consequence
no repository restore/build	listings remain compile-oriented
no real dependency fault run	timeout/circuit timing unproved
no provider/broker campaign	unknown-effect handling unproved
no representative load	permit and threshold values educational
no recovery campaign	restart and reconciliation evidence pending Ch54

EVIDENCE VERDICT

SPECIFIED | RUNTIME PENDING. Semantics, budgets, ordering, isolation, telemetry and campaign oracles are complete; no fresh runtime execution is claimed.

HANDOFF TO CHAPTER 54

Failure Injection and Recovery Evidence

Chapter 54 will execute the policies specified here against deliberate failure: kill a consumer, stop the broker, pause the database, duplicate a message, delay a projection and restart a service. Each scenario must retain the expected effect, actual observation, trace evidence and bounded recovery result.

BOUNDARY CARRIED FORWARD 53-BF

Chapter 53 leaves	Chapter 54 must add
named resilience profiles	fault-to-profile campaign matrix
attempt and state telemetry	retained incident chronology
unknown-effect outcomes	reconciliation and recovery proof
bulkhead/backpressure bounds	saturation and drain evidence
SPECIFIED evidence manifest	executed or honestly failed verdict

NEXT INVARIANT

A recovery claim is admitted only when the injected fault, expected behavior, observed telemetry, durable-state oracle and bounded return to service are retained under one campaign identity.

FINAL CHAPTER RULE

Resilience limits failure propagation; it does not rewrite reality

NEXUS-1 uses deadlines to bound waiting, retries to repeat only proven-safe attempts, circuits to refuse recently failing dependencies and concurrency limiters to isolate capacity. Every terminal result preserves completion, rejection, unavailability, cancellation or uncertainty exactly as observed.

PART IX CONTINUITY

Lineage locates the case; logs preserve observations; traces explain attempts; metrics aggregate behavior; Chapter 53 bounds failure propagation; Chapter 54 will inject failure and prove recovery.

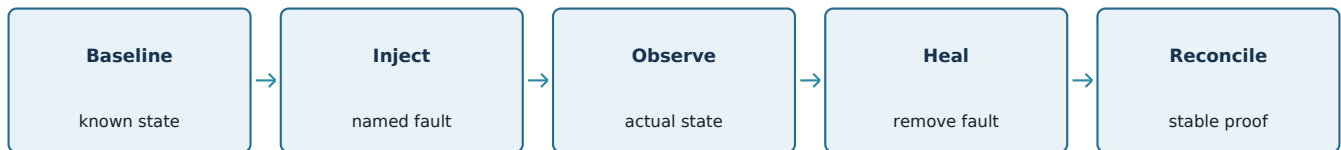
PART IX - OBSERVABILITY, RESILIENCE, AND FAILURE EVIDENCE

54. Failure Injection and Recovery Evidence

A restart is an action. Recovery is a retained argument that links the injected fault, expected degradation, actual durable state, trace evidence, reconciliation and bounded return to service.

CHAPTER DECISION

Execute six isolated experiments under one fail-closed harness: kill a consumer, stop the broker, pause a database, duplicate a message, delay a projection and restart a service. Admit a recovery claim only when the fault is proven active, removed, and followed by stable invariant-preserving state.

PROOF PATH 54-A - FROM HYPOTHESIS TO RECOVERY VERDICT**GUARANTEE LEDGER 54-A**

Evidence	Allowed claim	Required proof	Cannot claim
fault receipt	actuator applied named fault	target + command result	dependency failed as expected
observation	state was measured	owner snapshot + freshness	cause without chronology
trace	recorded path/attempt exists	query + pipeline health	complete path across crash
recovery	declared postconditions stabilized	bounded stable window	production RTO/RPO

CHAPTER EXIT CRITERIA 54-B

Question	Required answer
Can a fault escape the test?	no - allowlist, lease and final cleanup
Can a green health check prove recovery?	no - durable conservation also passes
Can missing trace data be hidden?	no - absent/unknown is explicit
Can one successful poll end recovery?	no - consecutive stable observations

ORDINARY DEVELOPER PROBLEM

The container restarted, but the workflow never recovered

A test kills a consumer, waits ten seconds and sees a green readiness endpoint. It passes. Later the team discovers one message remained unacknowledged, an outbox row never drained and a projection stayed behind. The test proved that a process could answer HTTP, not that distributed work recovered.

FAILURE LEDGER 54-1

Shortcut	False conclusion	Correction
process is running	service recovered	readiness + durable postconditions
sleep 10 seconds	system had time	deadline-bound convergence predicate
one trace found	flow complete	trace coverage + owner state
fault command returned 0	fault affected traffic	active-fault probe
queue became zero	no messages lost	accepted/effect conservation
rerun until green	experiment is reliable	retain first failure

PLAIN-LANGUAGE PRINCIPLE

A recovery experiment asks whether truth and progress survive turbulence, not whether orchestration commands happen to succeed.

CHAPTER BOUNDARY

Experiments here; reproducible runtime next

This chapter owns the experiment model, safety envelope, six campaigns, state/trace oracles and evidence verdict. Chapter 55 will package the services, databases, broker, telemetry and tooling into the one-command Docker Compose runtime that executes them reproducibly.

DECISION RECORD

ADR-054 - Recovery is a multi-source evidence claim

NEXUS-1 treats every fault experiment as a versioned hypothesis. The harness records the expected state before injection, actual state during the fault, trace and operational evidence, the explicit healing action, durable reconciliation and a bounded stable recovery result. Any missing mandatory source invalidates the claim.

EVIDENCE OWNERSHIP 54-C

Source	Owns	Does not own
fault actuator	requested target/action	business consequence
service database	durable owner state	broker delivery state
broker	ready/unacked/confirm state	consumer effect
telemetry	observed logs/spans/metrics	durable truth
orchestrator	chronology and verdict	service semantics

DECISION IDENTITY

ADR-054, experiment plans and oracle code are hashed with the build. A screenshot or manually typed timeline cannot replace machine-readable evidence.

INVARIANT CARRIED FROM CHAPTER 53

The policy and the experiment share one boundary identity

Timeout, retry, circuit and concurrency evidence is evaluated under the same named boundary profile used by the workload. A fault run with unknown or drifting resilience options is invalid.

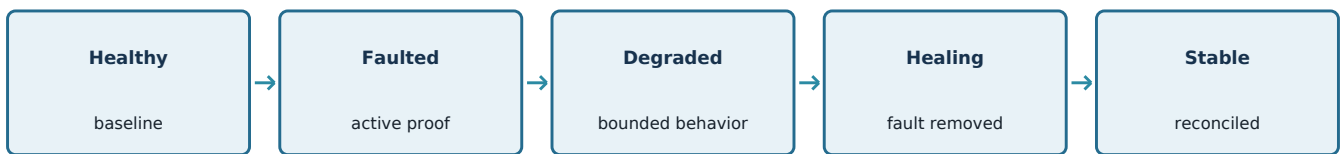
EXPERIMENT LIFECYCLE

Seven phases separate failure from recovery

LIFECYCLE 54-D

Phase	Required outcome
1. authorize	isolated environment and targets admitted
2. baseline	fresh healthy snapshots and control workload
3. arm	barriers/proxies configured; no fault yet
4. inject	fault receipt and active-fault proof retained
5. observe	expected and actual degraded state captured
6. heal	fault removed; cleanup independently verified
7. reconcile	durable invariants stabilize within budget

PROOF PATH 54-B - STATE TRANSITIONS

**NO SKIPPED PHASE**

If baseline is unhealthy, the fault does not run. If fault activity is unproved, observed degradation cannot be attributed to the experiment. If cleanup is unproved, the run ends ABORTED_UNHEALED.

SAFETY ENVELOPE

The harness cannot target production by construction

ADMISSION GATES 54-E

Gate	Required value
environment class	evidence-local or ephemeral CI only
Compose project	nexus1-evidence-<run> prefix
target labels	nexus1.fault-injection=allowed
service allowlist	six reviewed logical targets
data classification	synthetic deterministic corpus
exclusive lease	one campaign per runtime
outer timeout	hard campaign deadline

EXECUTABLE ASSET 54-A - FAIL-CLOSED ENVIRONMENT GUARD

```
public static void Authorize(EvidenceEnvironment env, ExperimentPlan plan)
{
    Require(env.Class is EnvironmentClass.LocalEvidence
            or EnvironmentClass.EphemeralCi, "unsafe environment");
    Require(env.ComposeProject.StartsWith("nexus1-evidence-",
            StringComparison.Ordinal), "unsafe Compose project");
    Require(plan.Targets.All(env.AllowedTargets.Contains), "target not allowlisted");
    Require(env.DataSet.Classification == "synthetic", "non-synthetic corpus");
    Require(plan.Deadline <= TimeSpan.FromMinutes(10), "campaign too long");
}
```

PRODUCTION RULE

There is no configuration flag that turns this harness on in production. The actuator package, credentials and Compose socket are absent from production composition.

CAMPAIGN IDENTITY

One ID joins plan, fault, telemetry, state and artifacts

EXECUTABLE ASSET 54-B - EXPERIMENT IDENTITY

```
public readonly record struct ExperimentId(Guid Value)
{
    public static ExperimentId New() => new(Guid.NewGuid());
    public override string ToString() => Value.ToString("D");
}

public sealed record ExperimentIdentity(
    ExperimentId Id,
    string ScenarioKey,
    string BuildDigest,
    string ConfigurationDigest,
    string ResilienceProfileDigest,
    int Seed,
    DateTimeOffset AuthorizedUtc);
```

IDENTITY PROPAGATION 54-F

Surface	Representation
orchestrator events	ExperimentId + step number
workload header/message	bounded baggage field
logs/traces	experiment.id attribute
metrics	scenario class only; ID excluded
artifact directory	opaque experiment ID
durable probe rows	campaign observation table

CARDINALITY BOUNDARY

ExperimentId is a locator in artifacts, logs and traces. It is never a metric label or circuit/bulkhead partition.

TYPED EXPERIMENT PLAN

Expected degradation and recovery are declared before injection

EXECUTABLE ASSET 54-C - PLAN CONTRACT

```
public sealed record ExperimentPlan(  
    ExperimentIdentity Identity,  
    FaultSpec Fault,  
    WorkloadSpec Workload,  
    ExpectedState ExpectedDuringFault,  
    ExpectedState ExpectedAfterRecovery,  
    TimeSpan ObservationBudget,  
    TimeSpan RecoveryBudget,  
    TimeSpan StableWindow,  
    ArtifactRequirement[] RequiredArtifacts);  
  
public sealed record FaultSpec(  
    FaultKind Kind,  
    string Target,  
    string InjectionPoint,  
    TimeSpan MaximumDuration);
```

PLAN REVIEW 54-G

Field	Gate
scenario key	one of six required experiments
fault target	allowlisted and uniquely resolved
injection point	externally observable or barrier-backed
expected states	predicates, not narrative only
budgets	positive and below outer deadline
artifacts	mandatory and digest-verifiable

PRE-REGISTRATION

The plan digest is retained before the baseline. Editing expected results after observing the run creates a new plan and a new experiment identity.

RESULT VOCABULARY

Passed, failed, invalid and unhealed are distinct

EXECUTABLE ASSET 54-D - RESULT MODEL

```
public enum ExperimentVerdict
{
    Passed, FailedExpectation, InvalidEvidence,
    AbortedHealed, AbortedUnhealed, RuntimePending
}

public sealed record ExperimentResult(
    ExperimentIdentity Identity,
    FaultReceipt Fault,
    StateObservation Baseline,
    StateObservation DuringFault,
    TraceEvidence Trace,
    RecoveryResult Recovery,
    EvidenceArtifact[] Artifacts,
    ExperimentVerdict Verdict,
    string[] Findings);
```

VERDICT MEANING 54-H

Verdict	Meaning
Passed	all required expectations and evidence passed
FailedExpectation	valid evidence contradicts hypothesis
InvalidEvidence	run cannot support a conclusion
AbortedHealed	stopped safely; fault removal proved
AbortedUnhealed	cleanup not proved; runtime quarantined
RuntimePending	design only; no fresh run retained

FAILURE IS EVIDENCE

A well-instrumented failed expectation is more valuable than a green run with missing state. The harness retains failures and never auto-reruns them into invisibility.

EXTERNAL ORCHESTRATION

The experiment runner survives the service it kills

PLACEMENT 54-I

Component	Process boundary	Reason
experiment runner	test host	survives service/container fault
fault actuator	test host	owns control-plane commands
state probes	owner APIs/DB/broker	independent observations
telemetry reader	collector/backend	queries retained evidence
barrier endpoint	target service, Evidence profile	exact crash window

CONTROL/OBSERVATION SEPARATION

The actuator's successful command is not reused as proof of impact. A second independent probe verifies that the consumer stopped, broker is unreachable, database is paused or proxy toxic is active.

CLOCK MODEL

UTC orders artifacts; monotonic time measures budgets

Every event records UTC for cross-process chronology and a runner-local monotonic offset for elapsed-time claims. Recovery duration is never computed from unsynchronized service wall clocks.

RUNNER SKELETON

Cleanup executes even when observation or assertion fails

EXECUTABLE ASSET 54-E - EXPERIMENT RUNNER

```
public async Task<ExperimentResult> RunAsync(
    ExperimentPlan plan, CancellationToken cancellationToken)
{
    Authorize(environment, plan);
    await using var exclusive = await leases.AcquireAsync(plan.Identity.Id,
        cancellationToken);
    using var outer = CancellationTokenSource.CreateLinkedTokenSource(
        cancellationToken);
    outer.CancelAfter(plan.Fault.MaximumDuration + plan.RecoveryBudget);

    var baseline = await observations.CaptureBaselineAsync(plan, outer.Token);
    await workload.StartAsync(plan.Workload, outer.Token);
    await using var fault = await actuator.InjectAsync(plan.Fault, outer.Token);
    try
    {
        await fault.ProveActiveAsync(outer.Token);
        var during = await observations.CaptureDuringFaultAsync(plan, outer.Token);
        return await recovery.HealObserveAndVerifyAsync(
            plan, fault, baseline, during, outer.Token);
    }
    finally
    {
        using var cleanup = new CancellationTokenSource(environment.CleanupTimeout);
        await fault.HealAsync(cleanup.Token);
        await fault.ProveAbsentAsync(cleanup.Token);
    }
}
```

CLEANUP TOKEN

Healing does not inherit an already-cancelled campaign token. It uses its own small hard timeout and emergency path; failure quarantines the runtime instead of pretending cleanup succeeded.

FAULT LEASE

Injection and healing are one idempotent resource

EXECUTABLE ASSET 54-F - IDEMPOTENT LEASE

```
public abstract class FaultLease : IAsyncDisposable
{
    private readonly SemaphoreSlim gate = new(1, 1);
    private bool healed;

    public abstract FaultReceipt Receipt { get; }
    public abstract Task ProveActiveAsync(Cancellation token);
    public abstract Task ProveAbsentAsync(Cancellation token);
    protected abstract Task HealCoreAsync(Cancellation token);

    public async Task HealAsync(Cancellation token)
    {
        await gate.WaitAsync(token);
        try
        {
            if (healed) return;
            await HealCoreAsync(token);
            healed = true;
        }
        finally { gate.Release(); }
    }

    public async ValueTask DisposeAsync()
    {
        using var budget = new CancellationTokenSource(TimeSpan.FromSeconds(15));
        await HealAsync(budget.Token);
    }
}
```

LEASE GUARANTEES 54-J

Guarantee	Evidence
one target	resolved container/proxy identity
active proof	independent state after injection
idempotent heal	multiple cleanup paths safe
maximum lifetime	watchdog expiry
final absence	post-heal control-plane query

CRASH OF THE RUNNER

An external watchdog owns the campaign lease expiry and best-effort healing. A stale lease prevents reuse of the runtime until a human-readable cleanup report passes.

OBSERVATION WITHOUT SLEEPS

Poll a predicate until it is stable or the budget expires

EXECUTABLE ASSET 54-G - STABLE CONVERGENCE

```
public async Task<T> AwaitStableAsync<T>(  
    Func<CancellationToken, Task<T>> probe,  
    Func<T, bool> predicate,  
    int consecutive,  
    TimeSpan interval,  
    TimeSpan timeout,  
    CancellationToken token)  
{  
    using var budget = CancellationTokenSource.CreateLinkedTokenSource(token);  
    budget.CancelAfter(timeout);  
    var stable = 0;  
    while (true)  
    {  
        var value = await probe(budget.Token);  
        stable = predicate(value) ? stable + 1 : 0;  
        if (stable >= consecutive) return value;  
        await Task.Delay(interval, timeProvider, budget.Token);  
    }  
}
```

CONVERGENCE PROFILE 54-K

Predicate	Stable requirement
readiness restored	3 observations over 1.5 s
outbox drained	3 fresh snapshots
projection caught up	checkpoint equal + no regression
no active fault	control plane twice
invariant conservation	two owner snapshots

TIMEOUT RESULT

Budget expiry records the last observed value, timestamp, freshness and mismatch. It does not reduce the predicate or extend the wait automatically.

DURABLE STATE SNAPSHOT

Recovery is judged across owners, not one table

EXECUTABLE ASSET 54-H - DISTRIBUTED SNAPSHOT

```
public sealed record DistributedStateSnapshot(  
    DateTimeOffset ObservedUtc,  
    long AcceptedAlarms,  
    long OutboxPending,  
    long OutboxDispatched,  
    long BrokerReady,  
    long BrokerUnacked,  
    long InboxProcessing,  
    long InboxCompleted,  
    long InboxDuplicates,  
    long RootCauseEffects,  
    long ProjectionCheckpoint,  
    long SourceHighWater,  
    FreshnessStamp[] Freshness);
```

SNAPSHOT SOURCES 54-L

Value	Authority
accepted/outbox	AlarmManagementDb
ready/unacked	RabbitMQ management/metrics
inbox/effects	RootCauseDb
checkpoint	projection database
source high-water	event owner/registry
readiness/epoch	service runtime

SNAPSHOT HONESTY

The values are not claimed atomic across databases. Each source retains its observation time; the verifier reasons over bounded intervals and stable convergence, not a fictional distributed transaction.

EXPECTED AND ACTUAL STATE

Predicates remain machine-readable and mismatches survive

EXECUTABLE ASSET 54-I - EXPECTATION EVALUATION

```
public sealed record StateExpectation(  
    string Key,  
    string Description,  
    Func<DistributedStateSnapshot, bool> Predicate,  
    EvidenceSeverity Severity);  
  
public sealed record ExpectationFinding(  
    string Key,  
    bool Matched,  
    string Expected,  
    string Actual,  
    DateTimeOffset ObservedUtc);  
  
public static ExpectationFinding Evaluate(  
    StateExpectation expected, DistributedStateSnapshot actual) =>  
    new(expected.Key, expected.Predicate(actual), expected.Description,  
        SnapshotFormatter.ValueFor(expected.Key, actual), actual.ObservedUtc);
```

COMPARISON RULES 54-M

Rule	Reason
evaluate original plan	prevents post-hoc success
retain every mismatch	failure remains diagnosable
freshness required	stale match is invalid
severity explicit	diagnostic vs invariant
actual value serialized	review without rerun

TRACE EVIDENCE

A crash can legitimately end a span before export

TRACE EVIDENCE CLASSES 54-N

Class	Meaning	Qualification
complete	expected spans/events present	collector fresh
partial-at-crash	prefix retained; target killed	container event + durable milestone
sampled-out	trace policy excluded path	metrics/state still required
pipeline-loss	collector/export failure	claim invalid for trace obligation
not-found	no matching trace	cause unresolved

EXECUTABLE ASSET 54-J - TRACE CORPUS

```
public sealed record TraceEvidence(  
    ExperimentId ExperimentId,  
    TraceEvidenceClass Class,  
    int TracesFound,  
    int SpansFound,  
    bool CollectorFresh,  
    TraceExpectationFinding[] Findings,  
    ArtifactRef QueryResult,  
    string Qualification);
```

NO SYNTHETIC CLOSURE

The harness never invents a terminal span after killing a process. It records partial-at-crash and uses durable state plus broker/container chronology to establish what happened.

EVIDENCE CHRONOLOGY

Every event has a sequence and monotonic offset

EXECUTABLE ASSET 54-K - TIMELINE EVENT

```
public sealed record ExperimentEvent(  
    long Sequence,  
    ExperimentId ExperimentId,  
    string Phase,  
    string Kind,  
    DateTimeOffset OccurredUtc,  
    TimeSpan RunnerOffset,  
    string Source,  
    string DetailCode,  
    ArtifactRef? Artifact);
```

REQUIRED CHRONOLOGY 54-O

Event	Source
baseline accepted	orchestrator
workload item admitted	edge/owner DB
fault command/active	actuator + probe
degradation observed	state/health/telemetry
fault removed	actuator + probe
readiness restored	service probe
backlog reconciled	owner snapshots

ORDERING QUALIFICATION

Cross-process UTC timestamps help investigation but do not create total order. The runner sequence orders its own actions; causal links and durable positions order domain events.

FAULT ACTUATOR INTERFACE

Mechanism is replaceable; semantics stay stable**EXECUTABLE ASSET 54-L - ACTUATOR PORTS**

```
public interface IFaultActuator
{
    Task<FaultLease> InjectAsync(FaultSpec fault, CancellationToken token);
}

public interface IContainerControl
{
    Task KillAsync(string service, string signal, CancellationToken token);
    Task StopAsync(string service, TimeSpan grace, CancellationToken token);
    Task StartAsync(string service, CancellationToken token);
    Task PauseAsync(string service, CancellationToken token);
    Task UnpauseAsync(string service, CancellationToken token);
    Task RestartAsync(string service, CancellationToken token);
    Task<ContainerState> InspectAsync(string service, CancellationToken token);
}
```

ACTUATOR MAPPING 54-P

Fault	Inject	Heal
consumer kill	SIGKILL consumer	start consumer
broker stop	stop broker	start broker
database pause	pause database	unpause database
duplicate message	publish retained envelope twice	none
projection delay	enable latency/barrier	remove toxic/release
service restart	restart named service	prove ready

SAFE COMPOSE COMMAND ADAPTER

Arguments are structured and targets are resolved twice**EXECUTABLE ASSET 54-M - COMMAND CONSTRUCTION**

```
private async Task RunComposeAsync(
    EvidenceEnvironment env,
    IReadOnlyList<string> arguments,
    CancellationToken token)
{
    Require(Path.GetFullPath(env.ComposeFile) == env.ApprovedComposeFile,
        "compose file drift");
    var start = new ProcessStartInfo("docker")
    {
        WorkingDirectory = env.WorkingDirectory,
        RedirectStandardOutput = true,
        RedirectStandardError = true,
        UseShellExecute = false
    };
    start.ArgumentList.Add("compose");
    start.ArgumentList.Add("--project-name");
    start.ArgumentList.Add(env.ComposeProject);
    start.ArgumentList.Add("--file");
    start.ArgumentList.Add(env.ComposeFile);
    foreach (var argument in arguments) start.ArgumentList.Add(argument);
    await processRunner.RunCheckedAsync(start, token);
}
```

COMMAND EXAMPLES 54-Q

Intent	Argument list
kill consumer	kill --signal SIGKILL alarm-consumer
stop broker	stop --timeout 1 broker
pause DB	pause alarm-db
heal DB	unpause alarm-db
restart service	restart --no-deps rootcause-api

NO SHELL INTERPOLATION

Scenario input never becomes a command string. Project, file and service are exact allowlisted values, and post-command inspection verifies the resolved container labels.

CONTAINER SEMANTICS

Kill, stop, pause and restart model different failures

DOCKER OPERATION SEMANTICS 54-R

Operation	Modeled condition	Important distinction
kill SIGKILL	abrupt process death	no graceful shutdown
stop	bounded graceful stop	container retained; start reverses
pause	process execution frozen	connections may appear hung
unpause	resume same processes	not a restart
restart	stop/start same configuration	Compose config changes not applied

MECHANISM MATCHING

The scenario states which failure is being modeled. A pause tests slow/unresponsive behavior; a stop tests dependency absence; a kill tests abrupt loss of in-process state.

READINESS AFTER START

Running is not ready

After start or restart, the harness waits for the application readiness contract and then for owner-state convergence. A listening TCP port or running container alone is insufficient.

NETWORK FAULT PATH

Proxy toxics model latency and cuts without stopping the dependency

PROXY USES 54-S

Toxic	Question
latency + jitter	do attempt/total timeouts remain bounded?
timeout	does the call become UNKNOWN_EFFECT when appropriate?
bandwidth	does body processing honor cancellation?
connection reset	does classifier distinguish transient failure?
disabled proxy	does circuit/backpressure contain load?

EXECUTABLE ASSET 54-N - NETWORK-FAULT LEASE

```
public interface INetworkFaultControl
{
    Task<NetworkFaultLease> AddLatencyAsync(
        string proxyKey,
        TimeSpan latency,
        TimeSpan jitter,
        double toxicity,
        CancellationToken token);
}

await using var toxic = await network.AddLatencyAsync(
    "projection-db", TimeSpan.FromSeconds(2),
    TimeSpan.Zero, toxicity: 1.0, cancellationToken);
```

TOPOLOGY PROOF

All application traffic must traverse the proxy for the campaign. A control request through the direct dependency address is rejected by configuration and network policy.

IN-PROCESS FAULT INJECTION

Use it for precise seams, never as proof of an external outage

MECHANISM COMPARISON 54-T

Mechanism	Best use	Cannot prove
Simmy fault/outcome	client predicate and policy behavior	broker/container failure
Simmy latency	timeout path at decorated callback	network path is slow
Toxiproxy	transport latency/cut	process crash
Compose kill/stop/pause	process/dependency lifecycle	exact instruction boundary
evidence barrier	crash window around commit/ack	natural production frequency

EXECUTABLE ASSET 54-O - EVIDENCE-ONLY CHAOS

```
if (environment.IsEnvironment("Evidence"))
{
    pipelineBuilder.AddChaosLatency(
        injectionRate: 1.0,
        latency: TimeSpan.FromSeconds(2));
}
```

DEFAULT-ACTIVE WARNING

Current Simmy chaos strategies are active when added. NEXUS-1 omits them entirely outside the Evidence composition and verifies the production descriptor contains zero chaos strategies.

EXACT CRASH BARRIERS

A deterministic handshake replaces lucky timing

EXECUTABLE ASSET 54-P - FAULT CHECKPOINT

```
public interface IEvidenceBarrier
{
    Task ReachAndBlockAsync(
        ExperimentId experimentId,
        string checkpoint,
        CancellationToken token);
}

// Evidence profile only: commit completed, broker ack not sent yet.
await inbox.CommitAsync(cancellationTokens);
await barriers.ReachAndBlockAsync(
    experimentId, "consumer.after-commit.before-ack", cancellationTokens);
await delivery.AckAsync(cancellationTokens);
```

BARRIER PROTOCOL 54-U

Step	Proof
arm	experiment/checkpoint allowlisted
reach	target emits durable/independent receipt
block	consumer remains before ack
kill	container event after barrier receipt
heal	new process epoch starts
cleanup	barrier state expires/removed

PRODUCTION ABSENCE

Barrier implementation and endpoint are excluded from production binaries or composition. A structural test rejects IEvidenceBarrier references outside the evidence adapter.

CONTROL CAMPAIGN

Healthy execution establishes the comparison corpus

BASELINE 54-V

Stimulus	Expected result
100 accepted alarms	100 durable accepted facts
outbox dispatch	100 confirmed messages
consumer processing	100 Inbox completed
root-cause effect	100 effects exactly once
projection	checkpoint reaches source high-water
telemetry	fresh trace/log/metric pipeline

CONTROL ORACLE 54-W

Proof	Gate
conservation	accepted = effects = projected terminal corpus
backlog	outbox pending = broker ready/unacked = 0
duplicates	zero for control corpus
readiness	all required services ready
fault census	no active fault/toxic/barrier

BASELINE INVALIDATION

A failed control campaign blocks all fault runs on that runtime. The harness does not attribute pre-existing backlog or telemetry loss to a new injection.

EXPERIMENT 1 - KILL A CONSUMER

Abrupt death must produce redelivery, not duplicate effect

HYPOTHESIS 54-X

Moment	Expected state
before kill	one delivery unacked at named barrier
kill active	consumer container stopped; readiness absent
broker	delivery returns ready or moves to new consumer
replacement starts	new process epoch; manual acknowledgement
redelivery	same MessageId admitted through Inbox
terminal	one durable effect; message acknowledged

REQUIRED EVIDENCE 54-Y

Source	Finding
barrier receipt	after-commit-before-ack reached
container events	SIGKILL after receipt
broker	redelivered=true / delivery chronology
Inbox	one completed + duplicate-completed branch
effect table	unique effect count = 1
trace	partial killed attempt + recovered attempt

MESSAGING CONTRACT

Consumer acknowledgement occurs only after the durable Inbox/effect transaction. Connection loss before acknowledgement permits redelivery; Inbox idempotency prevents a second effect.

CONSUMER CRASH WINDOWS

Before commit and after commit require different oracles

CRASH-WINDOW MATRIX 54-Z

Window	First attempt state	Recovery
before Inbox claim	no durable admission	redelivery processes normally
after claim, before effect	processing lease exists	lease expiry/repair then retry
after effect commit, before ack	effect + completed Inbox durable	duplicate-completed then ack
after ack	broker owns no delivery	no redelivery expected

EXECUTABLE ASSET 54-Q - CONSUMER ORACLE

```
Assert.Equal(1, evidence.EffectRows(messageId));
Assert.Equal(InboxState.Completed, evidence.InboxState(messageId));
Assert.True(evidence.DeliveryAttempts(messageId) >= 2);
Assert.Equal(1, evidence.DuplicateCompleted(messageId));
Assert.Equal(0, evidence.BrokerUnacked(messageId));
Assert.True(evidence.ConsumerEpochs >= 2);
```

NO EXACT-ONCE CLAIM

The campaign proves exactly one owner effect for this corpus under at-least-once delivery and Inbox deduplication. It does not claim exactly-once network delivery.

EXPERIMENT 2 - STOP THE BROKER

Accepted work remains durable while publication is unavailable

HYPOTHESIS 54-AA

Phase	Expected state
baseline	broker reachable; outbox empty
broker stopped	publisher failures classified; circuit may open
new commands	AlarmManagementDb commits accepted fact + outbox
during fault	outbox pending/oldest age rises within workload bound
broker started	readiness and publisher connection recover
drain	confirms mark rows dispatched; consumers catch up

NON-NEGOTIABLE ORACLES 54-AB

Oracle	Required result
acceptance	accepted command count matches owner facts
outbox	no pending row lost or marked dispatched without confirm
broker	stop/start proven by independent state
amplification	attempt count within Chapter 53 budget
terminal	all accepted facts published and consumed

AVAILABILITY CHOICE

If product policy rejects new commands while the broker is down, the expected state changes explicitly. This campaign uses the NEXUS-1 outbox contract: database acceptance can continue while dispatch is delayed.

BROKER RECOVERY CHRONOLOGY

Connection recovery is not backlog recovery

MILESTONES 54-AC

Milestone	Evidence
broker process started	container state
broker ready	health/management probe
publisher connected	client observation
circuit permits call	transition/attempt corpus
first confirm	publisher confirm
outbox zero	owner snapshot
consumer caught up	Inbox/effect/source positions

EXECUTABLE ASSET 54-R - BROKER DRAIN GATE

```
var recovered = await AwaitStableAsync(
    probe: token => snapshots.ReadAsync(token),
    predicate: s => s.OutboxPending == 0
        && s.BrokerReady == 0
        && s.BrokerUnacked == 0
        && s.InboxProcessing == 0
        && s.RootCauseEffects == s.AcceptedAlarms,
    consecutive: 3,
    interval: TimeSpan.FromMilliseconds(500),
    timeout: plan.RecoveryBudget,
    token: cancellationToken);
```

CIRCUIT QUALIFICATION

A process-local circuit may still be open briefly after the broker starts. The recovery clock includes that behavior; the harness does not manually close the circuit unless the plan explicitly tests operator control.

EXPERIMENT 3 - PAUSE A DATABASE

Frozen execution must trigger bounded waiting and honest readiness

HYPOTHESIS 54-AD

Surface	Expected state
database container	paused, not stopped
existing/new connections	calls hang until attempt/command timeout
service liveness	alive if process/event loop healthy
service readiness	not ready for DB-dependent work
request outcome	unavailable or UNKNOWN_COMMIT; never false success
bulkhead	active calls bounded; excess shed

REQUIRED EVIDENCE 54-AE

Proof	Gate
pause/unpause	independent container state
timing	attempt and total timeout within tolerance
capacity	maximum active DB operations within permit budget
health	liveness/readiness split visible
state	no invented commit outcome

PAUSE SEMANTICS

A paused container freezes its processes and can leave connections apparently established. It is deliberately different from stop and is useful for proving timeout, cancellation and resource isolation.

UNKNOWN DATABASE COMMIT

Loss of acknowledgement requires reconciliation

COMMIT OUTCOME MATRIX 54-AF

Injection point	Expected result
before transaction begins	unavailable; no owner rows
before commit sent	rollback/dispose; safe bounded replay if contract allows
after commit sent, before acknowledgement	UNKNOWN_COMMIT
after confirmed commit	completed

EXECUTABLE ASSET 54-S - RECONCILIATION ORACLE

```

var receipt = await operationReceipts.FindAsync(operationId, cancellationToken);
var effect = await ownerState.FindEffectAsync(operationId, cancellationToken);

var reconciled = (receipt, effect) switch
{
    ({ Status: ReceiptStatus.Completed }, not null) => Reconciled.Completed,
    (null, null) => Reconciled.NotCommitted,
    _ => Reconciled.Inconsistent
};
Assert.NotEqual(Reconciled.Inconsistent, reconciled);

```

TRUTH RULE

A client timeout or lost connection does not decide the database outcome. The durable operation receipt and owner effect do.

DATABASE RECOVERY

Unpause is followed by pool, readiness and state checks

RECOVERY SEQUENCE 54-AG

Step	Gate
1	database container no longer paused
2	fresh connection/query succeeds
3	service readiness returns
4	timed-out work classified/reconciled
5	connection and concurrency counts return baseline
6	new control command completes

EXECUTABLE ASSET 54-T - POST-RECOVERY PROBE

```

await AwaitStableAsync(
    probe: token => databaseProbe.ReadAsync(token),
    predicate: p => p.ContainerRunning
        && !p.ContainerPaused
        && p.QuerySucceeded
        && p.ServiceReady
        && p.ActiveConnections <= profile.ConnectionBudget,
    consecutive: 3,
    interval: TimeSpan.FromMilliseconds(500),
    timeout: plan.RecoveryBudget,
    token: cancellationToken);

```

POOL WARNING

The first successful query does not prove that abandoned calls released every connection. The campaign retains active-operation and connection-pool observations through the stable window.

EXPERIMENT 4 - DUPLICATE A MESSAGE

Identical MessageId, contract and payload exercise Inbox semantics

EXECUTABLE ASSET 54-U - DUPLICATE STIMULUS

```
var envelope = corpus.CreateAlarmAccepted(  
    messageId: MessageId.New(),  
    operationId: OperationId.New(),  
    occurredUtc: clock.GetUtcNow());  
  
await publisher.PublishConfirmedAsync(envelope, cancellationTokens);  
await publisher.PublishConfirmedAsync(envelope, cancellationTokens);
```

HYPOTHESIS 54-AH

Observation	Expected
broker deliveries	two accepted deliveries of same MessageId
Inbox first admission	processing then completed
Inbox duplicate	duplicate-completed disposition
domain effect	one effect only
acknowledgements	both deliveries terminally settled
telemetry	two process attempts; one logical effect

STIMULUS INTEGRITY

Both envelopes are byte-equivalent after canonical serialization. Same MessageId with a different digest is a separate conflict campaign and must be rejected, not treated as a benign duplicate.

DUPLICATE-MESSAGE ORACLE

At-least-once delivery converges to one durable effect

EXECUTABLE ASSET 54-V - CONSERVATION ASSERTIONS

```
Assert.Equal(2, evidence.BrokerDeliveries(messageId));  
Assert.Equal(1, evidence.InboxRows(messageId));  
Assert.Equal(InboxState.Completed, evidence.InboxState(messageId));  
Assert.Equal(1, evidence.DomainEffects(messageId));  
Assert.Equal(1, evidence.DuplicateCompleted(messageId));  
Assert.Equal(0, evidence.UnackedDeliveries(messageId));
```

FAILURE INTERPRETATIONS 54-AI

Finding	Meaning
two effects	idempotency failure
one effect, two Inbox rows	uniqueness defect
duplicate remains unacked	settlement defect
zero effect	processing/recovery failure
same ID/different digest accepted	identity-conflict defect

METRIC SPLIT

Delivery attempts, Inbox dispositions and domain effects remain separate measurements. Combining them would hide whether deduplication actually worked.

EXPERIMENT 5 - DELAY A PROJECTION

Write truth advances while read truth is explicitly stale

HYPOTHESIS 54-AJ

Surface	Expected during delay
write owner	acceptance and public facts continue
broker/consumer	events remain available or queued
projector	latency/barrier active; apply rate falls
read model	old value with visible checkpoint/age
API contract	staleness metadata, never false freshness
metrics	position/time lag rises

EXECUTABLE ASSET 54-W - STALE READ CONTRACT

```
public sealed record ProjectionRead<T>(  
    T Value,  
    long AppliedPosition,  
    long SourceHighWater,  
    DateTimeOffset ObservedUtc,  
    DateTimeOffset LastAppliedUtc)  
{  
    public long PositionLag => Math.Max(0, SourceHighWater - AppliedPosition);  
    public bool IsStale => PositionLag > 0;  
}
```

NO CROSS-MODEL LIE

The read API may return a stale projection when its contract permits, but it must expose provenance and lag. It cannot claim the latest write is absent from domain truth.

PROJECTION CATCH-UP

Recovery is monotonic convergence to source high-water

RECOVERY ORACLE 54-AK

Proof	Required result
toxic/barrier removed	independent absence proof
checkpoint sequence	never decreases
applied facts	no missing source positions
duplicate applies	idempotent/no second projection effect
position lag	converges to zero
time lag	returns within baseline range

EXECUTABLE ASSET 54-X - CHECKPOINT GATE

```
var checkpoints = evidence.ProjectionCheckpoints;  
Assert.NotEmpty(checkpoints);  
Assert.True(checkpoints.Zip(checkpoints.Skip(1))  
    .All(pair => pair.First <= pair.Second));  
Assert.Equal(evidence.SourceHighWater, checkpoints[^1]);  
Assert.Equal(0, evidence.MissingSourcePositions.Count);
```

CATCH-UP DURATION

The recovery clock ends only after checkpoint equality and a stable window. Removing latency is the start of healing, not the end of recovery.

EXPERIMENT 6 - RESTART A SERVICE

Durable work survives while process-local state resets

HYPOTHESIS 54-AL

State class	Expected after restart
domain/database	unchanged and authoritative
outbox/inbox	pending work resumes from durable rows
projection cursor	resumes from checkpoint
in-memory cache	cold/refilled under contract
circuit state	new process-local state unless externalized
readiness	false during startup; true after initialization
instance identity	new service epoch

STIMULUS 54-AM

Step	Action
1	admit bounded workload and retain receipts
2	wait for named in-flight/pending state
3	restart service --no-deps
4	observe readiness transition and new epoch
5	wait for durable backlog to drain
6	submit fresh control operation

CONFIGURATION CAVEAT

Docker Compose restart uses the existing service configuration; it does not apply changed environment variables or Compose edits. Configuration rollout is not part of this experiment.

RESTART RECOVERY ORACLE

The new process proves resume, not reinitialization from guesses

EXECUTABLE ASSET 54-Y - RESTART ASSERTIONS

```
Assert.NotEqual(evidence.Before.ServiceEpoch, evidence.After.ServiceEpoch);
Assert.True(evidence.ReadinessSequence.Contains(false));
Assert.True(evidence.After.ServiceReady);
Assert.Equal(evidence.Before.AcceptedAlarms, evidence.After.AcceptedAlarms);
Assert.Equal(evidence.After.SourceHighWater, evidence.After.ProjectionCheckpoint);
Assert.Equal(0, evidence.After.OutboxPending);
Assert.Equal(0, evidence.After.InboxProcessing);
Assert.True(evidence.ControlOperation.Completed);
```

RESTART DEFECTS 54-AN

Finding	Interpretation
same epoch	restart not proved or identity stale
ready before initialization	readiness contract defect
cursor reset	projection checkpoint ownership defect
pending row abandoned	worker resume defect
duplicate effect	replay/idempotency defect
control operation fails	recovery incomplete

RECOVERY BUDGET

Measure detection, degradation, healing and convergence separately

TIMING MODEL 54-AO

Interval	Start	End
fault activation	inject command	active-fault proof
detection	fault active	first correct degraded observation
fault duration	active proof	removal proof
readiness recovery	removal proof	readiness stable
state recovery	removal proof	durable invariants stable
total experiment	baseline accepted	artifacts sealed

EXECUTABLE ASSET 54-Z - RECOVERY RESULT

```
public sealed record RecoveryResult(
    DateTimeOffset FaultActiveUtc,
    DateTimeOffset FaultRemovedUtc,
    TimeSpan DetectionDuration,
    TimeSpan ReadinessRecoveryDuration,
    TimeSpan StateRecoveryDuration,
    int StableObservations,
    StateObservation FinalState,
    bool FaultRemovalProved,
    RecoveryVerdict Verdict);
```

BUDGET MEANING

These are campaign bounds under a synthetic workload. They are learning targets, not production recovery objectives or contractual RTOs.

LOSS AND DUPLICATION CONSERVATION

Counts close only when every difference has a reason

CONSERVATION EQUATIONS 54-AP

Ledger	Required relationship
command	$\text{admitted} = \text{rejected} + \text{accepted}$
outbox	$\text{accepted public facts} = \text{pending} + \text{dispatched}$
broker	$\text{confirmed} = \text{ready} + \text{unacked} + \text{consumed/settled}$
Inbox	$\text{deliveries} = \text{first-seen} + \text{duplicate} + \text{rejected-origin}$
effects	$\text{completed first-seen} = \text{owner effects}$
projection	$\text{applied unique positions} \leq \text{source high-water}$

EXECUTABLE ASSET 54-AA - CAMPAIGN CONSERVATION

```
Require(s.AcceptedPublicFacts == s.OutboxPending + s.OutboxDispatched,
    "outbox conservation failed");
Require(s.CompletedFirstSeen == s.RootCauseEffects,
    "effect conservation failed");
Require(s.ProjectionCheckpoint <= s.SourceHighWater,
    "projection advanced beyond source");
Require(s.MissingSourcePositions.Count == 0,
    "projection gap remains");
```

OBSERVATION GAP

Cross-system counts can be temporarily incomparable. The verifier evaluates them only after freshness and stable-window prerequisites pass, while retaining the intermediate mismatch timeline.

READINESS IS NECESSARY, NOT SUFFICIENT**Recovery combines runtime and durable postconditions****RECOVERY LAYERS 54-AQ**

Layer	Postcondition
control plane	no fault/toxic/barrier remains
process	target running with expected new/same epoch
liveness	process-local progress check healthy
readiness	required dependencies and initialization ready
messaging	ready/unacked/pending within terminal bounds
durable state	conservation and uniqueness pass
projection	checkpoint/lag converged
telemetry	pipeline fresh or explicitly qualified

FINAL GATE

The experiment passes only at the deepest required layer. A scenario may restore readiness quickly yet fail because state recovery exceeds its budget or a duplicate effect remains.

POST-RECOVERY CONTROL**A fresh operation proves the runtime can progress again**

After reconciliation, the harness submits one new deterministic command and follows it to terminal durable state. This control is distinct from draining pre-fault work.

ARTIFACT BUNDLE**Every claim points to immutable, digest-verified evidence****REQUIRED ARTIFACTS 54-AR**

Artifact	Purpose
plan.json	pre-registered hypothesis and budgets
environment.json	images, config and target identities
fault-events.jsonl	command/active/heal chronology
state-snapshots.jsonl	baseline/during/recovery owner state
traces.json	trace query and qualification
metrics.json	rates, age, lag, attempts and pipeline health
logs.jsonl	bounded operational observations
container-events.jsonl	process lifecycle proof
verdict.json	findings, residuals and digests

EXECUTABLE ASSET 54-AB - ARTIFACT REFERENCE

```
public sealed record EvidenceArtifact(  
    string Kind,  
    string RelativePath,  
    string Sha256,  
    long Length,  
    DateTimeOffset SealedUtc,  
    bool Required,  
    bool DigestVerified);
```

IMMUTABILITY

The verdict references sealed artifacts by digest. A later dashboard export or edited log becomes a new artifact and cannot silently replace campaign evidence.

EVIDENCE MANIFEST

One record binds the six campaigns to the runtime checkpoint

EXECUTABLE ASSET 54-AC - RECOVERY MANIFEST

```
public sealed record RecoveryEvidenceManifest(
    string BuildDigest,
    string PackageLockDigest,
    string ComposeDigest,
    string ConfigurationDigest,
    string ResilienceProfileDigest,
    string ExperimentCatalogueDigest,
    DateTimeOffset CampaignStartedUtc,
    DateTimeOffset CampaignEndedUtc,
    ExperimentResult[] Experiments,
    EvidenceArtifact[] SharedArtifacts,
    EvidenceVerdict Verdict);
```

CAMPAIGN COMPLETENESS 54-AS

Scenario	Required count
consumer kill	1 after-commit-before-ack + optional windows
broker stop	1
database pause	1 + unknown-commit subcase
duplicate message	1
projection delay	1
service restart	1
broken evidence control	1

NO AGGREGATE CAMOUFLAGE

The manifest retains each experiment verdict. Five passes and one failure never collapse into an overall green percentage.

FAIL-CLOSED VERIFIER

Missing fault proof or cleanup proof invalidates recovery

EXECUTABLE ASSET 54-AD - MANIFEST VERIFIER

```
public static EvidenceVerdict Verify(RecoveryEvidenceManifest m)
{
    if (m.Experiments.Select(e => e.Identity.ScenarioKey).ToHashSet()
        .IsSupersetOf(RequiredScenarios) is false)
        return EvidenceVerdict.Invalid("required scenario missing");
    if (m.Experiments.Any(e => !e.Fault.ActiveProved))
        return EvidenceVerdict.Invalid("fault activity unproved");
    if (m.Experiments.Any(e => !e.Recovery.FaultRemovalProved))
        return EvidenceVerdict.Invalid("fault removal unproved");
    if (m.Experiments.Any(e => e.Artifacts.Any(a =>
        a.Required && (!a.DigestVerified || a.Length == 0))))
        return EvidenceVerdict.Invalid("artifact missing or stale");
    if (m.Experiments.Any(e => e.Verdict != ExperimentVerdict.Passed))
        return EvidenceVerdict.Failed("one or more experiments failed");
    return EvidenceVerdict.Valid;
}
```

INVALID-EVIDENCE CAUSES 54-AT

Cause	Consequence
baseline stale	fault not run or claim invalid
active fault unproved	degradation unattributed
trace required but pipeline lost	trace obligation invalid
fault removal unproved	runtime quarantined
zero workload	no behavior exercised
missing owner snapshot	recovery unproved

DELIBERATELY BROKEN EVIDENCE CAMPAIGN

A green readiness result with stale state must fail

EXECUTABLE ASSET 54-AE - CONTROLLED EVIDENCE DEFECT

```
var stale = baseline with
{
    ObservedUtc = clock.GetUtcNow().Subtract(TimeSpan.FromMinutes(20)),
    Freshness = baseline.Freshness.Select(x => x with
    {
        IsFresh = false
    }).ToArray()
};

var result = verifier.VerifyRecovery(
    readiness: Ready,
    finalState: stale,
    faultRemoved: true);
Assert.Equal(RecoveryVerdict.InvalidEvidence, result.Verdict);
```

BROKEN ORACLE 54-AU

Layer	Expected finding
health	ready - deliberately misleading alone
freshness	owner snapshots stale
conservation	not evaluated as passed
verdict	INVALID_EVIDENCE
rerun	not automatic; defect retained
production composition	fault fixture absent

PURPOSE

The campaign proves that the evidence system rejects an attractive but unsupported recovery claim.

RUNNER UNIT TESTS

Order, cleanup and first failure are deterministic

EXECUTABLE ASSET 54-AF - CLEANUP TEST

```
[Fact]
public async Task Observation_failure_still_heals_fault()
{
    var actuator = new RecordingActuator();
    var runner = Fixture.Runner(actuator,
        observations: new ThrowingObservationSource());

    await Assert.ThrowsAsync<ObservationException>(() =>
        runner.RunAsync(Fixture.Plan(), CancellationToken.None));

    Assert.Equal(new[] { "inject", "prove-active", "heal", "prove-absent" },
        actuator.Calls);
}
```

UNIT MATRIX 54-AV

Test	Failure caught
baseline fails	fault injected too early
injection fails	cleanup state misreported
observation throws	fault leak
outer timeout	cancelled heal
double dispose	non-idempotent cleanup
expectation mismatch	result overwritten by retry

TEST SCOPE

Unit tests prove harness logic with recorded fakes. Container and dependency behavior requires the real component campaigns below.

COMPONENT HARNESS TESTS

Disposable infrastructure proves actuator semantics

COMPONENT LAYERS 54-AW

Layer	Environment	Proof
container lifecycle	throwaway container	kill/stop/start/pause state
network toxic	Toxiproxy network	latency/cut active and removed
broker	RabbitMQ container	confirm, ack and redelivery
database	SQL Server container	pause/timeout/unpause
service	real production image	readiness/epoch/resume

EXECUTABLE ASSET 54-AG - READINESS-AWARE FIXTURE

```
var service = new ContainerBuilder("nexus1/rootcause:evidence")
    .WithNetwork(network)
    .WithEnvironment("ASPNETCORE_ENVIRONMENT", "Evidence")
    .WithWaitStrategy(Wait.ForUnixContainer()
        .UntilHttpRequestIsSucceeded(request => request
            .ForPath("/health/ready")
            .ForStatusCode(HttpStatusCode.OK)))
    .Build();
```

RUNNING VS READY

The container fixture waits for the service readiness contract, not only a running state or open port. Durable recovery gates still execute separately.

STRUCTURAL AND SECURITY GATES

Fault capabilities cannot enter production composition

EXECUTABLE ASSET 54-AH - COMPOSITION CENSUS

```
Assert.DoesNotContain(production.Services,
    d => d.ServiceType == typeof(IFaultActuator));
Assert.DoesNotContain(production.Services,
    d => d.ServiceType == typeof(IEvidenceBarrier));
Assert.DoesNotContain(production.PipelineStrategies,
    s => s.Type.StartsWith("Chaos", StringComparison.Ordinal));

Assert.Contains(evidence.Services,
    d => d.ServiceType == typeof(IFaultActuator));
```

REPOSITORY GATES 54-AX

Scan	Rejects
Docker socket mount	application service access
fault credential	production secret/config
Evidence endpoint	production route inventory
chaos strategy	non-Evidence pipeline
shell command	unstructured target interpolation
unbounded campaign	missing timeout/watchdog

AUTHORITY BOUNDARY

Only the external test runner can mutate container/proxy state. Application services observe dependencies but cannot orchestrate their own recovery experiment.

CI EXECUTION DISCIPLINE

Destructive campaigns are serial inside one isolated runtime

CI RULES 54-AY

Rule	Reason
ephemeral project per job	no shared target
resource label verification	prevent wrong container
serial collection	faults do not overlap
hard job timeout	bounded fault lifetime
always-run cleanup	heal after assertion/cancellation
artifact upload on failure	retain first evidence
quarantine on unhealed	block runtime reuse

EXECUTABLE ASSET 54-AI - XUNIT ISOLATION

```
[CollectionDefinition("NEXUS-1 failure campaigns",
  DisableParallelization = true)]
public sealed class FailureCampaignCollection { }

[Collection("NEXUS-1 failure campaigns")]
[Trait("Category", "FailureEvidence")]
public sealed class RequiredFailureCampaigns(EvidenceRuntime fixture)
{
    // Six explicit scenario tests; no retry-on-failure attribute.
}
```

FLAKY-TEST RULE

A timing failure is investigated as a product, harness or environment finding. It is not hidden behind automatic retries; an explicit later rerun has a new experiment identity.

CAMPAIGN CATALOGUE

The six required experiments answer different questions

CATALOGUE 54-AZ

Scenario	Primary invariant	Primary recovery proof
kill consumer	one effect under redelivery	Inbox complete + ack
stop broker	accepted fact remains in outbox	confirmed drain
pause database	bounded wait; no false commit	reconciliation + pool
duplicate message	MessageId uniqueness	one effect, duplicate settled
delay projection	read staleness explicit	checkpoint catches high-water
restart service	durable work survives	new epoch + backlog drain

SHARED REQUIREMENTS 54-BA

Requirement	All six?
pre-registered expected state	yes
independent active-fault proof	yes
actual owner state	yes
trace evidence/qualification	yes
explicit heal/removal proof	yes
stable recovery result	yes

STANDARDS AND SOURCE LEDGER

Tools supply mechanisms; the book owns proof semantics

SOURCE LEDGER 54-BB

Source	Used for	Local qualification
Docker Compose CLI	kill/stop/pause/unpause/restart	allowlisted external actuator
Testcontainers for .NET	throwaway containers and readiness	component harness only
Toxiproxy module	latency and network-path faults	all traffic routed through proxy
RabbitMQ reliability	ack, confirm and redelivery semantics	Inbox/outbox owner proof
Polly Simmy	selective in-process fault/latency	Evidence composition only
OpenTelemetry	trace/log/metric observation	qualified by pipeline freshness

VERSION POLICY

Tool, image, CLI, package and broker versions are retained by digest. A mechanism's documentation cannot substitute for the executed campaign evidence.

ANTI-PATTERN REVIEW

Six attractive recovery claims remain invalid

REJECTED CLAIMS 54-BC

Claim	Why invalid
container is running	application may not be ready
readiness is green	durable backlog may remain
queue is empty	messages may have been lost
trace is missing, so call never happened	sampling/export/crash can remove evidence
retry made the experiment pass	first failure was hidden
one local run proves production recovery	workload/topology differ

COMPLETION LEDGER

What Chapter 54 now specifies

COMPLETION LEDGER 54-BD

Obligation	Asset	Status
experiment semantics	ADR, lifecycle and typed plan	SPECIFIED
safety/cleanup	allowlist, lease and watchdog	SPECIFIED
expected/actual state	fresh distributed snapshots	SPECIFIED
trace evidence	complete/partial/lost qualification	SPECIFIED
six required faults	campaigns and structural oracles	SPECIFIED
recovery	stable convergence and conservation	SPECIFIED
evidence verdict	manifest and fail-closed verifier	SPECIFIED
runtime execution	retained campaign artifacts	PENDING

OPEN RESIDUALS 54-BE

Residual	Consequence
no executable repository checkpoint	listings remain compile-oriented
no retained Compose campaign	all six runtime outcomes unproved
no image/package lock	exact APIs and images unproved
no representative workload	recovery budgets educational
no packaged one-command runtime	addressed by Chapter 55

EVIDENCE VERDICT

SPECIFIED | RUNTIME PENDING. The safety model, six experiment plans, state/trace oracles, recovery criteria and fail-closed evidence verifier are complete; no fresh failure campaign is claimed.

HANDOFF TO CHAPTER 55

The Reproducible Docker Compose Runtime

Chapter 55 will turn the deployment assumed by these experiments into a one-command local runtime containing the services, service-owned databases, RabbitMQ, telemetry, BFF and required development tooling. It must add deterministic seed data, pinned images, readiness-aware startup and an executable path for the Chapter 54 campaigns.

BOUNDARY CARRIED FORWARD 54-BF

Chapter 54 leaves	Chapter 55 must add
typed experiment catalogue	Compose profiles and target labels
actuator target names	canonical service/container mapping
state and telemetry probes	networking, ports and tool services
synthetic workload manifest	deterministic seed/bootstrap
RUNTIME PENDING campaigns	one-command executable entry point

NEXT INVARIANT

A local runtime is reproducible only when one pinned command creates the same topology, data, readiness order and evidence interfaces required to rerun a named failure campaign.

FINAL CHAPTER RULE

Recovery is proved when truth and progress stabilize after the fault

NEXUS-1 does not call a restart, reconnect or green endpoint recovery. The claim requires proven fault activity, truthful degradation, proven removal, fresh owner-state snapshots, qualified trace evidence, invariant conservation and bounded stable progress.

PART IX COMPLETION

Lineage locates the case; logs preserve observations; traces explain attempts; metrics aggregate behavior; resilience bounds propagation; failure campaigns prove or falsify recovery. Part X will make that runtime reproducible and deployable.